



PART 8

PARALLEL AND DISTRIBUTED DATABASES

Database systems can be centralized, where one server machine executes operations on the database. Database systems can also be designed to exploit parallel computer architectures. Distributed databases span multiple geographically separated machines.

Chapter 20 first outlines the architectures of database systems running on server systems, which are used in centralized and client-server architectures. The chapter then outlines parallel computer architectures, and parallel database architectures designed for different types of parallel computers. Finally, the chapter outlines architectural issues in building a distributed database system.

Chapter 21 discusses techniques for data storage and indexing in parallel and distributed database systems. These include data partitioning and replication. Key-value stores, which offer some but not all features of a full database system, are discussed along with their benefits and drawbacks.

Chapter 22 discusses algorithms for query processing in parallel and distributed database systems. This chapter focuses on query processing in decision-support systems. Such systems need to execute queries on very large amounts of data, and parallel processing of the query across multiple nodes is critical for processing queries within acceptable response times. The chapter covers parallel sort and join, pipelining, the implementation of MapReduce systems, and parallel stream processing.

Chapter 23 discusses how to carry out transaction processing in parallel and distributed database systems. In addition to supporting concurrency control and recovery, the system must deal with issues pertaining to replication of data and with failures that involve some, but not all, nodes. The chapter covers atomic commit protocols and consensus protocols designed for distributed databases, distributed concurrency control, replica consistency, and trade-offs of consistency for the sake of performance and availability.

CHAPTER 20



Database-System Architectures

The architecture of a database system is greatly influenced by the underlying computer system on which it runs, in particular by such aspects as processor and memory architecture, and networking, as well as by requirements of parallelism and distribution. In this chapter, we provide a high-level view of database architectures, with a focus on how they are influenced by the underlying hardware, as well as by requirements of parallel and distributed processing.

20.1 Overview

The earliest databases were built to run on a single physical machine supporting multitasking; such *centralized database systems* are still widely used. An enterprise-scale application that runs on a centralized database system today may have from tens to thousands of users and database sizes ranging from megabytes to hundreds of gigabytes.

Parallel database systems were developed, starting in the late 1980s to execute tasks in parallel on a large number of machines. These were developed to handle high-end enterprise applications whose requirements in terms of transaction processing performance, time to process decision support queries, and storage capacity could not be met by centralized databases. These databases were designed to run in parallel on hundreds of machines. Today, the growth of parallel databases is driven not just by enterprise applications, but even more so by web-scale applications, which may have millions to even hundreds of millions of users and may need to deal with many petabytes of data.

Parallel data storage systems are designed primarily to store and retrieve data based on keys. Unlike parallel databases, data storage systems typically provide very limited support for transactions, and they lack support for declarative querying. On the other hand, such systems can be run in parallel on very large numbers of machines (thousands to tens of thousands), a scale that most parallel databases cannot handle.

Further, data are often generated and stored on different database systems, and there is a need to execute queries and update transactions across multiple databases. This need led to the development of *distributed database systems*. Techniques developed for fault tolerance in the context of distributed databases today also play a key role in

ensuring the extremely high reliability and availability of massively parallel database and data storage systems.

We study the architecture of database systems in this chapter, starting with the traditional centralized architectures and covering parallel and distributed database architectures. We cover issues of parallel and distributed data storage and indexing in Chapter 21. Chapter 22 covers issues of parallel and distributed query processing, while Chapter 23 covers issues of parallel and distributed transaction processing.

20.2 Centralized Database Systems

Centralized database systems are those that run on a single computer system. Such database systems span a range from single-user database systems running on mobile devices or personal computers to high-performance database systems running on a server with multiple CPU cores and disks and a large amount of main memory that can be accessed by any of the CPU cores. Centralized database systems are widely used for enterprise-scale applications.

We distinguish two ways in which computers are used: as single-user systems and as multiuser systems. Smartphones and personal computers fall into the first category. A typical **single-user system** is a system used by a single person, usually with only one processor (usually with multiple cores), and one or two disks.¹ A typical **multiuser system**, on the other hand, has multiple disks, a large amount of memory, and multiple processors. Such systems serve a large number of users who are connected to the system remotely, and they are called **server systems**.

Database systems designed for single-user systems usually do not provide many of the facilities that a multiuser database provides. In particular, they may support very simple concurrency control schemes, since highly concurrent access to the database is very unlikely. Provisions for crash recovery in such systems may also be either very basic (e.g., making a copy of data before updating it), or even absent in some cases. Such systems may not support SQL and may instead provide an API for data access. Such database systems are referred to as **embedded databases**, since they are usually designed to be linked to a single application program and are accessible only from that application.

In contrast, multiuser database systems support the full transactional features that we have studied earlier. Such databases are usually designed as **servers**, which service requests received from application programs; the requests could be in the form of SQL queries, or they could be requests for retrieving, storing, or updating data specified using an API.

Most general-purpose computer systems in use today have a few multicore processors (typically one to four), with each multicore processor having a few cores (typically

¹Recall that we use the term *disk* to refer to hard disks as well as solid-state drives.

4 to 8). Main memory is shared across all the processors. Parallelism with such a small number of cores, and with shared memory, is referred to as **coarse-grained parallelism**.

Operating systems that run on single-processor systems support multitasking, allowing multiple processes to run on the same processor in a time-shared manner. Actions of different processes may thus be interleaved. Databases designed for single-processor machines have long been designed to allow multiple processes or threads to access shared database structures concurrently. Thus, many of the issues in handling multiple processes running truly in parallel, such as concurrent access to data, are already addressed by databases designed for single-processor machines. As a result, database systems designed for time-shared single-processor machines could be adapted relatively easily to run on coarse-grained parallel systems.

Databases running on coarse-grained parallel machines traditionally did not attempt to partition a single query among the processors; instead, they ran each query on a single processor, allowing multiple queries to run concurrently. Thus, such systems support a higher throughput; that is, they allow a greater number of transactions to run per second, although individual transactions do not run any faster. In recent years, with even mobile phones supporting multiple cores, such systems are evolving to support parallel processing of individual queries.

In contrast, machines with **fine-grained parallelism** have a large number of processors, and database systems running on such machines attempt to parallelize single tasks (queries, for example) submitted by users.

Parallelism has emerged as a critical issue in the design of software systems in general, and in particular in the design of database systems. As a result, parallel database systems, which once were specialized systems running on specially designed hardware, are increasingly becoming the norm. We study the architecture of parallel database systems in Section 20.4.

20.3 Server System Architectures

Server systems can be broadly categorized as transaction servers and data servers.

- **Transaction-server** systems, also called **query-server** systems, provide an interface to which clients can send requests to perform an action, in response to which they execute the action and send back results to the client. Usually, client machines ship transactions to the server systems, where those transactions are executed, and results are shipped back to clients that are in charge of displaying the data. Requests may be specified through the use of SQL or through a specialized application program interface.
- **Data-server systems** allow clients to interact with the servers by making requests to read or update data, in units such as files, pages, or objects. For example, file servers provide a file-system interface where clients can create, update, read, and

delete files. Data servers for database systems offer much more functionality; they support units of data—such as pages, tuples, or objects—that are smaller than a file. They provide indexing facilities for data, and they provide transaction facilities so that the data are never left in an inconsistent state if a client machine or process fails.

Of these, the transaction-server architecture is by far the more widely used architecture, although parallel data servers are widely used to handle traffic at web scale. We shall elaborate on the transaction-server and data-server architectures in Section 20.3.1 and Section 20.3.2.

20.3.1 Transaction-Server Architecture

A typical transaction-server system today consists of multiple processes accessing data in shared memory, as in Figure 20.1. The processes that form part of the database system include:

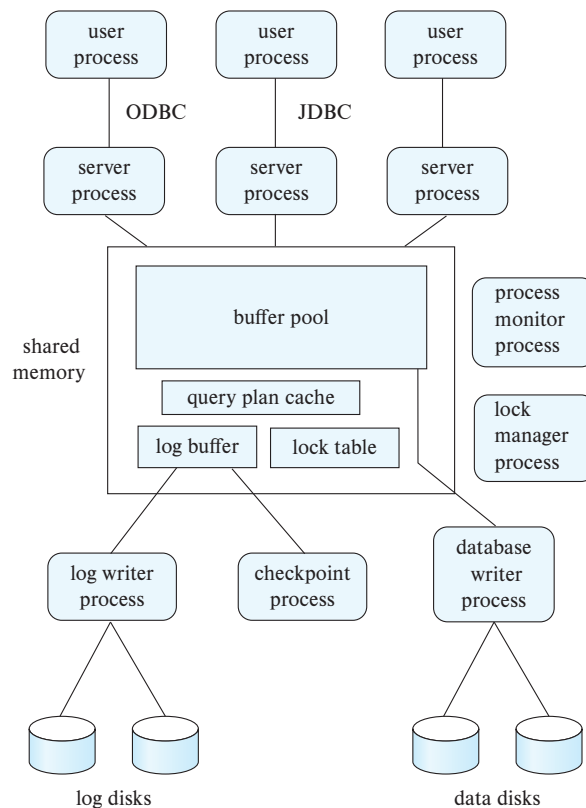


Figure 20.1 Shared memory and process structure.

- **Server processes:** These are processes that receive user queries (transactions), execute them, and send the results back. The queries may be submitted to the server processes from a user interface, or from a user process running embedded SQL, or via JDBC, ODBC, or similar protocols. Some database systems use a separate process for each user session, and a few use a single database process for all user sessions, but with multiple threads so that multiple queries can execute concurrently. (A **thread** is similar to a process, but multiple threads execute as part of the same process, and all threads within a process run in the same virtual-memory space. Multiple threads within a process can execute concurrently.) Many database systems use a hybrid architecture, with multiple processes, each one running multiple threads.
- **Lock manager process:** This process implements lock manager functionality, which includes lock grant, lock release, and deadlock detection.
- **Database writer process:** There are one or more processes that output modified buffer blocks back to disk on a continuous basis.
- **Log writer process:** This process outputs log records from the log record buffer to stable storage. Server processes simply add log records to the log record buffer in shared memory, and if a log force is required, they request the log writer process to output log records (recall that a log force causes the log contents in memory to be output to stable storage).
- **Checkpoint process:** This process performs periodic checkpoints.
- **Process monitor process:** This process monitors other processes, and if any of them fails, it takes recovery actions for the process, such as aborting any transaction being executed by the failed process and then restarting the process.

The shared memory contains all shared data, such as:

- Buffer pool.
- Lock table.
- Log buffer, containing log records waiting to be output to the log on stable storage.
- Cached query plans, which can be reused if the same query is submitted again.

All database processes can access the data in shared memory. Since multiple processes may read or perform updates on data structures in shared memory, there must be a mechanism to ensure **mutual exclusion**, that is, to ensure that a data structure is modified by at most one process at a time, and no process is reading a data structure while it is being written by other processes.

Such mutual exclusion can be implemented by means of operating system functions called semaphores. Alternative implementations, with less overhead, use one of

Note 20.1 ATOMIC INSTRUCTIONS

1. The instruction **test-and-set** (M) performs the following two actions atomically: (i) test, that is, read the value of memory location M , and then (ii) set it to 1; the test-and-set instruction returns the value that it read in step (i).

Suppose a memory location M representing an exclusive lock is initially set to 0. A process that wishes to get the lock executes the test-and-set (M). If it is the only process executing the instruction on M , the value that is read and returned would be 0, indicating to the process that it has acquired the lock, and M would be set to 1. When the process is done using the lock, it releases the lock by setting M back to 0.

If a second process executes test-and-set (M) before the lock is released, the value returned would be 1, indicating that some other process already has the lock. The process could repeat the execution of test-and-set on M periodically, until it gets a return value of 0, indicating that it has acquired the lock after it was released by another process.

Now, if two processes execute test-and-set (M) concurrently, one of them would see a return value of 0, while the other would see a return value of 1; this is because the read operation and the set operation are executed together, atomically. The first process to read the value would also set it to 1, and the second process would find that M is already set to 1. Thus, only one process acquires the lock, ensuring mutual execution.

2. The **compare-and-swap** instruction is another atomic instruction similar to the test-and-set instruction, but it takes the following operands: (M, V_o, V_n), where M is a memory location, and value V_o and V_n are two values (referred to as the old and new values). The instruction does the following atomically: it compares the value at M with V_o , and if it matches, it updates the value to V_n and returns success. If the values do not match, it does not update M , and it returns failure.

Similar to the case of test-and-set, we have a memory location M representing a lock, which is initially set to 0. A process that wants to acquire the lock executes compare-and-swap ($M, 0, id$) where id can be any nonzero value and is typically the process identifier. If no process has the lock, the compare-and-swap operation returns success, after storing the process identifier in M ; otherwise, the operation returns failure.

A benefit of compare-and-swap over the test-and-set implementation is that it is easy to find out which process has acquired the lock by just reading the content of M , if the process identifier is used as V_n .

the **atomic instructions**, test-and-set, or compare-and-swap, which are supported by the computer hardware. See Note 20.1 on page 966 for details of these instructions. All multiprocessor systems today support either the test-and-set or the compare-and-swap atomic instructions. Further details on these instructions may be found in operating systems textbooks.

Note that the atomic instructions can be used for mutual exclusion, which is equivalent to supporting exclusive locks, but they do not directly support shared locks. Thus, they cannot be used directly to implement general-purpose locking in databases. Atomic instructions are, however, used to implement short-duration locks, also known as latches, which are used for mutual exclusion in databases.

To avoid the overhead of message passing, in many database systems, server processes implement locking by directly updating the lock table (which is in shared memory) instead of sending lock request messages to a lock manager process. (The lock table is shown in Figure 18.10.)

Since multiple server processes may access the lock table in shared memory concurrently, processes must ensure mutual exclusion on access to the lock table. This is typically done by acquiring a mutex (also referred to as a latch) on the lock table, using the test-and-set or compare-and-swap instructions on a memory location representing a lock on the lock table.

A transaction that wants to acquire a lock by directly updating the lock table in shared memory executes the following steps.

1. Acquire a mutex (latch) on the lock table.
2. Check if the requested lock can be allocated, using the procedure we saw in Section 18.1.4. If it can, update the lock table to indicate the lock is allocated. Otherwise, update the lock table to indicate that the lock request is in the queue for that lock.
3. Release the mutex on the lock table.

If a lock cannot be obtained immediately because of a lock conflict, the transaction may periodically read the lock table to check if the lock has been allocated to it due to a lock release, which is described next.

Lock release is done as follows:

1. Acquire a mutex on the lock table
2. Remove the entry in the lock table for the lock being released.
3. If there are any other lock requests pending for the data item that can now be allocated to the lock, the lock table is updated to mark those requests as allocated. The rules on which lock requests may be granted are as described in Section 18.1.4.
4. Release the mutex on the lock table.

To avoid repeated checks on the lock table (an example of the phenomenon of *busy waiting*), operating system semaphores can be used by the lock request code to wait for a lock grant notification. The lock release code must then use the semaphore mechanism to notify waiting transactions that their locks have been granted.

Even if the system handles lock requests through shared memory, it still uses the lock manager process for deadlock detection.

20.3.2 Data Servers and Data Storage Systems

Data-server systems were originally developed to support data access from object-oriented databases; object-oriented databases allow programmers to use a programming language that allows creation, retrieval, and update of persistent objects.

Many of the target applications of object-oriented databases, such as computer-aided design (CAD) systems, required extensive computation on the retrieved data. For example, the CAD system may store a model of a computer chip or a building, and it may perform computations such as simulations on the retrieved model, which may be expensive in terms of CPU time.

If all the computation were done at the server, the server would be overloaded. Instead, in such an environment, it makes sense to store data on a separate data server machine, fetch data to client machines when needed, perform all processing at the client machines, and then to store new or updated data back on the data server machine. Thus, the processing power of client machines can be used to carry out the computation, while the server needs only to store and fetch data, without performing any computation.

More recently, a number of parallel data storage systems have been developed for handling very high volumes of data and transactions. Such systems do not necessarily support SQL, but instead provide APIs for storing, retrieving, and updating data items. Data items stored in such systems they could be tuples, or could be objects represented in formats such as JSON or XML, or they could even be files or documents.

We use the term **data item** to refer to tuples, objects, files, and documents. We also use the terms *data server* and *data storage system* interchangeably.

Data servers support communication of entire data items; in the case of very large data items, they may also support communication of only specified parts of the data item, for instance, specified blocks, instead of requiring that the entire data item be fetched or stored.

Data servers in earlier generations of storage systems supported a concept called *page shipping*, where the unit of communication is a database page that may potentially contain multiple data items. Page shipping is not used today, since storage systems do not expose the underlying storage layout to clients.

20.3.3 Caching at Clients

The time cost of communication between a client application and a server (whether a transaction server, or a data server) is high compared to that of a local memory

reference (milliseconds, versus less than 100 nanoseconds). The time to send a message over a network, and get a response back, called the *network round-trip time*, or *network latency*, can be nearly a millisecond even if the data server is in the same location as the client.

As a result, applications running at the clients adopt several optimization strategies to reduce the effects of network latency. The same strategies can also be useful in parallel database systems, where some of the data required for processing a query may be stored on a different machine from where it is consumed. The optimization strategies include the following:

- **Prefetching.** If the unit of communication is a single small item, the overhead of message passing is high compared to the amount of data transmitted. In particular, network latency can cause significant delays if a transaction makes repeated requests for data items across a network.

Thus, when an item is requested, it may make sense to also send other items that are likely to be used in the near future. Fetching items even before they are requested is called **prefetching**.

- **Data caching.** Data that are shipped to a client on behalf of a transaction can be **cached** at the client within the scope of a single transaction. Data can be cached even after the transaction completes, allowing successive transactions at the same client to make use of the cached data.

However, **cache coherency** is an issue: Even if a transaction finds cached data, it must make sure that those data are up to date, since they may have been updated, or even deleted, by a different client after they were cached. Thus, a message must still be exchanged with the server to check validity of the data and to acquire a lock on the data, unless the application is willing to live with potentially stale data. Further, new tuples may have been inserted after a transaction caches data, which may not be in the cache. The transaction may have to contact the server to find such tuples.

- **Lock caching.** If the usage of data is mostly partitioned among the clients, with clients rarely requesting data that are also requested by other clients, locks can also be cached at the client machine. Suppose that a client finds a data item in the cache, and that it also finds the lock required for an access to the data item in the cache. Then, the access can proceed without any communication with the server. However, the server must keep track of cached locks; if a client requests a lock from the server, the server must **call back** all conflicting locks on the data item from any other client machines that have cached the locks. The task becomes more complicated when machine failures are taken into account.
- **Adaptive lock granularity.** If a transaction requires locks on multiple data items, discovered in the course of a transaction, and each lock acquisition requires a round trip to a data server, the transaction may waste a good deal of time on

just lock acquisition. In such a situation, multi-granularity locking can be used to avoid multiple requests. For example, if multiple data items are stored in a page, a single page lock (which is at a coarser granularity) can avoid the need to acquire multiple data-item locks (which are at a finer granularity). This strategy works well if there is very little lock contention, but with higher contention, acquiring a coarse granularity lock can affect concurrency significantly.

Lock de-escalation, is a way of adaptively decreasing the lock granularity if there is higher contention. Lock de-escalation is initiated by the data server sending a request to the client to de-escalate a lock, and the client responds by acquiring finer-granularity locks and then releasing the coarser-granularity lock.

When switching to a finer granularity, if some of the locks were for cached data items that are not currently locked by any transaction at a client, the data item can be removed from the cache instead of acquiring a finer-granularity lock on it.

20.4 Parallel Systems

Parallel systems improve processing and I/O speeds by using a large number of computers in parallel. Parallel machines are becoming increasingly common, making the study of parallel database systems correspondingly more important.

In parallel processing, many operations are performed simultaneously, as opposed to serial processing, in which the computational steps are performed sequentially. A **coarse-grain** parallel machine consists of a small number of powerful processors; a **massively parallel** or **fine-grain parallel** machine uses thousands of smaller processors. Virtually all high-end server machines today offer some degree of coarse-grain parallelism, with up to two or four processors each of which may have 20 to 40 cores.

Massively parallel computers can be distinguished from the coarse-grain parallel machines by the much larger degree of parallelism that they support. It is not practical to share memory between a large number of processors. As a result, massively parallel computers are typically built using a large number of computers, each of which has its own memory, and often, its own set of disks. Each such computer is referred to as a **node** in the system.

Parallel systems at the scale of hundreds to thousands of nodes or more are housed in a **data center**, which is a facility that houses a large number of servers. Data centers provide high-speed network connectivity within the data center, as well as to the outside world. The numbers and sizes of data centers have grown tremendously in the last decade, and modern data centers may have several hundred thousand servers.

20.4.1 Motivation for Parallel Databases

The transaction requirements of organizations have grown with the increasing use of computers. Moreover, the growth of the World Wide Web has created many sites with millions of viewers, and the increasing amounts of data collected from these viewers has produced extremely large databases at many companies.

The driving force behind parallel database systems is the demands of applications that have to query extremely large databases (of the order of petabytes—that is, 1000 terabytes, or equivalently, 10^{15} bytes) or that have to process an extremely large number of transactions per second (of the order of thousands of transactions per second). Centralized and client-server database systems are not powerful enough to handle such applications.

Web-scale applications today often require hundreds to thousands of nodes (and in some cases, tens of thousands of nodes) to handle the vast number of users on the web.

Organizations are using these increasingly large volumes of data—such as data about what items people buy, what web links users click on, and when people make telephone calls—to plan their activities and pricing. Queries used for such purposes are called **decision-support queries**, and the data requirements for such queries may run into terabytes. Single-node systems are not capable of handling such large volumes of data at the required rates.

The set-oriented nature of database queries naturally lends itself to parallelization. A number of commercial and research systems have demonstrated the power and scalability of parallel query processing.

As the cost of computing systems has reduced significantly over the years, parallel machines have become common and relatively inexpensive. Individual computers have themselves become parallel machines using multicore architectures. Parallel databases are thus quite affordable even for small organizations.

Parallel database systems which can support hundreds of nodes have been available commercially for several decades, but the number of such products has seen a significant increase since the mid 2000s. Open-source platforms for parallel data storage such as the Hadoop File System (HDFS), and HBase, and for query processing, such as Hadoop Map-Reduce and Hive (among many others), have also seen extensive adoption.

It is worth noting that application programs are typically built such that they can be executed in parallel on a number of application servers, which communicate over a network with a database server, which may itself be a parallel system. The parallel architectures described in this section can be used not only for data storage and query processing in the database but also for parallel processing of application programs.

20.4.2 Measures of Performance for Parallel Systems

There are two main measures of performance of a database system: (1) **throughput**, the number of tasks that can be completed in a given time interval, and (2) **response time**, the amount of time it takes to complete a single task from the time it is submitted. A system that processes a large number of small transactions can improve throughput by processing many transactions in parallel. A system that processes large transactions can improve response time as well as throughput by performing subtasks of each transaction in parallel.

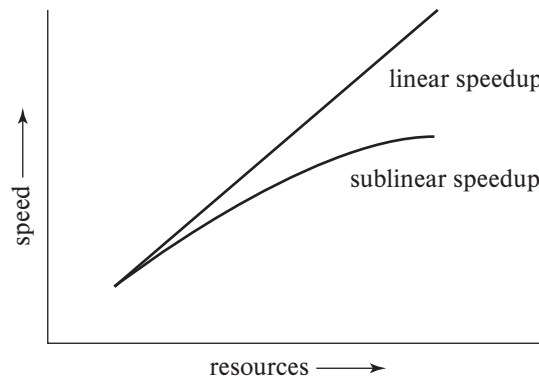


Figure 20.2 Speedup with increasing resources.

Parallel processing within a computer system allows database-system activities to be speeded up, allowing faster response to transactions, as well as more transactions to be executed per second. Queries can be processed in a way that exploits the parallelism offered by the underlying computer system.

Two important issues in studying parallelism are speedup and scaleup. Running a given task in less time by increasing the degree of parallelism is called **speedup**. Handling larger tasks by increasing the degree of parallelism is called **scaleup**.

Consider a database application running on a parallel system with a certain number of processors and disks. Now suppose that we increase the size of the system by increasing the number of processors, disks, and other components of the system. The goal is to process the task in time inversely proportional to the number of processors and disks allocated. Suppose that the execution time of a task on the larger machine is T_L , and that the execution time of the same task on the smaller machine is T_S . The speedup due to parallelism is defined as T_S/T_L . The parallel system is said to demonstrate **linear speedup** if the speedup is N when the larger system has N times the resources (processors, disk, and so on) of the smaller system. If the speedup is less than N , the system is said to demonstrate **sublinear speedup**. Figure 20.2 illustrates linear and sublinear speedup.²

Scaleup relates to the ability to process larger tasks in the same amount of time by providing more resources. Let Q be a task, and let Q_N be a task that is N times bigger than Q . Suppose that the execution time of task Q on a given machine M_S is T_S , and the execution time of task Q_N on a parallel machine M_L that is N times larger than M_S is T_L . The scaleup is then defined as T_S/T_L . The parallel system M_L is said to demonstrate **linear scaleup** on task Q if $T_L = T_S$. If $T_L > T_S$, the system is said

²In some cases, a parallel system may provide superlinear speedup, that is, an N times larger system may provide speedup greater than N . This could happen, for example, because data that did not fit in the main memory of a smaller system do fit in the main memory of a larger system, avoiding disk I/O. Similarly, data may fit in the cache of a larger system, reducing memory accesses compared to a smaller system, which could lead to superlinear speedup.

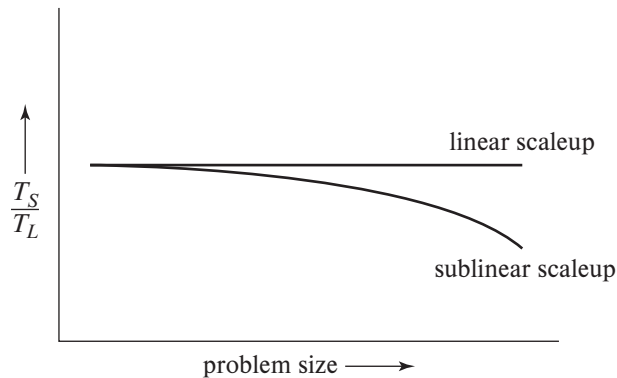


Figure 20.3 Scaleup with increasing problem size and resources.

to demonstrate **sublinear scaleup**. Figure 20.3 illustrates linear and sublinear scaleups (where the resources increase in proportion to problem size). There are two kinds of scaleup that are relevant in parallel database systems, depending on how the size of the task is measured:

- In **batch scaleup**, the size of the database increases, and the tasks are large jobs whose runtime depends on the size of the database. An example of such a task is a scan of a relation whose size is proportional to the size of the database. Thus, the size of the database is the measure of the size of the problem. Batch scaleup also applies in scientific applications, such as executing a weather simulation at an N -times finer resolution,³ or performing the simulation for an N -times longer period of time.
- In **transaction scaleup**, the rate at which transactions are submitted to the database increases, and the size of the database increases proportionally to the transaction rate. This kind of scaleup is what is relevant in transaction-processing systems where the transactions are small updates—for example, a deposit or withdrawal from an account—and transaction rates grow as more accounts are created. Such transaction processing is especially well adapted for parallel execution, since transactions can run concurrently and independently on separate nodes, and each transaction takes roughly the same amount of time, even if the database grows.

Scaleup is usually the more important metric for measuring the efficiency of parallel database systems. The goal of parallelism in database systems is usually to make sure that the database system can continue to perform at an acceptable speed, even as the size of the database and the number of transactions increases. Increasing the ca-

³For example, a weather simulation that divides the atmosphere in a particular region into cubes of side 200 meters may need to be modified to use a finer resolution, with cubes of side 100 meters; the number of cubes would thus be scaled up by a factor of 8.

capacity of the system by increasing the parallelism provides a smoother path for growth for an enterprise than does replacing a centralized system with a faster machine (even assuming that such a machine exists). However, we must also look at absolute performance numbers when using scaleup measures; a machine that scales up linearly may perform worse than a machine that scales less than linearly, simply because the latter machine is much faster to start off with.

A number of factors work against efficient parallel operation and can diminish both speedup and scaleup.

- **Sequential computation.** Many tasks have some components that can benefit from parallel processing, and some components that have to be executed sequentially. Consider a task that takes time T to run sequentially. Suppose the fraction of the total execution time that can benefit from parallelization is p , and that part is executed by n nodes in parallel. Then the total time taken would be $(1-p)T + (p/n)T$, and the speedup would be $\frac{1}{(1-p)+(p/n)}$. (This formula is referred to as **Amdahl's law**.) If the fraction p is, say $\frac{9}{10}$, then the maximum speedup possible, even with very large n , would be 10.

Now consider scaleup, where the problem size increases. If the time taken by the sequential part of a task increases along with the problem size, scaleup will be similarly limited. Suppose fraction p of the execution time of a problem benefits from increasing resources, while fraction $(1-p)$ is sequential and does not benefit from increasing resources. Then the scaleup with n times more resources on a problem that is n times larger will be $\frac{1}{n(1-p)+p}$. (This formula is referred to as **Gustafson's law**.) However, if the time taken by the sequential part does not increase with problem size, its impact on scaleup will be less as the problem sizes.

Start-up costs. There is a start-up cost associated with initiating a single process. In a parallel operation consisting of thousands of processes, the *start-up time* may overshadow the actual processing time, affecting speedup adversely.

- **Interference.** Since processes executing in a parallel system often access shared resources, a slowdown may result from the *interference* of each new process as it competes with existing processes for commonly held resources, such as a system bus, or shared disks, or even locks. Both speedup and scaleup are affected by this phenomenon.
- **Skew.** By breaking down a single task into a number of parallel steps, we reduce the size of the average step. Nonetheless, the service time for the single slowest step will determine the service time for the task as a whole. It is often difficult to divide a task into exactly equal-sized parts, and the way that the sizes are distributed is therefore *skewed*. For example, if a task of size 100 is divided into 10 parts, and the division is skewed, there may be some tasks of size less than 10 and some tasks of size more than 10; if even one task happens to be of size 20, the speedup obtained by running the tasks in parallel is only 5, instead of 10 as we would have hoped.

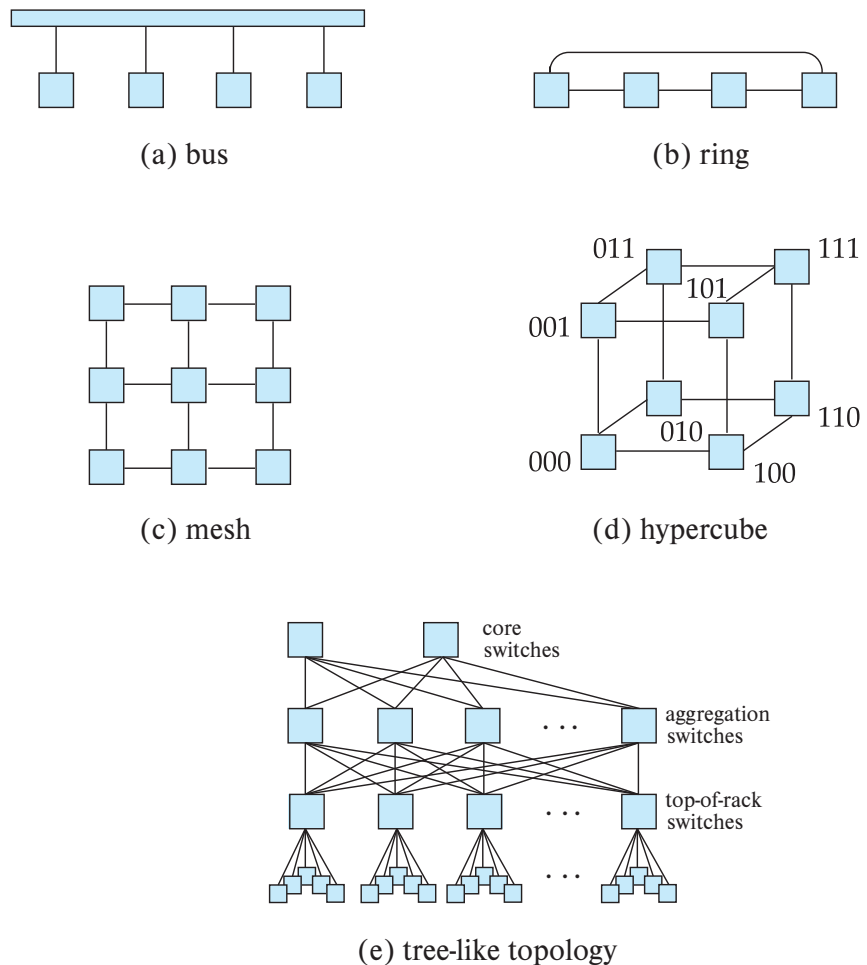


Figure 20.4 Interconnection networks.

20.4.3 Interconnection Networks

Parallel systems consist of a set of components (processors, memory, and disks) that can communicate with each other via an **interconnection network**. Figure 20.4 shows several commonly used types of interconnection networks:

- **Bus.** All the system components can send data on and receive data from a single communication bus. This type of interconnection is shown in Figure 20.4a. Bus interconnects were used in earlier days to connect multiple nodes in a network, but they are no longer used for this task. However, bus interconnections are still used for connecting multiple CPUs and memory units within a single node, and they work well for small numbers of processors. However, they do not scale well

with increasing parallelism, since the bus can handle communication from only one component at a time; with increasing numbers of CPUs and memory banks in a node, other interconnection mechanisms such as ring or mesh interconnections are now used even within a single node.

- **Ring.** The components are nodes arranged in a ring (circle), and each node is connected to its two adjacent nodes in the ring, as shown in Figure 20.4b. Unlike a bus, each link can transmit data concurrently with other links in the ring, leading to better scalability. However, to transmit data from one node to another node on the ring may require a large number of hops; specifically, up to $n/2$ hops may be needed on a ring with n nodes, assuming communication can be done in either direction on the ring. Furthermore, the transmission delay increases if the number of nodes in the ring is increased.
- **Mesh.** The components are nodes in a grid, and each component connects to all its adjacent components in the grid. In a two-dimensional mesh, each node connects to (up to) four adjacent nodes, while in a three-dimensional mesh, each node connects to (up to) six adjacent nodes. Figure 20.4c shows a two-dimensional mesh. Nodes that are not directly connected can communicate with one another by routing messages via a sequence of intermediate nodes that are directly connected to one another. The number of communication links grows as the number of components grows, and the communication capacity of a mesh therefore scales better with increasing parallelism.

Mesh interconnects are used to connect multiple cores in a processor, or processors in a single server, to each other; each processor core has direct access to a bank of memory connected to the processor core, but the system transparently fetches data from other memory banks by sending messages over the mesh interconnects.

However, mesh interconnects are no longer used for interconnecting nodes, since the number of hops required to transmit data increases significantly with the number of nodes (the number of hops required to transmit data from one node to another node in a mesh is proportional in the worst case to the square root of the number of nodes). Parallel systems today have very large numbers of nodes, and mesh interconnects would thus be impractically slow.

- **Hypercube.** The components are numbered in binary, and a component is connected to another if the binary representations of their numbers differ in exactly one bit. Thus, each of the n components is connected to $\log(n)$ other components. Figure 20.4d shows a hypercube with eight nodes. In a hypercube interconnection, a message from a component can reach any other component by going through at most $\log(n)$ links. In contrast, in a mesh architecture a component may be $2(\sqrt{n} - 1)$ links away from some of the other components (or $\sqrt{n}$ links away, if the mesh interconnection wraps around at the edges of the grid). Thus communication delays in a hypercube are significantly lower than in a mesh.

Hypercubes have been used to interconnect nodes in massively parallel computers in earlier days, but they are no longer commonly used.

- **Tree-like.** Server systems in a data center are typically mounted in racks, with each rack holding up to about 40 nodes. Multiple racks are used to build systems with larger numbers of nodes. A key issue is how to interconnect such nodes.

To connect nodes within a rack, there is typically a network switch mounted at the top of the rack; 48 port switches are commonly used, so a single switch can be used to connect all the servers in a rack. Current-generation network switches typically support a bandwidth of 1 to 10 gigabits per second (Gbps) simultaneously from/to each of the servers connected to the switch, although more expensive network interconnects with 40 to 100 Gbps bandwidths are available.

Multiple top-of-rack switches (also referred to as **edge switches**) can in turn be connected to another switch, called an **aggregation switch**, allowing interconnection between racks. If there are a large number of racks, the racks may be divided into groups, with one aggregation switch connecting a group of racks, and all the aggregation switches in turn connected to a **core switch**. Such an architecture is a **tree topology** with three tiers. The core switch at the top of the tree also provides connectivity to outside networks.

A problem with this basic tree structure, which is frequently used in local-area networks within organizations, is that the available bandwidth between racks is often not sufficient if multiple machines in a rack try to communicate significant amounts of data with machines from other racks. Typically, the interconnects of the aggregation switches support higher bandwidths of 10 to 40 Gbps, although interconnects of 100 Gbps are available. Interconnects of even higher capacity can be created by using multiple interconnects in parallel. However, even such high-speed links can be saturated if a large enough number of servers in a rack attempt to communicate at their full connection bandwidth to servers in other racks.

To avoid the bandwidth bottleneck of a tree structure, data centers typically connect each top-of-rack (edge) switch to multiple aggregation switches. Each aggregation switch in turn is linked to a number of core switches at the next layer. Such an interconnection topology is called a **tree-like topology**; Figure 20.4e shows a tree-like topology with three tiers. The tree-like topology is also referred to as a **fat-tree topology**, although originally the fat-tree topology referred to a tree topology where edges higher in the tree have a higher bandwidth than edges lower in the tree.

The benefit of the tree-like architecture is that each top-of-rack switch can route its messages through any of the aggregation switches that it is connected to, increasing the inter-rack bandwidth greatly as compared to the tree topology. Similarly, each aggregation switch can communicate with another aggregation switch via any of the core switches that it is connected to, increasing the bandwidth available between the aggregation switches. Further, even if an aggregation or edge switch fails, there are alternative paths through other switches. With appropriate

routing algorithms, the network can continue functioning even if a switch fails, making the network *fault-tolerant*, at least to failures of one or a few switches.

A tree-like architecture with three tiers can handle a **cluster** of tens of thousands of machines. Although a tree-like topology improves the inter-rack bandwidth greatly compared to a tree topology, parallel processing applications, including parallel storage and parallel database systems, perform best if they are designed in a way that reduces inter-rack traffic.

The tree-like topology and variants of it are widely used in data centers today. The complex interconnection networks in a data center are referred to as a **data center fabric**.

While network topologies are very important for scalability, a key to network performance is network technology used for individual links. The popular technologies include:

- **Ethernet:** The dominant technology for network connections today is the Ethernet technology. Ethernet standards have evolved over time, and the predominant versions used today are 1-gigabit Ethernet and 10-gigabit Ethernet, which support bandwidths of 1 and 10 gigabits per second respectively. Forty-gigabit Ethernet and 100-gigabit Ethernet technologies are also available at a higher cost and are seeing increasing usage. Ethernet protocols can be used over cheaper copper cables for short distances, and over optical fiber for longer distances.
- **Fiber channel:** The Fiber Channel Protocol standard was designed for high-speed interconnection between storage systems and computers, and it is predominantly used to implement storage area networks (described in Section 20.4.6). The different versions of the standard have supported increasing bandwidth over the years, with 16 gigabits per second available as of 2011, and 32 and 128 gigabits per second supported from 2016.
- **Infiniband:** The Infiniband standard was designed for interconnections with a data center; it was specifically designed for high-performance computing applications which need not just very high bandwidth, but also very low latency. The Infiniband standard has evolved, with link speeds of 8-gigabits per second available by 2007 and 24-gigabits per second available by 2014. Multiple links can be aggregated to give a bandwidth of 120 to 290-gigabits per second.

The latency associated with message delivery is as important as bandwidth for many applications. A key benefit of Infiniband is that it supports latencies as low as 0.7 to 0.5 microseconds. In contrast, Ethernet latencies can be up to hundreds of microseconds in an unoptimized local-area network, while latency-optimized Ethernet implementations still have latencies of several microseconds.

One of the important techniques used to reduce latency is to allow applications to send and receive messages by directly interfacing with the hardware, bypassing the operating system. With the standard implementations of the networking stack, applications

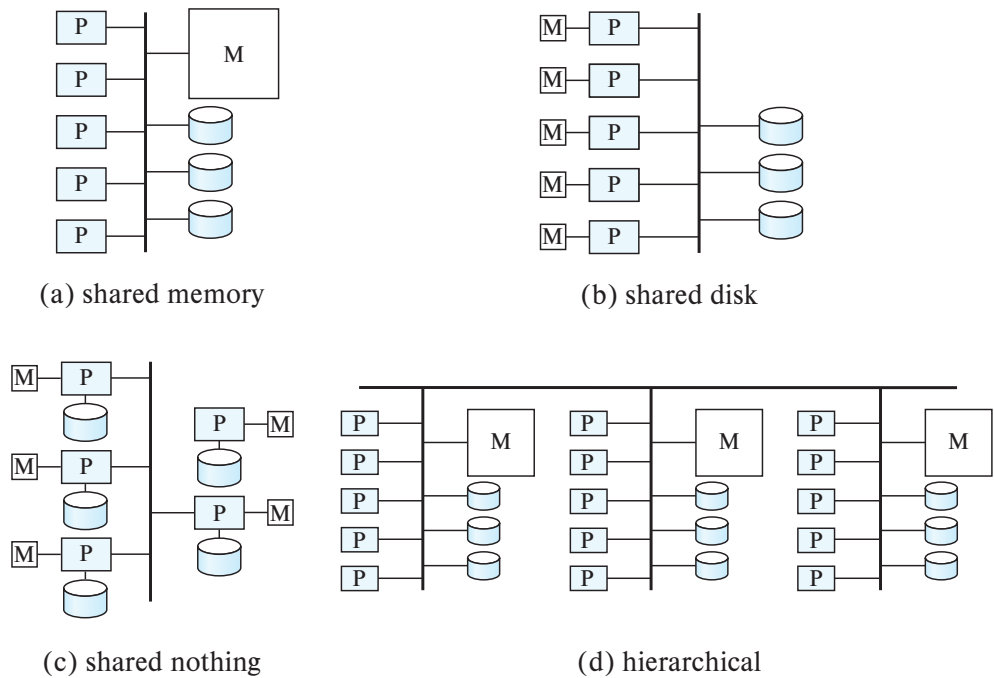


Figure 20.5 Parallel database architectures.

send messages to the operating system, which in turn interfaces with the hardware, which in turn delivers the message to the other computer, where again the hardware interfaces with the operating system, which then interfaces with the application to deliver the message. Support for direct access to the network interface, bypassing the operating system, reduces the communication latency significantly.

Another approach to reducing latency is to use **remote direct memory access (RDMA)**, a technology which allows a process on one node to directly read or write to memory on another node, without explicit message passing. Hardware support ensures that RDMA can transfer data at very high rates with very low latency. RDMA implementations can use Infiniband, Ethernet, or other networking technologies for physical communication between nodes.

20.4.4 Parallel Database Architectures

There are several architectural models for parallel machines. Among the most prominent ones are those in Figure 20.5 (in the figure, M denotes memory, P denotes a processor, and disks are shown as cylinders):

- **Shared memory.** All the processors share a common memory (Figure 20.5a).

- **Shared disk.** A set of nodes that share a common set of disks; each node has its own processor and memory (Figure 20.5b). Shared-disk systems are sometimes called **clusters**.
- **Shared nothing.** A set of nodes that share neither a common memory nor common disk (Figure 20.5c).
- **Hierarchical.** This model is a hybrid of the preceding three architectures (Figure 20.5d). This model is the most widely used model today.

In Section 20.4.5 through Section 20.4.8, we elaborate on each of these models.

Note that the interconnection networks are shown in an abstract manner in Figure 20.5. Do not interpret the interconnection networks shown in the figures as necessarily being a bus; in fact other interconnection networks are used in practice. For example, mesh networks are used within a processor, and tree-like networks are often used to interconnect nodes.

20.4.5 Shared Memory

In a **shared-memory** architecture, the processors have access to a common memory, typically through an interconnection network. Disks are also shared by the processors. The benefit of shared memory is extremely efficient communication between processes—data in shared memory can be accessed by any process without being moved with software. A process can send messages to other processes much faster by using memory writes (which usually take less than a microsecond) than by sending a message through a communication mechanism.

Multicore processors with 4 to 8 cores are now common not just in desktop computers, but even in mobile phones. High-end processing systems such as Intel’s Xeon processor have up to 28 cores per CPU, with up to 8 CPUs on a board, while the Xeon Phi coprocessor systems contain around 72 cores, as of 2018, and these numbers have been increasing steadily. The reason for the increasing number of cores is that the sizes of features such as logic gates in integrated circuits has been decreasing steadily, allowing more gates to be packed in a single chip. The number of transistors that can be accommodated on a given area of silicon has been doubling approximately every 1 1/2 to 2 years.⁴

Since the number of gates required for a processor core has not increased correspondingly, it makes sense to have multiple processors on a single chip. To maintain a distinction between on-chip multiprocessors and traditional processors, the term **core** is used for an on-chip processor. Thus, we say that a machine has a multicore processor.

⁴Gordon Moore, cofounder of Intel, predicted such an exponential growth in the number of transistors back in the 1960s; his prediction is popularly known as **Moore’s law**, even though, technically, it is not a *law*, but rather an observation and a prediction. In earlier decades, processor speeds also increased along with the decrease in the feature sizes, but that trend ended in the mid-2000s since processor clock frequencies beyond a few gigahertz could not be attained without unreasonable increase in power consumption and heat generation. Moore’s law is sometimes erroneously interpreted to have predicted exponential increases in processor speeds.

All the cores on a single processor typically access a shared memory. Further, a system can have multiple processors which can share memory. Another effect of the increasing number of gates has been the steady increase in the size of main memory as well as a decrease in cost, per-byte, of main memory.

Given the availability of multicore processors at a low cost, as well as the concurrent availability of very large amounts of memory at a low cost, shared-memory parallel processing has become increasingly important in recent years.

20.4.5.1 Shared-Memory Architectures

In earlier generation architectures, processors were connected to memory via a bus, with all processor cores and memory banks sharing a single bus. A downside of shared-memory accessed via a common bus is that the bus or the interconnection network becomes a bottleneck, since it is shared by all processors. Adding more processors does not help after a point, since the processors will spend most of their time waiting for their turn on the bus to access memory.

As a result, modern shared-memory architectures associate memory directly with processors; each processor has locally connected memory, which can be accessed very quickly; however, each processor can also access memory associated with other processors; a fast interprocessor communication network ensures that data are fetched with relatively low overhead. Since there is a difference in memory access speed depending on which part of memory is accessed, such an architecture is often referred to as **non-uniform memory architecture (NUMA)**.

Figure 20.6 shows a conceptual architecture of a modern shared-memory system with multiple processors; note that each processor has a bank of memory directly connected to it, and the processors are linked by a fast interconnect system; processors are also connected to I/O controllers which interface with external storage.

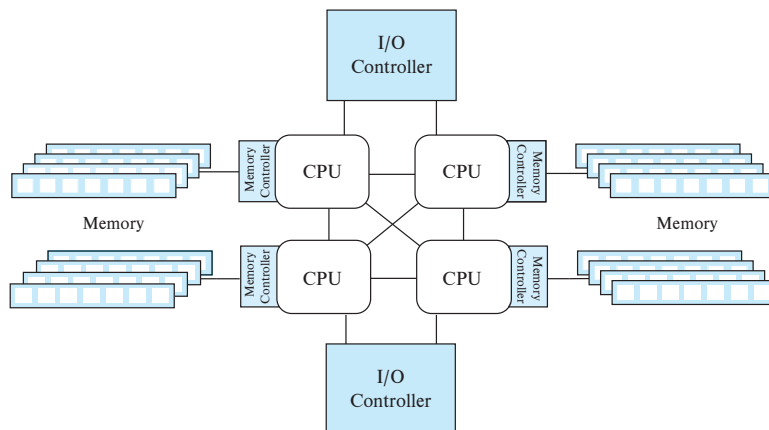


Figure 20.6 Architecture of a modern shared-memory system.

Because shared-memory architectures require specialized high-speed interconnects between cores and between processors, the number of cores/processors that can be interconnected in a shared-memory system is relatively small. As a result, the scalability of shared-memory parallelism is limited to at most a few hundred cores.

Processor architectures include cache memory, since access to cache memory is much faster than access to main memory (cache can be accessed in a few nanoseconds compared to nearly a hundred nanoseconds for main memory). Large cache memory is particularly important in shared-memory architectures, since a large cache can help minimize the number of accesses to shared memory.

If an instruction needs to access a data item that is not in cache, it must be fetched from main memory. Because main memory is much slower than processors, a significant amount of potential processing speed may be lost while a core waits for data from main memory. These waits are referred to as **cache misses**.

Many processor architectures support a feature called **hyper-threading**, or **hardware threads**, where a single physical core appears as two or more logical cores or threads. Different processes could be mapped to different logical cores. Only one of the logical cores corresponding to a single physical core can actually execute at any time. But the motivation for logical cores is that if the code running on one logical core blocks on a cache miss, waiting for data to be fetched from memory, the hardware of the physical core can start execution of one of the other logical cores instead of idling while waiting for data to be fetched from memory.

A typical multicore processor has multiple levels of cache, with the L1 cache being fastest to access, but also the smallest; lower cache levels such as L2 and L3 are slower (although still much faster than main memory) but considerably larger than the L1 cache. Lower cache levels are usually shared between multiple cores on a single processor. In the cache architecture shown in Figure 20.7, the L1 and L2 caches are local to each of the 4 cores, while the L3 cache is shared by all cores of the processor. data are read into, or written from, cache in units of a **cache line**, which typically consists of 64 consecutive bytes.

Core 0	Core 1	Core 2	Core 3
L1 Cache	L1 Cache	L1 Cache	L1 Cache
L2 Cache	L2 Cache	L2 Cache	L2 Cache
Shared L3 Cache			

Figure 20.7 Multilevel cache system.

20.4.5.2 Cache Coherency

Cache coherency is an issue whenever there are multiple cores or processors, each with its own cache. An update done on one core may not be seen by another core, if the local cache on the second core contains an old value of the affected memory location. Thus, whenever an update occurs to a memory location, copies of the content of that memory location that are cached on other caches must be invalidated.

Such invalidation is done lazily in many processor architectures; that is, there may be some time lag between a write to a cache and the dispatch of invalidation messages to other caches; in addition there may be a further lag in processing invalidation messages that are received at a cache. (Requiring immediate invalidation to be done always can cause a significant performance penalty, and thus it is not done in current-generation systems.) Thus, it is quite possible for a write to happen on one processor, and a subsequent read on another processor may not see the updated value.

Such a lack of cache coherency can cause problems if a process expects to see an updated memory location but does not. Modern processors therefore support **memory barrier** instructions, which ensure certain orderings between load/store operations before the barrier and those after the barrier. For example, the **store barrier** instruction (**sfence**) on the x86 architecture forces the processor to wait until invalidation messages are sent to all caches for all updates done prior to the instruction, before any further load/store operations are issued. Similarly, the **load barrier** instruction (**lfence**) ensures all received invalidation messages have been applied before any further load/store operations are issued. The **mfence** instruction does both of these tasks.

Memory barrier instructions must be used with interprocess synchronization protocols to ensure that the protocols execute correctly. Without the use of memory barriers, if the caches are not “strongly” coherent, the following scenario can happen. Consider a situation where a process *P1* updates memory location *A* first, then location *B*; a concurrent process *P2* running on a different core or processor reads *B* first and then reads *A*. With a coherent cache, if *P2* sees the updated value of *B*, it must also see the updated value of *A*. However, in the absence of cache coherence, the writes may be propagated out of order, and *P2* may thus see the updated value of *B* but the old value of *A*. While many architectures disallow out-of-order propagation of writes, there are other subtle errors that can occur due to lack of cache coherency. However, executing **sfence** instructions after each of these writes and **lfence** before each of the reads will always ensure that reads see a cache coherent state. As a result, in the above example, the updated value of *B* will be seen only if the updated value of *A* is seen.

It is worth noting that programs do not need to include any extra code to deal with cache coherency, as long as they acquire locks before accessing data, and release locks only after performing updates, since lock acquire and release functions typically include the required memory barrier instructions. Specifically, an **sfence** instruction is executed as part of the lock release code, before the data item is actually unlocked. Similarly an **lfence** is executed right after locking a data item, as part of the lock acquisition

function, and is thus executed before the item is read. Thus, the reader is guaranteed to see the most recent value written to the data item.

Synchronization primitives supported in a variety of languages also internally execute memory barrier instructions; as a result, programmers who use these primitives need not be concerned about lack of cache coherency.

It is also interesting to note that many processor architectures use a form of hardware-level shared and exclusive locking of memory locations to ensure cache coherency. A widely used protocol, called the MESI protocol, can be understood as follows: Locking is done at the level of cache lines, containing multiple memory locations, instead of supporting locks on individual memory locations, since cache lines are the units of cache access. Locking is implemented in the hardware, rather than in software, to provide the required high performance.

The MESI protocol keeps track of the state of each cache line, which can be Modified (updated after exclusive locking), Exclusive locked (locked but not yet modified, or already written back to memory), Share locked, or Invalid. A read of a memory location automatically acquires a shared lock on the cache line containing that location, while a memory write gets an exclusive lock on the cache line before performing the write. In contrast to database locks, memory lock requests do not wait; instead they immediately revoke conflicting locks. Thus, an exclusive lock request automatically invalidates all cached copies of the cache line and revokes all shared locks on the cache line. Symmetrically, a shared lock request causes any existing exclusive lock to be revoked and then fetches the latest copy of the memory location into cache.

In principle, it is possible to ensure “strong” cache coherency with such a locking-based cache coherence protocol, making memory barrier instructions redundant. However, many implementations include some optimizations that speed up processing, such as allowing delayed delivery of invalidation messages, at the cost of not guaranteeing cache coherence. As a result, memory barrier instructions are required on many processor architectures to ensure cache coherency.

20.4.6 Shared Disk

In the **shared-disk** model, each node has its own processors and memory, but all nodes can access all disks directly via an interconnection network. There are two advantages of this architecture over a shared-memory architecture. First, a shared-disk system can scale to a larger number of processors than a shared-memory system. Second, it offers a cheap way to provide a degree of **fault tolerance**: If a node fails, the other nodes can take over its tasks, since the database is resident on disks that are accessible from all nodes.

We can make the disk subsystem itself fault tolerant by using a RAID architecture, as described in Chapter 12, allowing the system to function even if individual disks fail. The presence of a large number of storage devices in a RAID system also provides some degree of I/O parallelism.

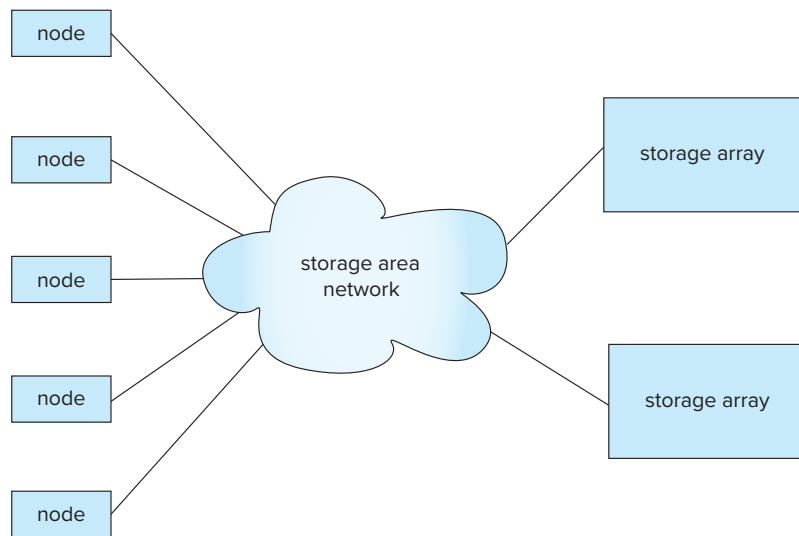


Figure 20.8 Storage-area network.

A **storage-area network (SAN)** is a high-speed local-area network designed to connect large banks of storage devices (disks) to nodes that use the data (see Figure 20.8). The storage devices physically consist of an array of multiple disks but provide a view of a logical disk, or set of disks, that hides the details of the underlying disks. For example, a logical disk may be much larger than any of the physical disks, and a logical disk's size can be increased by adding more physical disks. The processing nodes can access disks as if they are local disks, even though they are physically separate.

Storage-area networks are usually built with redundancy, such as multiple paths between nodes, so if a component such as a link or a connection to the network fails, the network continues to function.

Storage-area networks are well suited for building shared-disk systems. The shared-disk architecture with storage-area networks has found acceptance in applications that do not need a very high degree of parallelism but do require high availability.

Compared to shared-memory systems, shared-disk systems can scale to a larger number of processors, but communication across nodes is slower (up to a few milliseconds in the absence of special-purpose hardware for communication), since it has to go through a communication network.

One limitation of shared-disk systems is that the bandwidth of the network connection to storage in a shared-disk system is usually less than the bandwidth available to access local storage. Thus, storage access can become a bottleneck, limiting scalability.

20.4.7 Shared Nothing

In a **shared-nothing** system, each node consists of a processor, memory, and one or more disks. The nodes communicate by a high-speed interconnection network. A node

functions as the server for the data on the disk or disks that the node owns. Since local disk references are serviced by local disks at each node, the shared-nothing model overcomes the disadvantage of requiring all I/O to go through a single interconnection network.

Moreover, the interconnection networks for shared-nothing systems, such as the tree-like interconnection network, are usually designed to be scalable, so their transmission capacity increases as more nodes are added. Consequently, shared-nothing architectures are more scalable and can easily support a very large number of nodes.

The main drawbacks of shared-nothing systems are the costs of communication and of nonlocal disk access, which are higher than in a shared-memory or shared-disk architecture since sending data involves software interaction at both ends.

Due to their high scalability, shared-nothing architectures are widely used to deal with very large data volumes, supporting scalability to thousands of nodes, or in extreme cases, even to tens of thousands of nodes.

20.4.8 Hierarchical

The **hierarchical architecture** combines the characteristics of shared-memory, shared-disk, and shared-nothing architectures. At the top level, the system consists of nodes that are connected by an interconnection network and do not share disks or memory with one another. Thus, the top level is a shared-nothing architecture. Each node of the system could actually be a shared-memory system with a few processors. Alternatively, each node could be a shared-disk system, and each of the systems sharing a set of disks could be a shared-memory system. Thus, a system could be built as a hierarchy, with shared-memory architecture with a few processors at the base, and a shared-nothing architecture at the top, with possibly a shared-disk architecture in the middle. Figure 20.5d illustrates a hierarchical architecture with shared-memory nodes connected together in a shared-nothing architecture.

Parallel database systems today typically run on a hierarchical architecture, where each node supports shared-memory parallelism, with multiple nodes interconnected in a shared-nothing manner.

20.5 Distributed Systems

In a **distributed database system**, the database is stored on nodes located at geographically separated **sites**. The nodes in a distributed system communicate with one another through various communication media, such as high-speed private networks or the internet. They do not share main memory or disks. The general structure of a distributed system appears in Figure 20.9.

The main differences between shared-nothing parallel databases and distributed databases include the following:

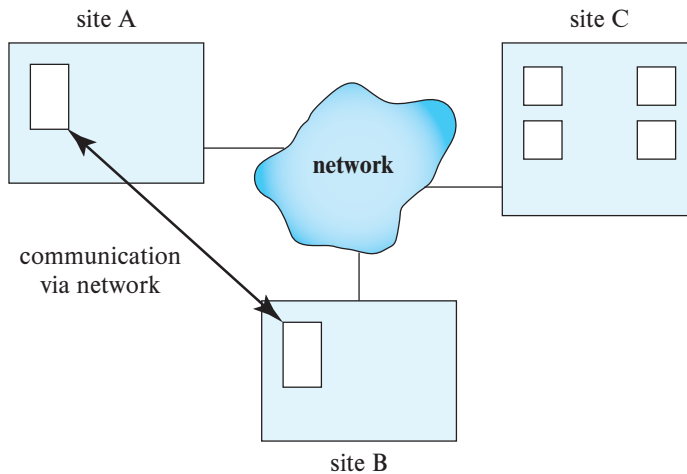


Figure 20.9 A distributed system.

- Distributed databases have sites that are geographically separated. As a result, the network connections have lower bandwidth, higher latency, and greater probability of failures, as compared to networks within a single data center. Systems built on distributed databases therefore need to be aware of network latency, and failures, as well as of physical data location. We discuss these issues later in this section. In particular, it is often desirable to keep a copy of the data at a data center close to the end user.
- Parallel database systems address the problem of node failure. However, some failures, particularly those due to earthquakes, fires, or other natural disasters, may affect an entire data center, causing failure of a large number of nodes. Distributed database systems need to continue working even in the event of failure of an entire data center, to ensure high availability. This requires replication of data across geographically separated data centers, to ensure that a common natural disaster does not affect all the data centers. Replication and other techniques to ensure high availability are similar in both parallel and distributed databases, although implementation details may differ.
- Distributed databases may be separately administered, with each site retaining some degree of autonomy of operation. Such databases are often the result of the integration of existing databases to allow queries and transactions to cross database boundaries. However, distributed databases that are built for providing geographic distribution, versus those built by integrating existing databases, may be centrally administered.
- Nodes in a distributed database tend to vary more in size and function, whereas parallel databases tend to have nodes that are of similar capacity.

- In a distributed database system, we differentiate between local and global transactions. A **local transaction** is one that accesses data only from nodes where the transaction was initiated. A **global transaction**, on the other hand, is one that either accesses data in a node different from the one at which the transaction was initiated, or accesses data in several different nodes.

Web-scale applications today run on data management systems that combine support for parallelism and distribution. Parallelism is used within a data center to handle high loads, while distribution across data centers is used to ensure high availability even in the event of natural disasters. At the lower end of functionality, such systems may be distributed data storage systems that support only limited functionality such as storage and retrieval of data by key, and they may not support schemas, query languages, or transactions; all such higher-level functionality has to be managed by the applications. At the higher end of functionality, there are distributed database systems that support schemas, query language, and transactions. However, one characteristic of such systems is that they are centrally administered.

In contrast, distributed databases that are built by integrating existing database systems have somewhat different characteristics.

- **Sharing data.** The major advantage in building a distributed database system is the provision of an environment where users at one site may be able to access the data residing at other sites. For instance, in a distributed university system, where each campus stores data related to that campus, it is possible for a user in one campus to access data in another campus. Without this capability, the transfer of student records from one campus to another campus would have to rely on some external mechanism.
- **Autonomy.** The primary advantage of sharing data by means of data distribution is that each site can retain a degree of control over data that are stored locally. In a centralized system, the database administrator of the central site controls the database. In a distributed system, there is a global database administrator responsible for the entire system. A part of these responsibilities is delegated to the local database administrator for each site. Depending on the design of the distributed database system, each administrator may have a different degree of **local autonomy**.

In a **homogeneous distributed database** system, nodes share a common global schema (although some relations may be stored only at some nodes), all nodes run the same distributed database-management software, and the nodes actively cooperate in processing transactions and queries.

However, in many cases a distributed database has to be constructed by linking together multiple already-existing database systems, each with its own schema and possibly running different database-management software. The sites may not be aware of one another, and they may provide only limited facilities for cooperation in query and transaction processing. Such systems are sometimes called **federated database systems** or **heterogeneous distributed database systems**.

Nodes in a distributed database communicate over *wide-area networks* (WAN). Although wide-area networks have bandwidth much greater than local-area networks, the bandwidth is usually shared by multiple users/applications and is expensive relative to local-area network bandwidth. Thus, applications that communicate across wide-area networks usually have a lower bandwidth.

Communication in a WAN must also contend with significant **latency**: a message may take up to a few hundred milliseconds to be delivered across the world, both due to speed-of-light delays, and due to queuing delays at a number of routers in the path of the message. Latency in a wide-area setting is a fundamental problem that cannot be reduced beyond a point. Thus, applications whose data and computing resources are distributed geographically have to be carefully designed to ensure that latency does not affect system performance excessively.

Wide-area networks also have to contend with network-link failures, a problem that is relatively rare in local-area networks. In particular, network-link failures may result in two sites that are both alive having no way to communicate with each other, a situation referred to as a **network partition**.⁵ In the event of a partition, it may not be possible for a user or an application to access required data. Thus, network partitioning affects the **availability** of a system. Tradeoffs between availability and consistency of data in the event of network partitions are discussed in Section 23.4.

20.6 Transaction Processing in Parallel and Distributed Systems

Atomicity of transactions is an important issue in building a parallel and distributed database system. If a transaction runs across two nodes, unless the system designers are careful, it may commit at one node and abort at another, leading to an inconsistent state. Transaction commit protocols ensure such a situation cannot arise. The *two-phase commit protocol* (2PC) is the most widely used of these protocols.

The 2PC protocol is described in detail in Section 23.2.1, but the key ideas are as follows: The basic idea behind 2PC is for each node to execute the transaction until it enters the partially committed state, and then leave the commit decision to a single coordinator node; the transaction is said to be in the *ready* state at a node at this point. The coordinator decides to commit the transaction only if the transaction reaches the ready state at every node where it executed; otherwise (e.g., if the transaction aborts at any node), the coordinator decides to abort the transaction. Every node where the transaction executed must follow the decision of the coordinator. If a node fails when a transaction is in ready state, when the node recovers from failure it should be in a position to either commit or abort the transaction, depending on the decision of the coordinator.

⁵Do not confuse the term *network partitioning* with the term *data partitioning*; data partitioning refers to dividing up of data items into partitions, which may be stored at different nodes.

Concurrency control is another issue in parallel and distributed databases. Since a transaction may access data items at several nodes, transaction managers at several nodes may need to coordinate to implement concurrency control. If locking is used, locking can be performed locally at the nodes containing accessed data items, but there is also a possibility of deadlock involving transactions originating at multiple nodes. Therefore deadlock detection needs to be carried out across multiple nodes. Failures are more common in distributed systems since not only may computers fail, but communication links may also fail. Replication of data items, which is the key to the continued functioning of distributed databases when failures occur, further complicates concurrency control. We describe concurrency-control techniques for distributed databases in Section 23.3 (which describes techniques based on locking) and Section 23.3.4 (which describes techniques based on timestamps).

The standard transaction models, based on multiple actions carried out by a single program unit, are often inappropriate for carrying out tasks that cross the boundaries of databases that cannot or will not cooperate to implement protocols such as 2PC. Alternative approaches, based on *persistent messaging* for communication, are generally used for such tasks; persistent messaging is discussed in Section 23.2.3.

When the tasks to be carried out are complex, involving multiple databases and/or multiple interactions with humans, coordination of the tasks and ensuring transaction properties for the tasks become more complicated. *Workflow management systems* are systems designed to help with carrying out such tasks.

20.7 Cloud-Based Services

Traditionally, enterprises purchased and ran servers that execute the database as well as the applications. There is a high cost to maintaining servers, including setting up server room infrastructure dealing with all kinds of failures such as air conditioning and power failures, not to mention failures of CPUs, disks, and other components of the servers. Further, if there is a sudden increase in demand, it is very difficult to add infrastructure to service the demand, and if demand falls, the infrastructure may lie idle.

In contrast, in the **cloud computing** model, applications of an enterprise are executed on an infrastructure that is managed by another company, typically at a data center that hosts a large number of machines used by many different enterprises/users. The service provider may provide not just hardware, but also support platforms such as databases, and application software.

A variety of vendors offer cloud services; these include major vendors such as Amazon, Microsoft, IBM, and Google, and a number of smaller vendors. One of the pioneers of cloud services, Amazon, originally built a large computing infrastructure purely for its internal use; then, seeing a business opportunity, it offered computing infrastructure as a service to other users. Cloud services became very popular within just a few years.

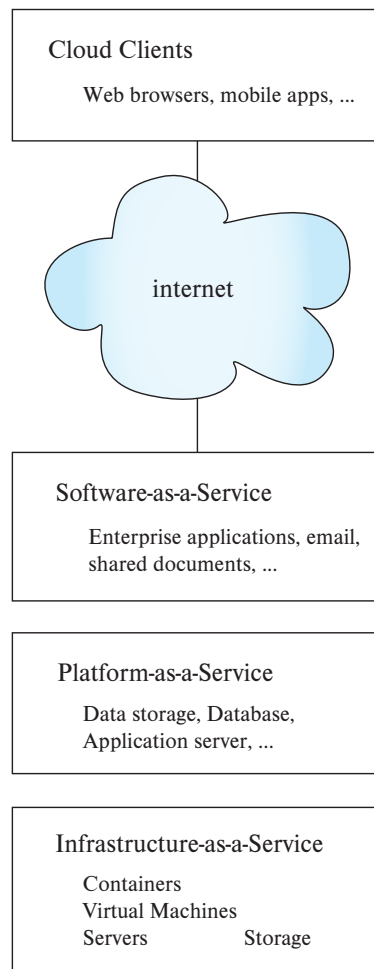


Figure 20.10 Cloud service models.

20.7.1 Cloud Service Models

There are several ways in which cloud computing can be utilized, which are summarized in Figure 20.10. These include infrastructure-as-a-service, platform-as-a-service, and software-as-a-service models.

- In the **infrastructure-as-a-service** model, an enterprise rents computing facilities; for example, an enterprise may rent one or more physical machines, along with disk storage space.

More frequently, cloud computing providers provide an abstraction of a **virtual machine (VM)**, which appears to the user to be a real machine. These machines are not “real” machines, but rather are simulated by software that allows a single

real computer to simulate several independent computers. *Containers* are a lower cost alternative to VMs and are described later in this section. Multiple VMs can run on a single server machine, and multiple containers can run on a single VM or server.

By running a very large data center with many machines, cloud-service providers can exploit economies of scale and deliver computing power at much lower cost than an enterprise can do using its own infrastructure.

Another major advantage of cloud computing is that the cloud-service provider usually has a large number of machines, with spare capacity, and thus an enterprise can rent more (virtual) machines as needed to meet demand and release them at times of light load. The ability to expand or contract capacity at short notice is often referred to as **elasticity**.

The above benefits of on-demand elastic provisioning of server systems have led to the widespread adoption of infrastructure-as-service platforms, especially by companies that anticipate rapid growth in their computing usage. However, due to the potential security risks of storing data outside the enterprise, the use of cloud computing is still limited in high-security enterprise needs, such as banking.

In the infrastructure-as-service model, the client enterprise runs its own software, including database systems, on virtual machines provided by the cloud-service provider; the client has to install the database system and deal with maintenance issues such as backup and restore.

- In the **platform-as-a-service** model, the service provider not only provides computing infrastructure, but it also deploys and manages platforms, such as data storage, databases, and application servers, that are used by application software. The client has to install and maintain application software, such as *enterprise resource planning (ERP)* systems, which run on such platform-provided services as application servers, database services, or data storage services.
 - **Cloud-based data storage** platforms provide a service that applications can use to store and retrieve data. The service provider takes care of provisioning sufficient amount of storage and computing power to support the load on the data storage platform. Such storage systems could support files, which are typically large, ranging in size from a few megabytes to thousands of megabytes, supporting millions of such files. Or such storage systems could support data items, which are typically small, ranging from hundreds of bytes to a few megabytes, but supporting billions of such data items. Such distributed file systems and data storage systems are discussed in Section 21.6 and Section 21.7. Database applications using cloud-based storage may run on the same cloud (i.e., the same set of machines), or on another cloud.

One of the main attractions of cloud-based storage is that it can be used by paying a fee without worrying about purchasing, maintaining, and managing the computer systems on which such a service runs. Further, if there is an

increase in demand, the number of servers on which the service runs can be increased by paying a larger fee, without having to actually purchase and deploy more servers. The service provider would of course have to deploy extra servers, but they benefit from economies of scale; the cost of deployment, and especially the time to deployment, are greatly reduced compared to what they would be if the end-users did it on their own.

The fees for cloud-based data storage are typically based on the amount of data stored, and amount of data input to, and the amount of data output from, the data storage system.

- **Database-as-a-service** platforms provide a database that can be accessed and queried by clients. Unlike storage services, database-as-a-service platforms provide database functionality such as querying using SQL or other query languages, which data storage systems do not provide. Early offerings of database-as-a-service only supported databases that run on a single node, although the node itself can have a substantial number of processors, memory, and storage. More recently, parallel database systems are being offered as a service on the cloud.
- In the **software-as-a-service** model, the service provider provides the application software as a service. The client does not need to deal with issues such as software installation or upgrades; these tasks are left to the service provider. The client can directly use interfaces provided by the software-as-a-service provider, such as web interfaces, or mobile app interfaces that provide a front end, with the application software acting as the back end.

The concept of virtual machines was developed in the 1960s to allow an expensive mainframe computer to be shared concurrently by users running different operating systems. Although computers are now much cheaper, there is still a cost associated with supplying electrical power to the computers and maintaining them; virtual machines allow this cost to be shared by multiple concurrent users. Virtual machines also ease the task of moving services to new machines: a virtual machine can be shut down on one physical server and restarted on another physical server with very little delay or downtime. This feature is particularly important for quick recovery in the event of hardware failure or upgrade.

Although multiple virtual machines can run on a single real machine, each VM has a high overhead, since it runs an entire operating system internally. When a single organization wishes to run a number of services, if it creates a separate VM for each service, the overhead can be very high. If multiple applications are run on one machine (or VM), two problems often arise: (1) applications conflict on network ports by each trying to listen to the same network port, and (2) applications require different versions of shared libraries, causing conflicts.

Containers solve both these problems; applications run in a container, which has its own IP address and its own set of shared libraries. Each application can consist of

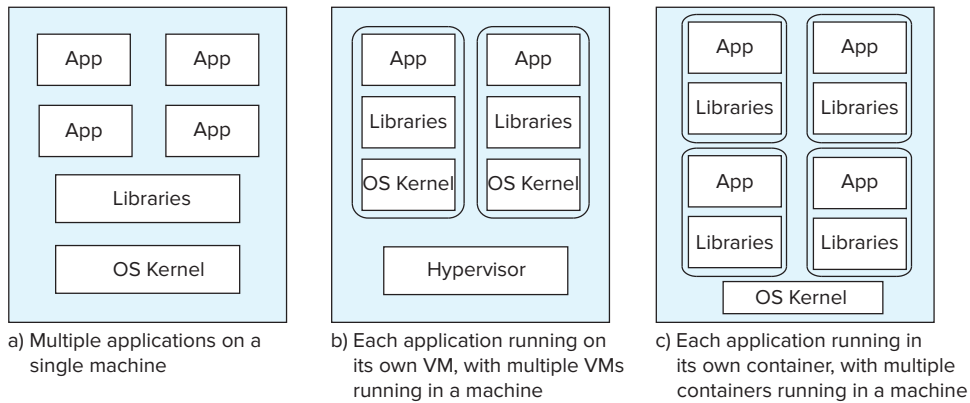


Figure 20.11 Application deployment alternatives.

multiple processes, all running within the same container. The cost of using containers to run applications is much less than the alternative of running each application in its own VM, since many containers can share the same operating system kernel. Each container appears to have its own file system, but the files are all stored in a common underlying file system across all containers. Processes within a container can interact with each other through the file system as well as interprocess communication, but they can interact with processes from other containers only via network connections.

Figure 20.11 depicts the different deployment alternatives for a set of applications. Figure 20.11a shows the alternative of multiple applications running in a single machine, sharing libraries and operating-system kernel. Figure 20.11b shows the alternative of running each application in its own VM, with multiple VMs running on a single machine. The different VMs running on a single real machine are managed by a software layer called the *hypervisor*. Figure 20.11c shows the alternative of using containers, with each container having its own libraries, and multiple containers running on a single machine. Since containers have lower overheads, a single machine can support more containers than VMs.

Containers provide low-cost support for elasticity, since more containers can be deployed very quickly on existing virtual machines, instead of starting up fresh virtual machines.

Many applications today are built as a collection of multiple **services**, each of which runs as a separate process, offering a network API; that is, the functions provided by the service are invoked by creating a network connection to the process and sending a service request over the network connection. Such an application architecture, which builds an application as a collection of small services, is called a **microservices architecture**. Containers fit the microservices architecture very well, since they provide a very low overhead mechanism to execute processes supporting each service.

Docker is a very widely used container platform, while **Kubernetes** is a very popular platform that provides not only containers, but also a microservices platform. Kubernetes allows applications to specify declaratively their container needs, and it automatically deploys and links multiple containers to execute the application. It can also manage a number of pods, which allow multiple containers to share storage (file system) and network (IP address) while allowing containers to retain their own copies of shared libraries. Furthermore, it can manage elasticity by controlling the deployment of additional containers when required. Kubernetes can support application scalability by load-balancing API requests across a collection of containers that all run copies of the same application. Users of the API do not need to know what IP addresses (each corresponding to a container) the service is running on, and they can instead connect to a single IP address. The load balancer distributes the requests from the common IP address to a set of containers (each with its own IP address) running the service.

20.7.2 Benefits and Limitations of Cloud Services

Many enterprises are finding the model of cloud computing and services beneficial. The cloud model saves client enterprises the need to maintain a large system-support staff and allows new enterprises to begin operation without having to make a large, upfront capital investment in computing systems. Further, as the needs of the enterprise grow, more resources (computing and storage) can be added as required; the cloud-computing vendor generally has very large clusters of computers, making it easy for the vendor to allocate resources on demand.

Users of cloud computing must be willing to accept that their data are held by another organization. This may present a variety of risks in terms of security and legal liability. If the cloud vendor suffers a security breach, client data may be divulged, causing the client to face legal challenges from its customers. Yet the client has no direct control over cloud-vendor security. These issues become more complex if the cloud vendor chooses to store data (or replicas of data) in a foreign country. Various legal jurisdictions differ in their privacy laws. So, for example, if a German company's data are replicated on a server in New York, then the privacy laws of the United States may apply instead of or in addition to those of Germany or the European Union. The cloud vendor might be required to release client data to the U.S. government even though the client never knew that its data would be stored in a location under U.S. jurisdiction. Specific cloud vendors offer their clients varying degrees of control over how their data are distributed geographically and replicated.

Despite the drawbacks, the benefits of cloud services are great enough that there is a rapidly growing market for such services.

20.8 Summary

- Centralized database systems run entirely on a single computer. Database systems designed for multiuser systems need to support the full set of transaction features.

Such systems are usually designed as servers that accept requests from applications via SQL or their own APIs.

- Parallelism with a small number of cores is referred to as coarse-grained parallelism. Parallelism with a large number of processors is referred to as fine-grained parallelism.
- Transaction servers have multiple processes, possibly running on multiple processors. So that these processes have access to common data, such as the database buffer, systems store such data in shared memory. In addition to processes that handle queries, there are system processes that carry out tasks such as lock and log management and checkpointing.
- Access to shared memory is controlled by a mutual-exclusion mechanism based on machine-level atomic instructions (test-and-set or compare-and-swap).
- Data-server systems supply raw data to clients. Such systems strive to minimize communication between clients and servers by caching data and locks at the clients. Parallel database systems use similar optimizations.
- Parallel database systems consist of multiple processors and multiple disks connected by a fast interconnection network. Speedup measures how much we can increase processing speed by increasing parallelism for a single transaction. Scaleup measures how well we can handle an increased number of transactions by increasing parallelism. Interference, skew, and start-up costs act as barriers to getting ideal speedup and scaleup.
- The components of a parallel system are connected via several possible types of interconnection networks: bus, ring, mesh, hypercube, or a tree-like topology.
- Parallel database architectures include the shared-memory, shared-disk, shared-nothing, and hierarchical architectures. These architectures have different trade-offs of scalability versus communication speed.
- Modern shared-memory architectures associate some memory with each processor, resulting in a non-uniform memory architecture (NUMA). Since each processor has its own cache, there is a problem of ensuring cache coherency, that is, consistency of data across the caches of multiple processors.
- Storage-area networks are a special type of local-area network designed to provide fast interconnection between large banks of storage devices and multiple computers.
- A distributed database system is a collection of partially independent database systems that (ideally) share a common schema and coordinate processing of transactions that access nonlocal data.

- Cloud services may be provided at a variety of levels: The infrastructure-as-a-service model provides clients with a virtual machine on which clients install their own software. The platform-as-a-service model provides data-storage, database, and application servers in addition to virtual machines, but the client needs to install and maintain application software. The software-as-a-service model provides that application software plus the underlying platform.
- Organizations using cloud services have to consider a wide variety of technical, economic, and legal issues in order to ensure the privacy and security of data and adequate performance despite the likelihood of data being stored at a remote location.

Review Terms

- Centralized Database Systems
 - Single-user system
 - Multiuser system
 - Server systems
 - Embedded databases
 - Servers
 - Coarse-grained parallelism
 - Fine-grained parallelism
- Server System Architectures
 - Transaction-server
 - Query-server
 - Data-server systems
 - Server processes
- Mutual exclusion
- Atomic instructions
- Data caching
- Parallel Systems
 - Coarse-grain parallel machine
 - Massively parallel machine
 - Fine-grain parallel machine
- Data center
- Decision-support queries
- Measure of performance
 - Throughput
 - Response time
 - Linear speedup
 - Sublinear speedup
 - Linear scaleup
 - Sublinear scaleup
- Sequential computation
- Amdahl's law
- Start-up costs
- Interconnection network
 - Bus
 - Ring
 - Mesh
 - Hypercube
 - Tree-like
 - Edge switches
- Aggregation switch

- Ethernet
- Fiber channel
- Infiniband
- Remote direct memory access (RDMA)
- Parallel Database Architectures
 - Shared memory
 - Shared disk
 - Clusters
 - Shared nothing
 - Hierarchical
- Moore's law
- NUMA
- Cache misses
- Hyper-threading
- Hardware threads
- Cache
- Shared-disk
- Fault tolerance
- Storage-area network (SAN)
- Distributed database system
- Local autonomy
- Homogeneous distributed database
- Federated database systems
- Heterogeneous distributed database systems
- Latency
- Network partition
- Availability
- Cloud computing
- Infrastructure-as-a-service
- Platform-as-a-service
- Cloud-based data storage
- Database-as-a-service
- Software-as-a-service
- Microservices architecture

Practice Exercises

- 20.1** Is a multiuser system necessarily a parallel system? Why or why not?
- 20.2** Atomic instructions such as compare-and-swap and test-and-set also execute a memory fence as part of the instruction on many architectures. Explain what is the motivation for executing the memory fence, from the viewpoint of data in shared memory that is protected by a mutex implemented by the atomic instruction. Also explain what a process should do before releasing a mutex.
- 20.3** Instead of storing shared structures in shared memory, an alternative architecture would be to store them in the local memory of a special process and access the shared data by interprocess communication with the process. What would be the drawback of such an architecture?
- 20.4** Explain the distinction between a *latch* and a *lock* as used for transactional concurrency control.
- 20.5** Suppose a transaction is written in C with embedded SQL, and about 80 percent of the time is spent in the SQL code, with the remaining 20 percent spent

in C code. How much speedup can one hope to attain if parallelism is used only for the SQL code? Explain.

- 20.6** Consider a pair of processes in a shared memory system such that process *A* updates a data structure, and then sets a flag to indicate that the update is completed. Process *B* monitors the flag, and starts processing the data structure only after it finds the flag is set.

Explain the problems that could arise in a memory architecture where writes may be reordered, and explain how the `sfence` and `lfence` instructions can be used to ensure the problem does not occur.

- 20.7** In a shared-memory architecture, why might the time to access a memory location vary depending on the memory location being accessed?
- 20.8** Most operating systems for parallel machines (i) allocate memory in a local memory area when a process requests memory, and (ii) avoid moving a process from one core to another. Why are these optimizations important with a NUMA architecture?
- 20.9** Some database operations such as joins can see a significant difference in speed when data (e.g., one of the relations involved in a join) fits in memory as compared to the situation where the data do not fit in memory. Show how this fact can explain the phenomenon of **superlinear speedup**, where an application sees a speedup greater than the amount of resources allocated to it.
- 20.10** What is the key distinction between homogeneous and federated distributed database systems?
- 20.11** Why might a client choose to subscribe only to the basic infrastructure-as-a-service model rather than to the services offered by other cloud service models?
- 20.12** Why do cloud-computing services support traditional database systems best by using a virtual machine, instead of running directly on the service provider's actual machine, assuming that data is on external storage?

Exercises

- 20.13** Consider a bank that has a collection of sites, each running a database system. Suppose the only way the databases interact is by electronic transfer of money between themselves, using persistent messaging. Would such a system qualify as a distributed database? Why?
- 20.14** Assume that a growing enterprise has outgrown its current computer system and is purchasing a new parallel computer. If the growth has resulted in many more transactions per unit time, but the length of individual transactions has

not changed, what measure is most relevant—speedup, batch scaleup, or transaction scaleup? Why?

- 20.15** Database systems are typically implemented as a set of processes (or threads) accessing shared memory.
- How is access to the shared-memory area controlled?
 - Is two-phase locking appropriate for serializing access to the data structures in shared memory? Explain your answer.
- 20.16** Is it wise to allow a user process to access the shared-memory area of a database system? Explain your answer.
- 20.17** What are the factors that can work against linear scale up in a transaction processing system? Which of the factors are likely to be the most important in each of the following architectures: shared-memory, shared disk, and shared nothing?
- 20.18** Memory systems today are divided into multiple modules, each of which can be serving a separate request at a given time, in contrast to earlier architectures where there was a single interface to memory. What impact has such a memory architecture have on the number of processors that can be supported in a shared-memory system?
- 20.19** Assume we have data items $d_1, d_2, \dots, d_n$ with each d_i protected by a lock stored in memory location M_i .
- Describe the implementation of $\text{lock-X}(d_i)$ and $\text{unlock}(d_i)$ via the use of the test-and-set instruction.
 - Describe the implementation of $\text{lock-X}(d_i)$ and $\text{unlock}(d_i)$ via the use of the compare-and-swap instruction.
- 20.20** In a shared-nothing system data access from a remote node can be done by remote procedure calls, or by sending messages. But remote direct memory access (RDMA) provides a much faster mechanism for such data access. Explain why.
- 20.21** Suppose that a major database vendor offers its database system (e.g., Oracle, SQL Server DB2) as a cloud service. Where would this fit among the cloud-service models? Why?
- 20.22** If an enterprise uses its own ERP application on a cloud service under the platform-as-a-service model, what restrictions would there be on when that enterprise may upgrade the ERP system to a new version?

Further Reading

[Hennessy et al. (2017)] provides an excellent introduction to the area of computer architecture, including the topics of shared-memory architectures and cache coherency, parallel computing architectures, and cloud computing, which we covered in this chapter. [Gray and Reuter (1993)] provides the classic textbook description of transaction processing, including the architecture of client-server and distributed systems. [Ozsu and Valduriez (2010)] provides textbook coverage of distributed database systems. [Abts and Felderman (2012)] provides an overview of data center networking.

Bibliography

- [Abts and Felderman (2012)] D. Abts and B. Felderman, “A Guided Tour of Datacenter Networking”, *Communications of the ACM*, Volume 55, Number 6 (2012), pages 44–51.
- [Gray and Reuter (1993)] J. Gray and A. Reuter, *Transaction Processing: Concepts and Techniques*, Morgan Kaufmann (1993).
- [Hennessy et al. (2017)] J. L. Hennessy, D. A. Patterson, and D. Goldberg, *Computer Architecture: A Quantitative Approach*, 6th edition, Morgan Kaufmann (2017).
- [Ozsu and Valduriez (2010)] T. Ozsu and P. Valduriez, *Principles of Distributed Database Systems*, 3rd edition, Prentice Hall (2010).

Credits

The photo of the sailboats in the beginning of the chapter is due to ©Pavel Nesvadba/Shutterstock.

CHAPTER 21



Parallel and Distributed Storage

As we discussed in Chapter 20, parallelism is used to provide speedup, where queries are executed faster because more resources, such as processors and disks, are provided. Parallelism is also used to provide scaleup, where increasing workloads are handled without increased response time, via an increase in the degree of parallelism.

In this chapter, we discuss techniques for data storage and indexing in parallel database systems.

21.1 Overview

We first describe, in Section 21.2 and Section 21.3, how to partition data amongst multiple nodes. We then discuss, in Section 21.4, replication of data and parallel indexing (in Section 21.5). Our description focuses on shared-nothing parallel database systems, but the techniques we describe are also applicable to distributed database systems, where data are stored in a geographically distributed manner.

File systems that run on a large number of nodes, called distributed file systems, are a widely used way to store data in a parallel system. We discuss distributed file systems in Section 21.6.

In recent years parallel data storage and indexing techniques have been extensively used for storage of nonrelational data, including unstructured text data, and semi-structured data in XML, JSON, or other formats. Such data are often stored in parallel **key-value stores**, which store data items with an associated key. The techniques we describe for parallel storage of relational data can also be used for key-value stores which are discussed in Section 21.7. We use the term **data storage system** to refer to both key-value stores, and the data storage and access layer of parallel database systems.

Query processing in parallel and distributed databases is discussed in Chapter 22, while and transaction processing in parallel and distributed databases is discussed in Chapter 23.

21.2 Data Partitioning

In its simplest form, **I/O parallelism** refers to reducing the time required to retrieve data from disk by partitioning the data over multiple disks.¹

At the lowest level, RAID systems allow blocks to be partitioned across multiple disks, allowing them to be accessed in parallel. Blocks are usually allocated to different disks in a round-robin fashion, as we saw in Section 12.5. For example, if there are n disks numbered 0 to $n - 1$, round-robin allocation assigns block i to disk $i \bmod n$. However, the block-level partitioning techniques supported by RAID systems do not offer any control in terms of which tuples of a relation are stored on which disk or node. Therefore, parallel database systems typically do not use block-level partitioning and instead perform partitioning at the level of tuples.

In systems with multiple nodes (computers), each with multiple disks, partitioning can potentially be specified to the level of individual disks. However, parallel database systems typically focus on partitioning data across nodes and leave it to the operating system on each node to decide on assigning blocks to disks within the node.

In a parallel storage system, the tuples of a relation are partitioned (divided) among many nodes, so that each tuple resides on one node; such partitioning is referred to as **horizontal partitioning**. Several partitioning strategies have been proposed for horizontal partitioning, which we study next.

We note that *vertical partitioning*, discussed in Section 13.6 in the context of columnar storage, is orthogonal to horizontal partitioning. (As an example of vertical partitioning, a relation $r(A, B, C, D)$ where A is a primary key, may be vertically partitioned into $r(A, B)$ and $r(A, C, D)$, if many queries require B values, while C and D values are large in size and not required for many queries.) Once tuples are horizontally partitioned, they may be stored in a vertically partitioned manner at each node.

We also note that several database vendors use the term *partitioning* to denote the partitioning of tuples of a relation r into multiple physical relations $r_1, r_2, \dots, r_n$, where all the physical relations r_i are stored in a single node. The relation r is not stored, but treated as a view defined by the query $r_1 \cup r_2 \cup \dots \cup r_n$. Such *intra-node partitioning* of a relation is typically used to ensure that frequently accessed tuples are stored separately from infrequently accessed tuples and is different from horizontal partitioning across nodes. Intra-node partitioning is described in more detail in Section 25.1.4.3.

In the rest of this chapter, as well as in subsequent chapters, we use the term *partitioning* to refer to *horizontal partitioning* across multiple nodes.

21.2.1 Partitioning Strategies

We present three basic *data-partitioning strategies* for partitioning tuples. Assume that there are n nodes, $N_1, N_1, \dots, N_n$, across which the data are to be partitioned.

¹As in earlier chapters, we use the term *disk* to refer to persistent storage devices, such as magnetic hard disks and solid-state drives.

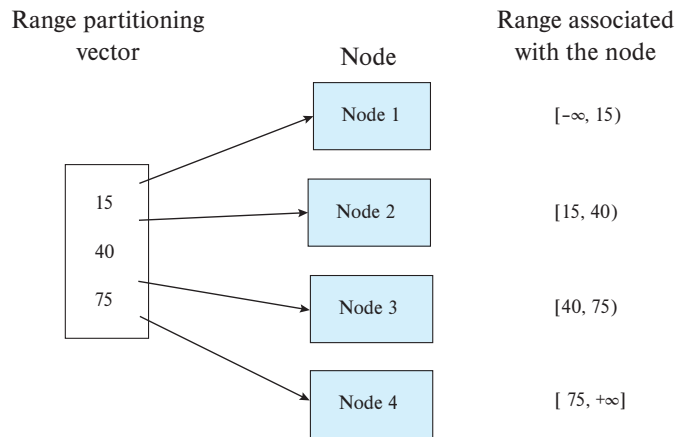


Figure 21.1 Example of range partitioning vector.

- **Round-robin.** This strategy scans the relation in any order and sends the i th tuple fetched during the scan to node number $N_{((i-1) \bmod n)+1}$. The round-robin scheme ensures an even distribution of tuples across nodes; that is, each node has approximately the same number of tuples as the others.
- **Hash partitioning.** This declustering strategy designates one or more attributes from the given relation's schema as the partitioning attributes. A hash function is chosen whose range is $\{1, 2, \dots, n\}$. Each tuple of the original relation is hashed on the partitioning attributes. If the hash function returns i , then the tuple is placed on node N_i .²
- **Range partitioning.** This strategy distributes tuples by assigning contiguous attribute-value ranges to each node. It chooses a partitioning attribute, A , and a **partitioning vector** $[v_1, v_2, \dots, v_{n-1}]$, such that, if $i < j$, then $v_i < v_j$. The relation is partitioned as follows: Consider a tuple t such that $t[A] = x$. If $x < v_1$, then t goes on node N_1 . If $x \geq v_{n-1}$, then t goes on node N_n . If $v_i \leq x < v_{i+1}$, then t goes on node N_{i+1} .

Figure 21.1 shows an example of a range partitioning vector. In the example in the figure, values less than 15 are mapped to Node 1. Values in the range $[15, 40)$, i.e., values ≥ 15 but < 40 , are mapped to Node 2; Values in the range $[40, 75)$, i.e., values ≥ 40 but < 75 , are mapped to Node 3, while values > 75 are mapped to Node 4.

We now consider how partitioning is maintained when a relation is updated.

1. When a tuple is inserted into a relation, it is sent to the appropriate node based on the partitioning strategy.

²Hash-function design is discussed in Section 24.5.1.1.

2. If a tuple is deleted, its location is first found based on the value of its partitioning attribute (for round-robin, all partitions are searched). The tuple is then deleted from wherever it is located.
3. If a tuple is updated, its location is not affected if either round-robin partitioning is used or if the update does not affect a partitioning attribute. However, if range partitioning or hash partitioning is used, and the update affects a partitioning attribute, the location of the tuple may be affected. In this case:
 - a. The original tuple is deleted from the original location, and
 - b. The updated tuple is inserted and sent to the appropriate node based on the partitioning strategy used.

21.2.2 Comparison of Partitioning Techniques

Once a relation has been partitioned among several nodes, we can retrieve it in parallel, using all the nodes. Similarly, when a relation is being partitioned, it can be written to multiple nodes in parallel. Thus, the transfer rates for reading or writing an entire relation are much faster with I/O parallelism than without it. However, reading an entire relation, or *scanning a relation*, is only one kind of access to data. Access to data can be classified as follows:

1. Scanning the entire relation.
2. Locating a tuple associatively (e.g., *employee_name* = “Campbell”); these queries, called **point queries**, seek tuples that have a specified value for a specific attribute.
3. Locating all tuples for which the value of a given attribute lies within a specified range (e.g., $10000 < salary < 20000$); these queries are called **range queries**.

The different partitioning techniques support these types of access at different levels of efficiency:

- **Round-robin.** The scheme is ideally suited for applications that wish to read the entire relation sequentially for each query. With this scheme, both point queries and range queries are complicated to process, since each of the n nodes must be used for the search.
- **Hash partitioning.** This scheme is best suited for point queries based on the partitioning attribute. For example, if a relation is partitioned on the *telephone_number* attribute, then we can answer the query “Find the record of the employee with *telephone_number* = 555-3333” by applying the partitioning hash function to 555-3333 and then searching that node. Directing a query to a single node saves the start-up cost of initiating a query on multiple nodes and leaves the other nodes free to process other queries.

Hash partitioning is also useful for sequential scans of the entire relation. If the hash function is a good randomizing function, and the partitioning attributes form a key of the relation, then the number of tuples in each of the nodes is approximately the same, without much variance. Hence, the time taken to scan the relation is approximately $1/n$ of the time required to scan the relation in a single node system.

The scheme, however, is not well suited for point queries on nonpartitioning attributes. Hash-based partitioning is also not well suited for answering range queries, since, typically, hash functions do not preserve proximity within a range. Therefore, all the nodes need to be scanned for range queries to be answered.

- **Range partitioning.** This scheme is well suited for point and range queries on the partitioning attribute. For point queries, we can consult the partitioning vector to locate the node where the tuple resides. For range queries, we consult the partitioning vector to find the range of nodes on which the tuples may reside. In both cases, the search narrows to exactly those nodes that might have any tuples of interest.

An advantage of this feature is that, if there are only a few tuples in the queried range, then the query is typically sent to one node, as opposed to all the nodes. Since other nodes can be used to answer other queries, range partitioning results in higher throughput while maintaining good response time. On the other hand, if there are many tuples in the queried range (as there are when the queried range is a larger fraction of the domain of the relation), many tuples have to be retrieved from a few nodes, resulting in an I/O bottleneck (hot spot) at those nodes. In this example of **execution skew**, all processing occurs in one—or only a few—partitions. In contrast, hash partitioning and round-robin partitioning would engage all the nodes for such queries, giving a faster response time for approximately the same throughput.

The type of partitioning also affects other relational operations, such as joins, as we shall see in Section 22.3 and Section 22.4.1.

Thus, the choice of partitioning technique also depends on the operations that need to be executed. In general, hash partitioning or range partitioning are preferred to round-robin partitioning.

Partitioning is important for large relations. Large databases that benefit from parallel storage often have some small relations. Partitioning is not a good idea for such small relations, since each node would end up with just a few tuples. Partitioning is worthwhile only if each node would contain at least a few disk blocks worth of data. Small relations are best left unpartitioned, while medium-sized relations could be partitioned across some of the nodes, rather than across all the nodes, in a large system.

21.3 Dealing with Skew in Partitioning

When a relation is partitioned (by a technique other than round-robin), there may be a skew in the distribution of tuples, with a high percentage of tuples placed in some

partitions and fewer tuples in other partitions. Such an imbalance in the distribution of data is called **data distribution skew**. Data distribution skew may be caused by one of two factors.

- **Attribute-value skew**, which refers to the fact that some values appear in the partitioning attributes of many tuples. All the tuples with the same value for the partitioning attribute end up in the same partition, resulting in skew.
- **Partition skew**, which refers to the fact that there may be load imbalance in the partitioning, even when there is no attribute skew.

Attribute-value skew can result in skewed partitioning regardless of whether range partitioning or hash partitioning is used. If the partition vector is not chosen carefully, range partitioning may result in partition skew. Partition skew is less likely with hash partitioning if a good hash function is chosen.

As Section 20.4.2 noted, even a small skew can result in a significant decrease in performance. Skew becomes an increasing problem with a higher degree of parallelism. For example, if a relation of 1000 tuples is divided into 10 parts, and the division is skewed, then there may be some partitions of size less than 100 and some partitions of size more than 100; if even one partition happens to be of size 200, the speedup that we would obtain by accessing the partitions in parallel is only 5, instead of the 10 for which we would have hoped. If the same relation has to be partitioned into 100 parts, a partition will have 10 tuples on an average. If even one partition has 40 tuples (which is possible, given the large number of partitions) the speedup that we would obtain by accessing them in parallel would be 25, rather than 100. Thus, we see that the loss of speedup due to skew increases with parallelism.

In addition to skew in the distribution of tuples, there may be **execution skew** even if there is no skew in the distribution of tuples, if queries tend to access some partitions more often than others. For example, suppose a relation is partitioned by the timestamp of the tuples, and most queries refer to recent tuples; then, even if all partitions contain the same number of tuples, the partition containing recent tuples would experience a significantly higher load.

In the rest of this section, we consider several approaches to handling skew.

21.3.1 Balanced Range-Partitioning Vectors

Data distribution skew in range partitioning can be avoided by choosing a **balanced range-partitioning vector**, which evenly distributes tuples across all nodes.

A balanced range-partitioning vector can be constructed by sorting, as follows: The relation is first sorted on the partitioning attributes. The relation is then scanned in sorted order. After every $1/n$ of the relation has been read, the value of the partitioning attribute of the next tuple is added to the partition vector. Here, n denotes the number of partitions to be constructed.

The main disadvantage of this method is the extra I/O overhead incurred in doing the initial sort. The I/O overhead for constructing balanced range-partitioning vectors can be reduced by using a precomputed frequency table, or **histogram**, of the attribute values for each attribute of each relation. Figure 21.2 shows an example of a histogram for an integer-valued attribute that takes values in the range 1 to 25. It is straightforward to construct a balanced range-partitioning function given a histogram on the partitioning attributes. A histogram takes up only a little space, so histograms on several different attributes can be stored in the system catalog. If the histogram is not stored, it can be computed approximately by sampling the relation, using only tuples from a randomly chosen subset of the disk blocks of the relation. Using a random sample allows the histogram to be constructed in much less time than it would take to sort the relation.

The preceding approach for creating range-partitioning vectors addresses data-distribution skew; extensions to handle execution skew are left as an exercise for the reader (Exercise 21.3).

A drawback of the above approach is that it is *static*: the partitioning is decided at some point and is not automatically updated as tuples are inserted, deleted, or updated. The partitioning vectors can be recomputed, and the data repartitioned, whenever the system detects skew in data distribution. However, the cost of repartitioning can be quite large, and doing it periodically would introduce a high load which can affect normal processing. Dynamic techniques for avoiding skew, which can adapt in a continuous and less disruptive fashion, are discussed in Section 21.3.2 and in Section 21.3.3.

21.3.2 Virtual Node Partitioning

Another approach to minimizing the effect of skew is to use *virtual nodes*. In the **virtual nodes** approach, we pretend there are several times as many *virtual nodes* as the number

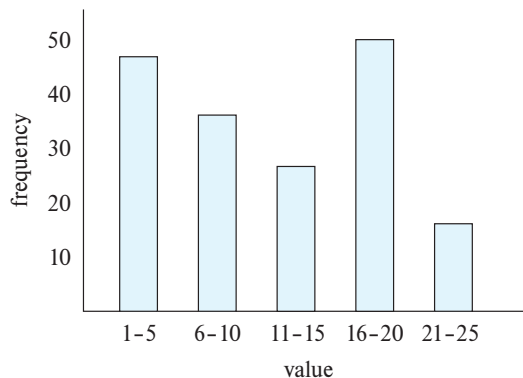


Figure 21.2 Example of histogram.

of real nodes. Any of the partitioning techniques described earlier can be used, but to map tuples and work to virtual nodes instead of to real nodes.³

Virtual nodes, in turn, are mapped to real nodes. One way to map virtual nodes to real nodes is round-robin allocation; thus, if there are n real nodes numbered 1 to n , virtual node i is mapped to real node $((i - 1) \bmod n) + 1$. The idea is that even if one range had many more tuples than the others because of skew, these tuples would get split across multiple virtual nodes ranges. Round-robin allocation of virtual nodes to real nodes would distribute the extra work among multiple real nodes, so that one node does not have to bear all the burden.

A more sophisticated way of doing the mapping is by tracking the number of tuples in each virtual node, and the load (e.g., the number of accesses per second) on each virtual node. Virtual nodes are then mapped to real nodes in a way that balances the number of stored tuples as well as the load across the real nodes. Thus, data-distribution skew and execution skew can be minimized.

The system must then record this mapping and use it to route accesses to the correct real node. If virtual nodes are numbered by consecutive integers, this mapping can be stored as an array *virtual_to_real_map[]*, with m entries, where there are m virtual nodes; the i th element of this array stores the real node to which virtual node i is mapped.

Yet another benefit of the virtual node approach is that it allows **elasticity of storage**, that is, as the load on the system increases it is possible to add more resources (nodes) to the system to handle the load. When a new node is added, some of the virtual nodes are *migrated* to the new real node, which can be done without affecting any of the other virtual nodes. If the amount of data mapped to each virtual node is small, the migration of a virtual node from one node to another can be done relatively fast, minimizing disruption.

21.3.3 Dynamic Repartitioning

While the virtual-node approach can reduce skew with range partitioning as well as hash partitioning, it does not work very well if the data distribution changes over time, resulting in some virtual nodes having a very large number of tuples, or a very high execution load. For example, if partitioning was done by timestamps of records, the last timestamp range would get an increasing number of records, as more records are inserted, while other ranges would not get any new records. Thus, even if the initial partitioning is balanced, it could become increasingly skewed over time.

Skew can be dealt with by recomputing the partitioning scheme entirely. However, repartitioning the data based on the new partitioning scheme would, in general, be a very expensive operation. In the preceding example, we would end up moving a significant number of records from each partition to a partition that precedes it in timestamp

³The virtual node approach is also called the **virtual processor** approach, a term used in earlier editions of this book; since the term *virtual processor* is now commonly used in a different sense in the context of virtual machines, we now use the term *virtual node*.

order. When dealing with large amounts of data, such repartitioning would be unreasonably expensive.

Dynamic repartitioning can be done in an efficient manner by instead exploiting the virtual node scheme. The basic idea is to split a virtual node into two virtual nodes when it has too many tuples, or too much load; the idea is very similar to a B⁺-tree node being split into two nodes when it is overfull. One of the newly created virtual nodes can then be migrated to a different node to rebalance the data stored at each node, or the load at each node.

Considering the preceding example, if the virtual node corresponding to a range of timestamps 2017-01-01 to *MaxDate* were to become overfull, the partition could be split into two partitions. For example, if half the tuples in this range have timestamps less than 2018-01-01, one partition would have timestamps from 2017-01-01 to less than 2018-01-01, and the other would have tuples with timestamps from 2018-01-01 to *MaxDate*. To rebalance the number of tuples in a real node, we would just need to move one of the virtual nodes to a new real node.

Dynamic repartitioning in this way is very widely used in parallel databases and parallel data storage systems today. In data storage systems, the term **table** refers to a collection of data items. Tables are partitioned into multiple **tablets**. The number of tablets into which a table is divided is much larger than the number of real nodes in the system; thus tablets correspond to virtual nodes.

The system needs to maintain a **partition table**, which provides a mapping from the partitioning key ranges to a tablet identifier, as well as the real node on which the tablet data reside. Figure 21.3 shows an example of a partition table, where the partition key is a date. Tablet0 stores records with key value < 2012-01-01. Tablet1 stores records with key values ≥ 2012-01-01, but < 2013-01-01. Tablet2 stores records with key values ≥ 2013-01-01, but < 2014-01-01, and so on. Finally, Tablet6 stores values ≥ 2017-01-01.

Read requests must specify a value for the partitioning attribute, which is used to identify the tablet which could contain a record with that key value; a request that does not specify a value for the partitioning attribute would have to be sent to all tablets. A read request is processed by using the partitioning key value *v* to identify the tablet

Value	Tablet ID	Node ID
2012-01-01	Tablet0	Node0
2013-01-01	Tablet1	Node1
2014-01-01	Tablet2	Node2
2015-01-01	Tablet3	Node2
2016-01-01	Tablet4	Node0
2017-01-01	Tablet5	Node1
MaxDate	Tablet6	Node1

Figure 21.3 Example of a partition table.

whose range of keys contains v , and then sending the request to the real node where the tablet resides. The request can be handled efficiently at that node by maintaining, for each tablet, an index on the partitioning key attribute.

Write, insert, and delete requests are processed similarly, by routing the requests to the correct tablet and real node, using the mechanism described above for reads.

The above scheme allows tablets to be split if they become too big; the key range corresponding to the tablet is split into two, with a newly created tablet getting half the key range. Records whose key range is mapped to the new tablet are then moved from the original tablet to the new tablet. The partition table is updated to reflect the split, so requests are then correctly directed to the appropriate tablet.

If a real node gets overloaded, either due to a large number of requests or due to too much data at the node, some of the tablets from the node can be moved to a different real node that has a lower load. Tablets can also be moved similarly in case one of the real nodes has a large amount of data, while another real node has less data. Finally, if a new real node joins a system, some tables can be moved from existing nodes to the new node. Whenever a tablet is moved to a different real node, the partition table is updated; subsequent requests will then be sent to the correct real node.

Figure 21.4 shows the partition table from Figure 21.3 after Tablet6, which had values $\geq 2017-01-01$, has been split into two: Tablet6 now has values $\geq 2017-01-01$, but $< 2018-01-01$, while the new tablet, Tablet7, has values $\geq 2018-01-01$. Such a split could be caused by a large number of inserts into Tablet6, making it very large; the split rebalances the sizes of the tablets.

Note also that Tablet1, which was in Node1, has now been moved to Node0 in Figure 21.4. Such a tablet move could be because Node1 is overloaded due to excessive data, or due to a high number of requests.

Most parallel data storage systems store the partition table at a **master** node. However, to support a large number of requests each second, the partition table is usually replicated, either to all client nodes that access data or to multiple **routers**. Routers accept read/write requests from clients and forward the requests to the appropriate

Value	Tablet ID	Node ID
2012-01-01	Tablet0	Node0
2013-01-01	Tablet1	Node0
2014-01-01	Tablet2	Node2
2015-01-01	Tablet3	Node2
2016-01-01	Tablet4	Node0
2017-01-01	Tablet5	Node1
2018-01-01	Tablet6	Node1
MaxDate	Tablet7	Node1

Figure 21.4 Example partition table after tablet split and tablet move.

real node containing the tablet/virtual nodes based on the key values specified in the request.

An alternative fully distributed approach is supported by a hash based partitioning scheme called **consistent hashing**. In the consistent hashing approach, keys are hashed to a large space, such as 32 bit integers. Further, node (or virtual node) identifiers are also hashed to the same space. A key k_i could be logically mapped to the node n_j whose hash value $h(n_j)$ is the highest value among all nodes satisfying $h(n_j) < h(k_i)$. But to ensure that every key is assigned to a node, hash values are treated as lying on a cycle similar to the face of a clock, where the maximum hash value *maxhash* is immediately followed by 0. Then, key k_i is then logically mapped to the node n_j whose hash value $h(n_j)$ is the closest among all nodes, when we move anti-clockwise in the circle from $h(k_i)$.

Distributed hash tables based on this idea have been developed where there is no need for either a master node or a router; instead each participating node keeps track of a few other peer nodes, and routing is implemented in a cooperative manner. New nodes can join the system, and integrate themselves by following specified protocols in a completely peer-to-peer manner, without the need for a master. See the Further Reading section at the end of the chapter for references providing further details.

21.4 Replication

With a large number of nodes, the probability that at least one node will malfunction in a parallel system is significantly greater than in a single-node system. A poorly designed parallel system will stop functioning if any node fails. Assuming that the probability of failure of a single node is small, the probability of failure of the system goes up linearly with the number of nodes. For example, if a single node would fail once every 5 years, a system with 100 nodes would have a failure every 18 days.

Parallel data storage systems must, therefore, be resilient to failure of nodes. Not only should data not be lost in the event of a node failure, but also, the system should continue to be *available*, that is, continue to function, even during such a failure.

To ensure tuples are not lost on node failure, tuples are replicated across at least two nodes, and often three nodes. If a node fails, the tuples that it stored can still be accessed from the other nodes where the tuples are replicated.⁴

Tracking the replicas at the level of individual tuples would result in significant overhead in terms of storage and query processing. Instead, replication is done at the level of partitions (tablets, nodes, or virtual nodes). That is, each partition is replicated; the locations of the partition replicas are recorded as part of the partition table.

Figure 21.5 shows a partition table with replication of tablets. Each tablet is replicated in two nodes.

⁴Caching also results in replication of data, but with the aim of speeding up access. Since data may be evicted from cache at any time, caching does not ensure availability in the event of failure.

Value	Tablet ID	Node ID
2012-01-01	Tablet0	Node0,Node1
2013-01-01	Tablet1	Node0,Node2
2014-01-01	Tablet2	Node2,Node0
2015-01-01	Tablet3	Node2,Node1
2016-01-01	Tablet4	Node0,Node1
2017-01-01	Tablet5	Node1,Node0
2018-01-01	Tablet6	Node1,Node2
MaxDate	Tablet7	Node1,Node2

Figure 21.5 Partition table of Figure 21.4 with replication.

The database system keeps track of failed nodes; requests for data stored at a failed node are automatically routed to the backup nodes that store a replica of the data. Issues of how to handle the case where one or more replicas are stored at a currently failed node are addressed briefly in Section 21.4.2, and in more detail later, in Section 23.4.

21.4.1 Location of Replicas

Replication to two nodes provides protection from data loss/unavailability in the event of single node failure, while replication to three nodes provides protection even in the event of two node failures. If all nodes where a partition is replicated fail, obviously there is no way to prevent data loss/unavailability. Systems that use low-cost commodity machines for data storage typically use three-way replication, while systems that use more reliable machines typically use two-way replication.

There are multiple possible failure modes in a parallel system. A single node could fail due to some internal fault. Further, it is possible for all the nodes in a rack to fail if there is some problem with the rack such as, for example, failure of power supply to the entire rack, or failure of the network switches in a rack, making all the nodes in the rack inaccessible. Further, there is a possibility of failure of an entire data center, for example, due to fire, flooding, or a large-scale power failure.

The location of the nodes where the replicas of a partition are stored must, therefore, be chosen carefully, to maximize the probability of at least one copy being accessible even during a failure. Such replication can be within a data center or across data centers.

- **Replication within a data center:** Since single node failures are the most common failure mode, partitions are often replicated to another node. With the tree-like interconnection topology commonly used in data center networks (described in Section 20.4.3) network bandwidth within a rack is much

higher than the network bandwidth between racks. As a result, replication to another node within the same rack as the first node reduces network demand on the network between racks. But to deal with the possibility of a rack failure, partitions are also replicated to a node on a different rack.

- **Replication across data centers:** To deal with the possibility of failure of an entire data center, partitions may also be replicated at one or more geographically separated data centers. Geographic separation is important to deal with disasters such as earthquakes or storms that may shut down all data centers in a geographic region.

For many web applications, round-trip delays across a long-distance network can affect performance significantly, a problem that is increasing with the use of Ajax applications that require multiple rounds of communication between the browser and the application. To deal with this problem, users are connected with application servers that are closest to them geographically, and data replication is done in such a way that one of the replicas is in the same data center as (or at least, geographically close to) the application server.

Suppose all the partitions at a node N_1 are replicated at a single node N_2 , and N_1 fails. Then, node N_2 will have to handle all the requests that would originally have gone to N_1 , as well as requests routed to node N_2 . As a result, node N_2 would have to perform twice as much work as other nodes in the system, resulting in execution skew during failure of node N_1 .

To avoid this problem, the replicas of partitions residing at a node, say N_1 , are spread across multiple other nodes. For example, consider a system with 10 nodes and two-way replication. Suppose node N_1 had one of the replicas of partitions p_1 through p_9 . Then, the other replica of partitions p_1 could be stored on N_2 , of p_2 on N_3 , and so on to p_9 on N_{10} . Then in the event of failure of N_1 , nodes N_2 through N_{10} would share the extra work equally, instead of burdening a single node with all the extra work.

21.4.2 Updates and Consistency of Replicas

Since each partition is replicated, updates made to tuples in a partition must be performed on all the replicas of the partition. For data that is never updated after it has been created, reads can be performed at any of the replicas, since all of them will have the same value. If a storage system ensures that all replicas are exclusive-locked and updated atomically (using, for example, the two-phase commit protocol which we will see in Section 23.2.1), reads of a tuple can be performed (after obtaining a shared lock) at any of the replicas and will see the most recent version of the tuple.

If data are updated, and replicas are not updated atomically, different replicas may temporarily have different values. Thus, a read may see a different value depending on which replica it accesses. Most applications require that read requests for a tuple must

receive the most recent version of the tuple; updates that are based on reading an older version could result in a *lost update problem*.

One way of ensuring that reads get the latest value is to treat one of the replicas of each partition as a **master replica**. All updates are sent to the master replica and are then propagated to other replicas. Reads are also sent to the master replica, so that reads get the latest version of any data item even if updates have not yet been applied to the other replicas.

If a master replica fails, a new master is assigned for that partition. It is important to ensure that every update operation performed by the old master has also been seen by the new master. Further, the old master may have updated some of the replicas, but not all, before it failed; the new master must complete the task of updating all the replicas. We discuss details in Section 23.6.2.

It is important to know which node is the (current) master for each partition. This information can be stored along with the partition table. Specifically, the partition table must record, in addition to the range of key values assigned to that partition, where the replicas of the partition are stored, and further which replica is currently the master.

Three solutions are commonly used to update replicas.

- The *two-phase commit* (2PC) protocol, which ensures that multiple updates performed by a transaction are applied atomically across multiple sites, is described in Section 23.2. This protocol can be used with replicas to ensure that an update is performed atomically on all replicas of a tuple.

We assume for now that all replicas are available and can be updated. Issues of how to allow two-phase commit to continue execution in the presence of failures, when some replicas may not be reachable, are discussed in Section 23.4.

- Persistent messaging systems, described in Section 23.2.3, which guarantee that a message is delivered once it is sent. Persistent messaging systems can be used to update replicas as follows: An update to a tuple is registered as a persistent message, sent to all replicas of the tuple. Once the message is recorded, the persistent messaging system ensures it will be delivered to all replicas. Thus, all replicas will get the update, eventually; the property is known as **eventual consistency** of replicas.

However, there may be a delay in message delivery, and during that time some replicas may have applied an update while others have not. To ensure that reads get a consistent version, reads are performed only at a master replica, where updates are made first. (If the site with a master replica of a tuple has failed, another replica can take over as the master replica after ensuring all pending persistent messages with updates have been applied.) Details are presented in Section 23.6.2.

- Protocols called *consensus protocols*, that allow updates of replicas to proceed even in the face of failures, when some replicas may not be reachable, can be used to manage update of replicas. Unlike the preceding protocols, consensus protocols can work even without a designated master replica. We study consensus protocols in Section 23.8.

21.5 Parallel Indexing

Indices in a parallel data storage system can be divided into two kinds: local and global indices. In the following discussion, when virtual node partitioning is used, the term *node* should be understood to mean virtual node (or equivalently, tablet).

- A **local index** is an index built on tuples stored in a particular node; typically, such an index would be built on all the partitions of a given relation. The index contents are stored on the same node as the data.
- A **global index** is an index built on data stored across multiple nodes; a global index can be used to efficiently find matching tuples, regardless of where the tuples are stored.

While the contents of a global index could be stored at a single central location, such a scheme would result in poor scalability; as a result, the contents of a global index should be partitioned across multiple nodes.

A **global primary index** on a relation is a global index on the attributes on which the tuples of the relation are partitioned. A global index on partitioning attribute K is constructed by merely creating local indices on K on each partition.

A query that is intended to retrieve tuples with a specific key value k_1 for K can be answered by first finding which partition could hold the key value k_1 , and then using the local index in that partition to find the required tuples.

For example, suppose the *student* relation is partitioned on the attribute ID, and a global index is to be constructed on the attribute ID. All that is required is to construct a local index on ID on each partition. Figure 21.6(a) shows a global primary index on the *student* relation, on attribute ID; the local indices are not shown explicitly in the figure.

A query that is intended to retrieve tuples with a specific value for ID, say 557, can be answered by first using the partitioning function on ID to first find which partition could contain the specified ID value 557; the query is then sent to the corresponding node, which uses the local index on ID to locate the required tuples.

Note that ID is the primary key for the relation *student*; however, the above scheme would work even if the partitioning attribute were not the primary key. The scheme can be extended to handle range queries, provided the partitioning function is itself based on ranges of values; a partitioning scheme based on hashing cannot support range queries.

A **global secondary index** on a relation is a global index whose index attributes do not match the attributes on which the tuples are partitioned.

Suppose the partitioning attributes are K_p , while the index attributes are K_i , with $K_p \neq K_i$. One approach for answering a selection query on attributes K_i is as follows: If a local index is created on K_i on each partition of the relation, the query is sent to each

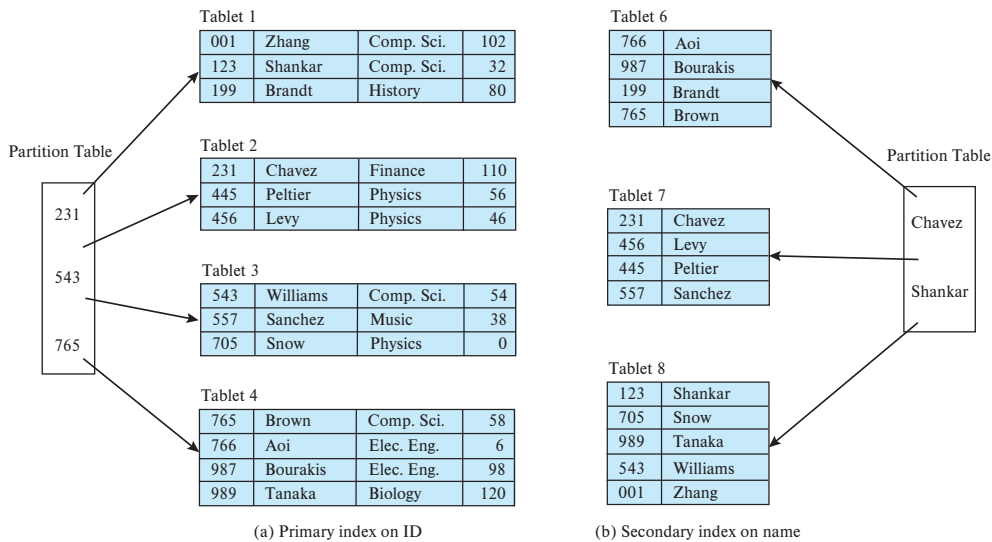


Figure 21.6 Global primary and secondary indices on *student* relation

partition, and the local index is used to find matching tuples. Such an approach using local indices is very inefficient if the number of partitions is large, since every partition has to be queried, even if only one or a few partitions contain matching tuples.

We now illustrate an efficient scheme for constructing a global secondary index by using an example. Consider again the *student* relation partitioned on the attribute *ID*, and suppose a global index is to be constructed on the attribute *name*. A simple way to construct such an index is to create a set of (*name*, *ID*) tuples, with one tuple per *student* tuple; let us call this set of tuples *index_name*. Now, the *index_name* tuples are partitioned on the attribute *name*. A local index on *name* is then constructed on each partition of *index_name*. In addition, a global index is created on the *ID*, which is the partitioning attribute. Figure 21.6(b) shows a secondary index on the *student* relation on attribute *name*; local indices are not explicitly shown in the figure.

Now, a query that needs to retrieve students with a given name can be handled by first examining the partition function of *index_name* to find which partition could store *index_name* tuples with the given name; the query is then sent to that partition, which uses the local index on *name* to find the corresponding *ID* value. Next, the global index on the *ID* value is used to find the required tuple.

Note that in the example above, the partitioning attribute *ID* does not have duplicates; hence it suffices to add only the index key *name* and the attribute *ID* to the *index_name* relation. Otherwise, further attributes would have to be added to ensure tuples can be uniquely identified, as described next.

In general, given a relation *r*, which is partitioned on a set of attributes K_p , if we wish to create a global secondary index on a set of attributes K_i , we create a new relation r_i^s , containing the following attributes:

1. K_i , and K_p
2. If the partitioning attributes K_p have duplicates, we would have to add further attributes K_u , such that $K_p \cup K_u$ is a key for the relation being indexed.

The relation r_i^s is partitioned on K_i , and a local index is created on K_i . In addition, a local index on attributes (K_p, K_u) is created on each partition of relation r .

We now consider how to use a global secondary index to answer a query. Consider a query that specifies a particular value v for K_i . The query is processed as follows:

1. The relevant partition of r_i^s for the value v is found using the partitioning function on K_i .
2. Use the local index on K_i at the above partition to find tuples of r_i^s that have the specified value v for K_i .
3. The tuples in the preceding result are partitioned based on the K_p value and sent to the corresponding nodes.
4. At each node, the tuples received from the preceding step are used along with the local index on r on attributes $K_p \cup K_u$, to find matching r tuples.

Note that relation r_i^s is basically a materialized view defined as $r_i^s = \Pi_{K_i, K_p, K_u}(r)$. Whenever r is modified by inserts, deletions, or updates to tuples, the materialized view r_i^s must be correspondingly updated.

Note also that updates to a tuple of r at some node may result in updates to tuples of r_i^s at other nodes. For example, in Figure 21.6, if the name of ID 001 is updated from Zhang to Yang, the tuple (Zhang, 001) at Tablet8 will be updated to (Yang, 001); since both Zhang and Yang belong in the same partition of the secondary index, no other partition is affected. On the other hand, if the name is updated from Zhang to Bolin, the tuple (Zhang, 001) will be deleted from Tablet8 and a new entry (Bolin, 001) added to Tablet6.

Performing the above updates to the secondary index as part of the same transaction that updates the base relation requires updates to be committed atomically across multiple nodes. Two-phase commit, discussed in Section 23.2, can be used for this task. Alternatives based on persistent messaging can also be used as described in Section 23.2.3, provided it is acceptable for the secondary index to be somewhat out of date.

21.6 Distributed File Systems

A **distributed file system** stores files across a large collection of machines while giving a single-file-system view to clients. As with any file system, there is a system of file names and directories, which clients can use to identify and access files. Clients do not need to bother about where the files are stored.

The goal of first-generation distributed file systems was to allow client machines to access files stored on one or more file servers. In contrast, later-generation distributed file systems, which we focus on, address distribution of file blocks across a very large number of nodes. Such distributed file systems can store very large amounts of data and support very large numbers of concurrent clients. A landmark system in this context was the Google File System (GFS), developed in the early 2000s, which saw widespread use within Google. The open-source Hadoop File System (HDFS) is based on the GFS architecture and is now very widely used.

Distributed file systems are generally designed to efficiently store large files whose sizes range from tens of megabytes to hundreds of gigabytes or more. However, they are designed to store moderate numbers of such files, of the order of millions; they are typically not designed to store billions of different files. In contrast, the parallel data storage systems we have seen earlier are designed to store very large numbers (billions or more) of data items, whose size can range from small (tens of bytes) to medium (a few megabytes).

As in parallel data storage systems, the data in a distributed file system are stored across a number of nodes. Since files can be much larger than data items in a data storage system, files are broken up into multiple blocks. The blocks of a single file can be partitioned across multiple machines. Further, each file block is replicated across multiple (typically three) machines, so that a machine failure does not result in the file becoming inaccessible.

File systems typically support two kinds of *metadata*:

1. A directory system, which allows a hierarchical organization of files into directories and subdirectories, and
2. A mapping from a file name to the sequence of identifiers of blocks that store the actual data in each file.

In the case of a centralized file system, the block identifiers help locate blocks in a storage device such as a disk. In the case of a distributed file system, in addition to providing a block identifier, the file system must provide the location (node identifier) where the block is stored; in fact, due to replication, the file system provides a set of node identifiers along with each block identifier.

In the rest of this section, we describe the organization of the Hadoop File System (HDFS), which is shown in Figure 21.7; the architecture of HDFS is derived from that of the Google File System (GFS). The nodes (machines) which store data blocks in HDFS are called **datanodes**. Blocks have an associated ID, and datanodes map the block ID to a location in their local file system where the block is stored.

The file system metadata too can be partitioned across many nodes, but unless carefully architected, this could lead to bad performance. GFS and HDFS took a simpler and more pragmatic approach of storing the file system metadata at a single node, called the **namenode** in HDFS.

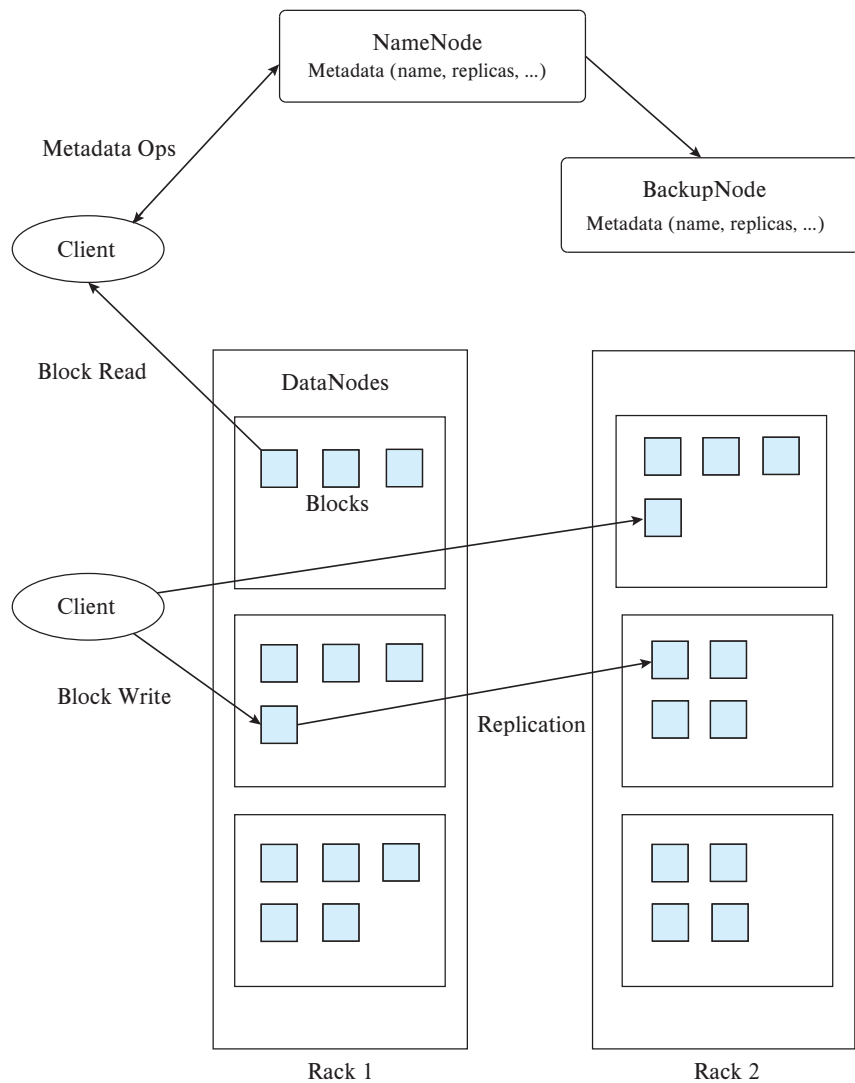


Figure 21.7 Hadoop Distributed File System (HDFS) architecture

Since all metadata reads have to go to the namenode, if a disk access were required to satisfy a metadata read, the number of requests that could be satisfied per second would be very small. To ensure acceptable performance, HDFS namenodes cache the entire metadata in memory; the size of memory then becomes a limiting factor in the number of files and blocks that the file system can manage. To reduce the memory size, HDFS uses very large block sizes (typically 64 MB) to reduce the number of blocks that the namenode must track for each file. Despite this, the limited amount of main memory on most machines constrains namenodes to support only a limited number of

files (of the order of millions). However, with main-memory sizes of many gigabytes, and block sizes of tens of megabytes, an HDFS system can comfortably handle many petabytes of data.

In any system with a large number of datanodes, datanode failures are a frequent occurrence. To deal with datanode failures, data blocks must be replicated to multiple datanodes. If a datanode fails, the block can still be read from one of the other datanodes that stores a replica of the block. Replication to three datanodes is widely used to provide high availability, without paying too high a storage overhead.

We now consider how a file open and read request is satisfied with HDFS. First, the client contacts the namenode, with the name of the file. The namenode finds the list of IDs of blocks containing the file data and returns to the client the list of block IDs, along with the set of nodes that contain replicas of each of the blocks. The client then contacts any one of the replicas for each block of the file, sending it the ID of the block, to retrieve the block. In case that particular replica does not respond, the client can contact any of the other replicas.

To satisfy a write request, the client first contacts the namenode, which allocates blocks, and decides which datanodes should store replicas of each block. The metadata are recorded at the namenode and sent back to the client. The client then writes the block to all the replicas. As an optimization to reduce network traffic, HDFS implementations may choose to store two replicas in the same rack; in that case, the block write is performed to one replica, which then copies the data to the second replica on the same rack. When all the replicas have processed the write of a block, an acknowledgment is sent to the client.

Replication introduces the problem of consistency of data across the replicas in case the file is updated. As an example, suppose one of the replicas of a data block is updated, but due to system failure, another replica does not get updated; then the system could end up with inconsistent states across the replicas. And what value is read would depend on which replica is accessed, which is not an acceptable situation.

While data storage systems in general need to deal with consistency, using techniques that we study in Chapter 23, some distributed file systems such as HDFS take a different approach: namely, not allowing updates. In other words, a file can be appended to, but data that are written cannot be updated. As each block of the file is written, the block is copied to the replicas. The file cannot be read until it is *closed*, that is, all data have been written to the file, and the blocks have been written successfully at all their replicas. The model of writing data to a file is sometimes referred to as **write-once-read-many** access model. Others such as GFS allow updates and detect certain inconsistent states caused by failures while writing to replicas; however, transactional (atomic) updates are not supported.

The restriction that files cannot be updated, but can only be appended to, is not a problem for many applications of a distributed file system. Applications that require updates should use a data-storage system that supports updates instead of using a distributed file system.

21.7 Parallel Key-Value Stores

Many Web applications need to store very large numbers (many billions) of relatively small records (of size ranging from a few kilobytes to a few megabytes). Storage would have to be distributed across thousands of nodes. Storing such records as files in a distributed file system is infeasible, since file systems are not designed to store such large numbers of small files. Ideally, a massively parallel relational database should be used to store such data. But the parallel relational databases available in the early 2000s were not designed to work at a massive scale; nor did they support the ability to easily add more nodes to the system without causing significant disruption to ongoing activities.

A number of parallel key-value storage systems were developed to meet the needs of such web applications. A **key-value store** provides a way to store or update a data item (value) with an associated key and to retrieve the data item with a given key. Some key-value stores treat the data items as uninterpreted sequences of bytes, while others allow a schema to be associated with the data item. If the system supports definition of a schema for data items, it is possible for the system to create and maintain secondary indices on specified attributes of data items.

Key-value stores support two very basic functions on tables: `put(table, key, value)`, used to store values, with an associated key, in a table, and `get(table, key)`, which retrieves the stored value associated with the specified key. In addition, they may support other functions, such as range queries on key values, using `get(table, key1, key2)`.

Further, many key-value stores support some form of flexible schema.

- Some allow column names to be specified as part of a schema definition, similar to relational data stores.
- Others allow columns to be added to, or deleted from, individual tuples; such key-value stores are sometimes referred to as **wide-column stores**. Such key-value stores support functions such as `put(table, key, columnname, value)`, to store a value in a specific column of a row identified by the key (creating the column if it does not already exist), and `get(table, key, columnname)`, which retrieves the value for a specific column of a specific row identified by the key. Further, `delete(table, key, columnname)` deletes a specific column from a row.
- Yet other key-value stores allow the value stored with a key to have a complex structure, typically based on JSON; they are sometimes referred to as **document stores**.

The ability to specify a (partial) schema of the stored value allows the key-value store to evaluate selection predicates at the data store; some stores also use the schema to support secondary indices.

We use the term *key-value store* to include all the above types of data stores; however, some people use the term *key-value store* to refer more specifically to those that

do not support any form of schema and treat the value as an uninterpreted sequence of bytes.

Parallel key-value stores typically support *elasticity*, whereby the number of nodes can be increased or decreased incrementally, depending on demand. As nodes are added, tablets can be moved to the new nodes. To reduce the number of nodes, tablets can be moved away from some nodes, which can then be removed from the system.

Widely used parallel key-value stores that support flexible columns (also known as wide-column stores) include Bigtable from Google, Apache HBase, Apache Cassandra (originally developed at Facebook), and Microsoft Azure Table Storage from Microsoft, among others. Key-value stores that support a schema include Megastore and Spanner from Google, and Sherpa/PNUTS from Yahoo!. Key-value stores that support semi-structured data (also known as document-stores) include Couchbase, DynamoDB from Amazon, and MongoDB, among others. Redis and Memcached are parallel in-memory key-value stores which are widely used for caching data.

Key-value stores are not full-fledged databases, since they do not provide many of the features that are viewed as standard on database systems today. Features that key-value stores typically do not support include declarative querying (using SQL or any other declarative query language), support for transactions, and support for efficient retrieval of records based on selections on nonkey attributes (traditional databases support such retrieval using secondary indices). In fact, they typically do not support primary-key constraints for attributes other than the key, and do not support foreign-key constraints.

21.7.1 Data Representation

As an example of data management needs of web applications, consider the profile of a user, which needs to be accessible to a number of different applications that are run by an organization. The profile contains a variety of attributes, and there are frequent additions to the attributes stored in the profile. Some attributes may contain complex data. A simple relational representation is often not sufficient for such complex data.

Many key-value stores support the *JavaScript Object Notation (JSON)* representation, which has found increasing acceptance for representing complex data (JSON is covered in Section 8.1.2). The JSON representation provides flexibility in the set of attributes that a record contains, as well as the types of these attributes. Yet others, such as Bigtable, define their own data model for complex data, including support for records with a very large number of optional columns.

In Bigtable, a record is not stored as a single value but is instead split into component attributes that are stored separately. Thus, the key for an attribute value conceptually consists of (record-identifier, attribute-name). Each attribute value is just a string as far as Bigtable is concerned. To fetch all attributes of a record, a range query, or more precisely a prefix-match query consisting of just the record identifier, is used. The `get()` function returns the attribute names along with the values. For efficient retrieval

of all attributes of a record, the storage system stores entries sorted by the key, so all attribute values of a particular record are clustered together.

In fact, the record identifier can itself be structured hierarchically, although to Bigtable itself the record identifier is just a string. For example, an application that stores pages retrieved from a web crawl could map a URL of the form:

`www.cs.yale.edu/people/silberschatz.html`

to the record identifier:

`edu.yale.cs.www/people/silberschatz.html`

so that pages are clustered in a useful order.

Data-storage systems often allow multiple versions of data items to be stored. Versions are often identified by timestamp, but they may be alternatively identified by an integer value that is incremented whenever a new version of a data item is created. Reads can specify the required version of a data item, or they can pick the version with the highest version number. In Bigtable, for example, a key actually consists of three parts: (record-identifier, attribute-name, timestamp).

Some key-value stores support *columnar storage* of rows, with each column of a row stored separately, with the row key and the column value stored for each row. Such a representation allows a scan to efficiently retrieve a specified column of all rows, without having to retrieve other columns from storage. In contrast, if rows are stored in the usual manner, with all column values stored with the row, a sequential scan of the storage would fetch columns that are not required, reducing performance.

Further, some key-value stores support the notion of a **column family**, which groups sets of columns into a column family. For a given row, all the columns in a specific column family are stored together, but columns from other column families are stored separately. If a set of columns are often retrieved together, storing them as a column family may allow more efficient retrieval, as compared to either columnar storage where these are stored and retrieved separately, or a row storage, which could result in retrieving unneeded columns from storage.

21.7.2 Storing and Retrieving Data

In this section, we use the term *tablet* to refer to partitions, as discussed in Section 21.3.3. We also use the term **tablet server** to refer to the node that acts as the server for a particular tablet; all requests related to a tablet are sent to the tablet server for that tablet.⁵ The tablet server would be one of the nodes that has a replica of the tablet and plays the role of master replica as discussed in Section 21.4.1.⁶

⁵HBase uses the terms *region* and *region server* in place of the terms *tablet* and *tablet server*

⁶In BigTable and HBase, replication is handled by the underlying distributed file system; tablet data are stored in files, and one of the nodes containing a replica of the tablet files is chosen as the tablet server.

We use the term **master** to refer to a site that stores a master copy of the partition information, including, for each tablet, the key ranges for the tablet, the sites storing the replicas of the tablet, and the current tablet server for that tablet.⁷ The master is also responsible for tracking the health of tablet servers; in case a tablet server node fails, the master assigns one of the other nodes that contains a replica of the tablet to act as the new tablet server for that tablet. The master is also responsible for reassigning tablets to balance the load in the system if some node is overloaded or if a new node is added to the system.

For each request coming into the system, the tablet corresponding to the key must be identified, and the request routed to the tablet server. If a single master site were responsible for this task, it would get overloaded. Instead, the routing task is parallelized in one of two ways:

- By replicating the partition information to the client sites; the key-value store API used by clients looks up the partition information copy stored at the client to decide where to route a request. This approach is used in Bigtable and HBase.
- By replicating the partition information to a set of router sites, which route requests to the site with the appropriate tablet. Requests can be sent to any one of the router sites, which forward the request to the correct tablet master. This approach is used, for example, in the PNUTS system.

Since there may be a gap between actually splitting or moving a tablet and updating the partition information at a router (or client), the partition information may be out of date when the routing decision is made. When the request reaches the identified tablet master node, the node detects that the tablet has been split, or that the site no longer stores a (master) replica of the tablet. In such a case, the request is returned to the router with an indication that the routing was incorrect; the router then retrieves up-to-date tablet mapping information from the master and reroutes the request to the correct destination.

Figure 21.8 depicts the architecture of a cloud data-storage system, based loosely on the PNUTS architecture. Other systems provide similar functionality, although their architecture may vary. For example, Bigtable/HBase do not have separate routers; the partitioning and tablet-server mapping information is stored in the Google File System/HDFS, and clients read the information from the file system and decide where to send their requests.

21.7.2.1 Geographically Distributed Storage

Several key-value stores support replication of data to geographically distributed locations; some of these also support partitioning of data across geographically distributed locations, allowing different partitions to be replicated in different sets of locations.

⁷The term *tablet controller* is used by PNUTS to refer to the master site.

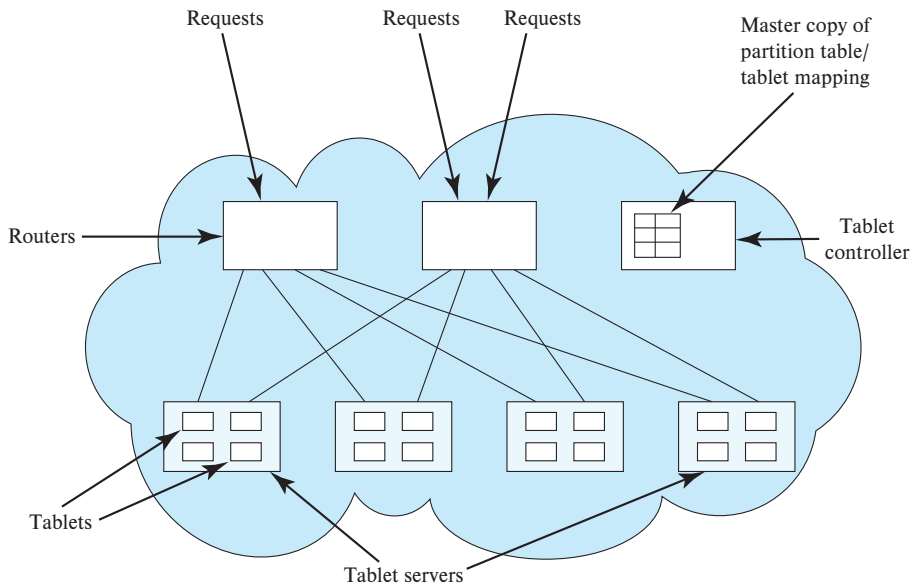


Figure 21.8 Architecture of a cloud data storage system.

One of the key motivations for geographic distribution is fault tolerance, which allows the system to continue functioning even if an entire data center fails due to a disaster such as a fire or an earthquake; in fact, earthquakes could cause all data centers in a region to fail. A second key motivation is to allow a copy of the data to reside at a geographic region close to the user; requiring data to be fetched from across the world could result in latencies of hundreds of milliseconds.

A key performance issue with geographical replication of data is that the latency across geographical regions is much higher than the latency within a data center. Some key-value stores nevertheless support geographically distributed replication, requiring transactions to wait for confirmation of updates from remote locations. Other key-value stores support asynchronous replication of updates to remote locations, allowing a transaction to commit without waiting for confirmation of updates from a remote location. There is, however, a risk of loss of updates in case of failure before the updates are replicated. Some key-value stores allow the application to choose whether to wait for confirmation from remote locations or to commit as soon as updates are performed locally.

Key-value stores that support geographic replication include Apache Cassandra, Megastore and Spanner from Google, Windows Azure storage from Microsoft, and PNUTS/Sherpa from Yahoo!, among others.

21.7.2.2 Index Structure

The records in each tablet in a key-value store are indexed on the key; range queries can be efficiently supported by storing records clustered on the key. A B⁺-tree file organization is a good option, since it supports indexing with clustered storage of records.

The widely used key-value stores BigTable and HBase are built on top of distributed file systems in which files are *immutable*; that is, files cannot be updated once they are created. Thus B⁺-tree indices or file organization cannot be stored in immutable files, since B⁺-trees require updates, which cannot be done on an immutable file.

Instead, the BigTable and HBase systems use the *stepped-merge* variant of the *log structured merge tree* (LSM tree), which we saw in Section 14.8.1, and is described in more detail in Section 24.2. The LSM tree does not perform updates on existing trees, but instead creates new trees either using new data or by merging existing trees. Thus, it is an ideal fit for use on top of distributed file systems that only support immutable files. As an extra benefit, the LSM tree supports clustered storage of records, and can support very high insert and update rates, which has been found very useful in many applications of key-value stores. Several key-value stores, such as Apache Cassandra and the WiredTiger storage structure used by MongoDB, use the LSM tree structure.

21.7.3 Support for Transactions

Most key-value stores offer limited support for transactions. For example, key-value stores typically support atomic updates on a single data item and ensure that updates on the data item are serialized, that is, run one after the other. Serializability at the level of individual operations is thus trivially satisfied, since the operations are run serially. Note that serializability at the level of transactions is not guaranteed by serial execution of updates on individual data items, since a transaction may access more than one data item.

Some key-value stores, such as Google's MegaStore and Spanner, provide full support for ACID transactions across multiple nodes. However, most key-value stores do not support transactions across multiple data items.

Some key-value stores provide a test-and-set operation that can help applications implement limited forms of concurrency control, as we see next.

21.7.3.1 Concurrency Control

Some key-value stores, such as the Megastore and Spanner systems from Google, support concurrency control via locking. Issues in distributed concurrency control are discussed in Chapter 23. Spanner also supports versioning and database snapshots based on timestamps. Details of the multiversion concurrency control technique implemented in Spanner are discussed in Section 23.5.1.

However, most of the other key-value stores, such as Bigtable, PNUTS/Sherpa, and MongoDB, support atomic operations on single data items (which may have multiple columns, or may be JSON documents in MongoDB).

Some key-value stores, such as HBase and PNUTS, provide an atomic test-and-set function, which allows an update to a data item to be conditional on the current version of the data item being the same as a specified version number; the check (test) and the update (set) are performed atomically. This feature can be used to implement a limited form of validation-based concurrency control, as discussed in Section 23.3.7.

Some data stores support atomic increment operations on data items and atomic execution of stored procedures. For example, HBase supports the `incrementColumnValue()` operation, which atomically reads and increments a column value, and a `checkAndPut()` which atomically checks a condition on a data item and updates it only if the check succeeds. HBase also supports atomic execution of stored procedures, which are called “coprocessors” in HBase terminology. These procedures run on a single data item and are executed atomically.

21.7.3.2 Atomic Commit

BigTable, HBase, and PNUTS support atomic commit of multiple updates to a single row; however, none of these systems supports atomic updates across different rows.

As one of the results of the above limitation, none of these systems supports secondary indices; updates to a data item would require updates to the secondary index, which cannot be done atomically.

Some systems, such as PNUTS, support secondary indices or materialized views with deferred updates; updates to a data item result in updates to the secondary index or materialized view being added to a messaging service to be delivered to the node where the update needs to be applied. These updates are guaranteed to be delivered and applied subsequently; however, until they are applied, the secondary index may be inconsistent with the underlying data. View maintenance is also supported by PNUTS in the same deferred fashion. There is no transactional guarantee on the updates of such secondary indices or materialized views, and only a best-effort guarantee in terms of when the updates reach their destination. Consistency issues with deferred maintenance are discussed in Section 23.6.3.

In contrast, the Megastore and Spanner systems developed by Google support atomic commit for transactions spanning multiple data items, which can be spread across multiple nodes. These systems use two-phase commit (discussed in Section 23.2) to ensure atomic commit across multiple nodes.

21.7.3.3 Dealing with Failures

If a tablet server node fails, another node that has a copy of the tablet should be assigned the task of serving the tablet. The master node is responsible for detecting node failures and reassigning tablet servers.

When a new node takes over as tablet server, it must recover the state of the tablet. To ensure that updates to the tablet survive node failures, updates to a tablet are logged, and the log is itself replicated. When a site fails, the tablets at the site are assigned to

other sites; the new master site of each tablet is responsible for performing recovery actions using the log to bring its copy of the tablet to an up-to-date state, after which updates and reads can be performed on the tablet.

In Bigtable, as an example, mapping information is stored in an index structure, and the index, as well as the actual tablet data, are stored in the file system. Tablet data updates are not flushed immediately, but log data are. The file system ensures that the file system data are replicated and will be available even in the face of failure of a few nodes in the cluster. Thus, when a tablet is reassigned, the new server for that tablet has access to up-to-date log data.

Yahoo!'s Sherpa/PNUTS system, on the other hand, explicitly replicates tablets to multiple nodes in a cluster and uses a persistent messaging system to implement the log. The persistent messaging system replicates log records at multiple sites to ensure availability in the event of a failure. When a new node takes over as the tablet server, it must apply any pending log records that were generated by the earlier tablet server before taking over as the tablet server.

To ensure availability in the face of failures, data must be replicated. As noted in Section 21.4.2, a key issue with replication is the task of keeping the replicas consistent with each other. Different systems implement atomic update of replicas in different fashions. Google BigTable and Apache HBase use replication features provided by an underlying file system (GFS for BigTable, and HDFS for HBase), instead of implementing replication on their own. Interestingly, neither GFS nor HDFS supports atomic updates of all replicas of a file; instead they support appends to files, which are copied to all replicas of the file blocks. An append is successful only when it has been applied to all replicas. System failures can result in appends that are applied to only some replicas; such incomplete appends are detected using sequence numbers and are cleaned up when they are detected.

Some systems such as PNUTS use a persistent messaging service to log updates; the messaging service guarantees that updates will be delivered to all replicas. Other systems, such as Google's Megastore and Spanner, use a technique called distributed consensus to implement consistent replication, as we discuss in Section 23.8. Such systems require a majority of replicas to be available to perform an update. Other systems, such as Apache Cassandra and MongoDB, allow the user control over how many replicas must be available to perform an update. Setting the value low could result in conflicting updates, which must be resolved later. We discuss these issues in Section 23.6.

21.7.4 Managing Without Declarative Queries

Key-value stores do not provide any query processing facility, such as SQL language support, or even lower-level primitives such as joins. Many applications that use key-value stores can manage without query language support. The primary mode of data access in such applications is to store data with an associated key and to retrieve data

with that key. In the user profile example, the key for user-profile data would be the user's identifier.

There are applications that require joins but implement the joins either in application code or by a form of view materialization. For example, in a social-networking application, each user should be shown new posts from all her friends, which conceptually requires a join.

One approach to computing the join is to implement it in the application code, by first finding the set of friends of a given user, and then querying the data object representing each friend, to find their recent posts.

An alternative approach is as follows: Whenever a user makes a post, for each friend of the user a message is sent to, the data object representing that friend and the data associated with the friend are updated with a summary of the new post. When that user checks for updates, all required data are available in one place and can be retrieved quickly.

Both approaches can be used without any underlying support for joins. There are trade-offs between the two alternatives such as higher cost at query time for the first alternative versus higher storage cost and higher cost at the time of writes for the second alternative.

21.7.5 Performance Optimizations

When using a data storage system, the physical location of data are decided by the storage system and hidden from the client. When storing multiple relations that need to be joined, partitioning each independently may be suboptimal in terms of communication cost. For example, if the join of two relations is computed frequently, it may be best if they are partitioned in exactly the same way, on their join attributes. As we will see in Section 22.7.4, doing so would allow the join to be computed in parallel at each storage site, without data transfer.

To support such scenarios, some data storage systems allow the schema designer to specify that tuples of one relation should be stored in the same partitions as tuples of another relation that they reference, typically using a foreign key. A typical use of this functionality is to store all tuples related to a particular entity together in the same partition; the set of such tuples is called an **entity group**.

Further, many data storage systems, such as HBase, support *stored functions* or *stored procedures*. Stored functions/procedures allow clients to invoke a function on a tuple (or an entity group) and instead of the tuples being fetched and executed locally, the function is executed at the partition where the tuple is stored. Stored functions/procedures are particularly useful if the stored tuples are large, while the function/procedure results are small, reducing data transfer.

Many data storage systems provide features such as support for automatically deleting old versions of data items after some period of time, or even deleting data items that are older than some specified period.

21.8 Summary

- Parallel databases have gained significant commercial acceptance in the past 20 years.
- Data storage and indexing are two important aspects of parallel database systems.
- Data partitioning involves the distribution of data among multiple nodes. In I/O parallelism, relations are partitioned among available disks so that they can be retrieved faster. Three commonly used partitioning techniques are round-robin partitioning, hash partitioning, and range partitioning.
- Skew is a major problem, especially with increasing degrees of parallelism. Balanced partitioning vectors, using histograms, and virtual node partitioning are among the techniques used to reduce skew.
- Parallel data storage systems must be resilient to failure of nodes. To ensure that data are not lost on node failure, tuples are replicated across at least two nodes, and often three nodes. If a node fails, the tuples that it stored can still be accessed from the other nodes where the tuples are replicated.
- Indices in a parallel data storage system can be divided into two kinds: local and global. A local index is an index built on tuples stored in a particular node; The index contents are stored on the same node as the data. A global index is an index built on data stored across multiple nodes.
- A distributed file system stores files across a large collection of machines, while giving a single-file-system view to clients. As with any file system, there is a system of file names and directories, which clients can use to identify and access files. Clients do not need to bother about where the files are stored.
- Web applications need to store very large numbers (many billions) of relatively small records (of size ranging from a few kilobytes to a few megabytes). A number of parallel key-value storage systems were developed to meet the needs of such web applications. A key-value store provides a way to store or update a data item (value) with an associated key, and to retrieve the data item with a given key.

Review Terms

- Key-value stores
- Data storage system
- I/O parallelism
- Data partitioning
- Horizontal partitioning
- Partitioning strategies
 - Round-robin
 - Hash partitioning
 - Range partitioning
- Partitioning vector

- Point queries
- Range queries
- Skew
 - Execution skew
 - Data distribution skew
 - Attribute-value skew
 - Partition skew
 - Execution skew
- Handling of skew
 - Balanced range-partitioning vector
 - Histogram
 - Virtual nodes
- Elasticity of storage
- Table
- Tablets
- Partition table
- Master node
- Routers
- Consistent hashing
- Distributed hash tables
- Replication
 - Replication within a data center
 - Replication across data center
 - Master replicas
 - Consistency of replicas
- Eventual consistency
- Global primary index
- Global secondary index
- Distributed file system
- Write-once-read-many access model
- Key-value store
- Wide-column stores
- Document stores
- Column family
- Tablet server

Practice Exercises

- 21.1** In a range selection on a range-partitioned attribute, it is possible that only one disk may need to be accessed. Describe the benefits and drawbacks of this property.
- 21.2** Recall that histograms are used for constructing load-balanced range partitions.
- a. Suppose you have a histogram where values are between 1 and 100, and are partitioned into 10 ranges, 1 – 10, 11 – 20, ..., 91 – 100, with frequencies 15, 5, 20, 10, 10, 5, 5, 20, 5, and 5, respectively. Give a load-balanced range partitioning function to divide the values into five partitions.
 - b. Write an algorithm for computing a balanced range partition with p partitions, given a histogram of frequency distributions containing n ranges.
- 21.3** Histograms are traditionally constructed on the values of a specific attribute (or set of attributes) of a relation. Such histograms are good for avoiding data

distribution skew but are not very useful for avoiding execution skew. Explain why.

Now suppose you have a workload of queries that perform point lookups. Explain how you can use the queries in the workload to come up with a partitioning scheme that avoids execution skew.

21.4 Replication:

- a. Give two reasons for replicating data across geographically distributed data centers.
- b. Centralized databases support replication using log records. How is the replication in centralized databases different from that in parallel/distributed databases?

21.5 Parallel indices:

- a. Secondary indices in a centralized database store the record identifier. A global secondary index too could potentially store a partition number holding the record, and a record identifier within the partition. Why would this be a bad idea?
- b. Global secondary indices are implemented in a way similar to local secondary indices that are used when records are stored in a B⁺-tree file organization. Explain the similarities between the two scenarios that result in a similar implementation of the secondary indices.

21.6 Parallel database systems store replicas of each data item (or partition) on more than one node.

- a. Why is it a good idea to distribute the copies of the data items allocated to a node across multiple other nodes, instead of storing all the copies in the same node (or set of nodes)?
- b. What are the benefits and drawbacks of using RAID storage instead of storing an extra copy of each data item?

21.7 Partitioning and replication.

- a. Explain why range-partitioning gives better control on tablet sizes than hash partitioning. List an analogy between this case and the case of B⁺-tree indices versus hash indices.
- b. Some systems first perform hashing on the key, and then use range partitioning on the hash values. What could be a motivation for this choice, and what are its drawbacks as compared to performing range partition direction on the key?
- c. It is possible to horizontally partition data, and then perform vertical partitioning locally at each node. It is also possible to do the converse,

where vertical partitioning is done first, and then each partition is then horizontally partitioned independently. What are the benefits of the first option over the second one?

- 21.8** In order to send a request to the master replica of a data item, a node must know which replica is the master for that data item.
- Suppose that between the time the node identifies which node is the master replica for a data item, and the time the request reaches the identified node, the mastership has changed, and a different node is now the master. How can such a situation be dealt with?
 - While the master replica could be chosen on a per-partition basis, some systems support a *per-record master replica*, where the records of a partition (or tablet) are replicated at some set of nodes, but each record's master replica can be on any of the nodes from within this set of nodes, independent of the master replica of other records. List two benefits of keeping track of master on a per-record basis.
 - Suggest how to keep track of the master replica for each record, when there are a large number of records.

Exercises

- 21.9** For each of the three partitioning techniques, namely, round-robin, hash partitioning, and range partitioning, give an example of a query for which that partitioning technique would provide the fastest response.
- 21.10** What factors could result in skew when a relation is partitioned on one of its attributes by:
- Hash partitioning?
 - Range partitioning?

In each case, what can be done to reduce the skew?

- 21.11** What is the motivation for storing related records together in a key-value store? Explain the idea using the notion of an entity group.
- 21.12** Why is it easier for a distributed file system such as GFS or HDFS to support replication than it is for a key-value store?
- 21.13** Joins can be expensive in a key-value store, and difficult to express if the system does not support SQL or a similar declarative query language. What can an application developer do to efficiently get results of join or aggregate queries in such a setting?

Tools

A wide variety of open-source Big Data tools are available, in addition to some commercial tools. In addition, a number of these tools are available on cloud platforms. Google File System (GFS) was an early generation parallel file system. Apache HDFS (hadoop.apache.org) is a widely used distributed file system implementation modeled after GFS. HDFS by itself does not define any internal format for files, but Hadoop implementations today support several optimized file formats such as Sequence files (which allow binary data), Avro (which supports semi-structured schemas) and Parquet and Orc (which support columnar data representation). Hosted cloud storage systems include the Amazon S3 storage system (aws.amazon.com/s3) and Google Cloud Storage (cloud.google.com/storage).

Google's Bigtable was an early generation parallel data storage system, architected as a layer on top of GFS. Amazon's Dynamo is an early generation parallel key-value store which is based on the idea of consistent hashing, developed initially for peer-to-peer data storage. Both are available hosted on the cloud as Google Bigtable (cloud.google.com/bigtable) and Amazon DynamoDB (aws.amazon.com/dynamodb). Google Spanner (cloud.google.com/spanner) is a hosted storage system that provides extensive transactional support. Apache HBase (hbase.apache.org) is a widely used open-source data storage system which is based on Bigtable and is implemented as a layer on top of HDFS. Apache Cassandra (cassandra.apache.org) which was developed at Facebook, Voldemort (www.project-voldemort.com) developed at LinkedIn, MongoDB (www.mongodb.com), CouchDB (couchdb.apache.org) and Riak (basho.com) are all open-source key-value stores. MongoDB and CouchDB use the JSON format for storing data. Aerospike (www.aerospike.com) is an open-source data storage system optimized for Flash storage. There are many other open-source parallel data storage systems available today.

Commercial parallel database systems include Teradata, Teradata Aster Data, IBM Netezza, and Pivotal Greenplum. IBM Netezza, Pivotal Greenplum, and Teradata Aster Data all use PostgreSQL as the underlying database, running independently on each node; each of these systems builds a layer on top, to partition data, and parallelize query processing across the nodes.

Further Reading

In the late 1970s and early 1980s, as the relational model gained reasonably sound footing, people recognized that relational operators are highly parallelizable and have good dataflow properties. Several research projects, including GAMMA ([DeWitt (1990)]), XPRS ([Stonebraker et al. (1988)]), and Volcano ([Graefe (1990)]) were launched to investigate the practicality of parallel storage of data and parallel execution of queries.

Teradata was one of the first commercial shared-nothing parallel database systems designed for decision support systems, and it continues to have a large market share.

Teradata supports partitioning and replication of data to deal with node failures. The Red Brick Warehouse was another early parallel database system designed for decision support (Red Brick was bought by Informix, and later IBM).

Information on the Google file system can be found in [Ghemawat et al. (2003)], while the Google Bigtable system is described in [Chang et al. (2008)]. The Yahoo! PNUTS system is described in [Cooper et al. (2008)], while Google Megastore and Google Spanner are described in [Baker et al. (2011)] and [Corbett et al. (2013)] respectively. Consistent hashing is described in [Karger et al. (1997)], while Dynamo, which is based on consistent hashing, is described in [DeCandia et al. (2007)].

Bibliography

- [Baker et al. (2011)] J. Baker, C. Bond, J. C. Corbett, J. J. Furman, A. Khorlin, J. Larson, J.-M. Leon, Y. Li, A. Lloyd, and V. Yushprakh, “Megastore: Providing Scalable, Highly Available Storage for Interactive Services”, In *Proceedings of the Conference on Innovative Data system Research (CIDR)* (2011), pages 223–234.
- [Chang et al. (2008)] F. Chang, J. Dean, S. Ghemawat, W. C. Hsieh, D. A. Wallach, M. Burrows, T. Chandra, A. Fikes, and R. E. Gruber, “Bigtable: A Distributed Storage System for Structured Data”, *ACM Trans. Comput. Syst.*, Volume 26, Number 2 (2008).
- [Cooper et al. (2008)] B. F. Cooper, R. Ramakrishnan, U. Srivastava, A. Silberstein, P. Bohannon, H.-A. Jacobsen, N. Puz, D. Weaver, and R. Yerneni, “PNUTS: Yahoo!’s Hosted Data Serving Platform”, *Proceedings of the VLDB Endowment*, Volume 1, Number 2 (2008), pages 1277–1288.
- [Corbett et al. (2013)] J. C. Corbett et al., “Spanner: Google’s Globally Distributed Database”, *ACM Trans. on Computer Systems*, Volume 31, Number 3 (2013).
- [DeCandia et al. (2007)] G. DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, S. Sivasubramanian, P. Voshall, and W. Vogels, “Dynamo: Amazons Highly Available Key-value Store”, In *Proc. of the ACM Symposium on Operating System Principles* (2007), pages 205–220.
- [DeWitt (1990)] D. DeWitt, “The Gamma Database Machine Project”, *IEEE Transactions on Knowledge and Data Engineering*, Volume 2, Number 1 (1990), pages 44–62.
- [Ghemawat et al. (2003)] S. Ghemawat, H. Gobioff, and S.-T. Leung, “The Google File System”, *Proc. of the ACM Symposium on Operating System Principles* (2003).
- [Graefe (1990)] G. Graefe, “Encapsulation of Parallelism in the Volcano Query Processing System”, In *Proc. of the ACM SIGMOD Conf. on Management of Data* (1990), pages 102–111.
- [Karger et al. (1997)] D. Karger, E. Lehman, T. Leighton, R. Panigrahy, M. Levine, and D. Lewin, “Consistent Hashing and Random Trees: Distributed Caching Protocols for Relieving Hot Spots on the World Wide Web”, In *Proc. of the ACM Symposium on Theory of Computing* (1997), pages 654–663.

[Stonebraker et al. (1988)] M. Stonebraker, R. H. Katz, D. A. Patterson, and J. K. Ousterhout, “The Design of XPRS”, In *Proc. of the International Conf. on Very Large Databases* (1988), pages 318–330.

Credits

The photo of the sailboats in the beginning of the chapter is due to ©Pavel Nesvadba/Shutterstock.

CHAPTER 22



Parallel and Distributed Query Processing

In this chapter, we discuss algorithms for query processing in parallel database systems. We assume that the queries are **read only**, and our focus is on query processing in decision support systems. Such systems need to execute queries on very large amounts of data, and parallel processing of the query across multiple nodes is critical for processing queries within acceptable response times.

Our focus in the early parts of this chapter is on relational query processing. However, later in the chapter, we examine issues in parallel processing of queries expressed in models other than the relational model.

Transaction processing systems execute large numbers of queries that perform updates, but each query affects only a small number of tuples. Parallel execution is key to handle large transaction processing loads; however, this topic is covered in Chapter 23.

22.1 Overview

Parallel processing can be exploited in two distinct ways in a database system. One approach is **interquery parallelism**, which refers to the execution of multiple queries in parallel with each other, across multiple nodes. The second approach is **intraquery parallelism**, which refers to the processing of different parts of the execution of a single query, in parallel across multiple nodes.

Interquery parallelism is essential for transaction processing systems. Transaction throughput can be increased by this form of parallelism. However, the response times of individual transactions are no faster than they would be if the transactions were run in isolation. Thus, the primary use of interquery parallelism is to scale up a transaction-processing system to support a larger number of transactions per second. Transaction processing systems are considered in Chapter 23.

In contrast, intraquery parallelism is essential for speeding up long-running queries, and it is the focus of this chapter.

Execution of a single query involves execution of multiple operations, such as selects, joins, or aggregate operations. The key to exploiting large-scale parallelism is to process each operation in parallel, across multiple nodes. Such parallelism is referred to as **intraoperation parallelism**. Since the number of tuples in a relation can be large, the degree of intraoperation parallelism is also potentially very large; thus, intraoperation parallelism is natural in a database system.

To illustrate the parallel evaluation of a query, consider a query that requires a relation to be sorted. Suppose that the relation has been partitioned across multiple disks by range partitioning on some attribute, and the sort is requested on the partitioning attribute. The sort operation can be implemented by sorting each partition in parallel, then concatenating the sorted partitions to get the final sorted relation. Thus, we can parallelize a query by parallelizing individual operations.

There is another source of parallelism in evaluating a query: The *operator tree* for a query can contain multiple operations. We can parallelize the evaluation of the operator tree by evaluating in parallel some of the operations that do not depend on one another. Further, as Chapter 15 mentions, we may be able to pipeline the output of one operation to another operation. The two operations can be executed in parallel on separate nodes, one generating output that is consumed by the other, even as it is generated. Both these forms of parallelism are examples of **interoperation parallelism**, which allows different operators of a query to be executed in parallel.

In summary, the execution of a single query can be parallelized in two different ways:

- **Intraoperation parallelism**, which we consider in detail in the next few sections, where we study parallel implementations of common relational operations such as sort, join, aggregate and other operations.
- **Interoperation parallelism**, which we consider in detail in Section 22.5.1.

The two forms of parallelism are complementary and can be used simultaneously on a query. Since the number of operations in a typical query is small, compared to the number of tuples processed by each operation, intraoperation parallelism can scale better with increasing parallelism. However, interoperation parallelism is also important, especially in shared memory systems with multiple cores.

To simplify the presentation of the algorithms, we assume a shared nothing architecture with n nodes, $N_1, N_2, \dots, N_n$. Each node may have one or more disks, but typically the number of such disks is small. We do not address how to partition the data between the disks at a node; RAID organizations can be used with these disks to exploit parallelism at the storage level, rather than at the query processing level.

The choice of algorithms for parallelizing query evaluation depends on the machine architecture. Rather than present algorithms for each architecture separately, we use a shared-nothing architecture in our description. Thus, we explicitly describe when data have to be transferred from one node to another.

We can simulate this model easily by using the other architectures, since transfer of data can be done via shared memory in a shared-memory architecture, and via shared disks in a shared-disk architecture. Hence, algorithms for shared-nothing architectures can be used on the other architectures too. In Section 22.6, we discuss how some of the algorithms can be further optimized for shared-memory systems.

Current-generation parallel systems are typically based on a hybrid architecture, where each computer has multiple cores with a shared memory, and there are multiple computers organized in a shared-nothing fashion. For the purpose of our discussion, with such an architecture, each core can be considered a node in a shared-nothing system. Optimizations to exploit the fact that some of the cores share memory with other cores can be performed as discussed in Section 22.6.

22.2 Parallel Sort

Suppose that we wish to sort a relation r that resides on n nodes $N_1, N_2, \dots, N_n$. If the relation has been range-partitioned on the attributes on which it is to be sorted, we can sort each partition separately and concatenate the results to get the full sorted relation. Since the tuples are partitioned on n nodes, the time required for reading the entire relation is reduced by a factor of n by the parallel access.

If relation r has been partitioned in any other way, we can sort it in one of two ways:

1. We can range-partition r on the sort attributes, and then sort each partition separately.
2. We can use a parallel version of the external sort-merge algorithm.

22.2.1 Range-Partitioning Sort

Range-partitioning sort, shown pictorially in Figure 22.1a, works in two steps: first range-partitioning the relation, then sorting each partition separately. When we sort by range-partitioning the relation, it is not necessary to range-partition the relation on the same set of nodes as those on which that relation is stored. Suppose that we choose nodes $N_1, N_2, \dots, N_m$ to sort the relation. There are two steps involved in this operation:

1. Redistribute the tuples in the relation, using a range-partition strategy, so that all tuples that lie within the i th range are sent to node n_i , which stores the relation temporarily on its local disk.

To implement range partitioning, in parallel every node reads the tuples from its disk and sends each tuple to its destination node based on the partition function. Each node $N_1, N_2, \dots, N_m$ also receives tuples belonging to its partition and stores them locally. This step requires disk I/O and network communication.

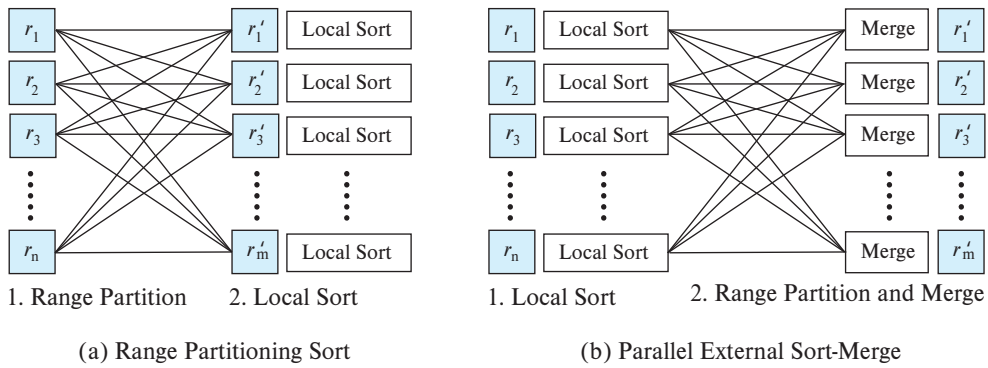


Figure 22.1 Parallel sorting algorithms.

- Each of the nodes sorts its partition of the relation locally, without interaction with the other nodes. Each node executes the same operation—namely, sorting—on a different data set. (Execution of the same operation in parallel on different sets of data are called **data parallelism**.)

The final merge operation is trivial, because the range partitioning in the first phase ensures that, for $1 \leq i < j \leq m$, the key values in node N_i are all less than the key values in N_j .

We must do range partitioning with a balanced range-partition vector so that each partition will have approximately the same number of tuples. We saw how to create such partition vectors in Section 21.3.1. Virtual node partitioning, as discussed in Section 21.3.2, can also be used to reduce skew. Recall that there are several times as many virtual nodes as real nodes, and virtual node partitioning creates a partition for each virtual node. Virtual nodes are then mapped to real nodes; doing so in a round-robin fashion tends to spread virtual nodes across real nodes in a way that reduces the degree of skew at real nodes.

22.2.2 Parallel External Sort-Merge

Parallel external sort-merge, shown pictorially in Figure 22.1b, is an alternative to range partitioning sort. Suppose that a relation has already been partitioned among nodes $N_1, N_2, \dots, N_n$ (it does not matter how the relation has been partitioned). Parallel external sort-merge then works this way:

- Each node N_i sorts the data available at N_i .
- The system then merges the sorted runs on each node to get the final sorted output.

The merging of the sorted runs in step 2 can be parallelized by this sequence of actions:

1. The system range-partitions the sorted partitions at each node N_i (all by the same partition vector) across the nodes $N_1, N_2, \dots, N_m$. It sends the tuples in sorted order, so each node receives the tuples as sorted streams.
2. Each node N_i performs a merge on the streams of tuples as they are received to get a single sorted run.
3. The system concatenates the sorted runs on nodes $N_1, N_2, \dots, N_m$ to get the final result.

As described, this sequence of actions results in an interesting form of **execution skew**, since at first every node sends all tuples of partition 1 to N_1 , then every node sends all tuples of partition 2 to N_2 , and so on. Thus, while sending happens in parallel, receiving tuples becomes sequential: First only N_1 receives tuples, then only N_2 receives tuples, and so on. To avoid this problem, the sorted sequence of tuples $S_{i,j}$ from any node i destined to any other node j is broken up into multiple blocks. Each node N_i sends the first block of tuples from $S_{i,j}$ node N_j , for each j ; it then sends the second block of tuples to each node N_j , and so on, until all blocks have been sent. As a result, all nodes receive data in parallel. (Note that tuples are sent in blocks, rather than individually, to reduce network overheads.)

22.3 Parallel Join

Parallel join algorithms attempt to divide the tuples of the input relations over several nodes. Each node then computes part of the join locally. Then, the system collects the results from each node to produce the final result. How exactly to divide the tuples depends on the join algorithm, as we see next.

22.3.1 Partitioned Join

For certain kinds of joins, it is possible to *partition* the two input relations across the nodes and to compute the join locally at each node. The partitioned join technique can be used for inner joins, where the join condition is an equi-join (e.g., $r \bowtie_{r.A=s.B} s$); the relations r and s are partitioned by the same partitioning function on their join attributes. The idea of partitioning is exactly the same as that behind the partitioning step of hash join. Partitioned join can also be used for outer joins, as we shall see shortly.

Suppose that we are using m nodes to perform the join, and that the relations to be joined are r and s . **Partitioned join** then works this way: The system partitions the relations r and s each into m partitions, denoted $r_1, r_2, \dots, r_m$ and $s_1, s_2, \dots, s_m$. In a partitioned join, however, there are two different ways of partitioning r and s :

- Range partitioning on the join attributes.
- Hash partitioning on the join attributes.

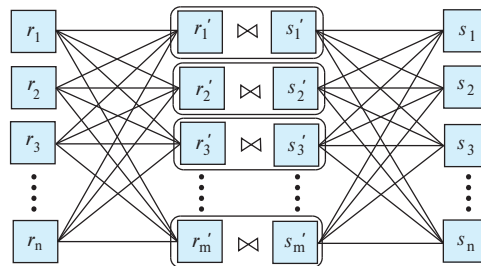
In either case, the same partitioning function must be used for both relations. For range partitioning, the same partition vector must be used for both relations. For hash partitioning, the same hash function must be used on both relations. Figure 22.2 depicts the partitioning in a partitioned parallel join.

The partitioned join algorithm first partitions one of the relations by scanning its tuples and sending them to the appropriate node based on the partition function and the join attribute values of each tuple. Specifically, each node N_i reads in the tuples of one of the relations, say r , from local disk, computes for each tuple t the partition r_j to which t belongs, and sends the tuple t to node N_j . Each node also simultaneously receives tuples that are sent to it and stores them on its local disk (this can be done by having separate threads for sending and receiving data). The process is repeated for all tuples from the other relation, s .

Once both relations are partitioned, we can use any join technique locally at each node N_i to compute the join of r_i and s_i . Thus, we can use partitioning to parallelize any join technique.

Partitioned join can be used not only for inner joins, but also for all three forms of outer join (left, right and full outer join). Each node computes the corresponding outer join locally, after partitioning is done on the join attributes. Further, since natural join can be expressed as an equijoin followed by a projection, natural joins can also be computed using partitioned join.

If one or both of the relations r and s are already partitioned on the join attributes (by either hash partitioning or range partitioning), the work needed for partitioning is reduced greatly. If the relations are not partitioned or are partitioned on attributes other than the join attributes, then the tuples need to be repartitioned.



Step 1: Partition r Step 2: Partition s
 Step 3: Each node N_i computes $r_i' \bowtie s_i'$

Figure 22.2 Partitioned parallel join.

We now consider issues specific to the join technique used locally at each node N_i . The local join operation can be optimized by performing some initial steps on tuples as they arrive at a node, instead of first storing the tuples to disk and then reading them back to perform these initial steps. These optimizations, which we describe below, are also used in nonparallel query processing, when results of an earlier operation are pipelined into a subsequent operation; thus, they are not specific to parallel query processing.

- If we use hash join locally, the resultant parallel join technique is called **partitioned parallel hash join**.

Recall that hash join first partitions both input relations into smaller pieces such that each partition of the smaller relation (the build relation) fits into memory. Thus, to implement hash join, the partitions r_i and s_i received by node N_i must be repartitioned using a hash function, say $h1()$. If the partitioning of r and s across the nodes was done by using a hash function $h0()$, the system must ensure that $h1()$ is different from $h0()$. Let the resultant partitions at node N_i be $r_{i,j}$ and $s_{i,j}$ for $j = 1 \dots n_i$, where n_i denotes the number of local partitions at node N_i .

Note that the tuples can be repartitioned based on the hash function used for the local hash join as they arrive and written out to the appropriate partitions, avoiding the need to write the tuples to disk and read them back in.

Recall also that hash join then loads each partition of the build relation into memory, builds an in-memory index on the join attributes, and finally probes the in-memory index using each tuple of the other relation, called the probe relation. Assume that relation s is chosen as the build relation. Then each partition $s_{i,j}$ is loaded in memory, with an index built on the join attributes, and the index is probed with each tuple of $r_{i,j}$.

Hybrid hash join (described in Section 15.5.5.5) can be used in case the partitions of one of the relations are small enough that a significant part of the partition fits in memory at each node. In this case, the smaller relation, say s , which is used as the build relation, should be partitioned first, followed by the larger relation, say r , which is used as the probe relation. Recall that with hybrid hash join, the tuples in the partition s_0 of the build relation s are retained in memory, and an in-memory index is built on these tuples. When the probe relation tuples arrive at the node, they are also repartitioned; tuples in the r_0 partition are used directly to probe the index on the s_0 tuples, instead of being written out to disk and read back in.

- If we use merge join locally, the resultant technique is called **partitioned parallel merge join**. Each of the partitions s_i and r_i must be sorted, and merged locally, at node N_i .

The first step of sorting, namely, run generation, can directly consume incoming tuples to generate runs, avoiding a write to disk before run generation.

- If we use nested-loops or indexed nested-loops join locally, the resultant technique is called **partitioned parallel nested-loop join** or **partitioned parallel indexed nested-**

loops join. Each node N_i performs a nested-loops (or indexed nested-loops) join on s_i and r_i .

22.3.2 Fragment-and-Replicate Join

Partitioning is not applicable to all types of joins. For instance, if the join condition is an inequality, such as $r \bowtie_{r.a < s.b} s$, it is possible that all tuples in r join with some tuple in s (and vice versa). Thus, there may be no nontrivial way of partitioning r and s so that tuples in partition r_i join with only tuples in partition s_i .

We can parallelize such joins by using a technique called *fragment-and-replicate*. We first consider a special case of fragment-and-replicate—**asymmetric fragment-and-replicate join**—which works as follows:

1. The system partitions one of the relations—say, r . Any partitioning technique can be used on r , including round-robin partitioning.
2. The system replicates the other relation, s , across all the nodes.
3. Node N_i then locally computes the join of r_i with all of s , using any join technique.

The asymmetric fragment-and-replicate scheme appears in Figure 22.3a. If r is already stored by partitioning, there is no need to partition it further in step 1. All that is required is to replicate s across all nodes.

The asymmetric fragment-and-replicate join technique is also referred to as **broadcast join**. It is a very useful technique, even for equi-joins, if one of the relations, say s , is small, and the other relation, say r , is large, since replicating the small relation s across all nodes may be cheaper than repartitioning the large relation r .

The general case of **fragment-and-replicate join** (also called the **symmetric fragment-and-replicate join**) appears in Figure 22.3b; it works this way: The system partitions relation r into n partitions, $r_1, r_2, \dots, r_n$, and partitions s into m partitions, $s_1, s_2, \dots, s_m$. As before, any partitioning technique may be used on r and on s . The values of m and n do not need to be equal, but they must be chosen so that there are at least $m * n$ nodes. Asymmetric fragment-and-replicate is simply a special case of general fragment-and-replicate, where $m = 1$. Fragment-and-replicate reduces the sizes of the relations at each node, compared to asymmetric fragment-and-replicate.

Let the nodes be $N_{1,1}, N_{1,2}, \dots, N_{1,m}, N_{2,1}, \dots, N_{n,m}$. Node $N_{i,j}$ computes the join of r_i with s_j . To ensure that each node $N_{i,j}$ gets all tuples of r_i and s_j , the system replicates r_i to nodes $N_{i,1}, N_{i,2}, \dots, N_{i,m}$ (which form a row in Figure 22.3b), and replicates s_j to nodes $N_{1,j}, N_{2,j}, \dots, N_{n,j}$ (which form a column in Figure 22.3b). Any join technique can be used at each node $N_{i,j}$.

Fragment-and-replicate works with any join condition, since every tuple in r can be tested with every tuple in s . Thus, it can be used where partitioning cannot be used. However, note that each tuple in r is replicated m times, and each tuple in s is replicated n times.

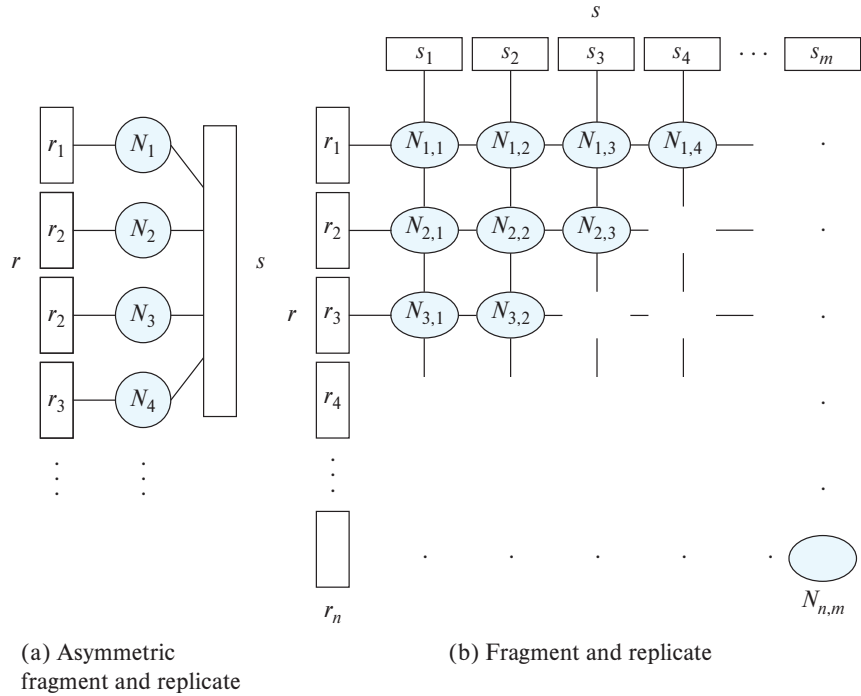


Figure 22.3 Fragment-and-replicate schemes.

Fragment-and-replicate join has a higher cost than partitioning, since it involves replication of both relations, and is therefore used only if the join does not involve equi-join conditions. Asymmetric fragment-and-replicate, on the other hand, is useful even for equi-join conditions, if one of the relations is small, as discussed earlier.

Note that asymmetric fragment-and-replicate join can be used to compute the left outer join operation $r \bowtie_{\theta} s$ if s is replicated, by simply computing the left outer join locally at each node. There is no restriction on the join condition θ .

However, $r \bowtie_{\theta} s$ cannot be computed locally if s is fragmented and r is replicated, since an r tuple may have no matching tuple in partition s_i , but may have a matching tuple in partition $s_j, j \neq i$. Thus, a decision on whether or not to output the r tuple with null values for s attributes cannot be made locally at node N_i . For the same reason, asymmetric fragment-and-replicate cannot be used to compute the full outer join operation, and symmetric fragment-and-replicate cannot be used to compute any of the outer join operations.

22.3.3 Handling Skew in Parallel Joins

Skew presents a special problem for parallel join techniques. If one of the nodes has a much heavier load than other nodes, the parallel join operation will take much longer to finish, with many idle nodes waiting for the heavily loaded node to finish its task.

When partitioning data for storage, to minimize skew in storage we use a balanced partitioning vector that ensures all nodes get the same number of tuples. For parallel joins, we need to instead balance the execution time of join operations across all nodes. Hash partitioning using any good hash function usually works quite well at balancing the load across nodes, unless some join attribute values occur very frequently. Range partitioning, on the other hand, is more vulnerable to join skew, unless the ranges are carefully chosen to balance the load.¹

Virtual-node partitioning with, say, round-robin distribution of virtual nodes to real nodes, can help in reducing skew at the level of real nodes even if there is skew at the level of virtual nodes, since the skewed virtual nodes tend to get spread over multiple real nodes.

The preceding techniques are examples of **join skew avoidance**. Virtual-node partitioning, in particular, is very effective at skew avoidance in most cases.

However, there are cases with high skew, for example where some join attribute values are very frequent in both input relations, leading to a large join result size. In such cases, there could be significant join skew, even with virtual-node partitioning.

Dynamic handling of join skew is an alternative to skew avoidance. A dynamic approach can be used to detect and handle skew in such situations. Virtual node partitioning is used, and the system then monitors the join progress at each real node. Each real node schedules one virtual node at a time. Suppose that some real node has completed join processing for all virtual nodes assigned to it, and is thus idle, while some other real node has multiple virtual nodes waiting to be processed. Then, the idle node can get a copy of the data corresponding to one of the virtual nodes at the busy node and process the join for that virtual node. This process can be repeated whenever there is an idle real node, as long as some real node has virtual nodes waiting to be processed.

This technique is an example of **work stealing**, where a processor that is idle takes work that is in the queue of another processor that is busy. Work stealing is inexpensive in a shared-memory system, since all data can be accessed quickly from the shared memory, as discussed further in Section 22.6. In a shared-nothing environment, data movement may be required to move a task from one processor to another, but it is often worth paying the overhead to reduce the completion time of a task.

22.4 Other Operations

In this section, we discuss parallel processing of other relational operations, as well as parallel processing in the MapReduce framework.

¹Cost estimation should be done using histograms on join attributes. A heuristic approximation is to estimate the join cost at each node N_i as the sum of the sizes of r_i and s_i , and choose range partitioning vectors to balance the sum of the sizes.

22.4.1 Other Relational Operations

The evaluation of other relational operations also can be parallelized:

- **Selection.** Let the selection be $\sigma_\theta(r)$. Consider first the case where θ is of the form $a_i = v$, where a_i is an attribute and v is a value. If the relation r is partitioned on a_i , the selection proceeds at a single node. If θ is of the form $l \leq a_i \leq u$ —that is, θ is a range selection—and the relation has been range-partitioned on a_i , then the selection proceeds at each node whose partition overlaps with the specified range of values. In all other cases, the selection proceeds in parallel at all the nodes.
- **Duplicate elimination.** Duplicates can be eliminated by sorting; either of the parallel sort techniques can be used, optimized to eliminate duplicates as soon as they appear during sorting. We can also parallelize duplicate elimination by partitioning the tuples (by either range or hash partitioning) and eliminating duplicates locally at each node.
- **Projection.** Projection without duplicate elimination can be performed as tuples are read in from disk in parallel. If duplicates are to be eliminated, either of the techniques just described can be used.
- **Aggregation.** Consider an aggregation operation. We can parallelize the operation by partitioning the relation on the grouping attributes, and then computing the aggregate values locally at each node. Either hash partitioning or range partitioning can be used. If the relation is already partitioned on the grouping attributes, the first step can be skipped.

We can reduce the cost of transferring tuples during partitioning by partly computing aggregate values before partitioning, at least for the commonly used aggregate functions. Consider an aggregation operation on a relation r , using the **sum** aggregate function on attribute B , with grouping on attribute A . The system can perform the **sum** aggregation at each node N_i on those r tuples stored at N_i . This computation results in tuples with partial sums at each node; the result at N_i has one tuple for each A value present in r tuples stored at N_i , with the sum of the B values of those tuples. The system then partitions the result of the local aggregation on the grouping attribute A and performs the aggregation again (on tuples with the partial sums) at each node N_i to get the final result.

As a result of this optimization, which is called **partial aggregation**, fewer tuples need to be sent to other nodes during partitioning. This idea can be extended easily to the **min** and **max** aggregate functions. Extensions to the **count** and **avg** aggregate functions are left for you to do in Exercise 22.2.

Skew handling for aggregation is easier than skew handling for joins, since the cost of aggregation is directly proportional to the input size. Usually, all that needs to be done is to use a good hash function to ensure the group-by attribute values are evenly distributed amongst the participating nodes. However, in some extreme cases, a few

values occur very frequently in the group-by attributes, and hashing can lead to uneven distribution of values. When applicable, partial aggregation is very effective in avoiding skew in such situations. However, when partial aggregation is not applicable, skew can occur with aggregation.

Dynamic detection and handling of such skew can be done in some such cases: in case a node is found to be overloaded, some of the key values that are not yet processed by the node can be reassigned to another node, to balance the load. Such reassignment is greatly simplified if virtual-node partitioning is used; in that case, if a real node is found to be overloaded, some virtual nodes assigned to the overloaded real node, but not yet processed, are identified, and reassigned to other real nodes.

More information on skew handling for join and other operators may be found in the Further Reading section at the end of the chapter.

22.4.2 Map and Reduce Operations

Recall the MapReduce paradigm, described in Section 10.3, which is designed to ease the writing of parallel data processing programs.

Recall that the `map()` function provided by the programmer is invoked on each input record and emits zero or more output data items, which are then passed on to the `reduce()` function. Each data item output by a `map()` function consists of a record (*key*, *value*); we shall call the key as the **intermediate key**. In general, a `map()` function can emit multiple such records and since there are many input records, there are potentially many output records overall.

The MapReduce system takes all the records emitted by the `map()` functions, and groups them such that all records with a particular intermediate key are gathered together. The `reduce()` function provided by the programmer is then invoked for each intermediate key and iterates over a collection of all values associated with that key.

Note that the `map` function can be thought of as a generalization of the project operation: both process a single record at a time, but for a given input record the project operation generates a single output record, whereas the `map` function can output multiple records (including, as a special case, 0 records). Unlike the project operation, the output of a `map` function is usually intended to become the input of a `reduce` function; hence, the output of a `map` function has an associated key that serves as a group by attribute. Recall that the `reduce` function takes as input a collection of values and outputs a result; with most of the `reduce` functions commonly in use, the result is an aggregate computed on the input values, and the `reduce` function is then essentially a user-defined aggregation function.

MapReduce systems are designed for parallel processing of data. A key requirement for parallel processing is the ability to parallelize file input and output across multiple machines; otherwise, the single machine storing the data will become a bottleneck. Parallelization of file input and output can be done by using a distributed file system, such as the *Hadoop File System (HDFS)*, discussed in Section 21.6, or by using a parallel/distributed storage system, discussed in Section 21.7. Recall that in such sys-

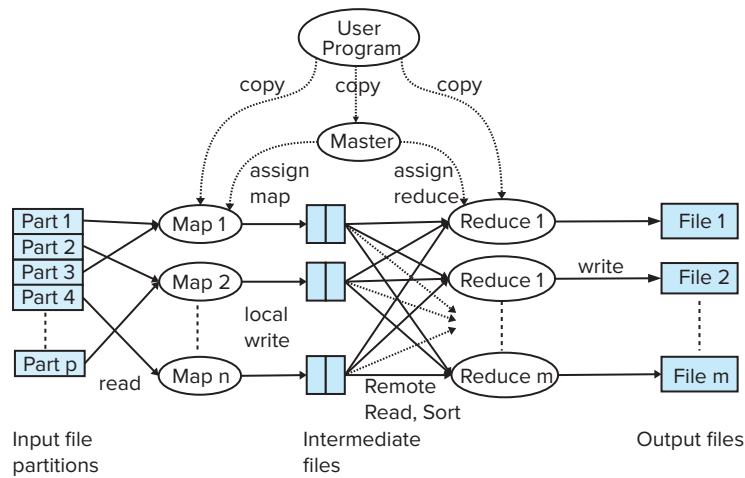


Figure 22.4 Parallel processing of MapReduce job.

tems, data are replicated (copied) across several (typically 3) machines, so that even if a few of the machines fail, the data are available from other machines that have copies of the data in the failed machine.

Conceptually, the map and reduce operations are parallelized in the same way that the relational operations project and aggregation are parallelized. Each node in the system has a number of concurrently executing **workers**, which are processes that execute map and reduce functions. The number of workers on one machine is often set to match the number of processor cores on the machine.

Parallel processing of MapReduce jobs is shown schematically in Figure 22.4. As shown in the figure, MapReduce systems split the input data into multiple pieces; the job of processing one such piece is called a **task**. Splitting can be done in units of files, and large files can be split into multiple parts. Tasks correspond to virtual nodes in our terminology, while workers correspond to real nodes. Note that with a multicore processor (as is standard today), MapReduce systems typically allocate one worker per core.

MapReduce systems also have a scheduler, which assigns tasks to workers.² Whenever a worker completes a task, it is assigned a new task, until all tasks have been assigned.

A key step between the map and reduce operations is the repartitioning of records output by the map step; these records are repartitioned based on their intermediate (reduce) key, such that all records with a particular key are assigned to the same reducer task. This could be done either by range-partitioning on the reduce key or by computing a hash function on the reduce key. In either case, the records are divided into multiple

²The scheduler is run on a dedicated node called the master node; the nodes that perform `map()` and `reduce()` tasks are called slave nodes in the Hadoop MapReduce terminology.

partitions, each of which is called a reduce task. A scheduler assigns reduce tasks to workers.

This step is identical to the repartitioning done for parallelizing the relational aggregation operation, with records partitioned into a number of virtual nodes based on their group-by key.

To process the records in a particular **reduce** task, the records are sorted (or grouped) by the reduce key, so that all records with the same reduce-key value are brought together, and then the `reduce()` is executed on each group of reduce-key values.

The reduce tasks are executed in parallel by the workers. When a worker completes a reduce task, another task is assigned to it, until all reduce tasks have been completed. A reduce task may have multiple different reduce key values, but a particular call to the `reduce()` function is for a single reduce key; thus, the `reduce()` function is called for each key in the reduce task.

Tasks correspond to virtual nodes in the virtual-node partitioning scheme. There are far more tasks than there are nodes, and tasks are divided among the nodes. As discussed in Section 22.3.3, virtual-node partitioning reduces skew. Also note that as discussed in Section 22.4.1, skew can be reduced by partial aggregation, which corresponds to *combiners* in the MapReduce framework.

Further, MapReduce implementations typically also carry out dynamic detection and handling of skew, as discussed in Section 22.4.1.

Most MapReduce implementations include techniques to ensure that processing can be continued even if some nodes fail during query execution. Details are discussed further in Section 22.5.4.

22.5 Parallel Evaluation of Query Plans

As discussed in Section 22.1, there are two types of parallelism: intraoperation and interoperation. Until now in this chapter, we have focused on intraoperation parallelism. In this section, we consider execution plans for queries containing multiple operations.

We first consider how to exploit interoperator parallelism. We then consider a model of parallel query execution which breaks parallel query processing into two types of steps: partitioning of data using the *exchange operator*, and execution of operations on purely local data, without any data exchange. This model is surprisingly powerful and is widely used in parallel database implementations.

22.5.1 Interoperation Parallelism

There are two forms of interoperation parallelism: pipelined parallelism and independent parallelism. We first describe these forms of parallelism, assuming each operator runs on a single node without intraoperation parallelism.

We then describe a model for parallel execution based on the exchange operator, in Section 22.5.2. Finally, in Section 22.5.3, we describe how a complete plan can be executed, combining all the forms of parallelism.

22.5.1.1 Pipelined Parallelism

Recall from Section 15.7.2 that in pipelining, the output tuples of one operation, A , are consumed by a second operation, B , even before the first operation has produced the entire set of tuples in its output. The major advantage of pipelined execution in a sequential evaluation is that we can carry out a sequence of such operations without writing any of the intermediate results to disk.

Parallel systems use pipelining primarily for the same reason that sequential systems do. However, pipelines are a source of parallelism as well, since it is possible to run operations A and B simultaneously on different nodes (or different cores of a single node), so that B consumes tuples in parallel with A producing them. This form of parallelism is called **pipelined parallelism**.

Pipelined parallelism is useful with a small number of nodes, but it does not scale up well. First, pipeline chains generally do not attain sufficient length to provide a high degree of parallelism. Second, it is not possible to pipeline relational operators that do not produce output until all inputs have been accessed, such as the set-difference operation. Third, only marginal speedup is obtained for the frequent cases in which one operator's execution cost is much higher than are those of the others.

All things considered, when the degree of parallelism is high, pipelining is a less important source of parallelism than partitioning. The real reason for using pipelining with parallel query processing is the same reason that pipelining is used with sequential query processing: namely, that pipelined executions can avoid writing intermediate results to disk.

Pipelining in centralized databases was discussed in Section 15.7.2; as mentioned there, pipelining can be done using a demand-driven, or pull, model of computation, or using a producer-driven, or push, model of computation. The pull model is widely used in centralized database systems.

However, the push model is greatly preferred in parallel database systems, since, unlike the pull model, the push model allows both the producer and consumer to execute in parallel.

Unlike the pull model, the push model requires a buffer that can hold multiple tuples, between the producer and consumer; without such a buffer, the producer would stall as soon as it generates one tuple. Figure 22.5 shows a producer and consumer with a buffer in-between. If the producer and consumer are on the same node, as shown in Figure 22.5a, the buffer can be in shared memory. However, if the producer and consumer are in different nodes, as shown in Figure 22.5b, there will be two buffers: one at the producer node to collect tuples as they are produced, and another at the consumer node to collect them as they are sent across the network.

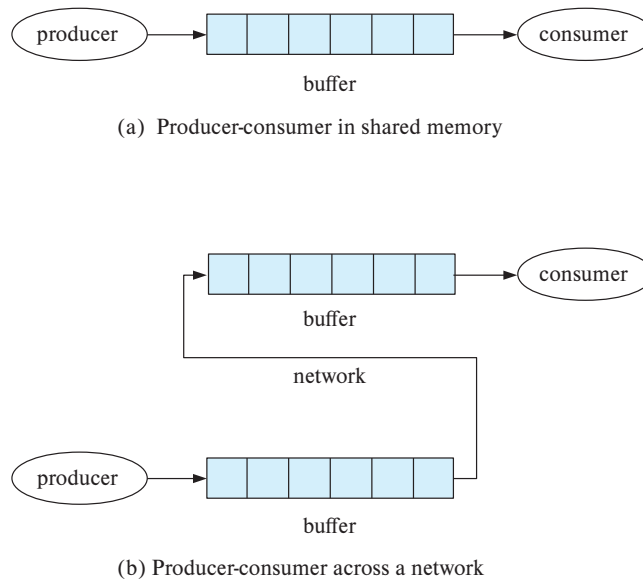


Figure 22.5 Producer and consumer with buffer.

When sending tuples across a network, it makes sense to collect multiple tuples and send them as a single *batch*, rather than send tuples one at a time, since there is usually a very significant overhead per message. Batching greatly reduces this overhead.

If the producer and consumer are on the same node and can communicate via a shared memory buffer, mutual exclusion needs to be ensured when inserting tuples into, or fetching tuples from, the buffer. Mutual exclusion protocols have some overhead, which can be reduced by inserting/retrieving a batch of tuples at a time, instead of one tuple at a time.

Note that with the pull model, either the producer or the consumer, but not both, can be executing at a given time; while this avoids the contention on the shared buffer that arises with the use of the push model, it also prevents the producer and consumer from running concurrently.

22.5.1.2 Independent Parallelism

Operations in a query expression that do not depend on one another can be executed in parallel. This form of parallelism is called **independent parallelism**.

Consider the join $r_1 \bowtie r_2 \bowtie r_3 \bowtie r_4$. One possible plan is to compute intermediate result $t_1 \leftarrow r_1 \bowtie r_2$ in parallel with intermediate result $t_2 \leftarrow r_3 \bowtie r_4$. Neither of these computations depends on each other, and hence they can be parallelized by independent parallelism. In other words, the execution of these two joins can be scheduled in parallel.

When these two computations complete, we can compute:

$$t_1 \bowtie t_2$$

Note that computation of the above join depends on the results of the first two joins, hence it cannot be done using independent parallelism.

Like pipelined parallelism, independent parallelism does not provide a high degree of parallelism and is less useful in a highly parallel system, although it is useful with a lower degree of parallelism.

22.5.2 The Exchange Operator Model

The Volcano parallel database popularized a model of parallelization called the *exchange-operator* model. The exchange operation repartitions data in a specified way; data interchange between nodes is done only by the exchange operator. All other operations work on local data, just as they would in a centralized database system; the data may be available locally either because it is already present, or because of the execution of a preceding exchange operator.

The **exchange operator** has two components: a scheme for *partitioning* outgoing data, applied at each source node, and a scheme for *merging* incoming data, applied at each destination node. The operator is shown pictorially in Figure 22.6, with the partitioning scheme denoted as “Partition,” and the merging scheme denoted as “Merge.”

The exchange operator can *partition* data in one of several ways:

1. By hash partitioning on a specified set of attributes.
2. By range partitioning on a specified set of attributes.
3. By replicating the input data at all nodes, referred to as **broadcasting**.
4. By sending all data to a single node.

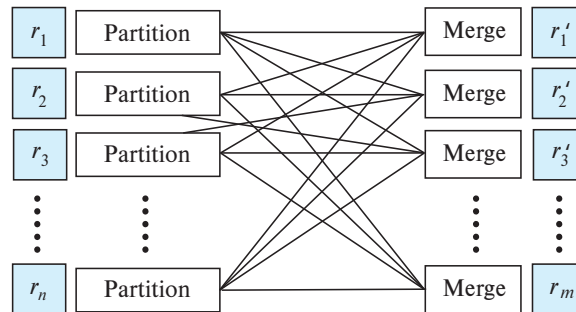


Figure 22.6 The exchange operator used for repartitioning.

Broadcasting data to all nodes is required for operations such as the asymmetric fragment-and-replicate join. Sending all data to a single node is usually done as a final step of parallel query processing, to get partitioned results together at a single site.

Note also that the input to the exchange operator can be at a single site (referred to as **unpartitioned**), or already partitioned across multiple sites. Repartitioning of already partitioned data results in each destination node receiving data from multiple source nodes, as shown in Figure 22.6.

Each destination node *merges* the data items received from the source nodes. This merge step can store data in the order received (which may be nondeterministic, since it depends on the speeds of the machines and unpredictable network delays); such merging is called **random merge**.

On the other hand, if the input data from each source is sorted, the merge step can exploit the sort order by performing an **ordered merge**. Suppose, for example, nodes $N_1, \dots, N_m$ first sort a relation locally, and then repartition the sorted relation using range partitioning. Each node performs an ordered merge operation on the tuples that it receives, to generate a sorted output locally.

Thus, the exchange operator performs the partitioning of data at the source nodes, as well the merging of data at the destination nodes.

All the parallel operator implementations we have seen so far can be modeled as a sequence of exchange operations, and local operators, at each node, that are completely unaware of parallelism.

- Range partitioning sort: can be implemented by an exchange operation that performs range partitioning, with random merge at the destination nodes, followed by a local sort operation at each destination node.
- Parallel external sort-merge: can be implemented by local sorting at the source nodes, followed by an exchange operation that performs range partitioning, along with ordered merging.
- Partitioned join: can be implemented by an exchange operation that performs the desired partitioning, followed by local join at each node.
- Asymmetric fragment-and-replicate join: can be implemented by an exchange operation that performs broadcast “partitioning” of the smaller relation, followed by a local join at each node.
- Symmetric fragment-and-replicate join: can be implemented by an exchange operation that partitions, and partially broadcasts each partition, followed by a local join at each node.
- Aggregation: can be implemented by an exchange operation that performs hash-partitioning on the grouping attributes, followed by a local aggregation operation at each node. The partial-aggregation optimization simply requires an extra local aggregation operation at each node, before the exchange operation.

Other relational operations can be implemented similarly, by a sequence of local operations running in parallel at each node, interspersed with exchange operations.

As noted earlier, parallel execution where data are partitioned, and operations are executed locally at each node, is referred to as *data parallelism*. The use of the exchange operator model to implement data parallel execution has the major benefit of allowing existing database query engines to be used at each of the local nodes, without any significant code changes. As a result, the exchange-operator model of parallel execution is widely used in parallel database systems.

There are, however, some operator implementations that can benefit from being aware of the parallel nature of the system they are running on. For example, an indexed nested-loops join where the inner relation is indexed on a parallel data-store would require remote access for each index lookup; the index lookup operation is thus aware of the parallel nature of the underlying system. Similarly, in a shared-memory system it may make sense to have a hash table or index in shared-memory, which is accessed by multiple processors (this approach is discussed briefly in Section 22.6); the operations running on each processor are then aware of the parallel nature of the system.

As we discussed in Section 22.5.1.1, while the demand-driven (or pull) iterator model for pipelined execution of operators is widely used in centralized database engines, the push model is preferred for parallel execution of operators in a pipeline.

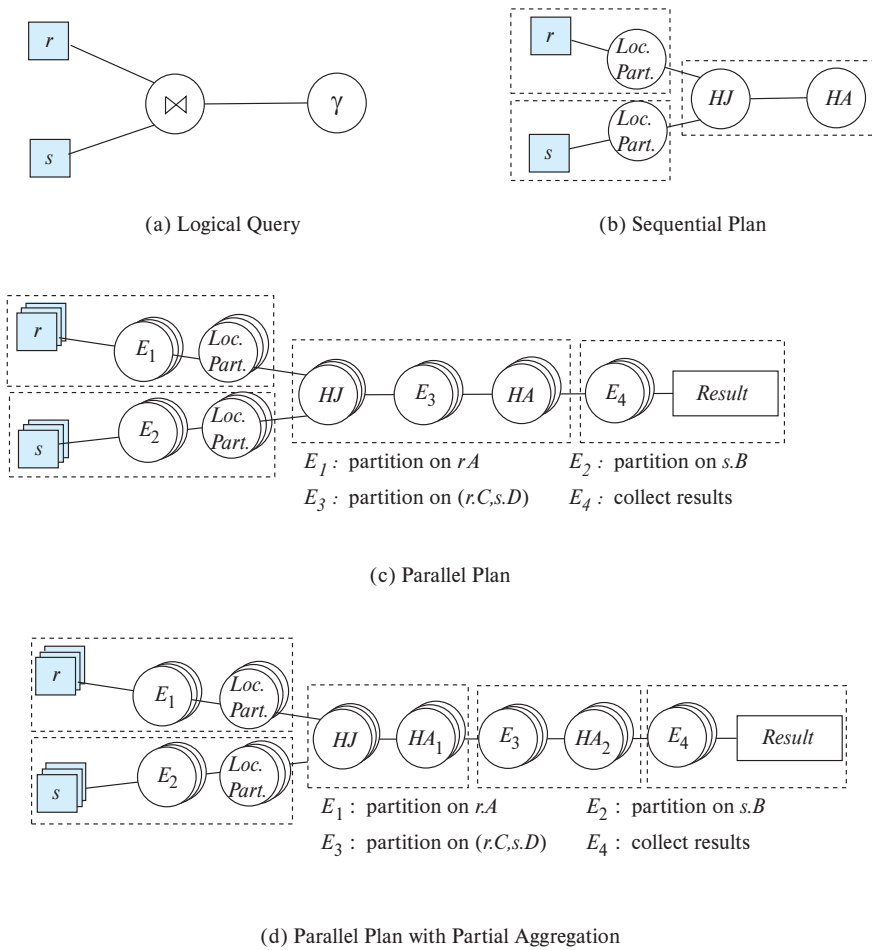
The exchange operator can be used to implement the push model between nodes in a parallel system, while allowing existing implementations of local operators to run using the pull model. To do so, at each source node of an exchange operator, the operator can pull multiple tuples from its input and create a batch of tuples destined for each destination node. The input may be computed by a local operation, whose implementation can use the demand-driven iterator model.

The exchange operator then sends batches of tuples to the destination nodes, where they are merged and kept in a buffer. The local operations can then consume the tuples in a demand-driven manner.

22.5.3 Putting It All Together

Figure 22.7 shows a query, along with a sequential and two alternative parallel query execution plans. The query, shown in Figure 22.7a, computes a join of two relations, r and s , and then computes an aggregate on the join result. Assume for concreteness that the query is $_{r.C,s.D}\gamma_{\text{sum}(s.E)}(r \bowtie_{r.A=s.B} s)$.

The sequential plan, shown in Figure 22.7b, uses a hash join (denoted as “HJ” in the figure), which executes in three separate stages. The first stage partitions the first input (r) locally on $r.A$; the second stage partitions the second input (s) locally on $s.B$; and the third stage computes the join of each of the corresponding partitions of r and s . The aggregate is computed using in-memory hash-aggregation, denoted by the operator HA; we assume that the number of groups is small enough that the hash table fits in memory.

**Figure 22.7** Parallel query execution plans.

The dashed boxes in the figure show which steps run in a pipelined fashion. In the sequential plan, the read of the relation r is pipelined to the first partitioning stage of the sequential hash join; similarly, the read of relations s is pipelined to the second partitioning stage of the hash join. The third stage of the hash join pipelines its output tuples to the hash aggregation operator.

The parallel query evaluation plan, shown in Figure 22.7c, starts with r and s already partitioned, but not on the required join attributes.³ The plan, therefore, uses the exchange operation E_1 to repartition r using attribute $r.A$; similarly, exchange operator E_2 repartitions s using $s.B$. Each node then uses hash join locally to compute the join

³Note the multiple boxes indicating a relation is stored in multiple nodes; similarly, multiple circles indicate that an operation is executed in parallel on multiple nodes.

of its partition of r and s . Note that these partitions are not assumed to fit in memory, so they must be further partitioned by the first two stages of hash join; this local partitioning step is denoted as “Loc. Part.” in the figure. The dashed boxes indicate that the output of the exchange operator can be pipelined to the local partitioning step. As in the sequential plan, there are two pipelined stages, one each for r and s . Note that exchange of tuples across nodes is done only by the exchange operator, and all other edges denote tuple flows within each node.

Subsequently, the hash join algorithm is executed in parallel at all participating nodes, and its output pipelined to the exchange operator E_3 . This exchange operator repartitions its input on the pair of attributes $(r.C, s.D)$, which are the grouping attributes of the subsequent aggregation. At the receiving end of the exchange operator, tuples are pipelined to the hash aggregation operator. Note that the above steps all run together as a single pipelined stage, even though there is an exchange operator as part of the stage. Note that the local operators computing hash join and hash aggregate need not be aware of the parallel execution.

The results of the aggregates are then collected together at a central location by the final exchange operator E_4 , to create the final result relation.

Figure 22.7d shows an alternative plan that performs partial aggregation on the results of the hash join, before partitioning the results. The partial aggregation is computed locally at each node by the operator HA_1 . Since no tuple is output by the partial aggregation operator until all its input is consumed, the pipelined stage contains only the local hash join and hash aggregation operators. The subsequent exchange operator E_3 which partitions its input on $(r.C, s.D)$ is part of a subsequent pipelined stage along with the hash aggregation operation HA_2 which computes the final aggregate values. As before, the exchange operator E_4 collects the results at a centralized location.

The above example shows how pipelined execution can be performed across nodes, as well as within nodes, and further how it can be done along with intra-operator parallel execution. The example also shows that some pipelined stages depend on the output of earlier pipelined stages; therefore their execution can start only after the previous step finishes. On the other hand, the initial exchange and partitioning of r and s occur in pipelined stages that are independent of each other; such independent stages can be scheduled concurrently, that is, at the same time, if desired.

To execute a parallel plan such as the one in our example, the different pipelined stages have to be scheduled for execution, in an order that ensures inter-stage dependencies are met. When executing a particular stage, the system must decide how many nodes an operation should be executed on. These decisions are usually made as part of the scheduling phase, before query execution starts.

22.5.4 Fault Tolerance in Query Plans

Parallel processing of queries across a moderate number of nodes, for example, hundreds of nodes, can be done without worrying about fault tolerance. If a fault occurs, the query is rerun, after removing any failed nodes from the system (replication of data

at the storage layer ensures that data continues to be available even in the event of a failure). However, this simple solution does not work well when operating at the scale of thousands or tens of thousands of nodes: if a query runs for several hours, there is a significant chance that there will be a failure while the query is being executed. If the query is restarted, there is a significant chance of another failure while it is executing, which is obviously an undesirable situation.

To deal with this problem, the query processing system should ideally just be able to redo the actions of a failed node, without redoing the rest of the computation.

Implementations of MapReduce that are designed to work at a massively parallel scale can be made fault tolerant as follows:

1. Each map operation executed at each node writes its output to local files.
2. The next operation, which is a reduce operation, executes at each node; the operation execution at a node reads data from the files stored at multiple nodes, collects the data, and starts processing the data only after it has got all its required data.
3. The reduce operation writes its output to a distributed file system (or distributed storage system) that replicates data, so that the data would be available even in the event of a failure.

Let us now examine the reason why things are done as above. First, if a particular map node fails, the work done at that node can be redone at a backup node; the work done at other map nodes is not affected. Work is not carried out by reduce nodes until all the required data has been fetched; the failure of a map node just means the reduce nodes fetch data from the backup map nodes. There is certainly a delay while the backup node does its work, but there is no need to repeat the entire computation.

Further, once a reduce node has finished its work, its output goes to replicated storage to ensure it is not lost even if a data storage node fails. This means that if a reduce node fails before it completes its work, it will have to be reexecuted at a backup node; other reduce nodes are not affected. Once a reduce node has finished its work, there is no need to reexecute it.

Note that it is possible to store the output of a map node in a replicated storage system. However, this increases the execution cost significantly, and hence map output is stored in local storage, even at the risk of having to reexecute the work done by a map node in case it fails before all the reduce nodes have fetched the data that they require from that map node.

It is also worth noting that sometimes nodes do not completely fail, but run very slowly; such nodes are called *straggler* nodes. Even a single straggler node can delay all the nodes in the next step (if there is a following step), or delay task completion (if it is in the last step). Straggler nodes can be dealt with by treating them similar to failed nodes, and reexecuting their tasks on other nodes (the original task on the straggler node can also be allowed to continue, in case it finishes first). Such reexecution to deal with

stragglers has been found to significantly improve time to completion of MapReduce tasks.

While the above scheme for fault tolerance is quite effective, there is an overhead that must be noted: a `reduce` stage cannot perform any work until the previous map stage has finished;⁴ and if multiple map and reduce steps are executed, the next map stage cannot perform any work until the preceding reduce stage has finished. In particular, this means that pipelining of data between stages cannot be supported; data are always materialized before it is sent to the next stage. Materialization carries a significant overhead, which can slow down computation.

Apache Spark uses an abstraction called *Resilient Distributed Datasets* (RDDs) to implement fault tolerance. As we have seen in Section 10.4.2, RDDs can be viewed as collections, and Spark supports algebraic operations that take RDDs as input, and generate RDDs as output. Spark keeps track of the operations used to create an RDD. In case of failures that result in loss of an RDD, the operations used to create the RDD can be reexecuted to regenerate the RDD. However, this may be time-consuming, so Spark also supports replication to reduce the chance of data loss, as well as storing of local copies of data when a shuffle (exchange) step is executed, to allow reexecution to be restricted to computation that was performed on failed nodes.

There has been a good deal of research on how to allow pipelining of data, while not requiring query execution to restart from the beginning in case of a single failure. Such schemes typically require nodes to track what data they have received from each source node. In the event of a source node failure, the work of the source node is redone on a backup node, which can result in some tuples that were received earlier being received again. Tracking the data received earlier is important to ensure duplicate tuples are detected and eliminated by the receiving node. The above ideas can also be used to implement fault tolerance for other algebraic operations, such as joins. In particular, if we use the exchange operator with data parallelism, fault tolerance can be implemented as an extension of the exchange operator.

References to more information on fault tolerant pipelining based on extensions of the exchange operator, as well as on fault tolerance schemes used in MapReduce and in Apache Spark, may be found in the Further Reading section at the end of the chapter.

22.6 Query Processing on Shared-Memory Architectures

Parallel algorithms designed for shared-nothing architectures can be used in shared-memory architectures. Each processor can be treated as having its own partition of memory, and we can ignore the fact that the processors have a common shared-

⁴Once a map node finishes its tasks, redistribution of results from that node to the reduce nodes can start even if other map nodes are still active; but the actual computation at the reduce node cannot start until all map tasks have completed and all map results redistributed.

memory. However, execution can be optimized significantly by exploiting the fast access to shared-memory from any of the processors.

Before we study optimizations that exploit shared-memory, we note that while many large-scale systems can execute on a single shared-memory system, the largest-scale systems today are typically implemented using a hierarchical architecture, with a shared-nothing architecture at the outer level, but with each node having a shared-memory architecture locally, as discussed in Section 20.4.8. The techniques we have studied so far for storing, indexing, and querying data in shared-nothing architectures are used to divide up storage, indexing, and query processing tasks among the different nodes in the system. Each node is a shared-memory parallel system, which uses parallel query processing techniques to execute the query processing tasks assigned to it. The optimizations we describe in this section can thus be used locally, at each node.

Parallel processing in a shared memory system is typically done by using threads, rather than separate processes. A **thread** is an execution stream that shares its entire memory⁵ with other threads. Multiple threads can be started up, and the operating system schedules threads on available processors.

We list below some optimizations that can be applied when parallel algorithms that we saw earlier are executed in a shared memory system.

1. If we use asymmetric fragment-and-replicate join, the smaller relation need not be replicated to each processor. Instead, only one copy needs to be stored in shared memory, which can be accessed by all the processors. This optimization is particularly useful if there are a large number of processors in the shared-memory system.
2. Skew is a significant problem in parallel systems, and it becomes worse as the number of processors grows. Handing off work from an overloaded node to an underloaded node is expensive in a shared-nothing system since it involves network traffic. In contrast, in a shared memory system, data assigned to a processor can be easily accessed from another processor.

To address skew in a shared-memory system, a good option is to use virtual-node partitioning, which allows work to be redistributed in order to balance load. Such redistribution could be done when a processor is found to be overloaded. Alternatively, whenever a processor finds that it has finished processing all the virtual nodes assigned to it, it can find other processors that still have virtual nodes left to be processed, and take over some of those tasks; as mentioned in Section 22.3.3, this approach is called *work stealing*. Note that such an approach to avoiding skew would be much more expensive in a shared-nothing environment since a significant amount of data movement would be involved, unlike in the shared-memory case.

⁵Technically, in operating-system terminology, its address space.

3. Hash join can be executed in two distinct ways.

- a. The first option is to partition both relations to each processor and then compute the joins of the partitions, in a manner similar to shared-nothing hash join. Each partition must be small enough that the hash index on a build-relation partition fits in the part of shared memory allocated to each processor.
- b. The second option is to partition the relations into fewer pieces, such that the hash index on a build-relation partition fits into common shared memory, rather than a fraction of the shared memory. The construction of the in-memory index, as well as probing of the index, must now be done in parallel by all the processors.

Parallelizing the probe phase is relatively easy, since each processor can work on some partition of the probe relation. In fact it makes sense to use the virtual node approach and partition the probe relation into many small pieces (sometimes called “morsels”), and have processors process a morsel at a time. When a processor is done with a morsel, it finds an unprocessed morsel and works on it, until there are no morsels left to be processed.

Parallelizing the construction of the shared hash index is more complicated, since multiple processors may attempt to update the same part of the hash index. Using locks is an option, but there are overheads due to locking. Techniques based on lock-free data structures can be used to construct the hash index in parallel.

References to more details on how to parallelize join implementations in shared memory may be found in the Further Reading section at the end of the chapter.

Algorithms designed for shared-memory systems must take into account the fact that in today’s processors, memory is divided into multiple memory banks, with each bank directly linked to some processor. The cost of accessing memory from a given processor is less if the memory is directly linked to the processor, and is more if it is linked to a different processor. Such memory systems are said to have a *Non-Uniform Memory Access* or *NUMA* architecture.

To get the best performance, algorithms must be *NUMA-aware*; that is, they must be designed to ensure that data accessed by a thread running on a particular processor is, as far as possible, stored in memory local to that processor. Operating systems support this task in two ways:

1. Each thread is scheduled, as far as possible, on the same processor core, every time it is executed.
2. When a thread requests memory from the operating system memory manager, the operating system allocates memory that is local to that processor core.

Note that the techniques for making the best use of shared memory are complementary to techniques that make the best use of processor caches, including cache-

conscious index structures (which we saw in Section 14.4.7) and cache-conscious algorithms for processing relational operators.

But in addition, since each processor core has its own cache, it is possible for a cache to have an old value that was subsequently updated on another processor core. Thus, query processing algorithms that update shared data structures should be careful to ensure that there are no bugs due to the use of outdated values, and due to race conditions on updating the same memory location from two processor cores. Locking and fence instructions to ensure cache consistency (Section 20.4.5) are used in combination to implement updates to shared data structures.

The form of parallelism we have studied so far allows each processor to execute its own code independently of other processors. However, some parallel systems support a different form of parallelism, called **Single Instruction Multiple Data (SIMD)**. With SIMD parallelism, the same instruction is executed on each of multiple data items, which are typically elements of an array. SIMD architectures became widely used in graphics processing units (GPUs), which were initially used for speeding up processing of computer graphics tasks. However, more recently, GPU chips have been used for parallelizing a variety of other tasks, one of which is parallel processing of relational operations using the SIMD support provided by GPUs. Intel's Xeon Phi coprocessor supports not only multiple cores in a single chip, but also several SIMD instructions that can operate in parallel on multiple words. There has been a good deal of research on how to process relational operations in parallel on such SIMD architectures; references to more information on this topic may be found in the bibliographic notes for this chapter, available online.

22.7 Query Optimization for Parallel Execution

Query optimizers for parallel query evaluation are more complicated than query optimizers for sequential query evaluation. First, the space of plan alternatives can be much larger than that for sequential plans. In particular, in a parallel setting, we need to take into account the different possible ways of partitioning the inputs and intermediate results, since different partitioning schemes can lead to different query execution costs, which is not an issue for a sequential plan.

Second, the cost models are more complicated, since the cost of partitioning must be taken into account, and issues such as skew and resource contention must be addressed.

22.7.1 Parallel Query Plan Space

As we have seen in Section 15.1, a sequential query plan can be expressed as an algebraic expression tree, where each node is a physical operator, such as a sort operator, a hash join operator, a merge-join operator, etc. Such a plan may be further annotated with instructions on what operations are to be pipelined and what intermediate results are to be materialized, as we saw in Section 15.7.

In addition to the above, a parallel query plan must specify

- How to parallelize each operation, including deciding what algorithm to use, and how to partition the inputs and intermediate results. Exchange operators can be used to partition inputs as well as intermediate results and final results.
- How the plan is to be **scheduled**; specifically:
 - How many nodes to use for each operation.
 - What operations to pipeline within the same node, or across different nodes.
 - What operations to execute sequentially, one after the other.
 - What operations to execute independently in parallel.

As an example of the partitioning decision, a join $r \bowtie_{r.A=s.A \wedge r.B=s.B} s$ can be parallelized by partitioning r and s on the attributes $r.A$ and $s.A$ alone, or on the attributes $r.B$ and $s.B$ alone, or on $(r.A, r.B)$ and $(s.A, s.B)$. The last option is likely to be the best if we consider only this join, since it minimizes the chances of skew which can arise if the cardinalities of $r.A$, $r.B$, $s.A$ or $s.B$ are low.

But consider now the query $_{r.A} \gamma_{\text{sum}(s.C)}(r \bowtie_{r.A=s.A \wedge r.B=s.B} s)$. If we perform the join by partitioning on $(r.A, r.B)$ (and $(s.A, s.B)$), we would then need to repartition the join result by $r.A$ to compute the aggregate. On the other hand, if we performed the join by partitioning on r and s on $r.A$ and $s.A$ respectively, both the join and the aggregate can be computed without any repartitioning, which could reduce the cost. This option is particularly likely to be a good option if $r.A$ and $s.A$ have high cardinality and few duplicates, since the chance of skew is less in this case.

Thus, the optimizer has to consider a larger space of alternatives, taking partitioning into account. References with more details about how to implement query optimization for parallel query processing systems, taking partitioning alternatives into account, may be found in the Further Reading section at the end of the chapter.

22.7.2 Cost of Parallel Query Evaluation

The cost of a sequential query plan is typically estimated based on the total resource consumption of the plan, adding up the CPU and I/O costs of the operators in a query plan. The *resource consumption* cost model can also be used in a parallel system, additionally taking into account the network cost, and adding it along with the other costs. As discussed in Section 15.2, even in a sequential system, the resource consumption cost model does not guarantee minimization of the execution time of an individual query. There are other cost models that are better at modeling the time to complete a query; however, the resource consumption cost model has the benefit of reducing the cost of query optimization, and is thus widely used. We return to the issue of other cost models later in the section.

We now study how the cost of a parallel query plan can be estimated based on the resource consumption model. If a query plan is data parallel, then each operation, other than the exchange operation, runs locally; the cost of such operations can be estimated using techniques we have seen earlier in Chapter 15, if we assume that the input relations are uniformly partitioned across n nodes, with each node receiving $1/n$ th of the overall input.

The cost of the exchange operation can be estimated based on the network topology, the amount of data transferred, and the network bandwidth; as before it is assumed that each node is equally loaded during the exchange operation. The cost of a query plan under the *resource-consumption* model can then be found by adding up the costs of the individual operations.

However, in a parallel system, two plans with the same resource consumption may have significantly different time to completion. A **response-time cost model** is a cost model that attempts to better estimate the time to completion of a query. If a particular operation is able to perform I/O operations in parallel with CPU execution, the response time would be better modeled as $\max(\text{CPU cost}, \text{I/O cost})$, rather than the resource consumption cost model of $(\text{CPU cost} + \text{I/O cost})$. Similarly, if two operations o_1 and o_2 are in a pipeline on a single node, and their CPU and I/O costs are c_1, io_1 and c_2, io_2 respectively, then their response time cost would be $\max(c_1 + c_2, io_1 + io_2)$. Similarly, if operations o_1 and o_2 are executed sequentially, then their response time cost would be $\max(c_1, io_1) + \max(c_2, io_2)$.

When executing operations in parallel across multiple nodes, the response time cost model would have to take into account:

- **Start-up costs** for initiating an operation at multiple nodes.
- **Skew** in the distribution of work among the nodes, with some nodes getting a larger number of tuples than others, and thus taking longer to complete. It is the time to completion of the slowest node that determines the time to completion of the operation.

Thus, any skew in the distribution of the work across nodes greatly affects performance.

Estimating the time to completion of the slowest node due to skew is not an easy task since it is highly data dependent. However, statistics such as number of distinct values of partitioning attributes, histograms on the distribution of values of partitioning attributes, and counts of most frequent values can be used to estimate the potential for skew. Partitioning algorithms that can detect and minimize the effect of skew, such as those discussed in Section 21.3, are important to minimize skew.

22.7.3 Choosing a Parallel Query Plan

The number of parallel evaluation plans from which to choose is much larger than the number of sequential evaluation plans. Optimization of parallel queries by considering all alternatives is therefore much more expensive than optimization of sequential

queries. Hence, we usually adopt heuristic approaches to reduce the number of parallel execution plans that we have to consider. We describe two popular approaches here.

1. A simple approach is to choose the most efficient sequential evaluation plan, and then to choose the optimal way to parallelize the operations in that evaluation plan. When choosing a sequential plan, the optimizer may use a basic sequential cost model; or it may use a simple cost model that takes some aspects of parallel execution into account but does not consider issues such as partitioning or scheduling. This option allows an existing sequential query optimizer to be used with minimal changes for the first step.

Next, the optimizer decides how to create an optimal parallel plan corresponding to the chosen sequential plan. At this point, choices of what partitioning techniques to use and how to schedule operators can be made in a cost-based manner.

The chosen sequential plan may not be optimal in the parallel context, since the exact cost formulae for parallel execution were not used when choosing it; nevertheless, the approach works reasonably well in many cases.

2. A more principled approach is to find the best plan, assuming that each operation is executed in parallel across all the nodes (operations with very small inputs may be executed on fewer nodes). Scheduling of independent operations in parallel on different nodes is not considered at this stage.

Partitioning of inputs and intermediate results is taken into consideration when estimating the cost of a query plan. Existing techniques for query optimization have been extended by considering partitioning as a physical property, in addition to physical properties such as sort orders that are already taken into account when choosing a sequential query plan. Just as sort operators are added to a query plan to get a desired sort order, exchange operators are added to get the desired partitioning property. The cost model used in practice is typically a resource consumption model, which we saw earlier. Although response-time cost models offer better estimates of query execution time, the cost of query optimization is higher when using a response-time cost model compared to the cost of optimization when using a resource-consumption cost model. References providing more information on the response-time cost model may be found in the Further Reading section at the end of the chapter.

Yet another dimension of optimization is the design of physical-storage organization to speed up queries. For example, a relation can be stored partitioned on any of several different attributes, and it may even be replicated and replicas can be stored partitioned on different attributes. For example, a relation $r(A, B, C)$ could be stored partitioned on A , and a replica could be partitioned on B . The query optimizer chooses the replica that is best suited for the query. The optimal physical organization differs

for different queries. The database administrator must choose a physical organization that appears to be good for the expected mix of database queries.

22.7.4 Colocation of Data

Even with parallel data storage and parallel processing of operations, the execution time of some queries can be too slow for the needs of some applications. In particular, queries that access small amounts of data stored at multiple nodes may run quite slowly when executed in parallel, as compared to the execution of the same query on a single node, if all the data were available at that node. There are many applications that need such queries to return answers with very low latency.

An important technique to speed up such queries is to colocate data that a query accesses in a single node. For example, suppose an application needs to access student information, along with information about courses taken by the student. Then, the *student* relation may be partitioned on the *ID*, and the *takes* relation also partitioned in exactly the same manner on *ID*. Tuples in the *course* relation, which is a small relation, may be replicated to all nodes. With such a partitioning, any query involving these three relations that retrieves data for a single *ID* can be answered locally at a single node. The query processing engine just detects which node is responsible for that *ID* and sends the query to that node, where it is executed locally.

Colocation of tuples from different relations is supported by many data storage systems. If the data storage system does not natively support colocation, an alternative is to create an object containing related tuples that share a key (e.g., *student* and *takes* records corresponding to a particular *ID*), and store it in the data storage system with the associated key (*ID*, in our example).

The colocation technique, however, does not work directly if different queries need a relation partitioned in different ways. For example, if the goal is to find all students who have taken a particular course section, the *takes* relation needs to be partitioned on the section information (*course_id*, *year*, *semester*, *sec_id*) instead of being partitioned on *ID*.

A simple technique to handle this situation is to allow multiple copies of a relation, partitioned on different attributes. These copies can be modeled as indices on the relation, partitioned on different attributes; when tuples in the relation are updated, so are the copies, to keep them consistent. In our example, one copy of *takes* can be partitioned on *ID* to be colocated with the *student* partitions, while another copy is partitioned on (*course_id*, *year*, *semester*, *sec_id*) to be colocated with the *section* relation.

Colocation helps optimize queries that compute joins between two relations; it can extend to three or more relations if either the remaining relations are replicated, or if all relations share a common set of join attributes. In the latter case, all tuples that would join together can be colocated by partitioning on the common join attributes. In either case, the join can be computed locally at a single node, if the query only wants the results for a specific value of the join attribute, as we saw earlier. However, not all join

queries are amenable to evaluation at a single node by using colocation. Materialized views, which we discuss next, offer a more general alternative.

22.7.5 Parallel Maintenance of Materialized Views

Materialized views, which we saw in Section 4.2.3 for speeding up queries in centralized databases, can also be used to speed up queries in parallel databases. Materialized views need to be maintained when the database is updated, as we saw in Section 16.5. Materialized views in a parallel database can have a very large amount of data, and must, therefore, be stored partitioned across multiple nodes.

As in the centralized case, materialized views in a parallel database speed up query answering at the cost of the overhead of view maintenance at the time of processing updates.

We consider first a very simple case of materialized views. It is often useful to store an extra copy of a relation, partitioned on different attributes, to speed up answering of some queries. Such a repartitioning can be considered a very simple case of a materialized view; view maintenance for such a view is straightforward, just requiring updates to be sent to the appropriate partition.

Indices can be considered a form of materialized views. Recall from Section 21.5 how parallel indices are maintained. Consider the case of maintenance of a parallel secondary index on an attribute B of a relation $r(A, B, C)$, with primary key A . The secondary index would be sorted on attribute B and would include unique key A ; assume it also includes attribute C . Suppose now that attribute B of a tuple $(a1, b1, c1)$ is updated from $b1$ to $b2$. This update results in two updates to the secondary index: delete the index entry with key $(b1, a1, c1)$, and add an entry $(b2, a1, c1)$. Since the secondary index is itself partitioned, these two updates need to be sent to the appropriate partition, based on the unique key attributes (B, A) .

In some cases, materialized view maintenance can be done by partitioning followed by local view maintenance. Consider a view that groups *takes* tuples by $(course_id, year, semester, sec_id)$, and then counts the number of students who have taken that course section. Such a materialized view would be stored partitioned on the grouping attributes $(course_id, year, semester, sec_id)$. It can be computed by maintaining a copy of the *takes* relation partitioned on $(course_id, year, semester, sec_id)$, and materializing the aggregates locally at each node. When there is an update, say an insert or delete to the *takes* relation, that update must be propagated to the appropriate node based on the partitioning chosen above. The materialized aggregate can be maintained locally at each node, as updates are received for the set of local tuples of the *takes* relation.

For more complex views, materialized view maintenance cannot be done by a single step of partitioning and local view maintenance. We consider a more general approach next.

First, consider an operator o , whose result is materialized, and an update (insert or delete) to one of the input relations of o that requires maintenance of the materialized result of o . Suppose the execution of operator o is parallelized using the exchange op-

erator model (Section 22.5.2) where the inputs are partitioned, and then operators are executed at each node on data made available locally by (re)partitioning. To support materialized view maintenance of the result of o , we materialize the output of o at each node and additionally materialize (store) the input partitions sent to the node when the materialized view result is initially computed.

Now, when there is an update (insert or delete) to an input to o , we send the update to the appropriate node using the same partition function used during the initial computation of o . Consider a node that has received such an update. Now, the maintenance of the locally materialized result at the node can be done using standard (nonparallel) view maintenance techniques using only locally available data.

Note that as we saw in Section 22.5.2, a variety of operations can be parallelized using the exchange operator model, and hence the preceding scheme provides a parallel view maintenance technique for all such operators.

Next, consider a query with multiple operators. Such a query can be parallelized using the exchange operator model. The exchange operators repartition data between nodes, and each node computing (possibly multiple) operations using data made available locally by the exchange operators, as we saw in Section 22.5.2.

We can materialize the inputs and results at each node. Now, when there is a change to an input at a node, we use standard view maintenance techniques locally to find the change to the materialized result, say v , at that node. If that result v is the final output of the query, we are done. Otherwise, there must be an exchange operator above it; we use the exchange operator to route the updates (inserts or delete) to v to the next operator. That operator in turn computes the change to its result, and propagates it further if required, until we get to the root of the original materialized view.

The issue of consistency of materialized views in the face of concurrent updates to the underlying relations is addressed in Section 23.6.3.

22.8 Parallel Processing of Streaming Data

We saw several applications of streaming data in Section 10.5. Many of the streaming data applications that we saw in that section, such as network monitoring or stock market applications, have very high rates of tuple arrival. Incoming tuples cannot be processed by a single computer, and parallel processing is essential for such systems. Streaming data systems apply a variety of operations on incoming data. We now see how some of these operations can be executed in parallel.

Parallelism is essential at all stages of query processing, starting with the entry of tuples from the sources. Thus, a parallel stream processing system needs to support a large number of entry points for data.

For example, a system that is monitoring queries posed on a search engine such as Google or Bing search has to keep up with a very high rate of queries. Search engines have a large number of machines across which user queries are distributed and executed. Each of these machines becomes a source for the query stream. The stream

processing system must have multiple entry points for the data, which receive data from the original sources and route them within the stream processing system.

Processing of data must be done by routing tuples from producers to consumers. We discuss routing of tuples in Section 22.8.1. Parallel processing of stream operations is discussed in Section 22.8.2, while fault tolerance is discussed in Section 22.8.3.

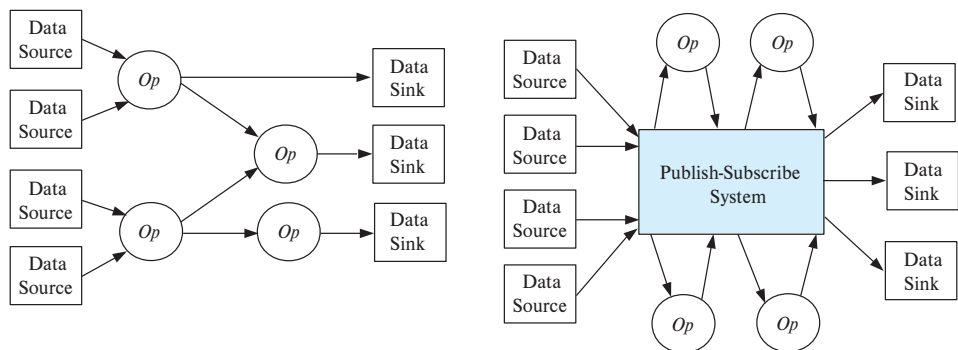
It is also worth noting that many applications that perform real-time analytics on streaming data also need to store the data and analyze it in other ways subsequently. Thus, many systems duplicate incoming data streams, sending one copy to a storage system for subsequent analysis and sending the other copy to a streaming data system; such an architecture is called the **lambda** architecture: the Greek symbol λ is used to pictorially denote that incoming data are forked into two copies, sent to two different systems.

While the lambda architecture allows streaming systems to be built quickly, it also results in duplication of effort: programmers need to write code to store and query the data in the format/language supported by a database, as well as to query the data in the language supported by a streaming data system. More recently, there have been efforts to perform stream processing as well as query processing on stored data within the same system to avoid this duplication.

22.8.1 Routing of Tuples

Since processing of data typically involves multiple operators, routing of data to operators is an important task. We first consider the logical structure of such routing, and address the physical structure, which takes parallel processing into account, later.

The *logical routing* of tuples is done by creating a directed acyclic graph (DAG) with operators as nodes. Edges between nodes define the flow of tuples. Each tuple output by an operator is sent along all the out-edges of the operator, to the consuming operators. Each operator receives tuples from all its in-edges. Figure 22.8a depicts the logical



(a) DAG representation of streaming data flow

(b) Publish-subscribe representation of streaming data flow

Figure 22.8 Routing of streams using DAG and publish-subscribe representations.

routing of stream tuples through a DAG structure. Operation nodes are denoted as “Op” nodes in the figure. The entry points to the stream processing system are the data source nodes of the DAG; these nodes consume tuples from the stream sources and inject them into the stream processing system. The exit points of the stream processing system are data sink nodes; tuples exiting the system through a data sink may be stored in a data store or file system or may be output in some other manner.

One way of implementing a stream processing system is by specifying the graph as part of the system configuration, which is read when the system starts processing tuples and is then used to route tuples. The Apache Storm stream processing system is an example of a system that uses a configuration file to define the graph, which is called a *topology* in the Storm system. (Data source nodes are called *spouts* in the Storm system, while operator nodes are called *bolts*, and edges connect these nodes.)

An alternative way of creating such a routing graph is by using **publish-subscribe** systems. A publish-subscribe system allows publication of documents or other forms of data, with an associated topic. Subscribers correspondingly subscribe to specified topics. Whenever a document is published to a particular topic, a copy of the document is sent to all subscribers who have subscribed to that topic. Publish-subscribe systems are also referred to as **pub-sub** systems for short.

When a publish-subscribe system is used for routing tuples in a stream processing system, tuples are considered documents, and each tuple is tagged with a topic. The entry points to the system conceptually “publish” tuples, each with an associated topic. Operators subscribe to one or more topics; the system routes all tuples with a specific topic to all subscribers of that topic. Operators can also publish their outputs back to the publish-subscribe system, with an associated topic.

A major benefit of the publish-subscribe approach is that operators can be added to the system, or removed from it, with relative ease. Figure 22.8b depicts the routing of tuples using a publish-subscribe representation. Each data source is assigned a unique topic name; the output of each operator is also assigned a unique topic name. Each operator subscribes to the topics of its inputs and publishes to the topics corresponding to its output. Data sources publish to their associated topic, while data sinks subscribe to the topics of the operators whose output goes to the sink.

The Apache Kafka system uses the publish-subscribe model to manage routing of tuples in streams. In the Kafka system, tuples published for a topic are retained for a specified period of time (called the retention period), even if there is currently no subscriber for the topic. Subscribers usually process tuples at the earliest possible time, but in case processing is delayed or temporarily stopped due to failures, the tuples are still available for processing until the retention time expires.

Many streaming data systems, such as Google’s Millwheel, and the Muppet stream processing system, use the term **stream** in place of the term *topic*. In such systems, streams are assigned names; operators can publish tuples to a stream, or subscribe to a stream, based on the name.

We now consider the *physical routing* of tuples. Regardless of the model used above, each logical operator must have multiple physical instances running in parallel on dif-

ferent nodes. Incoming tuples for a logical operator must be routed to the appropriate physical instance(s) of the operator. A partitioning function is used to determine which tuple goes to which instance of the operator.

In the context of a publish-subscribe system, each topic can be thought of as a separate logical operator that accepts tuples and passes them on to all subscribers of the topic. Since there may be a very large number of tuples for a given topic, they must be processed in parallel across multiple nodes in a parallel publish-subscribe system. In the Kafka system, for example, a topic is divided into multiple partitions, called a **topic-partition**; each tuple for that topic is sent to only one of the topic-partitions. Kafka allows a partition key to be attached to each tuple, and ensures that tuples with the same key are delivered to the same partition.

To allow processing by consumers, Kafka allows consumer operators register with a specified “consumer group.” The consumer group corresponds to a logical operator, while the individual consumers correspond to physical instances of the logical operator that run in parallel. Each tuple of a topic is sent to only one consumer in the consumer group. More precisely, all tuples in a particular topic-partition are sent to a single consumer in a consumer group; however, tuples from multiple partitions may be sent to the same consumer, leading to a many-to-one relationship from partitions to consumers.

Kafka is used in many streaming data processing implementations for routing tuples. Kafka Streams provides a client library supporting algebraic operations on streams, which can be used to build streaming applications on top of the Kafka publish-subscribe system.

22.8.2 Parallel Processing of Stream Operations

For standard relational operations, the techniques that we have seen for parallel evaluation of the operations can be used with streaming data.

Some of these, such as selection and projection operations, can be done in parallel on different tuples. Others, such as grouping, have to bring all tuples of a group together to one machine.⁶ When grouping is done with aggregation, optimizations such as pre-aggregation can be used to reduce the data transferred, but information about the tuples in a group must still be delivered to a single machine.

Windowing is an important operation in streaming data systems. Recall from Section 10.5.2.1 that incoming data are divided into windows, typically based on timestamps (windows can also be defined based on the number of tuples). Windowing is often combined with grouping/aggregation, with aggregates computed on groups of tuples within a window. The use of windows ensures that once the system can determine that new tuples will no longer belong to a particular window, aggregates for that window can be output. For example, suppose a window is based on time, say with each 5 minutes defining a window; once the system determines that future tuples will have a

⁶When grouping is combined with windowing (Section 10.5.2.1), a group contains all tuples in a window that have the same values for their grouping attributes.

timestamp larger than the end of a particular window, aggregates on that window can be output. Unlike grouping, windows can overlap each other.

When windowing and grouping are used together to compute aggregates, if there are overlapping windows, it is best to partition on just the grouping attributes. Otherwise, tuples which belong to multiple windows would have to be sent to multiple windows, an overhead that is avoided by partitioning on only the grouping attributes.

Many streaming systems allow users to create their own operators. It is important to be able to parallelize user-defined operators by allowing multiple instances of the operator to run concurrently. Such systems typically require each tuple to have an associated key, and all tuples with a particular key are sent to a particular instance of the operator. Tuples with different keys can be sent to different operator instances, allowing parallel processing.

Stream operations often need to store state. For example, a windowing operator may need to retain all tuples that it has seen in a particular window, as long as the window is active. Or, it may need to store aggregates computed at some resolution (say per minute) to later compute coarser resolution aggregates (say per hour). There are many other reasons for operators to store state. User-defined operators often define state internal to the operator (local variables), which needs to be stored.

Such state may be stored locally, at each node where a copy of the operator is executed. Alternatively, it may be stored centrally in a parallel data-storage system. The store-locally alternative has a lower cost but a higher risk of losing state information on failure, as compared to storing state in a parallel data-storage system. This aspect is discussed further in Section 22.8.3.

22.8.3 Fault Tolerance with Streaming Data

Fault tolerance when querying stored data can be achieved by reexecuting the query, or parts of the query, as we have seen in Section 22.5.4. However, such an approach to fault tolerance does not work well in a streaming setting for multiple reasons. First, many streaming data applications are latency sensitive, and delays in delivering results due to restarts are not desirable. Second, streaming systems provide a continuous stream of outputs. In the event of a failure, reexecuting the entire system or parts of it could potentially lead to duplicate copies of output tuples, which is not acceptable for many applications.

Thus, streaming data systems need to provide guarantees about delivery of output tuples, which can be one of: at-least once, at-most once, and exactly-once. The **at-least-once** semantics guarantees that each tuple is output at least once, but allows duplicate delivery during recovery from failures. The **at-most-once** semantics guarantees that each tuple is delivered at most once, without duplicates, but some tuples may be lost in the event of a failure. The **exactly-once** semantics guarantees that each tuple will be delivered exactly once, regardless of failures. This is the model that most applications require, although some applications may not care about duplicates and may accept at-least-once semantics.

To ensure such semantics, streaming systems must track what tuples have been processed at each operator and what tuples have been output. Duplicates can be detected by comparing against tuples output earlier and removed. (This can be done only if the system guarantees the absence of duplicates during normal processing, since otherwise the semantics of the streaming query may be affected by removal of genuine duplicates.)

One way to implement fault tolerance is to support it in the subsystem that routes tuples between operators. For example, in Kafka, tuples are published to topics, and each topic-partition can be stored in two or more nodes so that even if one of the nodes fails, the other one is available. Further, the tuples are stored on disk in each of the nodes so that they are not lost on power failure or system restart. Thus, the streaming data system can use this underlying fault tolerance and high availability mechanism to implement fault-tolerance and high availability at a higher level of the system.

In such a system, if an operator was executing on a failed node, it can be restarted on another node. The system must also (at least periodically) record up to what point each input stream had been consumed by the operator. The operator must be restarted and each input stream replayed from a point such that the operator can correctly output all tuples that it had not already output before the failure. This is relatively easy for operators without any state; operators without any state need to do extra work to restore the state that existed before failure. For example, a window operator needs to start from a point in the stream corresponding to the start of a window and replay tuples from that point.

If the window is very large, restarting from a very old point in the stream would be very inefficient. Instead, the operator may checkpoint its state periodically, along with points in the input stream up to which processing has been done. In the event of a failure, the latest checkpoint may be restored, and only input stream tuples that were processed since the last checkpoint need to be replayed.

The same approach can be for other operators that have state information; the state can be checkpointed periodically, and replay starts from the last checkpoint. The checkpointed state may be stored locally; however, this means that until the node recovers, stream processing cannot proceed. As an alternative, the state may be stored in a distributed file system or in a parallel data-storage system. Such systems replicate data to ensure high availability even in the event of failures. Thus, if a node has failed, its functions can be restarted on a different node, starting with the last checkpoint, and replaying the stream contents.

If the underlying system does not implement fault tolerance, operators can implement their own fault-tolerance mechanisms to avoid tuple loss. For example, each operator may store all tuples that it has output; a tuple can be discarded only after the operator knows that no consumer will need the tuple, even in the event of a failure.

Further, streaming systems must often guarantee low latency, even in the event of failures. To do so, some streaming systems have replicas of each operator, running concurrently. If one replica fails, the output can be fetched from the other replica. The system must make sure that duplicate tuples from the replicas are not output to

consumers. In such systems, one copy of an operator is treated as a primary copy, and the other copy as a hot-spare replica (recall that we discussed hot-spares in Section 19.7).

What we have described above is a high-level view of how streaming data systems implement fault tolerance. References to more information on how to implement fault tolerance in streaming systems may be found in the Further Reading section at the end of the chapter.

22.9 Distributed Query Processing

The need for distributed query processing originally arose when organizations needed to execute queries across multiple databases that were often geographically distributed. However, today the same need arises because organizations have data stored in multiple different databases and data storage systems, and they need to execute queries that access multiples of these databases and data storage systems.

22.9.1 Data Integration from Multiple Data Sources

Different parts of an enterprise may use different databases, either because of a legacy of how they were automated, or because of mergers of companies. Migrating an entire organization to a common system may be an expensive and time-consuming operation. An alternative is to keep data in individual databases, but to provide users with a logical view of integrated data. The local database systems may employ different logical models, different data-definition and data-manipulation languages, and may differ in their concurrency-control and transaction-management mechanisms. Some of the sources of data may not be full-fledged database systems but may instead be data storage systems, or even just files in a file system. Yet another possibility is that the data source may be on the cloud and accessed as a web service. Queries may need access to data stored across multiple databases and data sources.

Manipulation of information located in multiple databases and other data sources requires an additional software layer on top of existing database systems. This layer creates the illusion of logical database integration without requiring physical database integration and is sometimes called a **federated database system**.

Database integration can be done in several different ways:

- The **federated database** approach creates a common schema, called a **global schema**, for data from all the databases/data sources; each database has its own **local schema**. The task of creating a unified global schema from multiple local schemas is referred to as **schema integration**.

Users can issue queries against the global schema. A query on a global schema must be translated into queries on the local schemas at each of the sites where the query has to be executed. The query results have to be translated back into the global schema and combined to get the final result.

In general, updates to the common schema also need to be mapped to updates to the individual databases; systems that support a common schema and queries, but not updates, against the schema are sometimes referred to as **mediator** systems.

- The **data virtualization** approach allows applications to access data from multiple databases/data sources, but it does not try to enforce a common schema. Users have to be aware of the different schemas used in different databases, but they do not need to worry about which data are stored on which database system, or about how to combine information from multiple databases.
- The **external data** approach allows database administrators to provide schema information about data that are stored in other databases, along with other information, such as connection and authorization information needed to access the data. Data stored in external sources that can be accessed from a database are referred to as **external data**. **Foreign tables** are views defined in a database whose actual data are stored in an external data source. Such tables can be read as well as updated, depending on what operations the external data source supports. Updates on foreign tables, if supported, must be translated into updates on the external data source.

Unlike the earlier-mentioned approaches, the goal here is not to create a full-fledged distributed database, but merely to facilitate access to data from other data sources. The SQL Management of External Data (SQL MED) component of the SQL standard defines standards for accessing external data sources from a database.

If a data source is a database that supports SQL, its data can be easily accessed using ODBC or JDBC connections. Data in parallel data storage systems that do not support SQL, such as HBase, can be accessed using the API methods that they provide.

A **wrapper** provides a view of data stored at a data source, in a desired schema. For example, if the system has a global schema, and the local database schema is different from the global schema, a wrapper can provide a view of the data in the global schema. Wrappers can even be used to provide a relational view of nonrelational data sources, such as web services, flat files (e.g., web logs), and directory systems.

Wrappers can also translate queries on the global schema into queries on the local schema, and translate results back into the global schema. Wrappers may be provided by individual sites, they may be written as part of the federated database system. Many relational databases today support wrappers that provide a relational view of data stored in file systems; such wrappers are specific to the type of data stored in the files.

If the goal of data integration is solely to run decision support queries, *data warehouses*, which we saw in Section 11.2, are a widely used alternative to database integration. Data warehouses import data from multiple sources into a single (possibly parallel) system, with a centralized schema. Data are typically imported periodically,

for example, once in a day or once in a few hours, although continuous import is also increasingly used. Raw data imported from the data sources are typically processed and cleaned before being stored in the data warehouse.

However, with the scale at which data are generated by some applications, creating and maintaining such a warehouse can be expensive. Furthermore, queries cannot access the most recent data, since there is a delay between updates on the source database and import of the updates to the data warehouse. On the other hand, query processing can be done more efficiently in a data warehouse; further, queries at a data warehouse do not affect the performance of other queries and transactions at the data source. Whether to use a data warehouse architecture, or to directly access data from the data sources in response to individual queries, is a decision that each enterprise has to make based on its needs.

The term **data lake** is used to refer to an architecture where data are stored in multiple data storage systems and in different formats, including in file systems, but can be queried from a single system. Data lakes are viewed as an alternative to data warehouses, since they do not require up-front effort to preprocess data, although they do require more effort when creating queries.

22.9.2 Schema and Data Integration

The first task in providing a unified view of data lies in creating a unified conceptual schema, a task that is referred to as **schema integration**. Each local system provides its own conceptual schema. The database system must integrate these separate schemas into one common schema. Schema integration is a complicated task, mainly because of the semantic heterogeneity. The same attribute names may appear in different local databases but with different meanings.

Schema integration requires the creation of a **global schema**, which provides a unified view of data in different databases. Schema integration also requires a way to define how data are mapped from the local schema representation at each database, to the global schema. This step can be done by defining views at each site which, transform data from the local schema to the global schema. Data in the global schema is then treated as the union of the global views at the individual site. This approach is called the **global-as-view (GAV)** approach.

Consider an example with two sites which store student information in two different ways:

- Site s_1 which uses the relation $student1(ID, name, dept_name)$, and the relation $studentCreds(ID, tot_cred)$.
- Site s_2 which uses the relation $student2(ID, name, tot_cred)$, and the relation $studentDept(ID, dept_name)$.

Let the global schema chosen be $student(ID, name, dept_name, tot_cred)$.

Then, the global schema view at site *s1* would be defined as the view:

```
create view student_s1(ID, name, dept_name, tot_cred) as
select ID, name, dept_name, tot_cred
from student1, studentCreds
where student1.ID= studentCreds.ID;
```

While the global schema view at site *s2* would be defined as the view:

```
create view student_s2(ID, name, dept_name, tot_cred) as
select ID, name, dept_name, tot_cred
from student2, studentDept
where student2.ID= studentDept.ID;
```

Finally, the global schema *student* would be defined as the union of *student_s1* and *student_s2*.

Note that with the above view definition, a query on the global schema relation *student* can be easily translated into queries on the local schema relations at the sites *s1* and *s2*. It is harder to translate updates on the global schema into updates on the local schema, since there may not be a unique way to do so as discussed in Section 4.2.

There are more complex mapping schemes that are designed to deal with duplication of information across sites and to allow translation of updates on the global schema into updates on the local schema. The **local-as-view (LAV)** approach, which defines local data in each site as a view on a conceptual unified global relation, is one such approach.

Consider for example a situation where the *student* relation is partitioned between two sites based on the *dept_name* attribute, with all students in the “Comp. Sci.” department at site *s3* and all students in other departments in site *s4*. This can be specified using the local-as-view approach by defining the relations *student_s3* and *student_s4*, which are actually stored at the sites *s3* and *s4*, as equivalent to views defined on the global relation *student*.

```
create view student_s3 as
select *
from student
where student.dept_name = 'Comp. Sci.';
```

and

```
create view student_s4 as
select *
from student
where student.dept_name != 'Comp. Sci.';
```

With this extra information, the query optimizer can figure out that a query that seeks to retrieve students in the Comp. Sci. department need only be executed at site s_3 and need not be executed at site s_4 . More information on schema integration may be found in the bibliographic notes for this chapter, available online.

The second task in providing a unified view of data from multiple sources lies in dealing with differences in data types and values. For example, the data types used in one system may not be supported by other systems, and translation between types may not be simple. Even for identical data types, problems may arise from the physical representation of data: One system may use 8-bit ASCII, while another may use 16-bit Unicode; floating-point representations may differ; integers may be represented in *big-endian* or *little-endian* form. At the semantic level, an integer value for length may be inches in one system and millimeters in another; when integrating the data, a single representation must be used, and values converted to the chosen representation. Mapping between types can be done as part of the view definitions that translate data between the local schemas and the global schema.

A deeper problem is that the same conceptual entity may have different names in different systems. For example, a system based in the United States may refer to the city “Cologne,” whereas one in Germany refers to it as “Köln.” One approach to deal with this problem is to have a globally unique naming system, and map values to the unique names as part of the view definitions used for schema mappings. For example, the International Standards Organization has a unique code for country names, and for states/provinces within the countries. The GeoNames database (www.geonames.org) provides unique names for several million locations such as cities, geographical features, roads, buildings, and so forth.

When such standard naming systems are not available, some systems allow the specification of name equivalences; for example, such a system could allow a user to say that “Cologne” is the same as “Köln”. This approach is used in the [Linked Data](#) project, which supports the integration of a very large number of databases that use the RDF representation of data (the RDF representation is described in Section 8.1.4). However, querying is more complicated in such a scenario.

In our description of the view definitions above, we assumed that data are stored in local databases, and the view definitions are used to provide a global view of the data, without actually materializing the data in the global schema. However, such views can also be used to materialize the data in the global schema, which can then be stored in a data warehouse. In the latter case, updates on underlying data must be propagated to the data warehouse.

22.9.3 Query Processing Across Multiple Data Sources

A naive way to execute a query that accesses data from multiple data sources is to fetch all required data to one database, which then executes the query. But suppose, for example, that the query has a selection condition that is satisfied by only one or a few records out of a large relation. If the data source allows the selection to be performed at

the data source, it makes no sense to retrieve the entire relation; instead, the selection operation should be performed at the data source, while other operations, if any, may be performed at the database where the query was issued.

In general, different data sources may support different query capabilities. For example, if the source is a data storage system, it may support selections on key attributes only. Web data sources may restrict which fields selections are allowed on and may additionally require that selections be present on certain fields. On the other hand, if the source is a database that supports SQL, operations such as join or aggregation could be performed at the source and only the result brought over to the database that issues the query. In general, queries may have to be broken up and performed partly at the data source and partly at the site issuing the query.

The cost of processing a query that accesses multiple data sources depends on the local execution costs, as well as on the data transfer cost. If the network is a low bandwidth wide-area network, particular attention must be paid to minimizing data transfer.

In this section, we study issues in distributed query processing and optimization.

22.9.3.1 Join Locations and Join Ordering

Consider the following relational-algebra expression:

$$r_1 \bowtie r_2 \bowtie r_3$$

Assume that r_1 is stored at site S_1 , r_2 at S_2 , and r_3 at S_3 . Let S_I denote the site at which the query was issued. The system needs to produce the result at site S_I . Among the possible strategies for processing this query are these:

- Ship copies of all three relations to site S_I . Using the techniques of Chapter 16, choose a strategy for processing the entire query locally at site S_I .
- Ship a copy of the r_1 relation to site S_2 , and compute $temp_1 = r_1 \bowtie r_2$ at S_2 . Ship $temp_1$ from S_2 to S_3 , and compute $temp_2 = temp_1 \bowtie r_3$ at S_3 . Ship the result $temp_2$ to S_I .
- Devise strategies similar to the previous one, with the roles of S_1, S_2, S_3 exchanged.

There are several other possible strategies.

No one strategy is always the best one. Among the factors that must be considered are the volume of data being shipped, the cost of transmitting a block of data between a pair of sites, and the relative speed of processing at each site. Consider the first strategy. Suppose indices present at S_2 and S_3 are useful for computing the join. If we ship all three relations to S_I , we would need to either re-create these indices at S_I or use a different, possibly more expensive, join strategy. Re-creation of indices entails extra processing overhead and extra disk accesses. There are many variants of the second strategy, which process joins in different orders.

The cost of each of the strategies depends on the sizes of the intermediate results, the network transmission costs, and the costs of processing at each node. The query optimizer needs to choose the best strategy, based on cost estimates.

22.9.3.2 Semijoin Strategy

Suppose that we wish to evaluate the expression $r_1 \bowtie r_2$, where r_1 and r_2 are stored at sites S_1 and S_2 , respectively. Let the schemas of r_1 and r_2 be R_1 and R_2 . Suppose that we wish to obtain the result at S_1 . If there are many tuples of r_2 that do not join with any tuple of r_1 , then shipping r_2 to S_1 entails shipping tuples that fail to contribute to the result. We want to remove such tuples before shipping data to S_1 , particularly if network costs are high.

A possible strategy to accomplish all this is:

1. Compute $temp_1 \leftarrow \Pi_{R_1 \cap R_2}(r_1)$ at S_1 .
2. Ship $temp_1$ from S_1 to S_2 .
3. Compute $temp_2 \leftarrow r_2 \bowtie temp_1$ at S_2 .
4. Ship $temp_2$ from S_2 to S_1 .
5. Compute $r_1 \bowtie temp_2$ at S_1 . The resulting relation is the same as $r_1 \bowtie r_2$.

Before considering the efficiency of this strategy, let us verify that the strategy computes the correct answer. In step 3, $temp_2$ has the result of $r_2 \bowtie \Pi_{R_1 \cap R_2}(r_1)$. In step 5, we compute:

$$r_1 \bowtie r_2 \bowtie \Pi_{R_1 \cap R_2}(r_1)$$

Since join is associative and commutative, we can rewrite this expression as:

$$(r_1 \bowtie \Pi_{R_1 \cap R_2}(r_1)) \bowtie r_2$$

Since $r_1 \bowtie \Pi_{(R_1 \cap R_2)}(r_1) = r_1$, the expression is, indeed, equal to $r_1 \bowtie r_2$, the expression we are trying to evaluate.

This strategy is called a **semijoin strategy**, after the semijoin operator of the relational algebra, denoted $\ltimes$, which we saw in Section 16.4.4. The *natural semijoin* of r_1 with r_2 , denoted $r_1 \ltimes r_2$, is defined as:

$$r_1 \ltimes r_2 \stackrel{\text{def}}{=} \Pi_{R_1}(r_1 \bowtie r_2)$$

Thus, $r_1 \ltimes r_2$ selects those tuples of relation r_1 that contributed to $r_1 \bowtie r_2$. In step 3, $temp_2 = r_2 \ltimes r_1$. The semijoin operation is easily extended to theta-joins. The *theta semijoin* of r_1 with r_2 , denoted $r_1 \ltimes_\theta r_2$, is defined as:

$$r_1 \ltimes_\theta r_2 \stackrel{\text{def}}{=} \Pi_{R_1}(r_1 \bowtie_\theta r_2)$$

For joins of several relations, the semijoin strategy can be extended to a series of semijoin steps. It is the job of the query optimizer to choose the best strategy based on cost estimates.

This strategy is particularly advantageous when relatively few tuples of r_2 contribute to the join. This situation is likely to occur if r_1 is the result of a relational-algebra expression involving selection. In such a case, $temp_2$, that is, $r_2 \bowtie r_1$, may have significantly fewer tuples than r_2 . The cost savings of the strategy result from having to ship only $r_2 \bowtie r_1$, rather than all of r_2 , to S_1 .

Some additional cost is incurred in shipping $temp_1$, that is $\Pi_{R_1 \cap R_2}(r_1)$ to S_2 . If a sufficiently small fraction of tuples in r_2 contribute to the join, the overhead of shipping $temp_1$ will be dominated by the savings of shipping only a fraction of the tuples in r_2 . The overhead of sending $temp_1$ tuples from s_1 to s_2 can be reduced as follows. For the purpose of optimization of join processing, the semijoin operation can be implemented in a manner that overapproximates the true semijoin result. That is, the result should contain all the tuples in the actual semijoin result, but it may contain a few extra tuples. The extra tuples will get eliminated later by the join operation.

An efficient overapproximation of the semijoin result can be computed by using a probabilistic data structure called a **Bloom filter**, which uses bitmaps. Bloom filters are described in more detail in Section 24.1. To implement $r_2 \bowtie r_1$, a Bloom filter with a bitmap b of size m , initialized with all bits set to 0 is used. The join attributes of each tuple of r_1 are hashed to a value in the range $0 \dots (m - 1)$, and the corresponding bit of b is set to 1. The bitmap b , which is much smaller than the relation r_1 , can now be sent to the site containing r_2 . There, the same hash function is computed on the join attributes of each tuple of r_2 . If the corresponding bit is set to 1 in b , that r_2 tuple is accepted (added to the *result* relation), and otherwise it is rejected.

Note that it is possible for different join attribute values, say v_1 and v_2 to have the same hash value; even if r_1 has a tuple with value v_1 , but does not have any tuple with value v_2 , the result of the above procedure may include r_2 tuples whose join attribute value is v_2 . Such a situation is referred to as a **false positive**. However, if v_1 is present in r_1 , the technique will never reject a tuple in r_2 that has join attribute value v_1 , which is important for correctness.

The *result* relation computed above, which is a superset of or equal to $r_2 \bowtie r_1$, is sent to site s_1 . The join $r_1 \bowtie result$ is then computed at site s_1 to get the required join result. False positives may result in extra tuples in *result* that are not present in $r_2 \bowtie r_1$, but such tuples would be eliminated by the join.

To keep the probability of false positives low, the number of bits in the Bloom filter is usually set to a few times the estimated number of distinct join attribute values. Further, it is possible to use k independent hash functions, for some $k > 1$, to identify k bit positions for a given value, and set all of them to 1 when creating the bitmap. When querying it with a given value, the same k hash functions are used to identify k bit locations, and the value is determined to be absent if even one of the k bits has a 0 value. For example, if the bitmap has $10n$ bits, where n is the number of distinct join

attribute values, and $k = 7$ hash functions are used, the false positive rate would be about 1%.

22.9.3.3 Distributed Query Optimization

Several extensions need to be made to existing query optimization techniques in order to optimize distributed query plans.

The first extension is to record the location of data as a *physical property* of the data; recall that optimizers already deal with other physical properties such as the sort order of results. Just as the sort operation is used to create different sort orders, an exchange operation is used to transfer data between different sites.

The second extension is to track where an operator is executed; optimizers already consider different algorithms, such as hash join or merge join, for a given logical operator, in this case, the join operator. The optimizer is extended to additionally consider alternative sites for execution of each algorithm. Note that to execute an operator at a given site, its inputs must satisfy the physical property of being located at that site.

The third extension is to consider semijoin operations to reduce data transfer costs. Semijoin operations can be introduced as logical transformation rules; however, if done naively, the search space increases greatly, making this approach infeasible. Optimization cost can be reduced by restricting, as a heuristic, semijoins to be applied only on database tables, and never on intermediate join results.

A fourth extension is to use schema information to restrict the set of nodes at which a query needs to be executed. Recall from Section 22.9.2 that the local-as-view approach can be used to specify that a relation is partitioned in a particular way. In the example we saw there, site s_3 contains all student tuples with *dept_name* being Comp. Sci., while s_4 contains all the other student tuples. Suppose a query has a selection “*dept_name*='Comp. Sci.'” on *student*; then, the optimizer should recognize that there is no need to involve site s_4 when executing this query.

As another example, if the student data at site s_5 is a replica of the data at site s_3 , then the optimizer can choose to execute the query at either of the sites, depending on which is cheaper; there is no need to execute the query at both sites.

22.9.4 Distributed Directory Systems

A directory is a listing of information about some class of objects such as persons. Directories can be used to find information about a specific object, or in the reverse direction to find objects that meet a certain requirement. Several **directory access protocols** have been developed to provide a standardized way of accessing data in a directory.

A very widely used distributed directory system is the internet **Domain Name Service (DNS)** system, which provides a standardized way to map domain names (such as *db-book.com* or *www.cs.yale.edu*, to the IP addresses of the machines. (Although users see only the domain names, the underlying network routes messages based on IP addresses, and hence a way to convert domain names to IP addresses is critical for

the functioning of the internet.) The **Lightweight Directory Access Protocol (LDAP)** is another very widely used protocol designed for storing organizational data.

Data stored in directories can be represented in the relational model, stored in a relational database, and accessed through standard protocols such as JDBC or ODBC. The question then is, why come up with a specialized protocol for accessing directory information? There are several reasons.

- First, directory access protocols are simplified protocols that cater to a limited type of access to data. They evolved in parallel with the database access protocol standards.
- Second, directory systems were designed to support a hierarchical naming system for objects, similar to file system directory names. Such a naming system is important in many applications. For example, all computers whose names end in `yale.edu` belong to Yale, while those whose names end in `iitb.ac.in` belong to IIT Bombay. Within the `yale.edu` domain, there are subdomains such as `cs.yale.edu`, which corresponds to the CS department in Yale, and `math.yale.edu` which corresponds to the Math department at Yale.
- Third, and most important from a distributed systems perspective, the data in a distributed directory system are stored and controlled in a distributed, hierarchical, manner.

For example, a DNS server at Yale would store information about names of computers at Yale, along with associated information such as their IP addresses. Similarly, DNS servers at Lehigh and IIT Bombay would store information about computers in their respective domains. The DNS servers store information in a hierarchical fashion; for example, the information provided by the Yale DNS server may be stored in a distributed fashion across subdomains at Yale, such as the CS and Math DNS servers.

Distributed directory systems automatically forward queries submitted at a site to the site where the required information is actually stored, to give a unified view of data to users and applications.

Further, distributed directory implementations typically support replication to ensure the availability of data even if some nodes have failed.

Another example of usage of directory systems is for organization data. Such systems store information about employees, such as the employee identifier, name, email, organization unit (such as department), room number, phone number, and (encrypted) password of each employee. The schema of such organizational data are standardized as part of the Lightweight Directory Access Protocol (LDAP). Directory systems based on the LDAP protocol are widely used to authenticate users, using the encrypted passwords stored for each user. (More information about the LDAP data representation may be found in Section 25.5.)

Although distributed data storage across organizational units was important at one time, such directory systems are often centralized these days. In fact, several directory implementations use relational databases to store data, instead of creating special-purpose storage systems. However, the fact that the data representation and protocol to access the data are standardized has meant that these protocols continue to be widely used.

22.10 Summary

- Current generation parallel systems are typically based on a hybrid architecture, where each computer has multiple cores with a shared memory, and there are multiple computers organized in a shared-nothing fashion.
- Parallel processing in a database system can be exploited in two distinct ways.
 - Interquery parallelism—the execution of multiple queries in parallel with each other, across multiple nodes.
 - Intraquery parallelism—the processing of different parts of the execution of a single query, in parallel across multiple nodes.
- Interquery parallelism is essential for transaction-processing systems, while intraquery parallelism is essential for speeding up long-running queries.
- Execution of a single query involves execution of multiple relational operations. The key to exploiting large-scale parallelism is to process each operation in parallel, across multiple nodes (referred to as intraoperation parallelism).
- The operations that are the most amenable to parallelism are: sort, selection, duplicate elimination, projection, and aggregation.
- Range-partitioning sort works in two steps: first range-partitioning the relation, then sorting each partition separately.
- Parallel external sort-merge works in two steps: first, each node N_i sorts the data available at node N_i , then the system merges the sorted runs on each node to get the final sorted output.
- Parallel join algorithms divide the tuples of the input relations over several nodes. Each node then computes part of the join locally. Then the system collects the results from each node to produce the final result.
- Skew is a major problem, especially with increasing degrees of parallelism. Balanced partitioning vectors, using histograms, and virtual node partitioning are among the techniques used to reduce skew.

- The MapReduce paradigm is designed to ease the writing of parallel data processing programs using imperative programming languages that may not be expressible using SQL. Fault-tolerant implementations of the MapReduce paradigm are important for a variety of very large scale data processing tasks. Extensions of the MapReduce model based on algebraic operations is increasingly important. The Hive SQL system uses a MapReduce system as its underlying execution engine, compiling SQL queries to MapReduce code.
- There are two forms of interoperation parallelism: pipelined parallelism and independent parallelism. Pipelined parallelism is usually implemented using a push model, with buffers between operations.
- The exchange operation repartitions data in a specified way. Parallel query plans can be created in such a way that data interchange between nodes is done only by the exchange operator, while all other operations work on local data, just as they would in a centralized database system.
- In the event of failure, parallel query execution plans could be restarted. However, in very large systems where there is a significant chance of failure during the execution of a query, fault tolerance techniques are important to ensure that queries complete execution without restarting, despite failures.
- Parallel algorithms designed for shared-nothing architectures can be used in shared-memory architectures. Each processor can be treated as having its own partition of memory, and we can ignore the fact that the processors have a common shared memory. However, execution can be optimized significantly by exploiting the fast access to shared memory from any of the processors.
- Query optimization for parallel execution can be done using the traditional resource consumption cost model, or using the response time cost model. Partitioning of tables must be taken into account when choosing a plan, to minimize data exchange which is often a significant factor in query execution cost. Materialized views can be important in parallel environments since they can significantly reduce query execution cost.
- There are many streaming data applications today that require high performance processing, which can only be achieved by parallel processing of streaming data. Incoming tuples and results of operations need to be routed to other operations. The publish-subscribe model, implemented for example in Apache Kafka, has proven quite useful for such routing. Fault-tolerant processing of streaming data with exactly-once semantics for processing tuples is important in many applications. Persistence provided by publish-subscribe systems helps in this regard.
- Integration of schema and data from multiple databases is needed for many data processing tasks. The external data approach allows external data to be queries in a database as if it is locally resident. The global-as-view and local-as-view architec-

tures allows rewriting of queries from a global schema to individual local schemas. The semijoin strategy using Bloom filters can be useful to reduce data movement for joins in a distributed database. Distributed directory systems are a type of distributed database designed for distributed storage and querying of directories.

Review Terms

- Interquery parallelism
- Intraquery parallelism
 - Intraoperation parallelism
 - Interoperation parallelism
- Range-partitioning sort
- Data parallelism
- Parallel external sort-merge
- Partitioned join
- Partitioned parallel hash join
- Partitioned parallel merge join
- Partitioned parallel nested-loop join
- Partitioned parallel indexed nested-loops join
- Asymmetric fragment-and-replicate join
- Broadcast join
- Fragment-and-replicate join
- Symmetric fragment-and-replicate join
- Join skew avoidance
- Dynamic handling of join skew
- Work stealing
- Parallel selection
- Parallel duplicate elimination
- Parallel projection
- Parallel aggregation
- Partial aggregation
- Intermediate key
- Pipelined parallelism
- Independent parallelism
- Exchange operator
- Unpartitioned
- Random merge
- Ordered merge
- Parallel query execution plan
- Query processing in shared memory
- Thread
- Single Instruction Multiple Data (SIMD)
- Response-time cost model
- Parallel view maintenance
- Streaming data
- Lambda architecture
- Routing of streams
- Publish-subscribe
- Topic-partition
- At-least-once semantics
- At-most-once semantics
- Exactly-once semantics
- Federated database system
- Global schema
- Local schema
- Schema integration
- Mediator
- Data virtualization
- External data
- Foreign tables

- Data lake
- Global-as-view (GAV)
- Local-as-view (LAV)
- Linked Data
- Semijoin strategy
- Semijoin
- Theta semijoin
- Bloom filter
- False positive
- Directory access protocols
- Domain Name Service (DNS)
- Lightweight Directory Access Protocol (LDAP)

Practice Exercises

- 22.1** What form of parallelism (interquery, interoperation, or intraoperation) is likely to be the most important for each of the following tasks?
- Increasing the throughput of a system with many small queries
 - Increasing the throughput of a system with a few large queries when the number of disks and processors is large
- 22.2** Describe how partial aggregation can be implemented for the **count** and **avg** aggregate functions to reduce data transfer.
- 22.3** With pipelined parallelism, it is often a good idea to perform several operations in a pipeline on a single processor, even when many processors are available.
- Explain why.
 - Would the arguments you advanced in part *a* hold if the machine has a shared-memory architecture? Explain why or why not.
 - Would the arguments in part *a* hold with independent parallelism? (That is, are there cases where, even if the operations are not pipelined and there are many processors available, it is still a good idea to perform several operations on the same processor?)
- 22.4** Consider join processing using symmetric fragment and replicate with range partitioning. How can you optimize the evaluation if the join condition is of the form $|r.A - s.B| \leq k$, where k is a small constant? Here, $|x|$ denotes the absolute value of x . A join with such a join condition is called a **band join**.
- 22.5** Suppose relation r is stored partitioned and indexed on A , and s is stored partitioned and indexed on B . Consider the query:
- $$r.C \gamma_{\text{count}(s.D)} ((\sigma_{A>5}(r)) \bowtie_{r.B=s.B} s)$$
- Give a parallel query plan using the exchange operator, for computing the subtree of the query involving only the select and join operators.

- b. Now extend the above to compute the aggregate. Make sure to use pre-aggregation to minimize the data transfer.
 - c. Skew during aggregation is a serious problem. Explain how pre-aggregation as above can also significantly reduce the effect of skew during aggregation.
- 22.6 Suppose relation r is stored partitioned and indexed on A , and s is stored partitioned and indexed on B . Consider the join $r \bowtie_{r.B=s.B} s$. Suppose s is relatively small, but not small enough to make asymmetric fragment-and-replicate join the best choice, and r is large, with most r tuples not matching any s tuple. A hash-join can be performed but with a semijoin filter used to reduce the data transfer. Explain how semijoin filtering using Bloom filters would work in this parallel join setting.
- 22.7 Suppose you want to compute $r \bowtie_{r.A=s.A} s$.
 - a. Suppose s is a small relation, while r is stored partitioned on $r.B$. Give an efficient parallel algorithm for computing the left outer join.
 - b. Now suppose that r is a small relation, and s is a large relation, stored partitioned on attribute $s.B$. Give an efficient parallel algorithm for computing the above left outer join.
- 22.8 Suppose you want to compute $_{A,B} \gamma_{sum(C)}$ on a relation s which is stored partitioned on $s.B$. Explain how you would do it efficiently, minimizing/avoiding repartitioning, if the number of distinct $s.B$ values is large, and the distribution of number of tuples with each $s.B$ value is relatively uniform.
- 22.9 MapReduce implementations provide fault tolerance, where you can reexecute only failed mappers or reducers. By default, a partitioned parallel join execution would have to be rerun completely in case of even one node failure. It is possible to modify a parallel partitioned join execution to add fault tolerance in a manner similar to MapReduce, so failure of a node does not require full reexecution of the query, but only actions related to that node. Explain what needs to be done at the time of partitioning at the sending node and receiving node to do this.
- 22.10 If a parallel data-store is used to store two relations r and s and we need to join r and s , it may be useful to maintain the join as a materialized view. What are the benefits and overheads in terms of overall throughput, use of space, and response time to user queries?
- 22.11 Explain how each of the following join algorithms can be implemented using the MapReduce framework:
 - a. Broadcast join (also known as asymmetric fragment-and-replicate join).

- b. Indexed nested loop join, where the inner relation is stored in a parallel data-store.
- c. Partitioned join.

Exercises

- 22.12** Can partitioned join be used for $r \bowtie_{r.A < s.A \wedge r.B = s.B} s$? Explain your answer.
- 22.13** Describe a good way to parallelize each of the following:
- a. The difference operation
 - b. Aggregation by the **count** operation
 - c. Aggregation by the **count distinct** operation
 - d. Aggregation by the **avg** operation
 - e. Left outer join, if the join condition involves only equality
 - f. Left outer join, if the join condition involves comparisons other than equality
 - g. Full outer join, if the join condition involves comparisons other than equality
- 22.14** Suppose you wish to handle a workload consisting of a large number of small transactions by using shared-nothing parallelism.
- a. Is intraquery parallelism required in such a situation? If not, why, and what form of parallelism is appropriate?
 - b. What form of skew would be of significance with such a workload?
 - c. Suppose most transactions accessed one *account* record, which includes an *account_type* attribute, and an associated *account_type_master* record, which provides information about the account type. How would you partition and/or replicate data to speed up transactions? You may assume that the *account_type_master* relation is rarely updated.
- 22.15** What is the motivation for work-stealing with virtual nodes in a shared-memory setting? Why might work-stealing not be as efficient in a shared-nothing setting?
- 22.16** The attribute on which a relation is partitioned can have a significant impact on the cost of a query.
- a. Given a workload of SQL queries on a single relation, what attributes would be candidates for partitioning?

- b. How would you choose between the alternative partitioning techniques, based on the workload?
- c. Is it possible to partition a relation on more than one attribute? Explain your answer.

22.17 Consider system that is processing a stream of tuples for a relation r with attributes $(A, B, C, \text{timestamp})$. Suppose the goal of a parallel stream processing system is to compute the number of tuples for each A value in each 5 minute window (based on the timestamp of the tuple). What would be the topic and the topic partitions? Explain why.

Tools

A wide variety of open-source tools are available, in addition to some commercial tools, for parallel query processing. A number of these tools are also available on hosted cloud platforms.

Teradata was one of the first commercial parallel database systems, and it continues to have a large market share. The Red Brick Warehouse was another early parallel database system; Red Brick was acquired by Informix, which was itself acquired by IBM. Other commercial parallel database systems include Teradata Aster Data, IBM Netezza, and Pivotal Greenplum. IBM Netezza, Pivotal Greenplum, and Teradata Aster Data all use PostgreSQL as the underlying database, running independently on each node; each of these systems builds a layer on top, to partition data, and parallelize query processing across the nodes.

The open source Hadoop platform (hadoop.apache.org) includes the HDFS distributed file system and the Hadoop MapReduce platform. Systems that support SQL on a MapReduce platform include Apache Hive (hive.apache.org), which originated at Facebook, Apache Impala (impala.apache.org), which originated at Cloudera, and Apache HAWQ (hawq.apache.org), which originated at Pivotal. Apache Spark (spark.apache.org), which originated at the Univ. of California, Berkeley, and Apache Tez (tez.apache.org) are parallel execution frameworks that support a variety of operators beyond the basic map and reduce operators; and Hive SQL queries can be executed on both these platforms. Other parallel execution frameworks include Apache Pig (pig.apache.org), which originated at Yahoo!, the Asterix system (asterix.ics.uci.edu), which originated at the University of California, Irvine, the Apache Flink system (flink.apache.org), which originated as the Stratosphere project at the Technical University, Berlin, Humboldt University and the Hasso-Plattner Institute). Apache Spark and Apache Flink also support libraries for parallel machine learning.

These systems can access data stored in multiple different data formats, such as files in different formats in HDFS, or objects in a storage system such as HBase. Hadoop file formats were initially textual files, but today Hadoop implementations support several optimized file formats such as Sequence files (which allow binary data), Avro

(which supports semi-structured schemas) and Parquet (which supports columnar data representation).

Apache Kafka (kafka.apache.org) is widely used for routing tuples in streaming data systems. Systems designed for query processing on streaming data include Apache Storm (storm.apache.org), Kafka Streams (kafka.apache.org/documentation/streams/) and Heron (apache.github.io/incubator-heron/), developed by Twitter. Apache Flink (flink.apache.org), Spark Streaming (spark.apache.org/streaming/), the streaming component of Apache Spark, and Apache Apex (apex.apache.org) support analytics on streaming data along with analytics on stored data.

Many of the above systems are also offered as cloud-based services, as part of the cloud services offered by Amazon AWS, Google Cloud, Microsoft Azure, and other similar platforms.

Further Reading

Early work on parallel database systems include GAMMA ([DeWitt (1990)]), XPRS ([Stonebraker et al. (1989)] and Volcano ([Graefe (1990)]). [Graefe (1993)] presents an excellent survey of query processing, including parallel processing of queries. The exchange-operator model was advocated by [Graefe (1990)] and [Graefe and McKenna (1993)]. Skew handling in parallel joins is described by [DeWitt et al. (1992)].

[Ganguly et al. (1992)] describe query-optimization techniques based on the response-time cost model for parallel query execution, while [Zhou et al. (2010)] describe how to extend a query optimizer to account for partitioning properties and parallel plans. View maintenance in parallel data storage systems is described by [Agrawal et al. (2009)].

A fault-tolerant implementation of the MapReduce framework at Google, which lead to the widespread use of the MapReduce paradigm, is described by [Dean and Ghemawat (2010)]. [Kwon et al. (2013)] provide an overview of skew handling in the Hadoop MapReduce framework. [Herodotou and Babu (2013)] describe how to optimize a number of parameters for query execution in the MapReduce framework. An overview of the Apache Spark system is provided by [Zaharia et al. (2016)], while [Zaharia et al. (2012)] describe Resilient Distributed Datasets, a fault-tolerant abstraction which formed a basis for the Spark implementation. Extensions of the exchange operator to support fault-tolerance, are described by [Shah et al. (2004)], with a focus on fault-tolerant continuous queries. Fault-tolerant stream processing in the Google MillWheel system is described in [Akidau et al. (2013)].

The morsel-driven approach to parallel query evaluation in shared-memory systems with multi-core processors is described in [Leis et al. (2014)]. [Kersten et al. (2018)] provides a comparison of vectorwise query processing using optimizations such as SIMD instructions, with producer-driven pipelining.

[Carbone et al. (2015)] describe stream and batch processing in Apache Flink. [Ozsu and Valduriez (2010)] provides textbook coverage of distributed database systems.

Bibliography

- [Agrawal et al. (2009)] P. Agrawal, A. Silberstein, B. F. Cooper, U. Srivastava, and R. Ramakrishnan, “Asynchronous view maintenance for VLSD databases”, In *Proc. of the ACM SIGMOD Conf. on Management of Data* (2009), pages 179–192.
- [Akidau et al. (2013)] T. Akidau, A. Balikov, K. Bekiroğlu, S. Chernyak, J. Haberman, R. Lax, S. McVeety, D. Mills, P. Nordstrom, and S. Whittle, “MillWheel: Fault-tolerant Stream Processing at Internet Scale”, *Proceedings of the VLDB Endowment*, Volume 6, Number 11 (2013), pages 1033–1044.
- [Carbone et al. (2015)] P. Carbone, A. Katsifodimos, S. Ewen, V. Markl, S. Haridi, and K. Tzoumas, “Apache Flink: Stream and Batch Processing in a Single Engine”, *IEEE Data Eng. Bull.*, Volume 38, Number 4 (2015), pages 28–38.
- [Dean and Ghemawat (2010)] J. Dean and S. Ghemawat, “MapReduce: a flexible data processing tool”, *Communications of the ACM*, Volume 53, Number 1 (2010), pages 72–77.
- [DeWitt (1990)] D. DeWitt, “The Gamma Database Machine Project”, *IEEE Transactions on Knowledge and Data Engineering*, Volume 2, Number 1 (1990), pages 44–62.
- [DeWitt et al. (1992)] D. DeWitt, J. Naughton, D. Schneider, and S. Seshadri, “Practical Skew Handling in Parallel Joins”, In *Proc. of the International Conf. on Very Large Databases* (1992), pages 27–40.
- [Ganguly et al. (1992)] S. Ganguly, W. Hasan, and R. Krishnamurthy, “Query Optimization for Parallel Execution”, In *Proc. of the ACM SIGMOD Conf. on Management of Data* (1992), pages 9–18.
- [Graefe (1990)] G. Graefe, “Encapsulation of Parallelism in the Volcano Query Processing System”, In *Proc. of the ACM SIGMOD Conf. on Management of Data* (1990), pages 102–111.
- [Graefe (1993)] G. Graefe, “Query Evaluation Techniques for Large Databases”, *ACM Computing Surveys*, Volume 25, Number 2 (1993).
- [Graefe and McKenna (1993)] G. Graefe and W. McKenna, “The Volcano Optimizer Generator”, In *Proc. of the International Conf. on Data Engineering* (1993), pages 209–218.
- [Herodotou and Babu (2013)] H. Herodotou and S. Babu, “A What-if Engine for Cost-based MapReduce Optimization”, *IEEE Data Eng. Bull.*, Volume 36, Number 1 (2013), pages 5–14.
- [Kersten et al. (2018)] T. Kersten, V. Leis, A. Kemper, T. Neumann, A. Pavlo, and P. A. Boncz, “Everything You Always Wanted to Know About Compiled and Vectorized Queries But Were Afraid to Ask”, *Proceedings of the VLDB Endowment*, Volume 11, Number 13 (2018), pages 2209–2222.

- [Kwon et al. (2013)] Y. Kwon, K. Ren, M. Balazinska, and B. Howe, “Managing Skew in Hadoop”, *IEEE Data Eng. Bull.*, Volume 36, Number 1 (2013), pages 24–33.
- [Leis et al. (2014)] V. Leis, P. A. Boncz, A. Kemper, and T. Neumann, “Morsel-driven parallelism: a NUMA-aware query evaluation framework for the many-core age”, In *Proc. of the ACM SIGMOD Conf. on Management of Data* (2014), pages 743–754.
- [Ozsu and Valduriez (2010)] T. Ozsu and P. Valduriez, *Principles of Distributed Database Systems*, 3rd edition, Prentice Hall (2010).
- [Shah et al. (2004)] M. A. Shah, J. M. Hellerstein, and E. A. Brewer, “Highly-Available, Fault-Tolerant, Parallel Dataflows”, In *Proc. of the ACM SIGMOD Conf. on Management of Data* (2004), pages 827–838.
- [Stonebraker et al. (1989)] M. Stonebraker, P. Aoki, and M. Seltzer, “Parallelism in XPRS”, In *Proc. of the ACM SIGMOD Conf. on Management of Data* (1989).
- [Zaharia et al. (2012)] M. Zaharia, M. Chowdhury, T. Das, A. Dave, J. Ma, M. McCauly, M. J. Franklin, S. Shenker, and I. Stoica, “Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing”, In *Procs. USENIX Symposium on Networked Systems Design and Implementation, NSDI* (2012), pages 15–28.
- [Zaharia et al. (2016)] M. Zaharia, R. S. Xin, P. Wendell, T. Das, M. Armbrust, A. Dave, X. Meng, J. Rosen, S. Venkataraman, M. J. Franklin, A. Ghodsi, J. Gonzalez, S. Shenker, and I. Stoica, “Apache Spark: a unified engine for big data processing”, *Communications of the ACM*, Volume 59, Number 11 (2016), pages 56–65.
- [Zhou et al. (2010)] J. Zhou, P. Larson, and R. Chaiken, “Incorporating partitioning and parallel plans into the SCOPE optimizer”, In *Proc. of the International Conf. on Data Engineering* (2010), pages 1060–1071.

Credits

The photo of the sailboats in the beginning of the chapter is due to ©Pavel Nesvadba/Shutterstock.

CHAPTER 23



Parallel and Distributed Transaction Processing

We studied transaction processing in centralized databases earlier, covering concurrency control in Chapter 18 and recovery in Chapter 19. In this chapter, we study how to carry out transaction processing in parallel and distributed databases. In addition to supporting concurrency control and recovery, transaction processing in parallel and distributed databases must also deal with issues due to replication of data, and of failures of some nodes.

Both parallel and distributed databases have multiple nodes, which can fail independently. The main difference between parallel and distributed databases from the view point of transaction processing is that the latency of remote access is much higher, and bandwidth lower, in a distributed database than in a parallel database where all nodes are in a single data center. Failures such as network partitioning and message delays are much less likely within a data center than across geographically distributed sites, but nevertheless they can occur; transaction processing must be done correctly even if they do occur.

Thus, most techniques for transaction processing are common to both parallel and distributed databases. In the few cases where there is a difference, we explicitly point out the difference. And as a result, in this chapter, whenever we say that a technique is applicable to distributed databases, it should be interpreted to mean that it is applicable to distributed databases as well as to parallel databases, unless we explicitly say otherwise.

In Section 23.1, we outline a model for transaction processing in a distributed database. In Section 23.2, we describe how to implement atomic transactions in a distributed database by using special commit protocols.

In Section 23.3 we describe how to extend traditional concurrency control techniques to distributed databases. Section 23.4 describes concurrency control techniques for the case where data items are replicated, while Section 23.5 describes further extensions including how multiversion concurrency control techniques can be extended to deal with distributed databases, and concurrency control can be implemented with

heterogeneous distributed databases. Replication with weak degrees of consistency is discussed in Section 23.6.

Most techniques for dealing with distributed data require the use of coordinators to ensure consistent and efficient transaction processing. In Section 23.7 we discuss how coordinators can be chosen in a distributed fashion, robust to failures. Finally, Section 23.8 describes the distributed consensus problem, outlines solutions for the problem, and then discusses how these solutions can be used to implement fault-tolerant services by means of replication of a log.

23.1 Distributed Transactions

Access to the various data items in a distributed system is usually accomplished through transactions, which must preserve the ACID properties (Section 17.1). There are two types of transaction that we need to consider. The **local transactions** are those that access and update data in only one local database; the **global transactions** are those that access and update data in several local databases. Ensuring the ACID properties of the local transactions can be done as described in Chapter 17, Chapter 18, and Chapter 19. However, for global transactions, this task is much more complicated, since several nodes may be participating in the execution of the transaction. The failure of one of these nodes, or the failure of a communication link connecting these nodes, may result in erroneous computations.

In this section, we study the system structure of a distributed database and its possible failure modes. In later sections, we discuss how to ensure ACID properties are satisfied in a distributed database, despite failures. We reemphasize that these failure modes occur with parallel databases as well, and the techniques we describe are equally applicable to parallel databases.

23.1.1 System Structure

We now consider a system structure with multiple nodes, each of which can fail independently of the others. We note that the nodes may be within a single data center, corresponding to a parallel database system, or geographically distributed, in a distributed database system. The system structure is similar in either case; the problems with respect to transaction isolation and atomicity are the same in both cases, as are the solutions.

We note that the system structure we consider here is not applicable to a shared-memory parallel database system whose components do not have independent modes of failures. In such systems either the whole system is up, or the whole system is down. Further, there is usually only one transaction log used for recovery. Concurrency control and recovery techniques that are designed for centralized database systems can be used in such systems, and are preferable to techniques described in this chapter.

Each node has its own *local* transaction manager, whose function is to ensure the ACID properties of those transactions that execute at that node. The various trans-

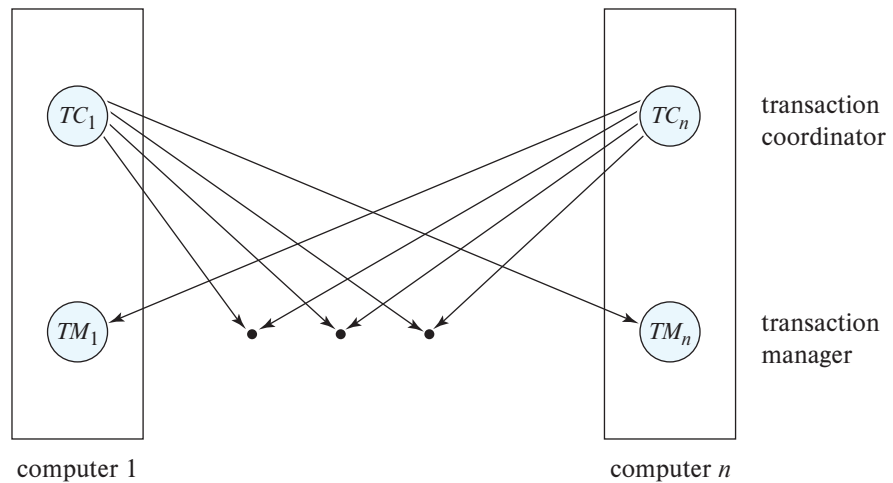


Figure 23.1 System architecture.

action managers cooperate to execute global transactions. To understand how such a manager can be implemented, consider an abstract model of a transaction system, in which each node contains two subsystems:

- The **transaction manager** manages the execution of those transactions (or subtransactions) that access data stored in the node. Note that each such transaction may be either a local transaction (i.e., a transaction that executes at only that node) or part of a global transaction (i.e., a transaction that executes at several nodes).
- The **transaction coordinator** coordinates the execution of the various transactions (both local and global) initiated at that node.

The overall system architecture appears in Figure 23.1.

The structure of a transaction manager is similar in many respects to the structure of a centralized system. Each transaction manager is responsible for:

- Maintaining a log for recovery purposes.
- Participating in an appropriate concurrency-control scheme to coordinate the concurrent execution of the transactions executing at that node.

As we shall see, we need to modify both the recovery and concurrency schemes to accommodate the distributed execution of transactions.

The transaction coordinator subsystem is not needed in the centralized environment, since a transaction accesses data at only a single node. A transaction coordinator, as its name implies, is responsible for coordinating the execution of all the transactions initiated at that node. For each such transaction, the coordinator is responsible for:

- Starting the execution of the transaction.
- Breaking the transaction into a number of subtransactions and distributing these subtransactions to the appropriate nodes for execution.
- Coordinating the termination of the transaction, which may result in the transaction being committed at all nodes or aborted at all nodes.

23.1.2 System Failure Modes

A distributed system may suffer from the same types of failure that a centralized system does (e.g., software errors, hardware errors, or disk crashes). There are, however, additional types of failure with which we need to deal in a distributed environment. The basic failure types are:

- Failure of a node.
- Loss of messages.
- Failure of a communication link.
- Network partition.

The loss or corruption of messages is always a possibility in a distributed system. The system uses transmission-control protocols, such as TCP/IP, to handle such errors. Information about such protocols may be found in standard textbooks on networking.

However, if two nodes *A* and *B* are not directly connected, messages from one to the other must be *routed* through a sequence of communication links. If a communication link fails, messages that would have been transmitted across the link must be rerouted. In some cases, it is possible to find another route through the network, so that the messages are able to reach their destination. In other nodes, a failure may result in there being no connection between some pairs of nodes. A system is **partitioned** if it has been split into two (or more) subsystems, called **partitions**, that lack any connection between them. Note that, under this definition, a partition may consist of a single node.

23.2 Commit Protocols

If we are to ensure atomicity, all the nodes in which a transaction *T* executed must agree on the final outcome of the execution. *T* must either commit at all nodes, or it must abort at all nodes. To ensure this property, the transaction coordinator of *T* must execute a commit protocol.

Among the simplest and most widely used commit protocols is the **two-phase commit protocol (2PC)**, which is described in Section 23.2.1.

23.2.1 Two-Phase Commit

We first describe how the two-phase commit protocol (2PC) operates during normal operation, then describe how it handles failures and finally how it carries out recovery and concurrency control.

Consider a transaction T initiated at node N_i , where the transaction coordinator is C_i .

23.2.1.1 The Commit Protocol

When T completes its execution—that is, when all the nodes at which T has executed inform C_i that T has completed— C_i starts the 2PC protocol.

- Phase 1.** C_i adds the record $\langle \text{prepare } T \rangle$ to the log and forces the log onto stable storage. It then sends a **prepare T** message to all nodes at which T executed. On receiving such a message, the transaction manager at that node determines whether it is willing to commit its portion of T . If the answer is no, it adds a record $\langle \text{no } T \rangle$ to the log and then responds by sending an **abort T** message to C_i . If the answer is yes, it adds a record $\langle \text{ready } T \rangle$ to the log and forces the log (with all the log records corresponding to T) onto stable storage. The transaction manager then replies with a **ready T** message to C_i .
- Phase 2.** When C_i receives **ready** responses to the **prepare T** message from all the nodes, or when it receives an **abort T** message from at least one participant node, C_i can determine whether the transaction T can be committed or aborted. Transaction T can be committed if C_i received a **ready T** message from all the participating nodes. Otherwise, transaction T must be aborted. Depending on the verdict, either a record $\langle \text{commit } T \rangle$ or a record $\langle \text{abort } T \rangle$ is added to the log and the log is forced onto stable storage. At this point, the fate of the transaction has been sealed.

Following this point, the coordinator sends either a **commit T** or an **abort T** message to all participating nodes. When a node receives that message, it records the result (either $\langle \text{commit } T \rangle$ or $\langle \text{abort } T \rangle$) in its log, and correspondingly either commits or aborts the transaction.

Since nodes may fail, the coordinator cannot wait indefinitely for responses from all the nodes. Instead, when a prespecified interval of time has elapsed since the **prepare T** message was sent out, if any node has not responded to the coordinator, the coordinator can decide to abort the transaction; the steps described for aborting the transaction must be followed, just as if a node had sent an **abort** message for the transaction.

Figure 23.2 shows an instance of successful execution of 2PC for a transaction T , with two nodes, N_1 and N_2 , that are both willing to commit transaction T . If any of the

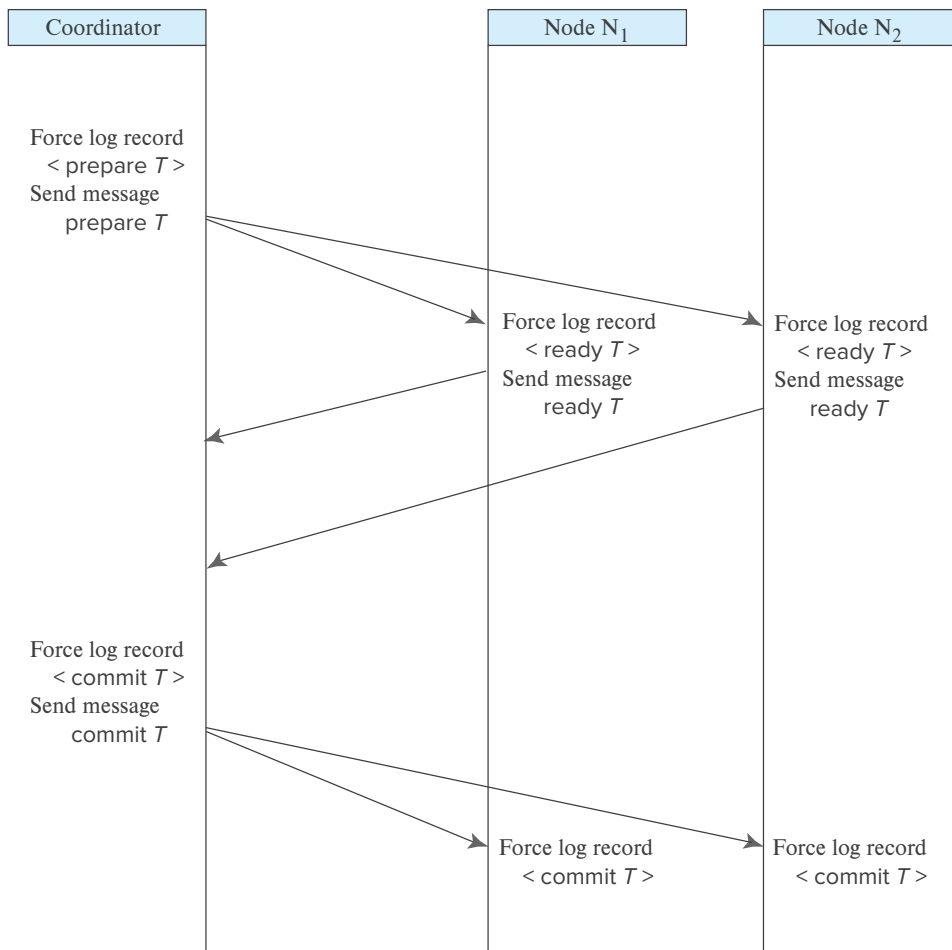


Figure 23.2 Successful execution of 2PC.

nodes sends a no T message, the coordinator will send an abort T message to all the nodes, which will then abort the transaction.

A node at which T executed can unconditionally abort T at any time before it sends the message `ready  $T$` to the coordinator. Once the `<ready  $T$ >` log record is written, the transaction T is said to be in the **ready state** at the node. The `ready  $T$` message is, in effect, a promise by a node to follow the coordinator's order to commit T or to abort T . To make such a promise, the needed information must first be stored in stable storage. Otherwise, if the node crashes after sending `ready  $T$` , it may be unable to make good on its promise. Further, locks acquired by the transaction must continue to be held until the transaction completes, even if there is an intervening node failure, as we shall see in Section 23.2.1.3.

Since unanimity is required to commit a transaction, the fate of T is sealed as soon as at least one node responds **abort** T . Since the coordinator node N_i is one of the nodes at which T executed, the coordinator can decide unilaterally to **abort** T . The final verdict regarding T is determined at the time that the coordinator writes that verdict (commit or abort) to the log and forces that verdict to stable storage.

In some implementations of the 2PC protocol, a node sends an **acknowledge** T message to the coordinator at the end of the second phase of the protocol. When the coordinator receives the **acknowledge** T message from all the nodes, it adds the record **<complete** T to the log. Until this step, the coordinator cannot forget about the commit or abort decision on T , since a node may ask for the decision. (A node that has not received a commit or abort for transaction T , perhaps due to a network failure or temporary node failure, may send such a request to the coordinator.) After this step, the coordinator can discard information about transaction T .

23.2.1.2 Handling of Failures

The 2PC protocol responds in different ways to various types of failure:

- **Failure of a participating node.** If the coordinator C_i detects that a node has failed, it takes these actions: If the node fails before responding with a **ready** T message to C_i , the coordinator assumes that it responded with an **abort** T message. If the node fails after the coordinator has received the **ready** T message from the node, the coordinator executes the rest of the commit protocol in the normal fashion, ignoring the failure of the node.

When a participating node N_k recovers from a failure, it must examine its log to determine the fate of those transactions that were in the midst of execution when the failure occurred. Let T be one such transaction. We consider each of the possible cases:

- The log contains a **<commit** T record. In this case, the node executes **redo**(T).
- The log contains an **<abort** T record. In this case, the node executes **undo**(T).
- The log contains a **<ready** T record. In this case, the node must consult C_i to determine the fate of T . If C_i is up, it notifies N_k regarding whether T committed or aborted. In the former case, it executes **redo**(T); in the latter case, it executes **undo**(T). If C_i is down, N_k must try to find the fate of T from other nodes. It does so by sending a **querystatus** T message to all the nodes in the system. On receiving such a message, a node must consult its log to determine whether T has executed there, and if T has, whether T committed or aborted. It then notifies N_k about this outcome. If no node has the appropriate information (i.e., whether T committed or aborted), then N_k can neither abort nor commit T . The decision concerning T is postponed until N_k can obtain the needed

information. Thus, N_k must periodically resend the `querystatus` message to the other nodes. It continues to do so until a node that contains the needed information has recovered. Note that the node at which C_i resides always has the needed information.

- The log contains no control records (`abort`, `commit`, `ready`) concerning T . Thus, we know that N_k failed before responding to the `prepare T` message from C_i . Since the failure of N_k precludes the sending of such a response, by our algorithm C_i must abort T . Hence, N_k must execute `undo(T)`.
- **Failure of the coordinator.** If the coordinator fails in the midst of the execution of the commit protocol for transaction T , then the participating nodes must decide the fate of T . We shall see that, in certain cases, the participating nodes cannot decide whether to commit or abort T , and therefore these nodes must wait for the recovery of the failed coordinator.
 - If an active node contains a `<commit T>` record in its log, then T must be committed.
 - If an active node contains an `<abort T>` record in its log, then T must be aborted.
 - If some active node does *not* contain a `<ready T>` record in its log, then the failed coordinator C_i cannot have decided to commit T , because a node that does not have a `<ready T>` record in its log cannot have sent a `ready T` message to C_i . However, the coordinator may have decided to abort T , but not to commit T . Rather than wait for C_i to recover, it is preferable to abort T .
 - If none of the preceding cases holds, then all active nodes must have a `<ready T>` record in their logs, but no additional control records (such as `<abort T>` or `<commit T>`). Since the coordinator has failed, it is impossible to determine whether a decision has been made, and if one has, what that decision is, until the coordinator recovers. Thus, the active nodes must wait for C_i to recover.

Since the fate of T remains in doubt, T may continue to hold system resources. For example, if locking is used, T may hold locks on data at active nodes. Such a situation is undesirable, because it may be hours or days before C_i is again active. During this time, other transactions may be forced to wait for T . As a result, data items may be unavailable not only on the failed node (C_i), but on active nodes as well. This situation is called the **blocking problem**, because T is blocked pending the recovery of node C_i .
- **Network partition.** When a network partition occurs, two possibilities exist:
 1. The coordinator and all its participants remain in one partition. In this case, the failure has no effect on the commit protocol.

2. The coordinator and its participants belong to several partitions. From the viewpoint of the nodes in one of the partitions, it appears that the nodes in other partitions have failed. Nodes that are not in the partition containing the coordinator simply execute the protocol to deal with the failure of the coordinator. The coordinator and the nodes that are in the same partition as the coordinator follow the usual commit protocol, assuming that the nodes in the other partitions have failed.

Thus, the major disadvantage of the 2PC protocol is that coordinator failure may result in blocking, where a decision either to commit or to abort T may have to be postponed until C_i recovers. We discuss how to remove this limitation shortly, in Section 23.2.2.

23.2.1.3 Recovery and Concurrency Control

When a failed node restarts, we can perform recovery by using, for example, the recovery algorithm described in Section 19.4. To deal with distributed commit protocols, the recovery procedure must treat **in-doubt transactions** specially; in-doubt transactions are transactions for which a **<ready T >** log record is found, but neither a **<commit T >** log record nor an **<abort T >** log record is found. The recovering node must determine the commit–abort status of such transactions by contacting other nodes, as described in Section 23.2.1.2.

If recovery is done as just described, however, normal transaction processing at the node cannot begin until all in-doubt transactions have been committed or rolled back. Finding the status of in-doubt transactions can be slow, since multiple nodes may have to be contacted. Further, if the coordinator has failed, and no other node has information about the commit–abort status of an incomplete transaction, recovery potentially could become blocked if 2PC is used. As a result, the node performing restart recovery may remain unusable for a long period.

To circumvent this problem, recovery algorithms typically provide support for noting lock information in the log. (We are assuming here that locking is used for concurrency control.) Instead of writing a **<ready T >** log record, the algorithm writes a **<ready T, L >** log record, where L is a list of all write locks held by the transaction T when the log record is written. At recovery time, after performing local recovery actions, for every in-doubt transaction T , all the write locks noted in the **<ready T, L >** log record (read from the log) are reacquired.

After lock reacquisition is complete for all in-doubt transactions, transaction processing can start at the node, even before the commit–abort status of the in-doubt transactions is determined. The commit or rollback of in-doubt transactions proceeds concurrently with the execution of new transactions. Thus, node recovery is faster and never gets blocked. Note that new transactions that have a lock conflict with any write locks held by in-doubt transactions will be unable to make progress until the conflicting in-doubt transactions have been committed or rolled back.

23.2.2 Avoiding Blocking During Commit

The blocking problem of 2PC is a serious concern for system designers, since the failure of a coordinator node could lead to blocking of a transaction that has acquired locks on a frequently used data item, which in turn prevents other transactions that need to acquire a conflicting lock from completing their execution.

By involving multiple nodes in the commit decision step of 2PC, it is possible to avoid blocking as long as a majority of the nodes involved in the commit decision are alive and can communicate with each other. This is done by using the idea of fault-tolerant distributed consensus. Details of distributed consensus are discussed in detail later, in Section 23.8, but we outline the problem and sketch a solution approach below.

The **distributed consensus problem** is as follows: A set of n nodes need to agree on a decision; in this case, whether or not to commit a particular transaction. The inputs to make the decision are provided to all the nodes, and then each node votes on the decision; in the case of 2PC, the decision is on whether or not to commit a transaction. The key goal of protocols for achieving distributed consensus is that the decision should be made in such a way that all nodes will “learn” the same value for the decision (i.e., all nodes will learn that the transaction is to be committed, or all nodes will learn that the transaction is to be aborted), even if some nodes fail during the execution of the protocol, or there are network partitions. Further, the distributed consensus protocol should not block, as long as a majority of the nodes participating remain alive and can communicate with each other.

There are several protocols for distributed consensus, two of which are widely used today (Paxos and Raft). We study distributed consensus in Section 23.8. A key idea behind these protocols is the idea of a vote, which succeeds only if a majority of the participating nodes agree on a particular decision.

Given an implementation of distributed consensus, the blocking problem due to coordinator failure can be avoided as follows: Instead of the coordinator locally deciding to commit or abort a transaction, it initiates the distributed consensus protocol, requesting that the value “committed” or “aborted” be assigned to the transaction T . The request is sent to all the nodes participating in the distributed consensus, and the consensus protocol is then executed by those nodes. Since the protocol is fault tolerant, it will succeed even if some nodes fail, as long as a majority of the nodes are up and remain connected. The transaction can be declared committed by the coordinator only after the consensus protocol completes successfully.

There are two possible failure scenarios:

- *The coordinator fails at any stage before informing all participating nodes of the commit or abort status of a transaction T .*

In this case, a new coordinator is chosen (we will see how to do so in Section 23.7). The new coordinator checks with the nodes participating in the distributed consensus to see if a decision was made, and if so informs the 2PC participants

of the decision. A majority of the nodes participating in consensus must respond, to check if a decision was made or not; the protocol will not block as long as the failed/disconnected nodes are in a minority.

If no decision was made earlier for transaction T , the new coordinator again checks with the 2PC participants to check if they are ready to commit or wish to abort the transaction, and follows the usual coordinator protocol based on their responses. As before, if no response is received from a participant, the new coordinator may choose to abort T .

- *The distributed consensus protocol fails to reach a decision.*

Failure of the protocol can occur due to the failure of some participating nodes. It could also occur because of conflicting requests, none of which gets a majority of “votes” during the consensus protocol. For 2PC, the request normally comes from a single coordinator, so such a conflict is unlikely. However, conflicting requests can arise in rare cases if a coordinator fails after sending out a commit message, but its commit message is delivered late; meanwhile, a new 2PC coordinator makes an abort decision since it could not reach some participating nodes. Even with such a conflict, the distributed consensus protocol guarantees that only one of the commit or abort requests can succeed, even in the presence of failures. But if some nodes are down, and neither the commit nor the abort request gets a majority vote from nodes participating in the distributed consensus, it is possible for the protocol to fail to reach a decision.

Regardless of the reason, if the distributed consensus protocol fails to reach a decision, the new coordinator just re-initiates the protocol.

Note that in the event of a network partition, a node that gets disconnected from the majority of the nodes participating in consensus may not learn about a decision, even if a decision was successfully made. Thus, transactions running at such a node may be blocked.

Failure of 2PC participants could make data unavailable, in the absence of replication. Distributed consensus can also be used to keep replicas of a data item in a consistent state, as we explain later in Section 23.8.4.

The idea of distributed consensus to make 2PC nonblocking was proposed in the 1980s; it is used, for example, in the Google Spanner distributed database system.

The **three-phase commit (3PC)** protocol is an extension of the two-phase commit protocol that avoids the blocking problem under certain assumptions. One variant of the protocol avoids blocking as long as network partitions do not occur, but it may lead to inconsistent decisions in the event of a network partition. Extensions of the protocol that work safely under network partitioning were developed subsequently. The idea behind these extensions is similar to the majority voting idea of distributed consensus, but the protocols are specifically tailored for the task of atomic commit.

23.2.3 Alternative Models of Transaction Processing

With two-phase commit, participating nodes agree to let the coordinator decide the fate of a transaction, and are forced to wait for the decision of the coordinator, while holding locks on updated data items. While such loss of autonomy may be acceptable within an organization, no organization would be willing to force its computers to wait, potentially for a long time, while a computer at another organization makes the decision.

In this section, we describe how to use *persistent messaging* to avoid the problem of distributed commit. To understand persistent messaging, consider how one might transfer funds between two different banks, each with its own computer. One approach is to have a transaction span the two nodes and use two-phase commit to ensure atomicity. However, the transaction may have to update the total bank balance, and blocking could have a serious impact on all other transactions at each bank, since almost all transactions at the bank would update the total bank balance.

In contrast, consider how funds transfer by a bank check occurs. The bank first deducts the amount of the check from the available balance and prints out a check. The check is then physically transferred to the other bank where it is deposited. After verifying the check, the bank increases the local balance by the amount of the check. The check constitutes a message sent between the two banks. So that funds are not lost or incorrectly increased, the check must not be lost and must not be duplicated and deposited more than once. When the bank computers are connected by a network, persistent messages provide the same service as the check (but much faster).

Persistent messages are messages that are guaranteed to be delivered to the recipient exactly once (neither less nor more), regardless of failures, if the transaction sending the message commits, and are guaranteed to not be delivered if the transaction aborts. Database recovery techniques are used to implement persistent messaging on top of the normal network channels, as we shall see shortly. In contrast, regular messages may be lost or may even be delivered multiple times in some situations.

Error handling is more complicated with persistent messaging than with two-phase commit. For instance, if the account where the check is to be deposited has been closed, the check must be sent back to the originating account and credited back there. Both nodes must, therefore, be provided with error-handling code, along with code to handle the persistent messages. In contrast, with two-phase commit, the error would be detected by the transaction, which would then never deduct the amount in the first place.

The types of exception conditions that may arise depend on the application, so it is not possible for the database system to handle exceptions automatically. The application programs that send and receive persistent messages must include code to handle exception conditions and bring the system back to a consistent state. For instance, it is not acceptable to just lose the money being transferred if the receiving account has been closed; the money must be credited back to the originating account, and if that is not possible for some reason, humans must be alerted to resolve the situation manually.

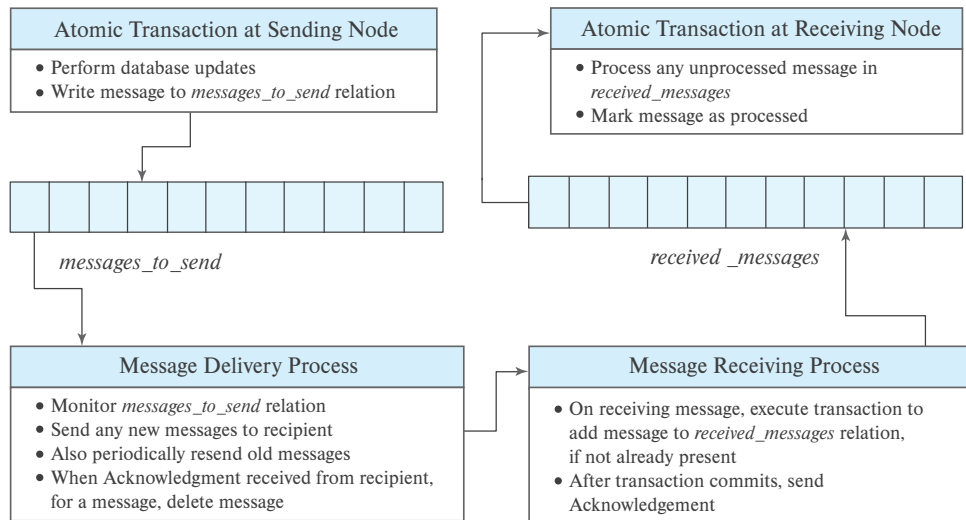


Figure 23.3 Implementation of persistent messaging.

There are many applications where the benefit of eliminating blocking is well worth the extra effort to implement systems that use persistent messages. In fact, few organizations would agree to support two-phase commit for transactions originating outside the organization, since failures could result in blocking of access to local data. Persistent messaging therefore plays an important role in carrying out transactions that cross organizational boundaries.

We now consider the **implementation** of persistent messaging. Persistent messaging can be implemented on top of an unreliable messaging infrastructure, which may lose messages or deliver them multiple times. Figure 23.3 shows a summary of the implementation, which is described in detail next.

- **Sending node protocol.** When a transaction wishes to send a persistent message, it writes a record containing the message in a special relation *messages_to_send*, instead of directly sending out the message. The message is also given a unique message identifier. Note that this relation acts as a message *outbox*.

A *message delivery process* monitors the relation, and when a new message is found, it sends the message to its destination. The usual database concurrency-control mechanisms ensure that the system process reads the message only after the transaction that wrote the message commits; if the transaction aborts, the usual recovery mechanism would delete the message from the relation.

The message delivery process deletes a message from the relation only after it receives an acknowledgment from the destination node. If it receives no acknowledgment from the destination node, after some time it sends the message again. It repeats this until an acknowledgment is received. In case of permanent failures,

the system will decide, after some period of time, that the message is undeliverable. Exception handling code provided by the application is then invoked to deal with the failure.

Writing the message to a relation and processing it only after the transaction has committed ensures that the message will be delivered if and only if the transaction commits. Repeatedly sending it guarantees it will be delivered even if there are (temporary) system or network failures.

- **Receiving node protocol.** When a node receives a persistent message, it runs a transaction that adds the message to a special *received_messages* relation, provided it is not already present in the relation (the unique message identifier allows duplicates to be detected). The relation has an attribute to indicate if the message has been processed, which is set to false when the message is inserted in the relation. Note that this relation acts as a message *inbox*.

After the transaction commits, or if the message was already present in the relation, the receiving node sends an acknowledgment back to the sending node.

Note that sending the acknowledgment before the transaction commits is not safe, since a system failure may then result in loss of the message. Checking whether the message has been received earlier is essential to avoid multiple deliveries of the message.

- **Processing of message.** Received messages must be processed to carry out the actions specified in the message. A process at the receiving node monitors the *received_messages* relation to check for messages that have not been processed. When it finds such a message, the message is processed, and as part of the same transaction that processes the message, the processed flag is set to true. This ensures that a message is processed exactly once after it is received.
- **Deletion of old messages.** In many messaging systems, it is possible for messages to get delayed arbitrarily, although such delays are very unlikely. Therefore, to be safe, the message must never be deleted from the *received_messages* relation. Deleting it could result in a duplicate delivery not being detected. But as a result, the *received_messages* relation may grow indefinitely. To deal with this problem, each message is given a timestamp, and if the timestamp of a received message is older than some cutoff, the message is discarded. All messages recorded in the *received_messages* relation that are older than the cutoff can be deleted.

Workflows provide a general model of distributed transaction processing involving multiple nodes and possibly human processing of certain steps, and they are supported by application software used by enterprises. For instance, as we saw in Section 9.6.1, when a bank receives a loan application, there are many steps it must take, including contacting external credit-checking agencies, before approving or rejecting a loan application. The steps, together, form a workflow. Persistent messaging forms the underlying basis for supporting workflows in a distributed environment.

23.3 Concurrency Control in Distributed Databases

We now consider how the concurrency-control schemes discussed in Chapter 18 can be modified so that they can be used in a distributed environment. We assume that each node participates in the execution of a commit protocol to ensure global transaction atomicity.

In this section, we assume that data items are not replicated, and we do not consider multiversion techniques. We discuss how to handle replicas later, in Section 23.4, and we discuss distributed multiversion concurrency control techniques in Section 23.5.

23.3.1 Locking Protocols

The various locking protocols described in Chapter 18 can be used in a distributed environment. We discuss implementation issues in this section.

23.3.1.1 Single Lock-Manager Approach

In the **single lock-manager** approach, the system maintains a *single* lock manager that resides in a *single* chosen node—say N_i . All lock and unlock requests are made at node N_i . When a transaction needs to lock a data item, it sends a lock request to N_i . The lock manager determines whether the lock can be granted immediately. If the lock can be granted, the lock manager sends a message to that effect to the node at which the lock request was initiated. Otherwise, the request is delayed until it can be granted, at which time a message is sent to the node at which the lock request was initiated. The transaction can read the data item from *any* one of the nodes at which a replica of the data item resides. In the case of a write, all the nodes where a replica of the data item resides must be involved in the writing.

The scheme has these advantages:

- **Simple implementation.** This scheme requires two messages for handling lock requests and one message for handling unlock requests.
- **Simple deadlock handling.** Since all lock and unlock requests are made at one node, the deadlock-handling algorithms discussed in Chapter 18 can be applied directly.

The disadvantages of the scheme are:

- **Bottleneck.** The node N_i becomes a bottleneck, since all requests must be processed there.
- **Vulnerability.** If the node N_i fails, the concurrency controller is lost. Either processing must stop, or a recovery scheme must be used so that a backup node can take over lock management from N_i , as described in Section 23.7.

23.3.1.2 Distributed Lock Manager

A compromise between the advantages and disadvantages can be achieved through the **distributed-lock-manager** approach, in which the lock-manager function is distributed over several nodes.

Each node maintains a local lock manager whose function is to administer the lock and unlock requests for those data items that are stored in that node. When a transaction wishes to lock a data item Q that resides at node N_i , a message is sent to the lock manager at node N_i requesting a lock (in a particular lock mode). If data item Q is locked in an incompatible mode, then the request is delayed until it can be granted. Once it has determined that the lock request can be granted, the lock manager sends a message back to the initiator indicating that it has granted the lock request.

The distributed-lock-manager scheme has the advantage of simple implementation, and it reduces the degree to which the coordinator is a bottleneck. It has a reasonably low overhead, requiring two message transfers for handling lock requests, and one message transfer for handling unlock requests. However, deadlock handling is more complex, since the lock and unlock requests are no longer made at a single node: There may be internode deadlocks even when there is no deadlock within a single node. The deadlock-handling algorithms discussed in Chapter 18 must be modified, as we shall discuss in Section 23.3.2, to detect global deadlocks.

23.3.2 Deadlock Handling

The deadlock-prevention and deadlock-detection algorithms in Chapter 18 can be used in a distributed system, with some modifications.

Consider first the deadlock-prevention techniques, which we saw in Section 18.2.1.

- Techniques for deadlock prevention based on lock ordering can be used in a distributed system, with no changes at all. These techniques prevent cyclic lock waits; the fact that locks may be obtained at different nodes has no effect on prevention of cyclic lock waits.
- Techniques based on preemption and transaction rollback can also be used unchanged in a distributed system. In particular, the *wait-die* technique is used in several distributed systems. Recall that this technique allows older transactions to wait for locks held by younger transactions, but if a younger transaction needs to wait for a lock held by an older transaction, the younger transaction is rolled back. The transaction that is rolled back may subsequently be executed again; recall that it retains its original start time; if it is treated as a new transaction, it could be rolled back repeatedly, and starve, even as other transactions make progress and complete.
- Timeout-based schemes, too, work without any changes in a distributed system.

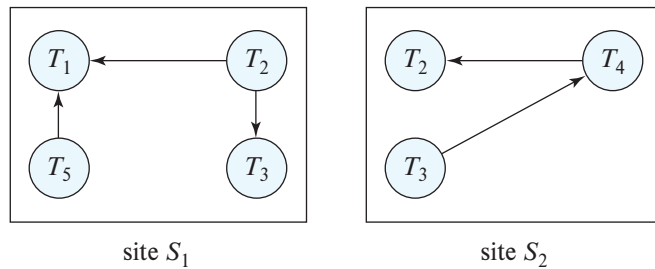


Figure 23.4 Local wait-for graphs.

Deadlock-prevention techniques may result in unnecessary waiting and rollback when used in a distributed system, just as in a centralized system,

We now consider deadlock-detection techniques that allow deadlocks to occur and detect them if they do. The main problem in a distributed system is deciding how to maintain the wait-for graph. Common techniques for dealing with this issue require that each node keep a **local wait-for graph**. The nodes of the graph correspond to all the transactions (local as well as nonlocal) that are currently either holding or requesting any of the items local to that node. For example, Figure 23.4 depicts a system consisting of two nodes, each maintaining its local wait-for graph. Note that transactions T_2 and T_3 appear in both graphs, indicating that the transactions have requested items at both nodes.

These local wait-for graphs are constructed in the usual manner for local transactions and data items. When a transaction T_i on node N_1 needs a resource in node N_2 , it sends a request message to node N_2 . If the resource is held by transaction T_j , the system inserts an edge $T_i \rightarrow T_j$ in the local wait-for graph of node N_2 .

If any local wait-for graph has a cycle, a deadlock has occurred. On the other hand, the fact that there are no cycles in any of the local wait-for graphs does not mean that there are no deadlocks. To illustrate this problem, we consider the local wait-for graphs of Figure 23.4. Each wait-for graph is acyclic; nevertheless, a deadlock exists in the system because the *union* of the local wait-for graphs contains a cycle. This graph appears in Figure 23.5.

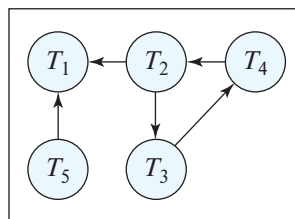


Figure 23.5 Global wait-for graph for Figure 23.4.

In the **centralized deadlock detection** approach, the system constructs and maintains a **global wait-for graph** (the union of all the local graphs) in a *single* node: the deadlock-detection coordinator. Since there is communication delay in the system, we must distinguish between two types of wait-for graphs. The *real* graph describes the real but unknown state of the system at any instance in time, as would be seen by an omniscient observer. The *constructed* graph is an approximation generated by the controller during the execution of the controller's algorithm. Obviously, the controller must generate the constructed graph in such a way that, whenever the detection algorithm is invoked, the reported results are correct. *Correct* means in this case that, if a deadlock exists, it is reported promptly, and if the system reports a deadlock, it is indeed in a deadlock state.

The global wait-for graph can be reconstructed or updated under these conditions:

- Whenever a new edge is inserted in or removed from one of the local wait-for graphs.
- Periodically, when a number of changes have occurred in a local wait-for graph.
- Whenever the coordinator needs to invoke the cycle-detection algorithm.

When the coordinator invokes the deadlock-detection algorithm, it searches its global graph. If it finds a cycle, it selects a victim to be rolled back. The coordinator must notify all the nodes that a particular transaction has been selected as the victim. The nodes, in turn, roll back the victim transaction.

This scheme may produce unnecessary rollbacks if:

- **False cycles** exist in the global wait-for graph. As an illustration, consider a snapshot of the system represented by the local wait-for graphs of Figure 23.6. Suppose that T_2 releases the resource that it is holding in node N_1 , resulting in the deletion of the edge $T_1 \rightarrow T_2$ in N_1 . Transaction T_2 then requests a resource held by T_3 at node N_2 , resulting in the addition of the edge $T_2 \rightarrow T_3$ in N_2 . If the insert $T_2 \rightarrow T_3$ message from N_2 arrives before the remove $T_1 \rightarrow T_2$ message from N_1 , the coordinator may discover the false cycle $T_1 \rightarrow T_2 \rightarrow T_3$ after the insert (but before the remove). Deadlock recovery may be initiated, although no deadlock has occurred.

Note that the false-cycle situation could not occur under two-phase locking. The likelihood of false cycles is usually sufficiently low that they do not cause a serious performance problem.

- A *deadlock* has indeed occurred and a victim has been picked, while one of the transactions was aborted for reasons unrelated to the deadlock. For example, suppose that node N_1 in Figure 23.4 decides to abort T_2 . At the same time, the coordinator has discovered a cycle and has picked T_3 as a victim. Both T_2 and T_3 are now rolled back, although only T_2 needed to be rolled back.

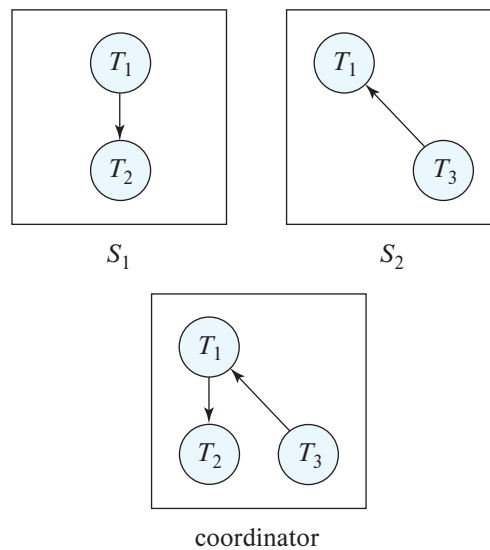


Figure 23.6 False cycles in the global wait-for graph.

Deadlock detection can be done in a distributed manner, with several nodes taking on parts of the task, instead of it being done at a single node. However, such algorithms are more complicated and more expensive. See the bibliographical notes for references to such algorithms.

23.3.3 Leases

One of the issues with using locking in a distributed system is that a node holding a lock may fail, and not release the lock. The locked data item could thus become (logically) inaccessible, until the failed node recovers and releases the lock, or the lock is released by another node on behalf of the failed node.

If an exclusive lock has been obtained on a data item, and the transaction is in the prepared state, the lock cannot be released until a commit/abort decision is made for the transaction. However, in many other cases it is acceptable for a lock that has been granted earlier to be revoked subsequently. In such cases, the concept of a lease can be very useful.

A **lease** is a lock that is granted for a specific period of time. If the process that acquires a lease needs to continue holding the lock beyond the specified period, it can **renew** the lease. A lease renewal request is sent to the lock manager, which extends the lease and responds with an acknowledgment as long as the renewal request comes in time. However, if the time expires, and the process does not renew the lease, the lease is said to **expire**, and the lock is released. Thus, any lease acquired by a node that either fails, or gets disconnected from the lock manager, is automatically released when the

lease expires. The node that holds a lease regularly compares the current lease expiry time with its local clock to determine if it still has the lease or the lease has expired.

One of the uses of leases is to ensure that there is only one coordinator for a protocol in a distributed system. A node that wants to act as coordinator requests an exclusive lease on a data item associated with the protocol. If it gets the lease, it can act as coordinator until the lease expires; as long as it is active, it requests lease renewal before the lease expires, and it continues to be the coordinator as long as the lock manager permits the lease renewal.

If a node N_1 acting as coordinator dies after the expiry of the lease period, the lease automatically expires, and another node N_2 that requests the lease can acquire it and become the coordinator. In most protocols it is important that there should be only one coordinator at a given time. The lease mechanism guarantees this, as long as clocks are synchronized. However, if the coordinator's clock runs slower than the lock manager's clock, a situation can arise where the coordinator thinks it still has the lease, while the lock manager thinks the lease has expired. While clocks cannot be exactly synchronized, in practice the inaccuracy is not very high. The lock manager waits for some extra wait time after the lease expiry time to account for clock inaccuracies before it actually treats the lease as expired.

A node that checks the local clock and decides it still has a lease may then take a subsequent action as coordinator. It is possible that the lease may have expired between when the clock was checked and when the subsequent action took place, which could result in the action taking place after the node is no longer the coordinator. Further, even if the action took place while the node had a valid lease, a message sent by the node may be delivered after a delay, by which time the node may have lost its lease. While it is possible for the network to deliver a message arbitrarily late, the system can decide on a maximum message delay time, and any message that is older is ignored by the recipient; messages have timestamps set by the sender, which are used to detect if a message needs to be ignored.

The time gaps due to the above two issues can be taken into account by checking that the lease expiry is at least some time t' into the future before initiating an action, where t' is a bound on how long the action will take after the lease time check, including the maximum message delay.

We have assumed here that while coordinators may fail, the lock manager that issues leases is able to tolerate faults. We study in Section 23.8.4 how to build a fault-tolerant lock manager; we note that the techniques described in that section are general purpose and can be used to implement fault-tolerant versions of any deterministic process, modeled as a "state machine."

23.3.4 Distributed Timestamp-Based Protocols

The principal idea behind the timestamp-based concurrency control protocols in Section 18.5 is that each transaction is given a *unique* timestamp that the system uses in

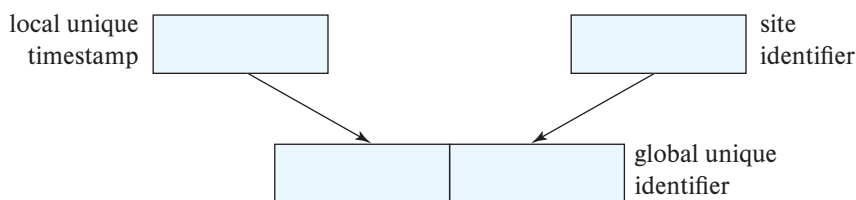


Figure 23.7 Generation of unique timestamps.

deciding the serialization order. Our first task, then, in generalizing the centralized scheme to a distributed scheme is to develop a scheme for generating unique timestamps. We then discuss how the timestamp-based protocols can be used in a distributed setting.

23.3.5 Generation of Timestamps

There are two primary methods for generating unique timestamps, one centralized and one distributed. In the centralized scheme, a single node distributes the timestamps. The node can use a logical counter or its own local clock for this purpose. While this scheme is easy to implement, failure of the node would potentially block all transaction processing in the system.

In the distributed scheme, each node generates a unique local timestamp by using either a logical counter or the local clock. We obtain the unique global timestamp by concatenating the unique local timestamp with the node identifier, which also must be unique (Figure 23.7). If a node has multiple threads running on it (as is almost always the case today), a thread identifier is concatenated with the node identifier, to make the timestamp unique. Further, we assume that consecutive calls to get the local timestamp within a node/thread will return different timestamps; if this is not guaranteed by the local clock, the returned local timestamp value may need to be incremented, to ensure two calls do not get the same local timestamp.

The order of concatenation is important! We use the node identifier in the least significant position to ensure that the global timestamps generated in one node are not always greater than those generated in another node.

We may still have a problem if one node generates local timestamps at a rate faster than that of the other nodes. In such a case, the fast node's logical counter will be larger than that of other nodes. Therefore, all timestamps generated by the fast node will be larger than those generated by other nodes. What we need is a mechanism to ensure that local timestamps are generated fairly across the system. There are two solution approaches for this problem.

1. Keep the clocks synchronized by using a network time protocol, which is a standard feature in computers today. The protocol periodically communicates with a

server to find the current time. If the local time is ahead of the time returned by the server, the local clock is slowed down, whereas if the local time is behind the time returned by the server it is speeded up, to bring it back in synchronization with the time at the server. Since all nodes are approximately synchronized with the server, they are also approximately synchronized with each other.

2. We define within each node N_i a **logical clock** (LC_i), which generates the unique local timestamp. The logical clock can be implemented as a counter that is incremented after a new local timestamp is generated. To ensure that the various logical clocks are synchronized, we require that a node N_i advance its logical clock whenever a transaction T_i with timestamp $\langle x, y \rangle$ visits that node and x is greater than the current value of LC_i . In this case, node N_i advances its logical clock to the value $x + 1$. As long as messages are exchanged regularly, the logical clocks will be approximately synchronized.

23.3.6 Distributed Timestamp Ordering

The timestamp ordering protocol can be easily extended to a parallel or distributed database setting. Each transaction is assigned a globally unique timestamp at the node where it originates. Requests sent to other nodes include the transaction timestamp. Each node keeps track of the read and write timestamps of the data items at that node. Whenever an operation is received by a node, it does the timestamp checks that we saw in Section 18.5.2, locally, without any need to communicate with other nodes.

Timestamps must be reasonably synchronized across nodes; otherwise, the following problem can occur. Suppose one node has a time significantly lagging the others, and a transaction T_1 gets its timestamp at that node n_1 . Suppose the transaction T_1 fails a timestamp test on a data item d_i because d_i has been updated by a transaction T_2 with a higher timestamp; T_1 would be restarted with a new timestamp, but if the time at node n_1 is not synchronized, the new timestamp may still be old enough to cause the timestamp test to fail, and T_1 would be restarted repeatedly until the time at n_1 advances ahead of the timestamp of T_2 .

Note that as in the centralized case, if a transaction T_i reads an uncommitted value written by another transaction T_j , T_i cannot commit until T_j commits. This can be ensured either by making reads wait for uncommitted writes to be committed, which can be implemented using locking, or by introducing commit dependencies, as discussed in Section 18.5. The waiting time can be exacerbated by the time required to perform 2PC, if the transaction performs updates at more than one node. While a transaction T_i is in the prepared state, its writes are not committed, so any transaction with a higher timestamp that reads an item written by T_i would be forced to wait.

We also note that the *multiversion timestamp ordering protocol* can be used locally at each node, without any need to communicate with other nodes, similar to the case of the timestamp ordering protocol.

23.3.7 Distributed Validation

We now consider the validation-based protocol (also called the optimistic concurrency control protocol) that we saw in Section 18.6. The protocol is based on three timestamps:

- The start timestamp $\text{StartTS}(T_i)$.
- The validation timestamp, $\text{TS}(T_i)$, which is used as the serialization order.
- The finish timestamp $\text{FinishTS}(T_i)$ which identifies when the writes of a transaction have completed.

While we saw a serial version of the validation protocol in Section 18.6, where only one transaction can perform validation at a time, there are extensions to the protocol to allow validations of multiple transactions to occur concurrently, within a single system.

We now consider how to adapt the protocol to a distributed setting.

1. Validation is done locally at each node, with timestamps assigned as described below.
2. In a distributed setting, the validation timestamp $\text{TS}(T_i)$ can be assigned at any of the nodes, but the same timestamp $\text{TS}(T_i)$ must be used at all nodes where validation is to be performed. Transactions must be serializable based on their timestamps $\text{TS}(T_i)$.
3. The validation test for a transaction T_i looks at all transactions T_j with $\text{TS}(T_j) < \text{TS}(T_i)$, to check if T_j either finished before T_i started, or has no conflicts with T_i . The assumption is that once a particular transaction enters the validation phase, no transaction with a lower timestamp can enter the validation phase. The assumption can be ensured in a centralized system by assigning the timestamps in a critical section, but cannot be ensured in a distributed setting.

A key problem in the distributed setting is that a transaction T_j may enter the validation phase after a transaction T_i , but with $\text{TS}(T_j) < \text{TS}(T_i)$. It is too late for T_i to be validated against T_j . However, this problem can be easily fixed by rolling back any transaction if, when it starts validation at a node, a transaction with a later timestamp had already started validation at that node.

4. The start and finish timestamps are used to identify transactions T_j whose writes would definitely have been seen by a transaction T_i . These timestamps must be assigned locally at each node, and must satisfy $\text{StartTS}(T_i) \leq \text{TS}(T_i) \leq \text{FinishTS}(T_i)$. Each node uses these timestamps to perform validation locally.
5. When used in conjunction with 2PC, a transaction must first be validated and then enter the prepared state. Writes cannot be committed at the database until

the transaction enters the committed state in 2PC. Suppose a transaction T_j reads an item updated by a transaction T_i that is in the prepared state and is allowed to proceed using the old value of the data item (since the value generated by T_i has not yet been written to the database). Then, when transaction T_j attempts to validate, it will be serialized after T_i and will surely fail validation if T_i commits. Thus, the read by T_j may as well be held until T_i commits and finishes its writes. The above behavior is the same as what would happen with locking, with write locks acquired at the time of validation.

Although full implementations of validation-based protocols are not widely used in distributed settings, optimistic concurrency control without read validation, which we saw in Section 18.9.3, is widely used in distributed settings. Recall that the scheme depends on storing a version number with each data item, a feature that is supported by many key-value stores.¹ Version numbers are incremented each time the data item is updated.

Validation is performed at the time of writing the data item, which can be done using a test-and-set function based on version numbers, that is supported by some key-value stores. This function allows an update to a data item to be conditional on the current version of the data item being the same as a specified version number. If the current version number of the data item is more recent than the specified version number, the update is not performed. For example, a transaction that read version 7 of a data item can perform a write, conditional on the version still being at 7. If the item has been updated meanwhile, the current version would not match, and the write would fail; however, if the version number is still 7, the write would be performed successfully, and the version number incremented to 8.

The test-and-set function can thus be used by applications to implement the limited form of validation-based concurrency control, discussed in Section 18.9.3, at the level of individual data items. Thereby, a transaction could read a value from a data item, perform computation locally, and update the data item at the end, as long as the value it read has not changed subsequently. This approach does not guarantee overall serializability, but it does prevent the lost-update anomaly.

HBase supports the test-and-set operation based on comparing values (similar to the hardware test-and-set operation), which is called `checkAndPut()`. Instead of comparing to a system-generated version number, the `checkAndPut()` invocation can pass in a column and a value; the update is performed only if the row has the specified value for the specified column. The check and the update are performed atomically. A variant, `checkAndMutate()`, allows multiple modifications to a row, such as adding or updating a column, deleting a column, or incrementing a column, after checking a condition, as a single atomic operation.

¹Note that this is not the same as multiversioning, since only one version needs to be stored.

23.4 Replication

One of the goals in using distributed databases is **high availability**; that is, the database must function almost all the time. In particular, since failures are more likely in large distributed systems, a distributed database must continue functioning even when there are various types of failures. The ability to continue functioning even during failures is referred to as **robustness**.

For a distributed system to be robust, data must be replicated, allowing the data to be accessible even if a node containing a replica of the data fails.

The database system must keep track of the locations of the replicas of each data item in the database catalog. Replication can be at the level of individual data items, in which the catalog will have one entry for each data item, recording the nodes where it is replicated. Alternatively, replication can be done at the level of partitions of a relation, with an entire partition replicated at two or more nodes. The catalog would then have one entry for each partition, resulting in considerably lower overhead than having one entry for each data item.

In this section we first discuss (in Section 23.4.1) issues with consistency of values between replicas. We then discuss (in Section 23.4.2) how to extend concurrency control techniques to deal with replicas, ignoring the issue of failures. Further extensions of the techniques to handle failures but modifying how reads and writes are executed are described in Section 23.4.3.

23.4.1 Consistency of Replicas

Given that a data item (or partition) is replicated, the system should ideally ensure that the copies have the same value. Practically, given that some nodes may be disconnected or may have failed, it is impossible to ensure that all copies have the same value. Instead, the system must ensure that even if some replicas do not have the latest value, reads of a data item get to see the latest value that was written.

More formally, the implementations of read and write operations on the replicas of a data item must follow a protocol that ensures the following property, called **linearizability**: Given a set of read and write operations on a data item,

1. there must be a linear ordering of the operations such that each read in the ordering should see the value written by the most recent write preceding the read (or the initial value if there is no such write), and
2. if an operation o_1 finishes before an operation o_2 begins (based on external time), then o_1 must precede o_2 in the linear order.

Note that linearizability only addresses what happens to a single data item, and it is orthogonal to serializability.

We first consider approaches that write all copies of a data item and discuss limitations of this approach; in particular, to ensure availability during failure, failed nodes need to be removed from the set of replicas, which can be quite tricky as we will see.

It is not possible, in general, to differentiate between node failure and network partition. The system can usually detect that a failure has occurred, but it may not be able to identify the type of failure. For example, suppose that node N_1 is not able to communicate with N_2 . It could be that N_2 has failed. However, another possibility is that the link between N_1 and N_2 has failed, resulting in network partition. The problem is partly addressed by using multiple links between nodes, so that even if one link fails the nodes will remain connected. However, multiple link failures can still occur, so there are situations where we cannot be sure whether a node failure or network partition has occurred.

There are protocols for data access that can continue working even if some nodes have failed, without any explicit actions to deal with the failures, as we shall see in Section 23.4.3.1. These protocols are based on ensuring a majority of nodes are written/read. With such protocols, actions to detect failed nodes and remove them from the system can be done in the background, and (re)integration of new or recovered nodes into the system can also be done without disrupting processing.

Although traditional database systems place a premium on consistency, there are many applications today that value availability more than consistency. The design of replication protocols is different for such systems and is discussed in Section 23.6.

In particular, one such alternative that is widely used for maintaining replicated data is to perform the update on a primary copy of the data item, and allow the transaction to commit without updating the other copies. However, the update is subsequently propagated to the other copies. Such propagation of updates, referred to as *asynchronous replication* or *lazy propagation of updates*, is discussed in Section 23.6.2.

One drawback of asynchronous replication is that replicas may be out of date for some time following each update. Another drawback is that if the primary copy fails after a transaction commits, but before the updates were propagated to the replicas, the updates of the committed transaction may not be visible to subsequent transactions, leading to an inconsistency.

On the other hand, a major benefit of asynchronous replication is that exclusive locks can be released as soon as the transaction commits on the primary copy. In contrast, if other replicas have to be updated before the transaction commits, there may be a significant delay in committing the transaction. In particular, if data is geographically replicated to ensure availability despite failure of an entire data center, the network round-trip time to a remote data center could range from tens of milliseconds to nearby locations, up to hundreds of milliseconds for data centers that are on the other side of the world. If a transaction were to hold a lock on a data item for this duration, the number of transactions that can update that data item would be limited to approximately 10 to 100 transactions per second. For certain applications, for example, user data in a web application, 10 to 100 transactions per second for a single data item is quite sufficient. However, for applications where some data items are updated

by a large number of transactions each second, holding locks for such a long time is not acceptable. Asynchronous replication may be preferred in such cases.

23.4.2 Concurrency Control with Replicas

We discuss several alternative ways of dealing with locking in the presence of replication of data items, in Section 23.4.2.1 to Section 23.4.2.4.

In this section, we assume updates are done on all replicas of a data item. If any node containing a replica of a data item has failed, or is disconnected from the other nodes, that replica cannot be updated. We discuss how to perform reads and updates in the presence of failures later, in Section 23.4.3.

23.4.2.1 Primary Copy

When a system uses data replication, we can choose one of the replicas of a data item as the **primary copy**. For each data item Q , the primary copy of Q must reside in precisely one node, which we call the **primary node** of Q .

When a transaction needs to lock a data item Q , it requests a lock at the primary node of Q . As before, the response to the request is delayed until it can be granted. The primary copy enables concurrency control for replicated data to be handled like that for unreplicated data. This similarity allows for a simple implementation. However, if the primary node of Q fails, lock information for Q would be lost, and Q would be inaccessible, even though other nodes containing a replica may be accessible.

23.4.2.2 Majority Protocol

The **majority protocol** works this way: If data item Q is replicated in n different nodes, then a lock-request message must be sent to more than one-half of the n nodes in which Q is stored. Each lock manager determines whether the lock can be granted immediately (as far as it is concerned). As before, the response is delayed until the request can be granted. The transaction does not operate on Q until it has successfully obtained a lock on a majority of the replicas of Q .

We assume for now that writes are performed on all replicas, requiring all nodes containing replicas to be available. However, the major benefit of the majority protocol is that it can be extended to deal with node failures, as we shall see in Section 23.4.3.1. The protocol also deals with replicated data in a decentralized manner, thus avoiding the drawbacks of central control. However, it suffers from these disadvantages:

- **Implementation.** The majority protocol is more complicated to implement than are the previous schemes. It requires at least $2(n/2 + 1)$ messages for handling lock requests and at least $(n/2 + 1)$ messages for handling unlock requests.
- **Deadlock handling.** In addition to the problem of global deadlocks due to the use of a distributed-lock-manager approach, it is possible for a deadlock to occur even if only one data item is being locked. As an illustration, consider a system with four nodes and full replication. Suppose that transactions T_1 and T_2 wish to lock

data item Q in exclusive mode. Transaction T_1 may succeed in locking Q at nodes N_1 and N_3 , while transaction T_2 may succeed in locking Q at nodes N_2 and N_4 . Each then must wait to acquire the third lock; hence, a deadlock has occurred. Luckily, we can avoid such deadlocks with relative ease by requiring all nodes to request locks on the replicas of a data item in the same predetermined order.

23.4.2.3 Biased Protocol

The **biased protocol** is another approach to handling replication. The difference from the majority protocol is that requests for shared locks are given more favorable treatment than requests for exclusive locks.

- **Shared locks.** When a transaction needs to lock data item Q , it simply requests a lock on Q from the lock manager at one node that contains a replica of Q .
- **Exclusive locks.** When a transaction needs to lock data item Q , it requests a lock on Q from the lock manager at all nodes that contain a replica of Q .

As before, the response to the request is delayed until it can be granted.

The biased scheme has the advantage of imposing less overhead on read operations than does the majority protocol. This savings is especially significant in common cases in which the frequency of read is much greater than the frequency of write. However, the additional overhead on writes is a disadvantage. Furthermore, the biased protocol shares the majority protocol's disadvantage of complexity in handling deadlock.

23.4.2.4 Quorum Consensus Protocol

The **quorum consensus** protocol is a generalization of the majority protocol. The quorum consensus protocol assigns each node a nonnegative weight. It assigns read and write operations on an item x two integers, called **read quorum** Q_r and **write quorum** Q_w , that must satisfy the following condition, where S is the total weight of all nodes at which x resides:

$$Q_r + Q_w > S \text{ and } 2 * Q_w > S$$

To execute a read operation, enough replicas must be locked that their total weight is at least Q_r . To execute a write operation, enough replicas must be locked so that their total weight is at least Q_w .

A benefit of the quorum consensus approach is that it can permit the cost of either read or write locking to be selectively reduced by appropriately defining the read and write quorums. For instance, with a small read quorum, reads need to obtain fewer locks, but the write quorum will be higher, hence writes need to obtain more locks. Also, if higher weights are given to some nodes (e.g., those less likely to fail), fewer

nodes need to be accessed for acquiring locks. In fact, by setting weights and quorums appropriately, the quorum consensus protocol can simulate the majority protocol and the biased protocols.

Like the majority protocol, quorum consensus can be extended to work even in the presence of node failures, as we shall see in Section 23.4.3.1.

23.4.3 Dealing with Failures

Consider the following protocol to deal with replicated data. Writes must be successfully performed at all replicas of a data item. Reads may read from any replica. When coupled with two-phase locking, such a protocol will ensure that reads will see the value written by the most recent write to the same data item. This protocol is also called the **read one, write all copies** protocol since all replicas must be written, and any replica can be read.

The problem with this protocol lies in what to do if some node is unavailable. To allow work to proceed in the event of failures, it may appear that we can use a “read one, write all available” protocol. In this approach, a read operation proceeds as in the **read one, write all** scheme; any available replica can be read, and a read lock is obtained at that replica. A write operation is shipped to all replicas, and write locks are acquired on all the replicas. If a node is down, the transaction manager proceeds without waiting for the node to recover. While this approach appears very attractive, it does not guarantee consistency of writes and reads. For example, a temporary communication failure may cause a node to appear to be unavailable, resulting in a write not being performed, but when the link is restored, the node is not aware that it has to perform some reintegration actions to catch up on writes it has lost. Further, if the network partitions, each partition may proceed to update the same data item, believing that nodes in the other partitions are all dead.

23.4.3.1 Robustness Using the Majority-Based Protocol

The majority-based approach to distributed concurrency control in Section 23.4.2.2 can be modified to work in spite of failures. In this approach, each data object stores with it a version number to detect when it was last written. Whenever a transaction writes an object it also updates the version number in this way:

- If data object a is replicated in n different nodes, then a lock-request message must be sent to more than one-half of the n nodes at which a is stored. The transaction does not operate on a until it has successfully obtained a lock on a majority of the replicas of a .

Updates to the replicas can be committed atomically using 2PC. (We assume for now that all replicas that were accessible stay accessible until commit, but we

relax this requirement later in this section, where we also discuss alternatives to 2PC.)

- Read operations look at all replicas on which a lock has been obtained and read the value from the replica that has the highest version number. (Optionally, they may also write this value back to replicas with lower version numbers.) Writes read all the replicas just like reads to find the highest version number (this step would normally have been performed earlier in the transaction by a read, and the result can be reused). The new version number is one more than the highest version number. The write operation writes all the replicas on which it has obtained locks and sets the version number at all the replicas to the new version number.

Failures (whether network partitions or node failures) can be tolerated as long as (1) the nodes available at commit contain a majority of replicas of all the objects written to and (2) during reads, a majority of replicas are read to find the version numbers. If these requirements are violated, the transaction must be aborted. As long as the requirements are satisfied, the two-phase commit protocol can be used, as usual, on the nodes that are available.

In this scheme, reintegration is trivial; nothing needs to be done. This is because writes would have updated a majority of the replicas, while reads will read a majority of the replicas and find at least one replica that has the latest version.

However, the majority protocol using version numbers has some limitations, which can be avoided by using extensions or by using alternative protocols.

1. The first problem is how to deal with the failure of participants during an execution of the two-phase commit protocol.

This problem can be dealt with by an extension of the two-phase commit protocol that allows commit to happen even if some replicas are unavailable, as long as a majority of replicas of a partition confirm that they are in prepared state. When participants recover or get reconnected, or otherwise discover that they do not have the latest updates, they need to query other nodes to catch up on missing updates. References that provide details of such solutions may be found in the bibliographic notes for this chapter, available online.

2. The second problem is how to deal with the failure of the coordinator during an execution of two-phase commit protocol, which could lead to the blocking problem. Consensus protocols, which we study in Section 23.8, provide a robust way of implementing two-phase commit without the risk of blocking even if the coordinator fails, as long as a majority of the nodes are up and connected, as we will see in Section 23.8.5.
3. The third problem is that reads pay a higher price, having to contact a majority of the copies. We study approaches to reducing the read overhead in Section 23.4.3.2.

23.4.3.2 Reducing Read Cost

One approach to dealing with this problem is to use the idea of read and write quorums from the quorum consensus protocol; reads can read from a smaller read quorum, while writes have to successfully write to a larger write quorum. There is no change to the version numbering technique described earlier. The drawback of this approach is that a higher write quorum increases the chance of blocking of update transactions, due to failure or disconnection of nodes. As a special case of quorum consensus, we give unit weights to all nodes, set the read quorum to 1, and set the write quorum to n (all nodes). This corresponds to the read-any-write-all protocol we saw earlier. There is no need to use version numbers with this protocol. However, if even a single node containing a data item fails, no write to the item can proceed, since the write quorum will not be available.

A second approach is to use the primary copy technique for concurrency control and force all updates to go through the primary copy. Reads can be satisfied by accessing only one node, in contrast to the majority or quorum protocols. However, an issue with this approach is how to handle failures. If the primary copy node fails, and another node is assigned to act as the primary copy, it must ensure that it has the latest version of all data items. Subsequently, reads can be done at the primary copy, without having to read data from other copies.

This approach requires that there be at most one node that can act as primary copy at a time, even in the event of network partitions. This can be ensured using leases as we saw earlier in Section 23.7. Furthermore, this approach requires an efficient way for the new coordinator to ensure that it has the latest version of all data items. This can be done by having a log at each node and ensuring the logs are consistent with each other. This problem is by itself a nontrivial process, but it can be solved using distributed consensus protocols which we study in Section 23.8. Distributed consensus internally uses a majority scheme to ensure consistency of the logs. But it turns out that if distributed consensus is used to keep logs synchronized, there is no need for version numbering.

In fact, consensus protocols provide a way of implementing fault-tolerant replication of data, as we see later in Section 23.8.4. Many fault-tolerant storage system implementations today are built using fault-tolerant replication of data based on consensus protocols.

There is a variant of the primary copy scheme, called the **chain replication** protocol, where the replicas are ordered. Each update is sent to the first replica, which records it locally and forwards it to the next replica, and so on. The update is completed when the last (tail) replica receives the update. Reads must be executed at the tail replica, to ensure that only updates that have been fully replicated are read. If a node in a replica chain fails, reconfiguration is required to update the chain; further, the system must ensure that any incomplete updates are completed before processing further updates. Optimized versions of the chain replication scheme are used in several storage systems.

References providing more details of the chain replication protocol may be found in the Further Reading section at the end of the chapter.

23.4.4 Reconfiguration and Reintegration

While nodes do fail, in most cases nodes recover soon, and the protocols described earlier can ensure that they will catch up with any updates that they missed.

However, in some cases a node may fail permanently. The system must then be **reconfigured** to remove failed nodes, and to allow other nodes to take over the tasks assigned to the failed node. Further, the database catalog must be updated to remove the failed node from the list of replicas of all data items (or relation partitions) that were replicated at that node.

As discussed earlier, a network failure may result in a node appearing to have failed, even if it has not actually failed. It is safe to remove such a node from the list of replicas; reads will no longer be routed to the node even though it may be accessible, but that will not cause any consistency problems.

If a failed node that was removed from the system eventually recovers, it must be **reintegrated** into the system. When a failed node recovers, if it had replicas of any partition or data item, it must obtain the current values of these data items it stores. The database recovery log at a live site can be used to find and perform all updates that happened when the node was down,

Reintegration of a node is more complicated than it may seem to be at first glance, since there may be updates to the data items processed during the time that the node is recovering. The database recovery log at a live site is used for catching up with the latest values for all data items at the node. Once it has caught up with the current value of all data items, the node should be added back into the list of replicas for the relevant partitions/data items, so it will receive all future updates. Locks are obtained on the partitions/data items, updates up to that point are applied from the log, and the node is added to the list of replicas for the partitions or data items, before releasing the locks. Subsequent updates will be applied directly to the node, since it will be in the list of replicas.

Reintegration is much easier with the majority-based protocols in Section 23.4.3.1, since the protocol can tolerate nodes with out-of-date data. In this case, a node can be reintegrated even before catching up on updates, and the node can catch up with missed updates subsequently.

Reconfiguration depends on nodes having an up-to-date version of the catalog that records what table partitions (or data items) are replicated at what nodes; thus information must be consistent across all nodes in a system. The replication information in the catalog could be stored centrally, and consulted on each access, but such a design would not be scalable since the central node would be consulted very frequently and would get overloaded. To avoid such a bottleneck, the catalog itself needs to be partitioned, and it may be replicated, for example, using the majority protocol.

23.5 Extended Concurrency Control Protocols

In this section, we describe further extensions to distributed concurrency control protocols. We first consider multiversion 2PL and how it can be extended to get globally consistent timestamps, in Section 23.5.1. Extensions of snapshot isolation to distributed settings are described in Section 23.5.2. Issues in concurrency control in heterogeneous distributed databases, where each node may have its own concurrency control technique, are described in Section 23.5.3.

23.5.1 Multiversion 2PL and Globally Consistent Timestamps

The multiversion two-phase locking (MV2PL) protocol, described in Section 18.7.2, combines the benefits of lock-free read-only transactions with the serializability guarantees of two-phase locking. Read-only transactions see a snapshot at a point in time, while update transactions use two-phase locking but create new versions of each data item that they update. Recall that with this protocol, each transaction T_i gets a unique timestamp $\text{CommitTS}(T_i)$ (which could be a counter, instead of actual time) at the time of commit. The transaction sets the timestamp of all items that it updates to $\text{CommitTS}(T_i)$. Only one transaction performs commit at a point in time; this guarantees that once T_i commits, a read-only transaction T_j whose $\text{StartTS}(T_j)$ is set to $\text{CommitTS}(T_i)$ will see committed values of all versions with timestamp $\leq \text{CommitTS}(T_i)$.

MV2PL can be extended to work in a distributed setting by having a central coordinator, that assigns start and commit timestamps and ensures that only one transaction can perform commit at a point in time. However, the use of a central coordinator limits scalability in a massively parallel data storage system.

The Google Spanner data storage system pioneered a version of the MV2PL system that is scalable and uses timestamps based on real clock time. We study the Spanner MV2PL implementation in the rest of this section.

Suppose every node has a perfectly accurate clock, and that commit processing can happen instantly with no delay between initiation of commit and its completion. Then, when a transaction wants to commit, it gets a commit timestamp by just reading the clock at any one node at any time after getting all locks, but before releasing any lock. All data item versions created by the transaction use this commit timestamp. Transactions can be serialized by this commit timestamp. Read-only transactions simply read the clock when they start and use it to get a snapshot of the database as of their start time.

If the clocks are perfectly accurate, and commit processing is instantaneous, this protocol can be used to implement MV2PL without any central coordination, making it very scalable.

Unfortunately, in the real world, the above assumptions do not hold, which can lead to the following problems:

1. Clocks are never perfectly accurate, and the clock at each node may be a little fast or a little slow compared to other clocks.

Thus, it is possible to have the following situation. Two update transactions T_1 and T_2 , both write a data item x , with T_1 writing it first, followed by T_2 but T_2 may end up with a lower commit timestamp because it got the timestamp at a different node than T_1 . This situation is not consistent with the serialization ordering of T_1 and T_2 , and it cannot happen with MV2PL in a centralized setting.

2. Commit processing takes time, which can cause read-only transactions to miss updates if the protocol is not carefully designed. Consider the following situation. A read-only transaction T_1 with start timestamp t_1 reads data item x at node N_1 , it is possible that soon after the read, another transaction T_2 with $\text{CommitTS}(T_2) \leq t_1$ (which got the commit timestamp at a different node N_2) may still perform a write on x . Then, T_1 should have read the value written by T_2 , but did not see it.

To deal with the first problem, namely, the lack of clock synchronization, Spanner uses the following techniques.

- Spanner has a few atomic clocks that are very accurate at each data center and uses the time they provide, along with time information from the Global Positioning system (GPS) satellites, which provides very accurate time information, to get a very good estimate of time at each node. We use the term *true time* to refer to the time that would have been given by an absolutely accurate clock.

Each node periodically communicates with time servers to synchronize its clock; if the clock has gone faster it is (logically) slowed down, and if it is slower, it is moved forward to the time from the server. In between synchronizations the local clock continues to tick, advancing the local time. A clock that ticks slower or faster than the correct rate results in local time at the node that is progressively behind or ahead of the true time.

- The second key technique is to measure local clock drift each time the node synchronizes with a time server and to use it to estimate the rate at which the local clock loses or gains time. Using this information, the Spanner system maintains a value ϵ such that if the local clock time is t' , the true time t is bounded by $t' - \epsilon \leq t \leq t' + \epsilon$. The Spanner system is able to keep the uncertainty value ϵ to less than 10 msec typically. The TrueTime API used by Spanner allows the system to get the current time value, along with an upper bound on the uncertainty in the time value.
- The next piece of the solution is an idea called **commit wait**. The idea is as follows: After all locks have been acquired at all nodes, the local time t' is read at a coordinator node. We would like to use the true time as timestamp, but we don't have the exact value. Instead, the highest possible value of true time, namely, $t' + \epsilon$, is used as a commit timestamp t_c . The transaction then waits, *while holding locks*, until it

is sure that the true time t is $\geq t_c$; this just requires waiting for a time interval 2ϵ , calculated as described earlier.

What the commit wait guarantees is that if a transaction T_1 has a commit timestamp t_c , at the true time t_c all locks were held by T_1 .

- Given the above, if a version x_t of a data item x has a timestamp t , we can say that that was indeed the value of x at true time t . This allows us to define a snapshot of the database at a time t , containing the latest versions of all data items as of time t . A database system is said to provide **external consistency** if the serialization order is consistent with the real-world time ordering in which the transactions commit. Spanner guarantees external consistency by ensuring that the timestamps used to define the transaction serialization order correspond to the true time when the transactions commit.
- One remaining issue is that transaction commit processing takes time (particularly so when 2PC is used). While a transaction with commit timestamp t is committing, a read of x by a read-only transaction with timestamp $t_1 \geq t$ may not see the version x_t , either because the timestamp has not yet been propagated to the node with the data item x , or the transaction is in prepared state.

To deal with this problem, reads that ask for a snapshot as of time t_1 are made to wait until the system is sure that no transactions with timestamp $\leq t_1$ are still in the process of committing. If a transaction with timestamp $t \leq t_1$ is currently in the prepared phase of 2PC, and we are not sure whether it will commit or abort, a read with timestamp t_1 would have to wait until we know the final commit status of the transaction.

Read-only transactions can be given a somewhat earlier timestamp, to guarantee that they will not have to wait; the trade-off here is that to avoid waiting, the transaction may not see the latest version of some data items.

23.5.2 Distributed Snapshot Isolation

Since snapshot isolation is widely used, extending it to work in a distributed setting is of significant practical importance. Recall from Section 18.8 that while snapshot isolation does not guarantee serializability, it avoids a number of concurrency anomalies.

If each node implements snapshot isolation independently, the resultant schedules can have anomalies that cannot occur in a centralized system. For example, suppose two transactions, T_1 and T_2 run concurrently on node N_1 , where T_1 writes x and T_2 reads x ; thus T_2 would not see updates made by T_1 to x . Suppose also that T_1 updates a data item y at node N_2 , and commits, and subsequently T_2 reads y at node N_2 . Then T_2 would see the value of y updated by T_1 at N_2 , but not see T_1 's update to x at N_1 . Such a situation could never occur when using snapshot isolation at a single node. Thus, just depending on local enforcement of snapshot isolation at each node is not sufficient to enforce snapshot isolation across nodes.

Several alternative distributed snapshot isolation protocols have been proposed in the literature. Since the protocols are somewhat complicated, we omit details, but references with more details may be found in the bibliographic notes for this chapter, available online. Some of these protocols allow local transactions at each node to execute without any global coordination step; an extra cost is paid only by global transactions, that is, transactions that execute at more than one node. These protocols have been prototyped on several databases/data storage systems, such as SAP HANA and HBase.

There has been some work on extending distributed snapshot isolation protocols to make them serializable. Approaches explored include adding timestamp checks similar to timestamp ordering, creating a transaction dependency graph at a central server, and checking for cycles in the graph, among other approaches.

23.5.3 Concurrency Control in Federated Database Systems

Recall from Section 20.5 that in many cases a distributed database has to be constructed by linking together multiple already-existing database systems, each with its own schema and possibly running different database-management software. Recall that such systems are called *federated database systems* or *heterogeneous distributed database systems*, and they consist of a layer of software on top of the existing database systems.

Transactions in a federated database may be classified as follows:

1. **Local transactions.** These transactions are executed by each local database system outside of the federated database system's control.
2. **Global transactions.** These transactions are executed under the control of the federated database system.

The federated database system is aware of the fact that local transactions may run at the local nodes, but it is not aware of what specific transactions are being executed, or of what data they may access.

Ensuring the local autonomy of each database system requires that no changes be made to its software. A database system at one node thus is not able to communicate directly with one at any other node to synchronize the execution of a global transaction active at several nodes.

Since the federated database system has no control over the execution of local transactions, each local system must use a concurrency-control scheme (e.g., two-phase locking or timestamping) to ensure that its schedule is serializable. In addition, in the case of locking, the local system must be able to guard against the possibility of local deadlocks.

The guarantee of local serializability is not sufficient to ensure global serializability. As an illustration, consider two global transactions T_1 and T_2 , each of which accesses and updates two data items, A and B , located at nodes N_1 and N_2 , respectively. Suppose that the local schedules are serializable. It is still possible to have a situation where, at

node N_1 , T_2 follows T_1 , whereas, at N_2 , T_1 follows T_2 , resulting in a nonserializable global schedule. Indeed, even if there is no concurrency among global transactions (i.e., a global transaction is submitted only after the previous one commits or aborts), local serializability is not sufficient to ensure global serializability (see Practice Exercise 23.11).

Depending on the implementation of the local database systems, a global transaction may not be able to control the precise locking behavior of its local subtransactions. Thus, even if all local database systems follow two-phase locking, it may be possible only to ensure that each local transaction follows the rules of the protocol. For example, one local database system may commit its subtransaction and release locks, while the subtransaction at another local system is still executing. If the local systems permit control of locking behavior and all systems follow two-phase locking, then the federated database system can ensure that global transactions lock in a two-phase manner and the lock points of conflicting transactions would then define their global serialization order. If different local systems follow different concurrency-control mechanisms, however, this straightforward sort of global control does not work.

There are many protocols for ensuring consistency despite the concurrent execution of global and local transactions in federated database systems. Some are based on imposing sufficient conditions to ensure global serializability. Others ensure only a form of consistency weaker than serializability but achieve this consistency by less restrictive means.

There are several schemes to ensure global serializability in an environment where update transactions as well as read-only transactions can execute. Several of these schemes are based on the idea of a **ticket**. A special data item called a ticket is created in each local database system. Every global transaction that accesses data at a node must write the ticket at that node. This requirement ensures that global transactions conflict directly at every node they visit. Furthermore, the global transaction manager can control the order in which global transactions are serialized, by controlling the order in which the tickets are accessed. References to such schemes appear in the bibliographic notes for this chapter, available online.

23.6 Replication with Weak Degrees of Consistency

The replication protocols we have seen so far guarantee consistency, even if there are node and network failures. However, these protocols have a nontrivial cost, and further they may block if a significant number of nodes fail or get disconnected due to a network partition. Further, in the case of a network partition, a node that is not in the majority partition would not only be unable to perform writes, but it would also be unable to perform even reads.

Many applications wish to have higher availability, even at the cost of consistency. We study the trade-offs between consistency and availability in this section.

23.6.1 Trading Off Consistency for Availability

The protocols we have seen so far require a (weighted) majority of nodes be in a partition for updates to proceed. Nodes that are in a minority partition cannot process updates; if a network failure results in more than two partitions, no partition may have a majority of nodes. Under such a situation, the system would be completely unavailable for updates, and depending on the read-quorum, may even become unavailable for reads. The write-all-available protocol which we saw earlier provides availability but not consistency.

Ideally, we would like to have consistency and availability, even in the face of partitions. Unfortunately, this is not possible, a fact that is crystallized in the so-called **CAP theorem**, which states that any distributed database can have at most two of the following three properties:

- Consistency.
- Availability.
- Partition-tolerance.

The proof of the CAP theorem uses the following definition of consistency, with replicated data: an execution of a set of operations (reads and writes) on replicated data is said to be **consistent** if its result is the same as if the operations were executed on a single node, in a sequential order that is consistent with the ordering of operations issued by each process (transaction). The notion of consistency is similar to atomicity of transactions, but with each operation treated as a transaction, and is weaker than the atomicity property of transactions.

In any large-scale distributed system, partitions cannot be prevented, and as a result, either availability or consistency has to be sacrificed. The schemes we have seen earlier sacrifice availability for consistency in the face of partitions.

Consider a web-based social-networking system that replicates its data on three servers, and a network partition occurs that prevents the servers from communicating with each other. Since none of the partitions has a majority, it would not be possible to execute updates on any of the partitions. If one of these servers is in the same partition as a user, the user actually has access to data, but would be unable to update the data, since another user may be concurrently updating the same object in another partition, which could potentially lead to inconsistency. Inconsistency is not as great a risk in a social-networking system as in a banking database. A designer of such a system may decide that a user who can access the system should be allowed to perform updates on whatever replicas are accessible, even at the risk of inconsistency.

In contrast to systems such as banking databases that require the ACID properties, systems such as the social-networking system mentioned above are said to require the **BASE** properties:

- Basically available.
- Soft state.
- Eventually consistent.

The primary requirement is availability, even at the cost of consistency. Updates should be allowed, even in the event of partitioning, following, for example, the write-all-available protocol (which is similar to multimaster replication described in Section 23.6). Soft state refers to the property that the state of the database may not be precisely defined, with each replica possibly having a somewhat different state due to partitioning of the network. Eventually consistent is the requirement that once a partitioning is resolved, eventually all replicas will become consistent with each other.

This last step requires that inconsistent copies of data items be identified; if one is an earlier version of the other, the earlier version can be replaced by the later version. It is possible, however, that the two copies were the result of independent updates to a common base copy. A scheme for detecting such inconsistent updates, called the version-vector scheme, is described in Section 23.6.4.

Restoring consistency in the face of inconsistent updates requires that the updates be merged in some way that is meaningful to the application. We discuss possible solutions for resolution of conflicting updates, in Section 23.6.5.

In general, no system designer wants to deal with the possibility of inconsistent updates and the resultant problems of detection and resolution. Where possible, the system should be kept consistent. Inconsistent updates are allowed only when a node is disconnected from the network, in applications that can tolerate inconsistency.

Some key-value stores such as Apache Cassandra and MongoDB allow an application to specify how many replicas need to be accessible to carry out a write operation or a read operation. As long as a majority of replicas are accessible, there is no problem with consistency for writes. However, if the application sets the required number at less than a majority, and many replicas are inaccessible, updates are allowed to go ahead; there is, however, a risk of inconsistent updates, which must be resolved later.

For applications where inconsistency can cause significant problems, or is harder to resolve, system designers prefer to build fault-tolerant systems using replication and distributed consensus that avoid inconsistencies, even at the cost of potential non-availability.

23.6.2 Asynchronous Replication

Many relational database systems support replication with weak consistency, which can take one of several forms.

With **asynchronous replication** the database allows updates at a *primary* node (also referred to as the **master** node) and propagates updates to replicas at other nodes subsequently; the transaction that performs the update can commit once the update is performed at the primary, even before replicas are updated. Propagation of updates after commit is also referred to as **lazy propagation**. In contrast, the term **synchronous replication** refers to the case where updates are propagated to other replicas as part of a single transaction.

With asynchronous replication, the system must guarantee that once the transaction commits at the primary, the updates are eventually propagated to all replicas, even if there are system failures in between. Later in this section, we shall see how this property is guaranteed using persistent messaging.

Since propagation of updates is done asynchronously, a read at a replica may not get the latest version of a data item. Asynchronous propagation of updates is commonly used to allow update transactions to commit quickly, even at the cost of consistency. A system designer may choose to use replicas only for fault tolerance. However, if the replica is available on a local machine, or another machine that can be accessed with low latency, it may be much cheaper to read the data item at the replica instead of reading it from the primary, as long as the application is willing to accept potentially stale data values.

Data storage systems based on asynchronous replication may allow data items to have versions, with associated timestamps. A transaction may then request a version with required freshness properties, for example not more than 10 minutes old. If a local replica has a version of the data item satisfying the freshness criterion, it can be used; otherwise, the read may have to be sent to the primary node.

Consider, for example, an airline reservation site that shows the prices of multiple flight options. Prices may vary frequently, and the system does not guarantee that a user will actually be able to book a ticket at the price shown initially. Thus, it is quite acceptable to show a price that is a few minutes old. Asynchronous replication is a good solution for this application: price data can be replicated to a large number of servers, which share the load of user queries; and price data are updated at a primary node and replicated asynchronously to all other replicas.

Multiversion concurrency control schemes can be used to give a *transaction-consistent snapshot* of the database to read-only transactions that execute at a replica; that is, the transaction should see all updates of all transactions up to some transaction in the serialization order and should not see any updates of transactions later in the serialization order. The multiversion 2PL scheme, described in Section 23.5.1, can be extended to allow a read-only transaction to access a replica that may not have up-to-date versions of some data items, but still get a transaction-consistent snapshot view of the database. To do so, replicas must be aware of what is the latest timestamp t_{safe} such that they have received all updates with commit timestamp before t . Any read of a snapshot with timestamp $t < t_{safe}$ can be processed by that replica. Such a scheme is used in the Google Spanner database,

Asynchronous replication is used in traditional (centralized) databases to create one or more replicas of the database, on which large queries can be executed, without interfering with transactions running on a primary node. Such replication is referred to **master-slave replication**, since the replicas cannot perform any updates on their own but must only perform updates that the master node asks them to perform.

In such systems, asynchronous propagation of updates is typically done in a continuous fashion to minimize delays until an update is seen at a replica. However, in data warehouses, updates may be propagated periodically—every night, for example—so that update propagation does not interfere with query processing.

Some database systems support **multimaster replication** (also called **update-anywhere replication**); updates are permitted at any replica of a data item and are propagated to all replicas either synchronously, using two-phase commit, or asynchronously.

Asynchronous replication is also used in some distributed storage systems. Such systems partition data, as we have seen earlier, but replicate each partition. There is a primary node for each partition, and updates are typically sent to the primary node, which commits the updates locally, and propagates them asynchronously to the other replicas of the partition. Some systems such as PNUTS even allow each data item in a partition to specify which node should act as the primary node for that data item; that node is responsible for committing updates to the data item, and propagating the update to the other replicas. The motivation is to allow a node that is geographically close to a user to act as the primary node for data items corresponding to that user.

In any system supporting asynchronous propagation of updates, it is important that once an update is committed at the primary, it must definitely be delivered to the other replicas. If there are multiple updates at a primary node, they must be delivered in the same order to the replicas; out-of-order delivery can cause an earlier update to arrive late and overwrite a later update.

Persistent messaging, which we saw in Section 23.2.3, provides guaranteed delivery of messages and is widely used for asynchronous replication. The implementation techniques for persistent messages described in Section 23.2.3 can be easily modified to ensure that messages are delivered in the order in which they were sent. With persistent messaging, each primary node needs to be aware of the location of all the replicas.

Publish-subscribe systems, which we saw in Section 22.8.1, offer a more flexible way of ensuring reliable message delivery. Recall that publish-subscribe systems allow messages to be published with an associated topic, and subscribers can subscribe to any desired topic. To implement asynchronous replication, a topic is created corresponding to each partition. All replicas of a partition subscribe to the topic corresponding to the partition. Any update (including inserts, deletes, and data item updates) to a partition is published as a message with the topic corresponding to the partition. The publish-subscribe system ensures that once such a message is published, it will be delivered to all subscribers in the order in which it was published.

Publish-subscribe systems designed for parallel systems, such as the Apache Kafka system, or the Yahoo Message Bus service used for asynchronous replication in the PNUTS distributed data storage system, allow a large number of topics, and use mul-

multiple servers to handle messages to different topics in parallel. Thus, asynchronous replication can be made scalable.

Fault tolerance is an issue with asynchronous propagation of updates. If a primary node fails, a new node must take over as primary; this can be done either using an election algorithm, as we saw earlier or by having a master node (which is itself chosen by election) decide which node takes over the job of a failed primary node.

Consider what happens if a primary copy records an update but fails before the update is sent to the replicas. The new primary node has no way of finding out what was the last update committed at the primary copy. It can either wait for the primary to recover, which is unacceptable, or it can proceed without knowing what updates were committed just before failure. In the latter case, there is a risk that a transaction on the new primary may read an old value of a data item or perform an update that conflicts with an earlier update on the old primary.

To reduce the chance of such problems, some systems replicate the log records of the primary node to a backup node and allow the transaction to commit at the primary only after the log record has been successfully replicated at the backup node; if the primary node fails, the backup node takes over as the primary. Recall that this is the two-safe protocol from Section 19.7. This protocol is resilient to failure of one node, but not to the failure of two nodes.

If an application is built on top of a storage system using asynchronous replication, applications may potentially see some anomalous behaviors such as a read not seeing the effect of an earlier write done by the same application, or a later read seeing an earlier version of a data item than an earlier read, if different reads and writes are sent to different replicas. While such anomalies cannot be completely prevented in the event of failures, they can be avoided during normal operation by taking some precautions. For example, if read and write requests for a data item from a particular node are always sent to the same replica, the application will see any writes it has performed, and if two reads are performed on the same data item, the later read will see a version at least as new as the earlier read. This property is guaranteed if a primary replica is used to perform all actions on a data item.

23.6.3 Asynchronous View Maintenance

Indices and materialized views are forms of data derived from underlying data, and can thus be viewed as forms of replicated data. Just like replicas, indices and materialized views could be updated (maintained) as part of each transaction that updates the underlying data; doing so would ensure consistency of the derived data with the underlying data.

However, many systems prefer to perform index and view maintenance in an asynchronous manner, to reduce the overhead on transactions that update the underlying data. As a result, the indices and materialized views could be out of date. Any transac-

tion that uses such indices or materialized views must be aware that these structures may be out of date.

We now consider how to maintain indices and materialized views in the face of concurrent updates.

- The first requirement for view maintenance is for the subsystem that performs maintenance to receive information about updates to the underlying data in such a way that each update is delivered exactly once, despite failures.

Publish-subscribe systems are a good match for the first requirement above. All updates to any underlying relation are published to the pub-sub system with the relation name as the topic; the view maintenance subsystem subscribes to the topics corresponding to its underlying relations and received all relevant updates. As we saw in Section 22.8.1, we can have topics corresponding to each tablet of a stored relation. For a nonmaterialized intermediate relation that is partitioned, we can have a topic corresponding to each partition.

- The second requirement is for the subsystem to update the derived data in such a way that the derived data will be consistent with the underlying data, despite concurrent updates to the underlying data.

Since the underlying data may receive further updates as an earlier update is being processed, no asynchronous view maintenance technique can guarantee that the view state is consistent with the state of the underlying data at all times. However, the consistency requirement can be formalized as follows: if there are no updates to the underlying data for a sufficient amount of time, asynchronous maintenance must ensure that the derived data is consistent with the underlying data; such a requirement is known as an **eventual consistency** requirement.

The technique for parallel maintenance of materialized views which we saw in Section 22.7.5 uses the exchange operator model to send updates to nodes and allows view maintenance to be done locally. Techniques designed for view maintenance in a centralized setting can be used at each node, on locally materialized input data. Recall from Section 16.5.1 that view maintenance may be deferred, that is, it may be done after the transaction commits. Techniques for deferred view maintenance in a centralized setting already need to deal with concurrent updates; such techniques can be used locally at each node.

- A third requirement is for reads to get a consistent view of data. In general, a query that reads data from multiple nodes may not observe the updates of a transaction T on node N_1 , but may see the updates that T performed on node N_2 , thus seeing a transactionally inconsistent view of data. Systems that use asynchronous replication typically do not support transactionally consistent views of the database.

Further, scans of the database may not see an operation-consistent view of the database. (Recall the notion of operation consistency from Section 18.9, which requires that any operation should not see a database state that reflects only some of the updates of another operation. In Section 18.9 we saw an example of a scan

using an index that could see two versions, or neither version, of a record updated by a concurrent transaction, if the relation scan does not follow two-phase locking. A similar problem occurs with asynchronous propagation of updates, even if both the relation scan and the update transaction follow two-phase locking.

For example, consider a relation $r(A, B, C)$, with primary key A , which is partitioned on attribute B . Now consider a query that is scanning the relation r . Suppose there is a concurrent update to a tuple $t_1 \in r$, which updates attribute $t_1.B$ from v_1 to v_2 . Such an update requires deletion of the old tuple from the partition corresponding to value v_1 , and insertion of the new tuple in the partition corresponding to v_2 . These updates are propagated asynchronously.

Now, the scan of r could possibly scan the node corresponding to v_1 after the old tuple is deleted there but visit the node corresponding to v_2 before the asynchronous propagation inserts the updated tuple in that node. Then, the scan would completely miss the tuple, even though it should have seen either the old value or the new value of t_1 . Further, the scan could visit the node corresponding to v_1 before the delete is propagated to that node, and the node corresponding to v_2 after the insert is propagated to that node, and thereby see two versions of t_1 , one from before the update and one from after the update. Neither case would be possible with two-phase locking, if updates are propagated synchronously to all copies.

If a multiversion concurrency control technique is used, where data items have timestamps, snapshot reads are a good way to get a consistent scan of a relation; the snapshot timestamp should be set a sufficiently old value that all updates as of that timestamp have reached all replicas.

23.6.4 Detecting Inconsistent Updates

Many applications developed for such high availability are designed to continue functioning locally even when the node running the application is disconnected from the other nodes.

As an example, when data are replicated, and the network gets partitioned, if a system chooses to trade off consistency to get availability, updates may be done concurrently at multiple replicas. Such conflicting updates need to be detected and resolved. When a connection is re-established, the application needs to communicate with a storage system to send any updates done locally and fetch updates performed elsewhere. There is a potential for conflicting updates from different nodes. For example, node N_1 may update a locally cached copy of a data item while it is disconnected; concurrently another node may have updated the data item on the storage system, or may have updated its own local copy of the data item. Such conflicting updates must be detected, and resolved.

As another example, consider an application on a mobile device that supports offline updates (i.e., permits updates even if the mobile device is not connected to the

network). To give the user a seamless usage experience, such applications perform the updates on a locally cached copy, and then apply the update to the data store when the device goes back online. If the same data item may be updated from multiple devices, the problem of conflicting updates arises here, too. The schemes described below can be used in this context too, with nodes understood to also refer to mobile devices.

A mechanism for detecting conflicting updates is described in this section. How to resolve conflicting updates once they are detected is application dependent, and there is no general technique for doing so. However, some commonly used approaches are discussed in Section 23.6.5.

For data items updated by only one node, it is a simple matter to propagate the updates when the node gets reconnected to the storage system. If the node only caches read-only copies of data that may be updated by other nodes, the cached data may become inconsistent. When the node gets reconnected, it can be sent **invalidation reports** that inform it of out-of-date cache entries.

However, if updates can occur at more than one node, detecting conflicting updates is more difficult. Schemes based on **version numbering** allow updates of shared data from multiple nodes. These schemes do not guarantee that the updates will be consistent. Rather, they guarantee that, if two nodes independently update the same version of a data item, the clash will be detected eventually, when the nodes exchange information either directly or through a common node.

The **version-vector scheme** detects inconsistencies when replicas of a data item are independently updated. This scheme allows copies of a data item to be stored at multiple nodes.

The basic idea is for each node i to store, with its copy of each data item d , a **version vector**—that is, a set of version numbers $\{V[j]\}$, with one entry for each other node j on which the data item could potentially be updated. When a node i updates a data item d , it increments the version number $V[i]$ by one.

For example, suppose a data item is replicated at nodes N_1 , N_2 and N_3 . If the item is initially created at N_1 , the version vector could be $[1, 0, 0]$. If it is then replicated at N_2 , and then updated at node N_2 , the resultant version vector would be $[1, 1, 0]$. Suppose now that this version of the data item is replicated to N_3 , and then both N_2 and N_3 concurrently update the data item. Then, the version vector of the data item at N_2 would be $[1, 2, 0]$, while the version vector at N_3 would be $[1, 1, 1]$.

Whenever two nodes i and j connect with each other, they exchange updated data items, so that both obtain new versions of the data items. However, before exchanging data items, the nodes have to discover whether the copies are consistent:

1. If the version vectors V_i and V_j of the copy of the data item at nodes i and j are the same—that is, for each k , $V_i[k] = V_j[k]$ —then the copies of data item d are identical.
2. If, for each k , $V_i[k] \leq V_j[k]$ and the version vectors are not identical, then the copy of data item d at node i is older than the one at node j . That is, the copy of

data item d at node j was obtained by one or more modifications of the copy of the data item at node i . Node i replaces its copy of d , as well as its copy of the version vector for d , with the copies from node j .

In our example above, if N_1 had the vector $[1, 0, 0]$ for a data item, while N_2 had the vector $[1, 1, 0]$, then the version at N_2 is newer than the version at N_1 .

3. If there are a pair of values k and m such that $V_i[k] < V_j[k]$ and $V_i[m] > V_j[m]$, then the copies are *inconsistent*; that is, the copy of d at i contains updates performed by node k that have not been propagated to node j , and, similarly, the copy of d at j contains updates performed by node m that have not been propagated to node i . Then, the copies of d are inconsistent, since two or more updates have been performed on d independently.

In our example, after the concurrent updates at N_2 and N_3 , the two version vectors show the updates are inconsistent. Let V_2 and V_3 denote the version vectors at N_2 and N_3 . Then $V_2[2] = 2$ while $V_3[2] = 1$, whereas $V_2[3] = 0$, while $V_3[3] = 1$.

Manual intervention may be required to merge the updates. After merging the updates (perhaps manually), the version vectors are merged, by setting $V[k]$ to the maximum of $V_i[k]$ and $V_j[k]$ for each k . The node l that performs the write then increments $V[l]$ by 1 and then writes the data item and its version vector V .

The version-vector scheme was initially designed to deal with failures in distributed file systems. The scheme gained importance because mobile devices often store copies of data that are also present on server systems. The scheme is also widely used in distributed storage systems that allow updates to happen even if a node is not in a majority partition.

The version-vector scheme cannot solve the problem of how to reconcile inconsistent copies of data detected by the scheme. We discuss reconciliation in Section 23.6.5.

The version-vector scheme works well for detecting inconsistent updates to a single data item. However, if a storage system has a very large number of replicated items, finding which items have been inconsistently updated can be quite expensive if done naively. In Section 23.6.6 we study a data structure called a Merkle tree that can efficiently detect differences between sets of data items.

23.6.5 Resolving Conflicting Updates

Detection of conflicting updates may happen when a read operation fetches copies of a data item from multiple replicas or when the system executes a background process that compares data item versions.

At that point, conflicting updates on the same data item need to be **resolved**, to create a single common version. Resolution of conflicting updates is also referred to as **reconciliation**.

There is no technique for resolution so that can be used across all applications. We discuss some techniques that have been used in several commonly used applications.

Many applications can perform reconciliation automatically by executing on each node all update operations that had been performed on other nodes during a period of disconnection. This solution requires that the system keep track of operations, for example, adding an item to a shopping cart, or deleting an item from a shopping cart. This solution works if operations **commute**—that is, they generate the same result, regardless of the order in which they are executed. The addition of items to a shopping cart clearly commutes. Deletions do not commute with additions in general, which should be clear if you consider what happens if an addition of an item is exchanged with a delete of the same item. However, as long as deletion always operates only on items already present in the cart, this problem does not arise.

As another example, many banks allow customers to withdraw money from an ATM even if it is temporarily disconnected from the bank network. When the ATM gets reconnected, the withdrawal operation is applied to the account. Again, if there are multiple withdrawals, they may get merged in an order different from the order in which they happened in the real world, but the end result (balance) is the same. Note that since the operation already took place in the physical world, it cannot be rejected because of a negative balance; the fact that an account has a negative balance has to be dealt with separately.

There are other application-specific solutions for resolving conflicting updates. In the worst case, however, a system may need to alert humans to the conflicting updates, and let the humans decide how to resolve the conflict.

Dealing with such inconsistency automatically, and assisting users in resolving inconsistencies that cannot be handled automatically, remains an area of research.

23.6.6 Detecting Differences Between Collections Using Merkle Tree

The **Merkle tree** (also known as **hash tree**) is a data structure that allows efficient detection of differences between sets of data items that may be stored at different replicas. (To avoid confusion between tree nodes and system nodes, we shall refer to the latter as replicas in this section.)

Detecting items that have inconsistent values across replicas due to weak consistency is merely one of motivations for Merkle trees. Another motivation is performing sanity checks of replicas that are synchronously updated, and should be consistent, but may be inconsistent due to bugs or other failures. We consider below a binary version of the Merkle tree.

We assume that each data item has a key and a value; in case we are considering collections that do not have an explicit key, the data item value itself can be used as a key.

Each data item key k_i is hashed by a function $h_1()$ to get a hash value with n bits, where n is chosen such that 2^n is within a small factor of the number of data items. Each data item value v_i is hashed by another function $h_2()$ to get a hash value (which

is typically much longer than n bits). Finally, we assume a hash function $h_3()$ which takes as input a collection of hash values and returns a hash value computed from the collection (this hash function must be computed in a way that does not depend on the input order of the hash values, which can be done, for example, by sorting the collection before computing the hash function).

Each node of a Merkle tree has associated with it an identifier and stores a hash value. Each leaf of the tree can be identified by an n -bit binary number. For a given leaf identified by number k , consider the set of all data items i whose key k_i is such that $h_1(k_i) = k$. Then, the hash value v_k stored at leaf k is computed by applying $h_2()$ on each of the data item values v_i , and then applying $h_3()$ on the resultant collection of hash values. The system also maintains an index that can retrieve all the data items with a given hash value computed by function $h_2()$.

Figure 23.8 shows an example of a Merkle tree on 8 data items. The hash value of these data items on h_1 are shown on the left. Note that if for an item i_j , $h_1(i_j) = k$, then the data item i_j is associated with the leaf with identifier k .

Each internal node of the Merkle tree is identified by a hash value that is j bits long if the node is at depth j ; leaves are at depth n , and the root at depth 0. The internal node identified by a number k has as children nodes identified by $2k$ and $2k + 1$. The hash value stored v_k at node k is computed by applying $h_3()$ to the hash value stored at nodes $2k$ and $2k + 1$.

Now, suppose this Merkle tree is constructed on the data at two replicas (the replicas may be whole database replicas, or replicas of a partition of the database). If all items at the two replicas are identical, the stored hash values at the root nodes will also be identical.

As long as $h_2()$ computes a long enough hash value, and is suitably chosen, it is very unlikely that $h_2(v_1) = h_2(v_2)$ if $v_1 \neq v_2$, and similarly for $h_3()$. The SHA1 hash function with a 160-bit hash value is an example of a hash function that satisfies this

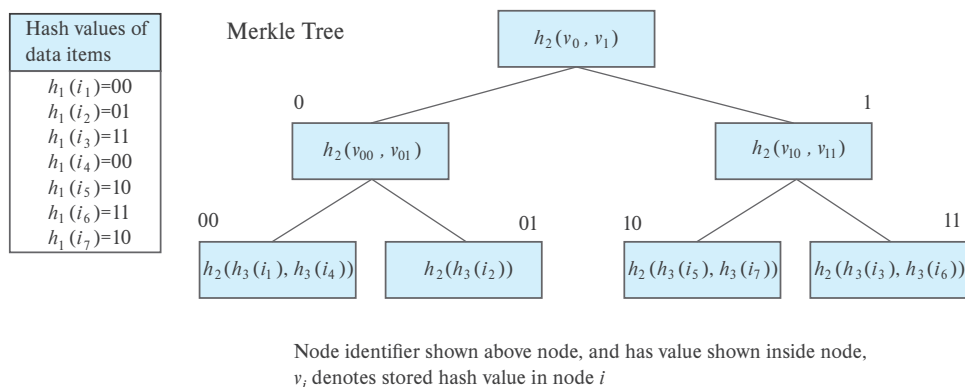


Figure 23.8 Example of Merkle tree.

requirement. Thus, we can assume that if two nodes have the same stored hash values, all the data items under the two nodes are identical.

If, in fact, there is a difference in the value of any items at the two replicas, or if an item is present at one replica but not at the other, the stored hash values at the root will be different, with high probability.

Then, the stored hash values at each of the children are compared with the hash values at the corresponding child in the other tree. Search traverses down each child whose hash value differs, until a leaf is reached. The traversal is done in parallel on both trees and requires communication to send tree node contents from one replica to the other.

At the leaf, if the hash values differ, the list of data item keys associated with the leaves, and the corresponding data item values are compared across the two trees, to find data items whose values differ as well as data items that are present in one of the trees but not in the other.

One such traversal takes time at most logarithmic in the number of leaf nodes of the tree; since the number of leaf nodes is chosen to be close to the number of data items, the traversal time is also logarithmic in the number of data items. This cost is paid at most once for each data item that differs between the two replicas. Furthermore, a path to a leaf is traversed only if there is, in fact, a difference at the leaf.

Thus, the overall cost for finding differences between two (potentially very large) sets is $O(m \log_2 N)$, where m is the number of data items that differ and N is the total number of data items. Wider trees can be used to reduce the number of nodes encountered in a traversal, which would be $\log_K N$ if each node has K children, at the cost of more data being transferred for each node. Wider trees are preferred if network latency is high compared to the network bandwidth.

Merkle trees have many applications; they can be used to find the difference in contents of two databases that are almost identical without transferring large amounts of data. Such inconsistencies can occur due to the use of protocols that only guarantee weak consistency. They could also occur because of message or network failures that result in differences in replicas, even if consensus or other protocols that guarantee consistent reads are used.

The original use of Merkle trees was for **verification of the contents** of a collection that may have potentially been corrupted by malicious users. Here, the Merkle tree leaf nodes must store the hash values of all data items that map to it, or a tree variant that only stores one data item at a leaf may be used. Further, the stored hash value at the root is digitally signed, meaning its contents cannot be modified by a malicious user who does not have the private key used for signing the value.

To check an entire relation, the hash values can be recomputed from the leaves upwards, and the recomputed hash value at the root can be compared with the digitally signed hash value stored at the root.

To check consistency of a single data item, its hash value is recomputed; and then so is the hash value for its leaf node n_i , using existing hash values for other data items

that hash to the same leaf. Next consider the parent node n_j of node n_i in the tree. The hash value of n_j is computed using the recomputed hash value of n_i and the already stored hash values of other children of n_j . This process is continued upward until the root of the tree. If the recomputed hash value at the root matches the signed hash value stored with the root, the contents of the data item can be determined to be uncorrupted.

The above technique works for detecting corruption since with suitably chosen hash functions, it is very hard for a malicious user to create replacement values for data items in a way that the recomputed hash value is identical to the signed hash value stored at the root.

23.7 Coordinator Selection

Several of the algorithms that we have presented require the use of a coordinator. If the coordinator fails because of a failure of the node at which it resides, the system can continue execution by restarting a new coordinator on another node. One way to continue execution is by maintaining a backup to the coordinator that is ready to assume responsibility if the coordinator fails. Another way is to “elect” a coordinator from among the nodes that are alive. We outline these options in this section. We then briefly describe fault-tolerant distributed services that have been developed to help developers of distributed applications perform these tasks.

23.7.1 Backup Coordinator

A **backup coordinator** is a node that, in addition to other tasks, maintains enough information locally to allow it to assume the role of coordinator with minimal disruption to the distributed system. All messages directed to the coordinator are received by both the coordinator and its backup. The backup coordinator executes the same algorithms and maintains the same internal state information (such as, for a concurrency coordinator, the lock table) as does the actual coordinator. The only difference in function between the coordinator and its backup is that the backup does not take any action that affects other nodes. Such actions are left to the actual coordinator.

In the event that the backup coordinator detects the failure of the actual coordinator, it assumes the role of coordinator. Since the backup has all the information available to it that the failed coordinator had, processing can continue without interruption.

The prime advantage of the backup approach is the ability to continue processing immediately. If a backup were not ready to assume the coordinator’s responsibility, a newly appointed coordinator would have to seek information from all nodes in the system so that it could execute the coordination tasks. Frequently, the only source of some of the requisite information is the failed coordinator. In this case, it may be necessary to abort several (or all) active transactions and to restart them under the control of the new coordinator.

Thus, the backup-coordinator approach avoids a substantial amount of delay while the distributed system recovers from a coordinator failure. The disadvantage is the overhead of duplicate execution of the coordinator's tasks. Furthermore, a coordinator and its backup need to communicate regularly to ensure that their activities are synchronized.

In short, the backup-coordinator approach incurs overhead during normal processing to allow fast recovery from a coordinator failure.

23.7.2 Election of Coordinator

In the absence of a designated backup coordinator, or in order to handle multiple failures, a new coordinator may be chosen dynamically by nodes that are live.

One possible approach is to have a designated node choose a new coordinator, when the current coordinator has failed. However, this raises the question of what to do if the node that chooses replacement coordinators itself fails.

If we have a fault-tolerant lock manager, a very effective way of choosing a new coordinator for a task is to use *lock leases*. The current coordinator has a lock lease on a data item associated with the task. If the coordinator fails, the lease will expire. If a participant determines that the coordinator may have failed, it attempts to get a lock lease for the task. Note that multiple participants may attempt to get a lease, but the lock manager ensures that only one of them can get the lease. The participant that gets the lease becomes the new coordinator. As discussed in Section 23.3.3, this ensures that only one node that can be the coordinator at a given time. Lock leases are widely used to ensure that a single node gets chosen as coordinator. However, observe that there is an underlying assumption of a fault-tolerant lock manager.

A participant determines that the coordinator may have failed if it is unable to communicate with the coordinator. Participants send periodic **heart-beat** messages to the coordinator and wait for an acknowledgment; if the acknowledgment is not received within a certain time, the coordinator is assumed to have failed.

Note that the participant cannot definitively distinguish a situation where the coordinator is dead from a situation where the network link between the node and the coordinator is cut. Thus, the system should be able to work correctly even if the current coordinator is alive, but another participant determines that the coordinator is dead. Lock leases ensure that at most one node can be the coordinator at any time; once a coordinator dies, another node can become the coordinator. However, lock leases work only if a fault-tolerant lock manager is available.

This raises the question of how to implement such a lock manager. We return later, in Section 23.8.4, to the question of how to implement a fault-tolerant lock manager. But it turns out that to do so efficiently, we need to have a coordinator. And, lock leases cannot be used to choose the coordinator for the lock manager! The problem of how to choose a coordinator without depending on a lock manager is solved by **election algorithms**, which enable the participating nodes to choose a new coordinator in a decentralized manner.

Suppose the goal is to elect a coordinator just once. Then, each node that wishes to become the coordinator proposes itself as a candidate to all the other nodes; such nodes are called *proposers*. The participating nodes then vote on which node among the candidates is to be chosen. If a majority of the participating nodes (called *acceptors*) vote for a particular candidate, it is chosen. A subset of nodes called *learners* ask the acceptor nodes for their vote and determine if a majority have voted for a particular candidate.

The problem with the above idea is that if there are multiple candidates, none of them may get a majority of votes. The question is what to do in such a situation. There are at least two approaches that have been proposed:

- Nodes are given unique numbers; if more than one candidate proposes itself, acceptors choose the highest-numbered candidate. Even then votes may be split with no majority decision, due to delayed or missing messages; in such a case, the election is run again. But if a node N_1 that was a candidate finds that a higher-numbered node N_2 has proposed itself as a candidate, then N_1 withdraws from the next round of the election. The highest-numbered candidate will win the election. The **bully algorithm** for election is based on this idea.

There are some subtle details due to the possibility that the highest-numbered candidate in one round may fail during a subsequent round, leading to there being no candidates at all! If a proposer observes that no coordinator was selected in a round where it withdrew itself as a candidate, it proposes itself as a candidate again in the next round.

Note also that the election has multiple rounds; each round has a number, and a candidate attaches a round number with the proposal. The round number is chosen to be the maximum round that it has seen, plus 1. A node can give a vote to only one candidate in a particular round, but it may change its vote in the next round.

- The second approach is based on *randomized retry*, which works as follows: If there is no majority decision in a particular round, all participants wait for a randomly chosen amount of time; if by that time a coordinator has been chosen by a majority of nodes, it is accepted as a coordinator. Otherwise, after the timeout, the node proposes itself as a candidate. As long as the timeouts are chosen properly (large enough compared to network latency) with high likelihood only one node proposes itself at a particular time and will get votes from a majority of nodes in a particular round.

If no candidate gets a majority vote in a round, the process is repeated. With very high probability, after a few rounds, one of the candidates gets a majority and is thus chosen as coordinator.

The randomized-retry approach was popularized by the Raft consensus algorithm, and it is easier to reason about it and show not just correctness, but also bounds on the expected time for an election round to succeed in choosing a coordinator, as compared to the node-numbering-based approach.

Note that the above description assumed that choosing a coordinator is a one-time activity. However, the chosen coordinator may fail, requiring a fresh election algorithm. The notion of a **term** is used to deal with this situation. As mentioned above, each time a node proposes itself as a coordinator, it associates the proposal with a round number, which is 1 more than the highest round number it has seen earlier, after ensuring that in the previous round no coordinator was chosen, or the chosen coordinator has subsequently failed. The round number is henceforth referred to as a **term**. When the election succeeds, the chosen coordinator is the coordinator for the corresponding term. If the election fails, the corresponding term does not have any coordinator chosen, but the election should succeed in a subsequent term.

Note also that there are subtle issues that arise since a node n may be disconnected from the network for a while, and it may get reconnected without ever realizing that it was disconnected. In the interim, the coordinator may have changed. In particular, if the node n was the coordinator, it may continue to think it is the coordinator, and some other node, say N_1 , which was also disconnected may think that n is still coordinator. However, if a coordinator was successfully elected, the majority of the nodes agree that some other node, say N_2 , is the coordinator.

In general, it is possible for more than one node to think that is the coordinator at the same time, although at most one of them can have the majority vote at that point in time.

To avoid this problem, each coordinator can be given a lease for a specified period. The coordinator can extend the lease by requesting an extension from other nodes and getting confirmation from a majority of the nodes. But if the coordinator is disconnected from a majority of the nodes, it cannot renew its lease, and the lease expires. A node can vote for a new coordinator only if the last lease time that it confirmed to the earlier coordinator has expired. Since a new coordinator needs a majority vote, it cannot get the vote until the lease time of the previous coordinator has expired.

However, even if leases are used to ensure that two nodes cannot be coordinators at the same time, delayed messages can result in a node getting a message from an old coordinator after a new one has been elected.

To deal with this problem, the current term of the sender is included with each message exchanged in the system. Note that when a node n is elected as coordinator, it has an associated term t ; participant nodes that learn that n is the coordinator are aware of the current term t . A node may receive a message with an old term either because an old coordinator did not realize it has been replaced or because of message delivery delay; the latter problem can occur even if leases or other mechanisms ensure that only one node can be the coordinator at a time. In either case, a node that receives a **stale message**, that is, one with a term older than the current term of the node, it can ignore the message. If a node receives a message with a higher number, it is behind the rest of the system, and it needs to find out the current term and coordinator by contacting other nodes.

Some protocols do not require the coordinator to store any state information; in such cases, the new coordinator can take over without any further actions. However,

other protocols require coordinators to retain state information. In such cases, the new coordinator has to reconstruct the state information from persistent data and recovery logs created by the previous coordinator. Such logs, in turn, need to be replicated to multiple nodes so that the loss of a node does not result in the loss of access to the recovery data. We shall see how to ensure availability by means of data replication in subsequent sections.

23.7.2.1 Distributed Coordination Services

There are a very large number of distributed applications that are in daily use today. Instead of each one having to implement its own mechanism for electing coordinators (among other tasks), it makes sense to develop a fault-tolerant coordination service that can be used by multiple distributed applications.

The **ZooKeeper** service is one such very widely used fault-tolerant distributed coordination service. The **Chubby** service developed earlier at Google is another such service, which is widely used for applications developed by Google. These services internally use consensus protocols to implement fault tolerance; we study consensus protocols in Section 23.8.

These services provide a file-system-like API, which supports the following features, among others:

- *Store (small amounts of) data* in files, with a hierarchical namespace. A typical use for such storage is to store configuration information that can be used to start up a distributed application, or for new nodes to join a distributed application by finding out which node is currently the coordinator.
- *Create and delete files*, which can be used to implement locking. For example, to get a lock, a process can attempt to create a file with a name corresponding to the lock. If another process has already created the file, the coordination service will return an error, so the process knows it could not get the lock.

For example, a node that acts as a master for a tablet in a key-value store would get a lock on a file whose name is the identifier of the tablet. This ensures that two nodes cannot be masters for the tablet at the same time.

If an overall application master detects that a tablet master has died, it could release the lock. If the service supports lock leases, this could happen automatically, if the tablet master does not renew its lease.

- *Watch for changes on a file*, which can be used by a process to check if a lock has been released, or to be informed about other changes in the system that require action by the process.

23.8 Consensus in Distributed Systems

In this section we first describe the consensus problem in a distributed system, that is, how a set of nodes agree on a decision in a fault-tolerant way. Distributed consensus is

a key building block for protocols that update replicated data in a fault-tolerant manner. We outline two consensus protocols, Paxos and Raft. We then describe replicated state machines, which can be used to make services, such as data storage systems and lock managers, fault tolerant. We end the section by describing how consensus can be used to make two-phase commit nonblocking.

23.8.1 Problem Overview

Software systems need to make decisions, such as the coordinator's decision on whether to commit or abort a transaction when using 2PC, or a decision on which node is to act as coordinator, in case a current coordinator fails.

If the decision is made by a single node, such as the commit/abort decision made by a coordinator node in 2PC, the system may block in case the node fails, since other nodes have no way of determining what decision was made. Thus, to ensure fault tolerance, multiple nodes must participate in the decision protocol; even if some of these nodes fail, the protocol must be able to reach a decision. A single node may make a proposal for a decision, but it must involve the other nodes to reach a decision in a fault-tolerant manner.

The most basic form of the **distributed consensus problem** is thus as follows: a set of n nodes (referred to as *participants*) need to agree on a decision by executing a protocol such that:

- All participants must “learn” the same value for the decision even if some nodes fail during the execution of the protocol, or messages are lost, or there are network partitions.
- The protocol should not block, and must terminate, as long as some majority of the nodes participating remain alive and can communicate with each other.

Any real system cannot just make a single decision once, but needs to make a series of decisions. A good abstract of the process of making multiple consensus decisions is to treat each decision as *adding a record to a log*. Each node has a copy of the log, and records are appended to the log at each node. There can potentially be conflicts on what record is added at what point in a log. The **multiple consensus protocol** viewed from this perspective needs to ensure that the log is uniquely defined.

Most consensus protocols allow temporary divergence of logs across nodes while the protocol is being executed; that is, the same log position at different nodes may have different records, and the end of the log may be different at different nodes. Shared-log consensus protocols keep track of an index into the log such that any entry before that index has definitely been agreed upon. Any entries after that index may be in the process of being agreed upon, or may be entries from failed attempts at consensus. However, the protocols subsequently bring the inconsistent parts of the log logs back in synchronization. To do this, log records at some nodes may be deleted after being inserted; such log records are viewed as not yet committed and cannot be used to make

decisions. Only log records in the prefix of the log that are in the committed prefix may be used to make decisions.

Several protocols have been proposed for distributed consensus. Of these, the Paxos family of protocols is one of the most popular, and it has been implemented in many systems. While the basic Paxos protocol is intuitively easy to understand at a high level, there are a number of details in its implementation that are rather complicated, and particularly so in the multiple consensus version. To address this issue, the Raft consensus protocol was developed, with ease of understanding and implementation being key goals, and it has been adopted by many systems. We outline the intuition behind these protocols in this section.

A key idea behind distributed consensus protocols is the idea of *voting* to make a decision; a particular decision succeeds only if a majority of the participating nodes vote for it. Note that if two or more different values are proposed for a particular decision, at most one of them can be voted for by a majority; thus, it is not possible for two different values to be chosen. Even if some nodes fail, if a majority of the participants vote for a value, it gets chosen, thus making the voting fault tolerant as long as a majority of the participants are up and connected to each other. There is, however, a risk that votes may get split between the proposed values, and some nodes may not vote if they fail; as a result, no value may be decided on. In such a case the voting procedure has to be executed again.

While the above intuition is easy enough to understand, there are many details that make the protocols nontrivial. We study some of these issues in the following sections.

We note that although we study some of the features of the Paxos and Raft consensus protocols, we omit a number of details that are needed for correct operation to keep our description concise.

We also note that a number of other consensus protocols have been proposed, and some of them are widely used, such as the Zab protocol which is part of the ZooKeeper distributed coordination service.

23.8.2 The Paxos Consensus Protocol

The basic Paxos protocol for making a single decision has the following participants.

1. One or more nodes that can propose a value for the decision; such nodes are called **proposers**.
2. One or more nodes that act as **acceptors**. An acceptor may get proposals with different values from different proposers and must choose (vote for) only one of the values.

Note that failure of an acceptor does not cause a problem, as long as a majority of the acceptors are live and reachable. Failure or disconnection of a majority would block the consensus protocol.

3. A set of nodes, called **learners**, query the acceptors to find what value each acceptor voted for in a particular round. (Acceptors could also send the value they accepted to the learners, without waiting for a query from the learner.)

Note that the same node can play the roles of proposer, acceptor, and learner.

If a majority of the acceptors voted for a particular value, that value is the chosen (consensus) value for that decision. But there are two problems:

1. It is possible for votes to be split among multiple proposals, and no proposal is accepted by a majority of the acceptors.

If any proposed value is to get a majority, at least some acceptors must change their decision. Thus, we must allow another round of decision making, where acceptors may choose a new value. This may need to be repeated as long as required until one value wins a majority vote.

2. Even if a majority of nodes do accept a value, it is possible that some of these nodes die or get disconnected after accepting a value, but before any learner finds out about their acceptance, and the remaining acceptors of that value do not constitute a majority.

If this is treated as the failure of a round, and a different value is chosen in a subsequent round, we have a problem. In particular, a learner that learned about the earlier majority would conclude that a particular value was chosen, while another learner could conclude that a different value was chosen, which is not acceptable.

Note also that acceptors must log their decision so when they recover they know what decision they made earlier.

The first problem above, namely, split votes, does not affect correctness, but it affects performance. To avoid this problem, Paxos makes use of a coordinator node. Proposers send a proposal to the coordinator, which picks one of the proposed values and follows the preceding steps to get a majority vote. If proposals come from only one coordinator, there is no conflict, and the lone proposed value gets a majority vote (modulo network and node failures).

Note that if the coordinator dies or is unreachable, a new coordinator can be elected, using techniques we saw earlier in Section 23.7, and the new coordinator can then do the same job as the earlier coordinator. Coordinators have no local state, so the new one can take over without any recovery steps.

The second problem, namely, different values getting majorities in different rounds, is a serious problem and must be avoided by the consensus protocol. To do so, Paxos uses the following steps:

1. Each proposal in Paxos has a number; different proposals must have different numbers.

2. In phase 1a of the protocol, a proposer sends a *prepare* message to acceptors, with its proposal number n .
3. In phase 1b of the protocol, an acceptor that receives a prepare message with number n checks if it has already responded to a message with a number higher than n . If so, it ignores the message. Otherwise, it remembers the number n and responds with the highest proposal number $m < n$ that it has already accepted, along with the corresponding value v ; if it has not accepted any value earlier, it indicates so in its response. (Note that responding is different from accepting.)
4. In phase 2a, the proposer checks if it got a response from a majority of the acceptors. If it does, it chooses a value v as follows: If none of the acceptors has already accepted any value, the proposer may use whatever value it intended to propose. If at least one of the acceptors responded that it accepted a value v with some number m , the proposer chooses the value v that has the highest associated number m (note that m must be $< n$).

The proposer now sends an *accept* request with the chosen value v and number n .

5. In phase 2b, when an acceptor gets an accept request with value v and number n , it checks if it has responded to a prepare message with number $n_1 > n$; if so it ignores the accept request. Otherwise, it accepts the proposed value v with number n .

The above protocol is quite clever, since it ensures the following: if a majority of acceptors accepted a value v (with any number n), then even if there are further proposals with number $n_1 > n$, the value proposed will be value v . Intuitively, the reason is that a value can be accepted with number n only if a majority of nodes respond to a prepare message with number n ; let us call this set of acceptors P . Suppose a value v had been accepted earlier by a majority of nodes with number m ; call this set of nodes A . Then A and P must have a node in common, and the common node will respond with value v and number m .

Note that some other proposal with a number $p > n$ may have been made earlier, but if it had been accepted by even one node, then a majority of nodes would have responded to the proposal with number p , and thus will not respond to the proposal with number n . Thus, if a proposal with value v is accepted by a majority of nodes, we can be sure that any further proposal will be for the already chosen value v .

Note that if a learner finds that no proposal was accepted by a majority of nodes, it can ask any proposer to issue a fresh proposal. If a value v had been accepted by a majority of nodes, it would be found and accepted again, and the learner would now learn about the value. If no value was accepted by a majority of nodes earlier, the new proposal could be accepted.

The above algorithm is for a single decision. Paxos has been extended to allow a series of decisions; the extended algorithm is called Multi-Paxos. Real implementations

also need to deal with other issues, such as how to add a node to the set of acceptors, or to remove a node from the set of acceptors if it is down for a long time, without affecting the correctness of the protocol. References with more details about Paxos and Multi-Paxos may be found in the bibliographic notes for this chapter, available online.

23.8.3 The Raft Consensus Protocol

There are several consensus protocols whose goal is to maintain a log, to which records can be appended in a fault-tolerant manner. Each node participating in such a protocol has a replica of the log. Log-based protocols simplify the handling of multiple decisions. The Raft consensus protocol is an example of such a protocol, and it was designed to be (relatively) easy to understand.

A key goal of log-based protocols is to keep the log replicas in sync by presenting a logical view of appending records atomically to all copies of the log. In fact, atomically appending the same entry to all replicas is not possible, due to failures. Recall that failure modes may include a node being temporarily disconnected and missing some updates, without ever realizing it was disconnected. Further, a log append may be done at just a few nodes, and the append process may fail subsequently, leaving other replicas without the record. Thus, ensuring all copies of the log are identical at all times is impossible. Such protocols must ensure the following:

- Even if a log replica is temporarily inconsistent with another, the protocol will bring it back in sync eventually by deleting and replacing log records on some copies.
- A log entry will not be treated as committed until the algorithm guarantees that it will never be deleted.

Protocols such as Raft that are based on log replication can allow each node to run a “state machine,” with log entries used as commands to the state machine; state machines are described in Section 23.8.4.

The Raft algorithm is based on having a coordinator, which is called a **leader** in Raft terminology. The other participating nodes are called **followers**. Since leaders may die and need to be replaced, time is divided into terms, which are identified by integers. Each term has a unique leader, although some terms may not have any associated leader. Later terms have higher identifiers than earlier terms.

Leaders are elected in Raft using the randomized-retry algorithm outlined in Section 23.7.2. Recall that the randomized-retry algorithm already incorporates the notion of a term. A node that votes for a leader does so for a specific term. Nodes keep track of the currentTerm based on messages from leaders or requests for votes.

Note that a leader N_1 may get temporarily disconnected, but get reconnected after other nodes find the leader cannot be reached, and elect a new leader N_2 . Node N_1

log index	1	2	3	4	5	6	7
leader	1 $x \leftarrow 2$	1 $z \leftarrow 2$	1 $x \leftarrow 3$	2 $x \leftarrow 4$	3 $x \leftarrow 1$	3 $y \leftarrow 6$	3 $z \leftarrow 4$
follower 1	1 $x \leftarrow 2$	1 $z \leftarrow 2$	1 $x \leftarrow 3$	2 $x \leftarrow 4$	3 $x \leftarrow 1$		
follower 2	1 $x \leftarrow 2$	1 $z \leftarrow 2$	1 $x \leftarrow 3$	2 $x \leftarrow 4$	3 $x \leftarrow 1$	3 $y \leftarrow 6$	3 $z \leftarrow 4$
follower 3	1 $x \leftarrow 2$	1 $z \leftarrow 2$	1 $x \leftarrow 3$				
follower 4	1 $x \leftarrow 2$	1 $z \leftarrow 2$	1 $x \leftarrow 3$	2 $x \leftarrow 4$	3 $x \leftarrow 1$	3 $y \leftarrow 6$	



committed entries

Figure 23.9 Example of Raft logs.

does not know that there is a new leader and may continue to execute the actions of a leader. The protocol should be robust to such situations.

Figure 23.9 shows an example of Raft logs at a leader and four followers. Note that the log index denotes the position of a particular record in a log. The number at the top of each log record is the term in which the log record was created, while the part below it shows the log entry, assumed here to record assignments to different variables.

Any node that wishes to append a record to the replicated log sends a log append request to the current leader. The leader adds its term as a field of the log records and appends the record to its log; it then sends an *AppendEntries* remote procedure call to the other nodes; the call contains several parameters, including these:

- *term*: the term of the current leader.
- *previousLogEntryPosition*: the position in the log of the preceding log entry.
- *previousLogEntryTerm*: the term associated with the preceding log entry.
- *logEntries*: an array of log records, allowing the call to append multiple log records at the same time.
- *leaderCommitIndex*: an index such that all log records at that index or earlier are committed. Recall that a log entry is not considered committed until a leader has confirmed that a majority of nodes have accepted that log entry. The leader keeps, in *leaderCommitIndex*, a position in the log such that all log records at that index and earlier are committed; this value is sent along with the *AppendEntries* call so that the nodes learn which log records have been committed.

If a majority of the nodes respond to the call with a return value true, the leader can report successful log append (along with the position in the log) to the node that initiated the log append. We will shortly see what happens if a majority do not respond with true.

Each follower that receives an AppendEntries message does the following:

1. If the term in the message is less than `currentTerm`, then Return false.
2. If the log does not contain an entry at a previous log entry position, whose term matches the term in the message, then Return false.
3. If there is an existing entry at the log position that is different from the first log record in the AppendEntries message, the existing entry and all subsequent log entries are deleted.
4. Any log records in the `logEntries` parameter that are not already in the log are appended to the log.
5. The follower also keeps track of a local `commitIndex` to track which records are committed. If the `leaderCommitIndex > commitIndex`, set `commitIndex = min(leaderCommitIndex, index of last entry in log)`.
6. Return true.

Note that the last step keeps track of the last committed log record. It is possible that the leader's log is ahead of the local log, so `commitIndex` cannot be blindly set to `leaderCommitIndex`, and it may need to be set to the local end of log if the `leaderCommitIndex` is ahead of the local end of log.

Figure 23.9 shows that different followers may have different log states, since some AppendEntries messages may not have reached those nodes. The part of the log up to entry 6 is present at a majority of nodes (namely, the leader, follower 2 and follower 4). On receipt of a true response to the AppendEntries call for the log record at position 6 from these followers, the leader can set `leaderCommitIndex` to 6.

It is possible for a node N_1 to be a leader in some term, and on temporary disconnection it may get replaced by a new leader N_2 in the next term. N_1 may not realize that there is a new leader for some time and may send `appendEntry` messages to other nodes. However, a majority of the other nodes will know about the new leader, and would have a term higher than that of N_1 . Thus, these nodes would return false, and include their current term in the response. Node N_1 would then realize that there is a leader with a new term; it then switches from the role of leader to that of follower.

The protocol must deal with the fact that some nodes may have outdated logs. Note that in step 2 of the follower protocol, the follower returns false if its log is outdated. In such a case, the leader will retry an AppendEntries, sending it all log records from an even earlier point in the log. This may happen several times, until the leader sends log

records from a point that is already in the follower log. At this point, the `AppendEntries` command would succeed.

A key remaining problem is to ensure that if a leader dies, and another one takes over, the log is brought to a consistent state. Note that the leader may have appended some log entries locally and replicated some of them to some other nodes, but the new leader may or may not have all these records. To deal with this situation, the Raft protocol includes steps to ensure the following:

1. The protocol ensures that any node elected as leader has all the committed log entries. To do so, any candidate must contact a majority of the nodes and send information about its log state when seeking a vote. A node will vote for a candidate in the election only if it finds that the candidate's log state is *at least as up-to-date* as its own; note that the definition of "at least as up-to-date" is a little complicated since it involves term identifiers in log records, and we omit details.

Since the above check is done by a majority of the nodes that voted for the new leader, any committed entry would certainly be present in the log of the newly elected leader.

2. The protocol then forces all other nodes to replicate the leader's log.

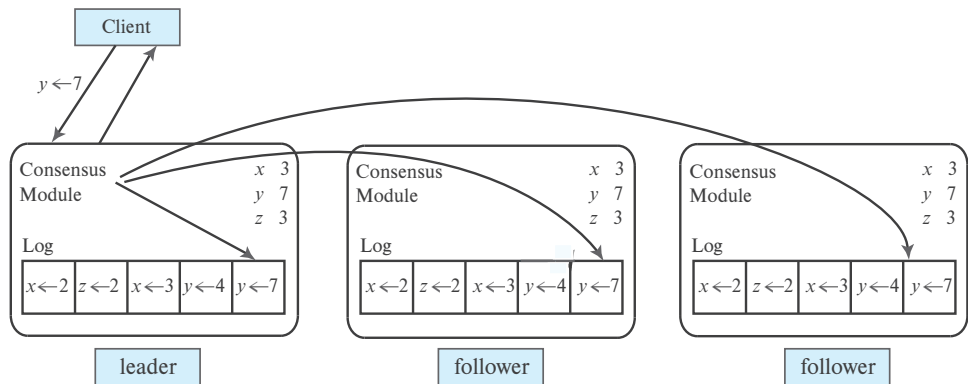
Note that the first step above does not actually find up to what log record is committed. Some of the log records at the new leader may not have been committed earlier, but may get committed when the new leader's log is replicated in this step.

There is also a subtle detail in that the new leader cannot count the number of replicas with a particular record from an earlier term, and declare it committed if it is at a majority of the nodes. Intuitively, the problem is because of the definition of "at least as up-to-date" and the possibility that a leader may fail, recover, and be elected as leader again. We omit details, but note that the way this problem is solved is for the new leader to replicate a new log record in its current term; when that log record is determined to be at a majority of the replicas, it and all earlier log records can be declared to be committed.

It should be clear that although the protocol, like Paxos, seems simple at a high level, there are many subtle details that need to be taken care of to ensure consistency even in the face of multiple failures and restarts. There are further details to be taken care of, including how to change the *cluster membership*, that is, the set of nodes that form the system, while the system is running (doing so carelessly can result in inconsistencies). Details of the above steps, including proofs of correctness, may be found in references in the bibliographic notes for this chapter, available online.

23.8.4 Fault-Tolerant Services Using Replicated State Machines

A key requirement in many systems is for a service to be made fault tolerant. A lock manager is an example of such a service, as is a key-value storage system.



Leader declares log record committed after it is replicated at a majority of nodes. Update of state machine at each replica happens only after log record has been committed.

Figure 23.10 Replicated state machine.

A very powerful approach to making services fault tolerant is to model them as “state machines” and then use the idea of replicated state machines that we describe next.

A **state machine** receives inputs and has a stored state; it makes state transitions on each input and may output some results along with the state transition. A **replicated state machine** is a state machine that is replicated on multiple nodes to make it fault tolerant. Intuitively, even if one of the nodes fails, the state and output can be obtained from any of the nodes that are alive, provided all the state machines are in a consistent state. The key to ensuring that the state machine replicas are consistent is to (a) require the state machines to be deterministic, and (b) ensure that all replicas get exactly the same input in the same order.

To ensure that all replicas get exactly the same input in the same order, we just append the inputs to a replicated log, using, for example, techniques we saw earlier in Section 23.8.3. As soon as a log entry is determined to be committed, it can be given as input to the state machine, which can then process it.

Figure 23.10 depicts a replicated state machine based on a replicated log. When a client issues a command, such as $y \leftarrow 7$ in the figure, the command is sent to the leader, where the command is appended to the log. The leader then replicates the command to the logs at the followers. Once a majority have confirmed that the command has been replicated in their logs, the leader declares the command committed and applies the command to its state machine. It also informs the followers of the commit, and the followers then apply the command to their state machine.

In the example in Figure 23.10, the state machine merely records the value of the updated variable; but in general, the state machine may execute any other actions. The actions are, however, required to be deterministic, so all state machines are in exactly

the same state when they have executed the same set of commands; the order of execution of commands will be the same since commands are executed in the log order.

Commands such as lock request must return a status to the caller. The status can be returned from any one of the replicas where the command is performed. Most implementations return the status from the leader node, since the request is sent to the leader, and the leader is also the first node to know when a log record has been committed (replicated to a majority of the nodes).

We now consider two applications that can be made fault-tolerant using the replicated state machine concept.

We first consider how to implement a **fault-tolerant lock manager**. A lock manager gets commands, namely, lock requests and releases, and maintains a state (lock table). It also gives output (lock grants or rollback requests on deadlock) on processing inputs (lock requests or releases). Lock managers can easily be coded to be deterministic, that is, given the same input, the state and output will be the same even if the code is executed again on a different node.

Thus, we can take a centralized implementation of a lock manager and run it on each node. Lock requests and releases are appended to a replicated log using, for example, the Raft protocol. Once a log entry is committed, the corresponding command (lock request or release) can be processed, in order, by the lock manager code at each replica. Even if some of the replicas fail, the other replicas can continue processing as long as a majority are up and connected.

Now consider the issue of implementing a **fault-tolerant key-value store**. A single-node storage system can be modeled as a state machine that supports `put()` and `get()` operations. The storage system is treated as a state machine, and the state machine is run on multiple nodes.

The `put()` operations are appended to the log using a consensus protocol and are processed when the consensus protocol declares the corresponding log records to be committed (i.e., replicated to a majority of the nodes).

If the consensus protocol uses leaders, `get()` operations need not be logged, and need to be executed only on the leader. To ensure that a `get()` operation sees the most recent `put()` on the same data item, all `put()` operations on the same data item that precede the `get()` operation in the log must be committed before the `get()` operation is processed. (If a consensus protocol does not use a leader, `get()` operations can also be logged and executed by at least one of the replicas which returns the value to the caller.)

Google's Spanner is an example of a system that uses the replicated state machine approach to creating a fault-tolerant implementation of a key-value storage system and a lock manager.

To ensure scalability, Spanner breaks up data into partitions, each of which has a subset of the data. Each partition has its data replicated across multiple nodes. Each node runs two state machines: one for the key-value storage system, and one for the lock manager. The set of replicas for a particular partition are called a *Paxos group*; one

of the nodes in a Paxos group acts as the Paxos group leader. Lock manager operations, as well as key-value store operations for a particular partition, are initiated at the Paxos group leader for that partition. The operations are appended to a log, which is replicated to the other nodes in the Paxos group using the Paxos consensus protocol.² Requests are applied in order at each member of the Paxos group, once they are committed.

As an optimization, `get()` operations are not logged, and executed only at the leader as described earlier. As a further optimization, Spanner allows reads to run as of a particular point in time, allowing reads to be executed at any replica of the partition (in other words, any other member of the Paxos group) that is sufficiently up to date, based on the multiversion two-phase locking protocol described earlier in Section 23.5.1.

23.8.5 Two-Phase Commit Using Consensus

Given a consensus protocol implementation, we can use it to create a **non-blocking two-phase commit** implementation. The idea is simple: instead of a coordinator recording its commit or abort decision locally, it uses a consensus protocol to record its decision in a replicated log. Even if the coordinator subsequently fails, other participants in the consensus protocol know about the decision, so the blocking problem is avoided.

In case the coordinator fails before making a decision for a transaction, a new coordinator can first check the log to see if a decision was made earlier, and if not it can make a commit/abort decision and use the consensus protocol to record the decision.

For example, in the Spanner system developed by Google, a transaction may span multiple partitions. Two-phase commit is initiated by a client and coordinated by the Paxos group leader at one of the partitions where the transaction executed. All other partitions where an update was performed acts as a participant in the two-phase commit protocol. Prepare and commit messages are sent to the Paxos group leader node of each of the partitions; recall that two-phase commit participants as well as coordinators record decisions in their local logs. These decisions are recorded by each leader, using consensus involving all the other nodes in its Paxos group.

If a Paxos group member other than the leader dies, the leader can continue processing the two-phase commit steps, as long as a majority of the group nodes are up and connected. If a Paxos group leader fails, one of the other group members takes over as the group leader. Note that all the state information required to continue commit processing is available to the new leader. Log records written during commit processing are available since the log is replicated. Also, recall from Section 23.8.4 that Spanner makes the lock manager fault tolerant by using the replicated state machine concept. Thus, a consistent replica of the lock table is also available with the new leader. Thus, the two-phase commit steps of both the coordinator and the participants can continue to be executed even if some nodes fail.

²The Multi-Paxos version of Paxos is used, but we shall just refer to it as Paxos for simplicity.

23.9 Summary

- A distributed database system consists of a collection of sites or nodes, each of which maintains a local database system. Each node is able to process local transactions: those transactions that access data in only that single node. In addition, a node may participate in the execution of global transactions: those transactions that access data in several nodes. Transaction managers at each node manage access to local data, while the transaction coordinator coordinates execution of global transactions across multiple nodes.
- A distributed system may suffer from the same types of failure that can afflict a centralized system. There are, however, additional failures with which we need to deal in a distributed environment, including the failure of a node, the failure of a link, loss of a message, and network partition. Each of these problems needs to be considered in the design of a distributed recovery scheme.
- To ensure atomicity, all the nodes in which a transaction T executed must agree on the final outcome of the execution. T either commits at all nodes or aborts at all nodes. To ensure this property, the transaction coordinator of T must execute a commit protocol. The most widely used commit protocol is the two-phase commit protocol.
- The two-phase commit protocol may lead to blocking, the situation in which the fate of a transaction cannot be determined until a failed node (the coordinator) recovers. We can use distributed consensus protocols, or the three-phase commit protocol, to reduce the risk of blocking.
- Persistent messaging provides an alternative model for handling distributed transactions. The model breaks a single transaction into parts that are executed at different databases. Persistent messages (which are guaranteed to be delivered exactly once, regardless of failures), are sent to remote nodes to request actions to be taken there. While persistent messaging avoids the blocking problem, application developers have to write code to handle various types of failures.
- The various concurrency-control schemes used in a centralized system can be modified for use in a distributed environment. In the case of locking protocols, the only change that needs to be incorporated is in the way that the lock manager is implemented. Centralized lock managers are vulnerable to overloading and to failures. Deadlock detection in a distributed-lock-manager environment requires cooperation between multiple nodes, since there may be global deadlocks even when there are no local deadlocks.
- The timestamp ordering and validation based protocols can also be extended to work in a distributed setting. Timestamps used to order transactions need to be made globally unique.

- Protocols for handling replicated data must ensure consistency of data. Linearizability is a key property that ensures that concurrent reads and writes to replicas of a single data item can be serialized.
- Protocols for handling replicated data include the primary copy, majority, biased, and quorum consensus protocols. These have different trade-offs in terms of cost and ability to work in the presence of failures.
- The majority protocol can be extended by using version numbers to permit transaction processing to proceed even in the presence of failures. While the protocol has a significant overhead, it works regardless of the type of failure. Less-expensive protocols are available to deal with node failures, but they assume network partitioning does not occur.
- To provide high availability, a distributed database must detect failures, reconfigure itself so that computation may continue, and recover when a processor or a link is repaired. The task is greatly complicated by the fact that it is hard to distinguish between network partitions and node failures.
- Globally consistent and unique timestamps are key to extending multiversion two-phase locking and snapshot isolation to a distributed setting.
- The CAP theorem indicates that one cannot have consistency and availability in the face of network partitions. Many systems tradeoff consistency to get higher availability. The goal then becomes eventual consistency, rather than ensuring consistency at all times. Detecting inconsistency of replicas can be done by using version vector schemes and Merkle trees.
- Many database systems support asynchronous replication, where updates are propagated to replicas outside the scope of the transaction that performed the update. Such facilities must be used with great care, since they may result in nonserializable executions.
- Some of the distributed algorithms require the use of a coordinator. To provide high availability, the system must maintain a backup copy that is ready to assume responsibility if the coordinator fails. Another approach is to choose the new coordinator after the coordinator has failed. The algorithms that determine which node should act as a coordinator are called election algorithms. Distributed coordination services such as ZooKeeper support coordinator selection in a fault-tolerant manner.
- Distributed consensus algorithms allow consistent updates of replicas, even in the presence of failures, without requiring the presence of a coordinator. Coordinators may still be used for efficiency, but failure of a coordinator does not affect correctness of the protocols. Paxos and Raft are widely used consensus protocols. Replicated state machines, which are implemented using consensus algorithms, can be used to build a variety of fault-tolerant services.

Review Terms

- Distributed transactions
 - Local transactions
 - Global transactions
- Transaction manager
- Transaction coordinator
- System failure modes
- Network partition
- Commit protocols
- Two-phase commit protocol (2PC)
 - Ready state
 - In-doubt transactions
 - Blocking problem
- Distributed consensus
- Three-phase commit protocol (3PC)
- Persistent messaging
- Concurrency control
- Single lock manager
- Distributed lock manager
- Deadlock handling
 - Local wait-for graph
 - Global wait-for graph
 - False cycles
- Lock leases
- Timestamping
- Replicated data
- Linearizability
- Protocols for replicas
 - Primary copy
 - Majority protocol
 - Biased protocol
 - Quorum consensus protocol
- Robustness
 - Majority-based approach
 - Read one, write all
 - Read one, write all available
 - Node/Site reintegration
- External consistency
- Commit wait
- CAP theorem
- BASE properties
- Asynchronous replication
- Lazy propagation
- Master – slave replication
- Multimaster (update-anywhere) replication
- Asynchronous view maintenance
- Eventual consistency
- Version-vector scheme
- Merkle tree
- Coordinator selection
- Backup coordinator
- Election algorithms
- Bully algorithm
- Term
- Distributed consensus protocol
- Paxos
 - Proposers
 - Acceptors
 - Learners
- Raft

- Leaders
- Followers
- Replicated state machine
- Fault tolerant lock manager
- Non-blocking two-phase commit

Practice Exercises

- 23.1** What are the key differences between a local-area network and a wide-area network, that affect the design of a distributed database?
- 23.2** To build a highly available distributed system, you must know what kinds of failures can occur.
- List possible types of failure in a distributed system.
 - Which items in your list from part a are also applicable to a centralized system?
- 23.3** Consider a failure that occurs during 2PC for a transaction. For each possible failure that you listed in Exercise 23.2a, explain how 2PC ensures transaction atomicity despite the failure.
- 23.4** Consider a distributed system with two sites, A and B . Can site A distinguish among the following?
- B goes down.
 - The link between A and B goes down.
 - B is extremely overloaded and response time is 100 times longer than normal.

What implications does your answer have for recovery in distributed systems?

- 23.5** The persistent messaging scheme described in this chapter depends on time-stamps. A drawback is that they can discard received messages only if they are too old, and may need to keep track of a large number of received messages. Suggest an alternative scheme based on sequence numbers instead of time-stamps, that can discard messages more rapidly.
- 23.6** Explain the difference between data replication in a distributed system and the maintenance of a remote backup site.
- 23.7** Give an example where lazy replication can lead to an inconsistent database state even when updates get an exclusive lock on the primary (master) copy if data were read from a node other than the master.
- 23.8** Consider the following deadlock-detection algorithm. When transaction T_i , at site S_1 , requests a resource from T_j , at site S_3 , a request message with time-

stamp n is sent. The edge (T_i, T_j, n) is inserted in the local wait-for graph of S_1 . The edge (T_i, T_j, n) is inserted in the local wait-for graph of S_3 only if T_j has received the request message and cannot immediately grant the requested resource. A request from T_i to T_j in the same site is handled in the usual manner; no timestamps are associated with the edge (T_i, T_j) . A central coordinator invokes the detection algorithm by sending an initiating message to each site in the system.

On receiving this message, a site sends its local wait-for graph to the coordinator. Note that such a graph contains all the local information that the site has about the state of the real graph. The wait-for graph reflects an instantaneous state of the site, but it is not synchronized with respect to any other site.

When the controller has received a reply from each site, it constructs a graph as follows:

- The graph contains a vertex for every transaction in the system.
- The graph has an edge (T_i, T_j) if and only if:
 - There is an edge (T_i, T_j) in one of the wait-for graphs.
 - An edge (T_i, T_j, n) (for some n) appears in more than one wait-for graph.

Show that, if there is a cycle in the constructed graph, then the system is in a deadlock state, and that, if there is no cycle in the constructed graph, then the system was not in a deadlock state when the execution of the algorithm began.

23.9 Consider the chain-replication protocol, described in Section 23.4.3.2, which is a variant of the primary-copy protocol.

- a. If locking is used for concurrency control, what is the earliest point when a process can release an exclusive lock after updating a data item?
- b. While each data item could have its own chain, give two reasons it would be preferable to have a chain defined at a higher level, such as for each partition or tablet.
- c. How can consensus protocols be used to ensure that the chain is uniquely determined at any point in time?

23.10 If the primary copy scheme is used for replication, and the primary gets disconnected from the rest of the system, a new node may get elected as primary. But the old primary may not realize it has got disconnected, and may get reconnected subsequently without realizing that there is a new primary.

- a. What problems can arise if the old primary does not realize that a new one has taken over?
- b. How can leases be used to avoid these problems?

- c. Would such a situation, where a participant node gets disconnected and then reconnected without realizing it was disconnected, cause any problem with the majority or quorum protocols?
- 23.11** Consider a federated database system in which it is guaranteed that at most one global transaction is active at any time, and every local site ensures local serializability.
- a. Suggest ways in which the federated database system can ensure that there is at most one active global transaction at any time.
 - b. Show by example that it is possible for a nonserializable global schedule to result despite the assumptions.
- 23.12** Consider a federated database system in which every local site ensures local serializability, and all global transactions are read only.
- a. Show by example that nonserializable executions may result in such a system.
 - b. Show how you could use a ticket scheme to ensure global serializability.
- 23.13** Suppose you have a large relation $r(A, B, C)$ and a materialized view $v = \pi_{A \gamma_{\text{sum}(B)}}(r)$. View maintenance can be performed as part of each transaction that updates r , on a parallel/distributed storage system that supports transactions across multiple nodes. Suppose the system uses two-phase commit along with a consensus protocol such as Paxos, across geographically distributed data centers.
- a. Explain why it is not a good idea to perform view maintenance as part of the update transaction, if some values of attribute A are “hot” at certain points in time, that is, many updates pertain to those values of A .
 - b. Explain how operation locking (if supported) could solve this problem.
 - c. Explain the tradeoffs of using asynchronous view maintenance in this context.

Exercises

- 23.14** What characteristics of an application make it easy to scale the application by using a key-value store, and what characteristics rule out deployment on key-value stores?
- 23.15** Give an example where the read one, write all available approach leads to an erroneous state.

- 23.16** In the majority protocol, what should the reader do if it finds different values from different copies, to (a) decide what is the correct value, and (b) to bring the copies back to consistency? If the reader does not bother to bring the copies back to consistency, would it affect correctness of the protocol?
- 23.17** If we apply a distributed version of the multiple-granularity protocol of Chapter 18 to a distributed database, the site responsible for the root of the DAG may become a bottleneck. Suppose we modify that protocol as follows:
- Only intention-mode locks are allowed on the root.
 - All transactions are given the strongest intention-mode lock (IX) on the root automatically.
- Show that these modifications alleviate this problem without allowing any non-serializable schedules.
- 23.18** Discuss the advantages and disadvantages of the two methods that we presented in Section 23.3.4 for generating globally unique timestamps.
- 23.19** Spanner provides read-only transactions a snapshot view of data, using multi-version two-phase locking.
- a. In the centralized multi-version 2PL scheme, read-only transactions never wait. But in Spanner, reads may have to wait. Explain why.
 - b. Using an older timestamp for the snapshot can reduce waits, but has some drawbacks. Explain why, and what the drawbacks are.
- 23.20** Merkle trees can be made short and fat (like B⁺-trees) or thin and tall (like binary search trees). Which option would be better if you are comparing data across two sites that are geographically separated, and why?
- 23.21** Why is the notion of term important when an election is used to choose a coordinator? What are the analogies between elections with terms and elections used in a democracy?
- 23.22** For correct execution of a replicated state machine, the actions must be deterministic. What could happen if an action is non-deterministic?

Further Reading

Textbook coverage of distributed transaction processing, including concurrency control and the two-phase and three-phase commit protocols, is provided by [Bernstein and Goodman (1981)] and [Bernstein and Newcomer (2009)]. Textbook discussions of distributed databases are offered by [Ozsu and Valduriez (2010)]. A collection of papers on data management on cloud systems is in [Ooi and Parthasarathy (2009)].

The implementation of the transaction concept in a distributed database is presented by [Gray (1981)] and [Traiger et al. (1982)]. The 2PC protocol was developed by [Lampson and Sturgis (1976)]. The three-phase commit protocol is from [Skeen (1981)]. Techniques for non-blocking two-phase commit based on consensus, called Paxos Commit, are described in [Gray and Lamport (2004)].

Chain replication was initially proposed by [van Renesse and Schneider (2004)] and an optimized version of was proposed by [Terrace and Freedman (2009)].

Distributed optimistic concurrency control is described in [Agrawal et al. (1987)], while distributed snapshot isolation is described in [Binnig et al. (2014)] and [Schenkel et al. (1999)]. The externally consistent distributed multi-version 2PL scheme used in Spanner is described in [Corbett et al. (2013)].

The CAP theorem was conjectured by [Brewer (2000)], and was formalized and proved by [Gilbert and Lynch (2002)]. [Cooper et al. (2008)] describe Yahoo!'s PNUTS system, including its support for asynchronous maintenance of replicas using a publish-subscribe system. Parallel view maintenance is described in [Chen et al. (2004)] and [Zhang et al. (2004)], while asynchronous view maintenance is described in [Agrawal et al. (2009)]. Transaction processing in federated database systems is discussed in [Mehrotra et al. (2001)].

Paxos is described in [Lamport (1998)]; Paxos is based on features from several earlier protocols, reference in [Lamport (1998)]. Google's Chubby lock service, which is based on Paxos, is described by [Burrows (2006)]. The widely used ZooKeeper system for distributed coordination is described in [Hunt et al. (2010)], and the consensus protocol (also known as atomic broadcast protocol) used in ZooKeeper is described in [Junqueira et al. (2011)]. The Raft consensus protocol is described in [Ongaro and Ousterhout (2014)].

Bibliography

- [Agrawal et al. (1987)] D. Agrawal, A. Bernstein, P. Gupta, and S. Sengupta, "Distributed optimistic concurrency control with reduced rollback", *Distributed Computing*, Volume 2, Number 1 (1987), pages 45–59.
- [Agrawal et al. (2009)] P. Agrawal, A. Silberstein, B. F. Cooper, U. Srivastava, and R. Ramakrishnan, "Asynchronous view maintenance for VLSD databases", In *Proc. of the ACM SIGMOD Conf. on Management of Data* (2009), pages 179–192.
- [Bernstein and Goodman (1981)] P. A. Bernstein and N. Goodman, "Concurrency Control in Distributed Database Systems", *ACM Computing Surveys*, Volume 13, Number 2 (1981), pages 185–221.
- [Bernstein and Newcomer (2009)] P. A. Bernstein and E. Newcomer, *Principles of Transaction Processing*, 2nd edition, Morgan Kaufmann (2009).
- [Binnig et al. (2014)] C. Binnig, S. Hildenbrand, F. Färber, D. Kossmann, J. Lee, and N. May, "Distributed snapshot isolation: global transactions pay globally, local transactions

pay locally”, *VLDB Journal*, Volume 23, Number 6 (2014), pages 987–1011.

- [Brewer (2000)] E. A. Brewer, “Towards Robust Distributed Systems (Abstract)”, In *Proc. of the ACM Symposium on Principles of Distributed Computing* (2000), page 7.
- [Burrows (2006)] M. Burrows, “The Chubby Lock Service for Loosely-Coupled Distributed Systems”, In *Symp. on Operating Systems Design and Implementation (OSDI)* (2006), pages 335–350.
- [Chen et al. (2004)] S. Chen, B. Liu, and E. A. Rundensteiner, “Multiversion-based view maintenance over distributed data sources”, *ACM Transactions on Database Systems*, Volume 29, Number 4 (2004), pages 675–709.
- [Cooper et al. (2008)] B. F. Cooper, R. Ramakrishnan, U. Srivastava, A. Silberstein, P. B. Hannon, H.-A. Jacobsen, N. Puz, D. Weaver, and R. Yerneni, “PNUTS: Yahoo!’s Hosted Data Serving Platform”, *Proceedings of the VLDB Endowment*, Volume 1, Number 2 (2008), pages 1277–1288.
- [Corbett et al. (2013)] J. C. Corbett et al., “Spanner: Google’s Globally Distributed Database”, *ACM Trans. on Computer Systems*, Volume 31, Number 3 (2013).
- [Gilbert and Lynch (2002)] S. Gilbert and N. Lynch, “Brewer’s Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services”, *SIGACT News*, Volume 33, Number 2 (2002), pages 51–59.
- [Gray (1981)] J. Gray, “The Transaction Concept: Virtues and Limitations”, In *Proc. of the International Conf. on Very Large Databases* (1981), pages 144–154.
- [Gray and Lamport (2004)] J. Gray and L. Lamport, “Consensus on transaction commit”, *ACM Transactions on Database Systems*, Volume 31, Number 1 (2004), pages 133–160.
- [Hunt et al. (2010)] P. Hunt, M. Konar, F. Junqueira, and B. Reed, “ZooKeeper: Wait-free Coordination for Internet-scale Systems”, In *USENIX Annual Technical Conference (USENIX ATC)* (2010), pages 11–11.
- [Junqueira et al. (2011)] F. P. Junqueira, B. C. Reed, and M. Serafini, “Zab: High-performance broadcast for primary-backup systems”, In *IEEE/IFIP 41st International Conference on Dependable Systems Networks (DSN)* (2011), pages 245–256.
- [Lamport (1998)] L. Lamport, “The Part-Time Parliament”, *ACM Trans. Comput. Syst.*, Volume 16, Number 2 (1998), pages 133–169.
- [Lampson and Sturgis (1976)] B. Lampson and H. Sturgis, “Crash Recovery in a Distributed Data Storage System”, Technical report, Computer Science Laboratory, Xerox Palo Alto Research Center, Palo Alto (1976).
- [Mehrotra et al. (2001)] S. Mehrotra, R. Rastogi, Y. Breitbart, H. F. Korth, and A. Silberchatz, “Overcoming Heterogeneity and Autonomy in Multidatabase Systems.”, *Inf. Comput.*, Volume 167, Number 2 (2001), pages 137–172.
- [Ongaro and Ousterhout (2014)] D. Ongaro and J. K. Ousterhout, “In Search of an Understandable Consensus Algorithm”, In *USENIX Annual Technical Conference (USENIX ATC)* (2014), pages 305–319.

- [**Ooi and Parthasarathy (2009)**] B. C. Ooi and S. Parthasarathy, “Special Issue on Data Management on Cloud Computing Platforms”, *IEEE Data Engineering Bulletin*, Volume 32, Number 1 (2009).
- [**Ozsu and Valduriez (2010)**] T. Ozsu and P. Valduriez, *Principles of Distributed Database Systems*, 3rd edition, Prentice Hall (2010).
- [**Schenkel et al. (1999)**] R. Schenkel, G. Weikum, N. Weisenberg, and X. Wu, “Federated Transaction Management with Snapshot Isolation”, In *Eight International Workshop on Foundations of Models and Languages for Data and Objects, Transactions and Database Dynamics* (1999), pages 1–25.
- [**Skeen (1981)**] D. Skeen, “Non-blocking Commit Protocols”, In *Proc. of the ACM SIGMOD Conf. on Management of Data* (1981), pages 133–142.
- [**Terrace and Freedman (2009)**] J. Terrace and M. J. Freedman, “Object Storage on CRAQ: High-Throughput Chain Replication for Read-Mostly Workloads”, In *USENIX Annual Technical Conference (USENIX ATC)* (2009).
- [**Traiger et al. (1982)**] I. L. Traiger, J. N. Gray, C. A. Galtieri, and B. G. Lindsay, “Transactions and Consistency in Distributed Database Management Systems”, *ACM Transactions on Database Systems*, Volume 7, Number 3 (1982), pages 323–342.
- [**van Renesse and Schneider (2004)**] R. van Renesse and F. B. Schneider, “Chain Replication for Supporting High Throughput and Availability”, In *Symp. on Operating Systems Design and Implementation (OSDI)* (2004), pages 91–104.
- [**Zhang et al. (2004)**] X. Zhang, L. Ding, and E. A. Rundensteiner, “Parallel multisource view maintenance”, *VLDB Journal*, Volume 13, Number 1 (2004), pages 22–48.

Credits

The photo of the sailboats in the beginning of the chapter is due to ©Pavel Nesvadba/Shutterstock.

